SQL 전문가 가이드 요약

목차

1. 챕터별 핵심 내용 요약 4

제1장 데이터 모델링의 이해 4

제2장 데이터 모델과 SQL 4

제1장 SQL 기본 5

제2장 SQL 활용 5

제3장 관리 구문 6

제1장 SQL 수행 구조 6

제2장 SQL 분석 도구 6

제3장 인덱스 튜닝 7

제4장 조인 튜닝 7

제5장 SQL 옵티마이저 8

제6장 고급 SQL 튜닝 8

제7장 Lock과 트랜잭션 동시성 제어 9

2. 챕터별 상세 학습 노트 10

제1장 데이터 모델링의 이해 10

제2장 데이터 모델과 SQL 17

제1장 SQL 기본 21

제2장 SQL 활용 32

제3장 관리 구문 38

제1장 SQL 수행 구조 42

제2장 SQL 분석 도구 46

제3장 인덱스 튜닝 50

제4장 조인 튜닝 54

제5장 SQL 옵티마이저 58

제6장 고급 SQL 튜닝 63

제7장 Lock과 트랜잭션 동시성 제어 71

3. SQL 문법과 원문 코드 해설 75

제1장 데이터 모델링의 이해 75

제2장 데이터 모델과 SQL 76

제1장 SQL 기본 80

제2장 SQL 활용 . . . 88
제3장 관리 구문 . . . 99
제1장 SQL 수행 구조 . . . 105
제2장 SQL 분석 도구 . . . 108
제3장 인덱스 튜닝 . . . 111
제4장 조인 튜닝 . . . 114
제5장 SQL 옵티마이저 . . . 119
제6장 고급 SQL 튜닝 . . . 124
제7장 Lock과 트랜잭션 동시성 제어 . . . 131

4. 핵심 개념 비교 및 구조화 자료 . . . 135
제1장 데이터 모델링의 이해 . . . 135
제2장 데이터 모델과 SQL . . . 135
제1장 SQL 기본 . . . 136
제2장 SQL 활용 . . . 137
제3장 관리 구문 . . . 138
제1장 SQL 수행 구조 . . . 139
제2장 SQL 분석 도구 . . . 140
제3장 인덱스 튜닝 . . . 140
제4장 조인 튜닝 . . . 141
제5장 SQL 옵티마이저 . . . 141
제6장 고급 SQL 튜닝 . . . 142
제7장 Lock과 트랜잭션 동시성 제어 . . . 143

5. 전체 핵심 내용 복습 자료 . . . 145
전체 핵심 요약 . . . 145
SQL 문법 요약표 . . . 148
주요 개념 비교표 . . . 156
주요 코드 모음 . . . 159
핵심 내용 체크리스트 . . . 167

수록 범위는 제공된 786쪽 PDF에서 확인되는 본문 12장입니다. 목차에만 기재된 부록은 파일에 없어 포함하지 않았습니다. 책 소개·저자·출판 정보와 연습문제 재출제는 제외했습니다. 출처의 쪽수는 인쇄면 번호가 아닌 원문 PDF의 파일 쪽수입니다. DBMS 지원 범위와 동작 비교는 원문의 설명 기준이며, 코드는 원문 식별자·상수·조건을 보존하고 공백과 줄 배치만 정리했습니다. 원문에서 문장·코드가 어긋나는 부분은 해당 해설에서 구분했습니다.

1. 챕터별 핵심 내용 요약

제1장 데이터 모델링의 이해

과목 I. 데이터 모델링의 이해

원문 PDF 9-76쪽

핵심 주제와 주요 개념

- 데이터 모델링의 목적과 단계
- 데이터 독립성과 3단계 스키마
- 엔터티·속성·관계·식별자
- ERD 해석과 식별자관계의 선택

핵심 내용의 흐름

1. 현실의 업무를 추상화·단순화·명확화한다.
2. 관리할 대상을 엔터티로, 성격을 속성으로, 연관성을 관계로 표현한다.
3. 인스턴스를 구분할 식별자와 관계의 제약을 정하고, SQL과 데이터 구조에 미칠 영향을 검토한다.

학습 집중 포인트

- 집합인 엔터티와 개별 요소인 인스턴스를 구분한다.
- 관계차수와 필수·선택 참여를 함께 읽는다.
- 식별자관계와 비식별자관계의 장단점을 업무·조회 조건과 연결한다.

제2장 데이터 모델과 SQL

과목 I. 데이터 모델링의 이해

원문 PDF 78-118쪽

핵심 주제와 주요 개념

- 제1·2·3정규형과 반정규화의 성능 영향
- 관계의 식별자 상속과 조인
- 모델에 표현되는 트랜잭션
- NULL 연산과 집계
- 본질식별자·인조식별자의 장단점

핵심 내용의 흐름

1. 정규화로 속성과 엔터티를 올바르게 배치한다.
2. 분리된 데이터를 관계의 매핑키로 연결하여 SQL을 구성한다.
3. 함께 발생하는 데이터는 한 트랜잭션으로 처리하고 NULL과 식별자 제약으로 데이터 품질을 검토한다.

학습 집중 포인트

- 정규화의 각 기준을 데이터 중복·변경 부하·정합성과 연결한다.
- 반정규화의 조회 이득뿐 아니라 추가 갱신과 데이터 불일치 비용을 판단한다.
- NULL 처리 위치와 평균의 분모를 구분한다.

- 유일한 인조번호가 업무상 중복까지 막는 것은 아니라는 점을 기억한다.

제1장 SQL 기본

과목 II. SQL 기본과 활용

원문 PDF 122-270쪽

핵심 주제와 주요 개념

- 관계형 데이터베이스·테이블·SQL의 기본 개념
- SELECT와 표현식·별칭
- 문자·숫자·날짜·변환·NULL 함수와 CASE
- WHERE 조건과 연산 우선순위
- 집계·GROUP BY·HAVING
- 정렬과 SELECT의 논리 처리 순서
- 등가·비등가·외부 조인 및 표준 조인

핵심 내용의 흐름

1. 테이블과 데이터 유형을 이해하고 SELECT로 필요한 칼럼과 표현식을 정한다.
2. WHERE로 개별 행을 제한하고 함수를 이용하여 값을 변환한다.
3. GROUP BY와 집계함수로 그룹을 만들고 HAVING으로 그룹을 제한한 뒤 ORDER BY로 정렬한다.
4. 여러 테이블의 관계와 조인 조건을 이용하여 필요한 정보 집합을 연결한다.

학습 집중 포인트

- NULL인 값과 행이 없는 공집합을 구분한다.
- WHERE·HAVING·SELECT·ORDER BY의 역할과 사용 가능한 칼럼 범위를 구분한다.
- 연산자 우선순위와 문자 상수 인용부호를 정확하게 적용한다.
- 조인 조건에 맞는 행과 보존해야 하는 행을 구분하고 ON·USING·NATURAL의 제약을 확인한다.

제2장 SQL 활용

과목 II. SQL 기본과 활용

원문 PDF 272-388쪽

핵심 주제와 주요 개념

- 서브쿼리로 메인 집합을 유지하며 조건·값을 공급하고, 집합 연산자로 결과 집합을 결합한다.
- 그룹 함수는 여러 수준의 집계 결과를, 윈도우 함수는 행을 유지한 분석 결과를 만든다.
- Top N, 계층 전개, PIVOT·UNPIVOT, 정규 표현식으로 조회·분석의 표현 범위를 넓힌다.

핵심 내용의 흐름

1. 서브쿼리의 종속 관계와 반환 형태 → 결과 집합의 합·교·차 → 집계 수준 확장 → 행 간 분석 → 상위 행 제한 → 계층 전개 → 행·열 변환 → 패턴 검색·치환

학습 집중 포인트

- 단일 행과 다중 행 연산자의 대응, 조인과 서브쿼리의 결과 단위를 구분한다.
- 집계 수준·파티션·정렬·윈도우 범위를 각각 확인한다.

- 정렬 후 행 제한, 계층 전개 조건과 사후 필터, 변환 대상 밖의 암묵적 집계 열을 주의한다.

제3장 관리 구문

과목 II. SQL 기본과 활용

원문 PDF 390-461쪽

핵심 주제와 주요 개념

- DML로 행을 입력·수정·삭제하고 MERGE로 입력과 수정을 분기한다.
- TCL로 연관 작업의 확정·취소 범위를 관리한다.
- DDL로 테이블·칼럼·제약조건을 정의하고 DCL로 사용자·권한·ROLE을 관리한다.

핵심 내용의 흐름

1. 행의 변경 → 논리적 작업 단위의 확정·취소 → 저장 구조와 무결성 제약 정의 → 접근 권한 관리

학습 집중 포인트

- WHERE 없는 UPDATE·DELETE의 전체 행 적용, 서브쿼리 UPDATE의 NULL 대입을 확인한다.
- DBMS별 자동 커밋과 명시적 트랜잭션 차이를 구분한다.
- DELETE·TRUNCATE·DROP, 시스템 권한·객체 권한, 로그인·사용자·스키마를 구별한다.

제1장 SQL 수행 구조

과목 III. SQL 고급 활용 및 튜닝

원문 PDF 465-503쪽

핵심 주제와 주요 개념

- 데이터베이스·인스턴스·프로세스·저장공간·메모리의 관계
- SQL을 실행계획으로 바꾸는 과정과 옵티마이저 힌트
- 블록 단위 I/O와 논리적 읽기 최소화

핵심 내용의 흐름

1. 저장 구조와 메모리 이해 → SQL 파싱·최적화·실행 → 실제 블록 접근 → I/O 효율화

학습 집중 포인트

- 커밋과 데이터 파일 기록을 구분한다.
- 비용은 예상값이고 실행통계는 실측값임을 구분한다.
- 캐시 히트율이 높아도 논리적 읽기가 많으면 비효율적이다.

제2장 SQL 분석 도구

과목 III. SQL 고급 활용 및 튜닝

원문 PDF 504-529쪽

핵심 주제와 주요 개념

- 예상 실행계획과 실제 실행통계의 구분
- Oracle·SQL Server 트레이스 수집과 해석
- 대기 이벤트와 AWR을 이용한 응답 시간 분석

핵심 내용의 흐름

1. 실행 전 계획 확인 → 실제 수행량·처리 단계 측정 → CPU·대기 원인 분리 → 병목 구간 분석

학습 집중 포인트

- 측정 도구가 SQL을 실제 수행하는지 먼저 구분한다.
- 예상 행 수와 실제 행 수, 누적값과 마지막 실행값을 구분한다.
- 대기 이벤트 순위만 보고 장애를 단정하지 않는다.

제3장 인덱스 튜닝

과목 III. SQL 고급 활용 및 튜닝

원문 PDF 530-579쪽

핵심 주제와 주요 개념

- B*Tree 구조와 스캔 방식별 조건
- 인덱스 종류와 저장 방식의 차이
- 테이블 랜덤 접근 및 인덱스 스캔량 최소화
- 조회 패턴과 DML을 함께 고려한 인덱스 설계

핵심 내용의 흐름

1. 수직 탐색으로 시작점 찾기 → 리프 범위 스캔 → 필요 시 ROWID로 테이블 접근 → 두 단계의 비효율을 각각 줄이기

학습 집중 포인트

- 인덱스 사용 여부보다 읽는 범위와 테이블 방문 횟수를 본다.
- NULL 저장, 클러스터·클러스터형·IOT, 키·INCLUDE 칼럼을 구분한다.
- 선행 등치 조건과 후행 범위 조건이 스캔 범위를 어떻게 정하는지 이해한다.

제4장 조인 튜닝

과목 III. SQL 고급 활용 및 튜닝

원문 PDF 580-617쪽

핵심 주제와 주요 개념

- NL·소트 머지·해시 조인의 처리 구조와 선택 조건
- 스칼라 서브쿼리의 캐시·다중 값·Unnesting
- 집계 후 조인·배타적 관계·부등호·선분이력·ROWID 활용

핵심 내용의 흐름

1. 선행 집합과 반복 탐색 이해 → 정렬·해시를 이용한 대량 처리 → 쿼리 구조와 이력 모델에 맞는 조인 최적화

학습 집중 포인트

- 전체 행 수보다 필터 후 집합 크기와 조인 시도 횟수를 본다.
- 정렬·해시 구축의 선행 비용과 부분범위처리 가능 구간을 구분한다.
- 시간 이력의 조회 기준이 과거 거래 시점인지 현재 시점인지 명확히 한다.

제5장 SQL 옵티마이저

과목 III. SQL 고급 활용 및 튜닝

원문 PDF 618-665쪽

핵심 주제와 주요 개념

- 최적화 목표와 비용 추정의 근거, 옵티마이저의 한계
- SQL·실행계획 공유, 바인드 변수와 애플리케이션 커서 캐싱
- 결과의 의미를 유지하며 처리 구조를 바꾸는 쿼리 변환

핵심 내용의 흐름

1. 통계정보에서 선택도와 카디널리티를 추정하고 액세스·조인 비용을 비교한다.
2. 비용을 들여 만든 실행계획을 공유하고, 반복 파싱까지 줄여 시스템 자원을 절약한다.
3. 서브쿼리·뷰·조건식을 변환해 더 다양한 실행계획을 검토하되 결과와 예상 비용을 확인한다.

학습 집중 포인트

- 비용은 실측값이 아니라 통계와 가정에 의한 예상치다.
- SQL의 Static/Dynamic 구분과 바인드 변수 사용 여부는 다른 기준이다.
- 쿼리 변환의 우선 조건은 결과 보장이며, 변환한다고 항상 더 빨라지는 것은 아니다.

제6장 고급 SQL 튜닝

과목 III. SQL 고급 활용 및 튜닝

원문 PDF 667-762쪽

핵심 주제와 주요 개념

- 소트·DML·데이터베이스 Call의 불필요한 일량 줄이기
- 파티션 Pruning과 병렬 처리로 대용량 작업 범위 나누기
- CASE·복제·UNION ALL·페이징·윈도우 함수·WITH를 이용한 효율적인 집합 처리

핵심 내용의 흐름

1. 소트를 없애거나 대상과 메모리 사용량을 줄이고, DML의 인덱스·로그·조인 비용을 조정한다.
2. 반복 호출을 One SQL·Array Processing으로 바꾸고, 실제 필요한 결과만 가져온다.
3. 파티션과 병렬화는 데이터를 나누는 기준 및 통신량을 고려해 적용한다.
4. 온라인의 첫 응답과 배치의 전체 완료라는 서로 다른 목표에 맞춰 SQL 형태를 선택한다.

학습 집중 포인트

- 소트를 없애는 것과 소트 메모리를 줄이는 것, Top N Stopkey와 Top N Sort를 구분한다.
- 속도가 빠른 적재 방식에는 Lock과 복구 제약이 함께 있다.
- 병렬도만 높여서는 전체 배치 성능이 보장되지 않는다.
- One SQL 자체보다 중복 액세스·I/O·Call을 얼마나 줄이는지가 핵심이다.

제7장 Lock과 트랜잭션 동시성 제어

과목 III. SQL 고급 활용 및 튜닝

원문 PDF 764-786쪽

핵심 주제와 주요 개념

- Lock 호환성, 블로킹·교착상태, SQL Server와 Oracle의 구현 차이
- 트랜잭션 ACID와 격리성 수준에서 허용하는 읽기 현상
- 비관적·낙관적 동시성 제어와 MVCC의 읽기 일관성

핵심 내용의 흐름

1. 같은 데이터의 동시 접근을 직렬화하는 Lock과 유지 기간을 이해한다.
2. 격리 수준에 따라 다른 트랜잭션의 변경이 읽기에 어떻게 보이는지 구분한다.
3. 업무 단위의 충돌을 제어하고, 과거 버전을 이용해 읽기와 쓰기의 간섭을 줄인다.

학습 집중 포인트

- Lock 이름이 같아도 제품별 구현·모드·해제 시점은 다르다.
- 동시성을 높이면서 데이터 무결성을 유지하는 것이 목표이며, 격리성을 무조건 올리는 것이 해답은 아니다.
- MVCC의 문장 기준과 트랜잭션 기준을 구분하고 Undo 재사용의 제약을 기억한다.

2. 챕터별 상세 학습 노트

제1장 데이터 모델링의 이해

과목 I. 데이터 모델링의 이해

원문 PDF 9-76쪽

제1절 데이터 모델의 이해

원문 PDF 10-32쪽

모델링은 복잡한 현실의 현상을 일정한 표기법으로 표현하는 과정이다. 추상화는 현실을 정해진 형식으로 바꾸는 것, 단순화는 제한된 규약과 언어로 이해하기 쉽게 만드는 것, 명확화는 애매함을 제거하여 정확하게 기술하는 것이다. 모델은 의사소통과 기록을 위한 수단이며 시스템의 계획·분석·설계뿐 아니라 구축 후 변경과 관리에도 사용된다.

관점	파악하는 내용
데이터 관점	업무가 어떤 데이터와 관련되는지, 데이터 사이의 관계는 무엇인지 파악한다.
프로세스 관점	실제로 수행하는 업무와 수행해야 하는 일을 파악한다.
상관 관점	업무 처리 방법에 따라 데이터가 어떤 영향을 받는지 파악한다.

데이터 모델링은 정보시스템을 구축하기 위해 업무에 존재하는 데이터와 필요한 정보를 분석하고 설계하는 과정이다. 누가 접근하는지, 어떻게 처리하는지, 전산화하는지와 독립적으로 업무 규칙을 표현한다. 목적은 첫째 업무의 정보 요구를 정확하게 분석하고 설명하는 것, 둘째 분석 결과로 데이터베이스를 생성하여 개발과 데이터 관리에 활용하는 것이다. 따라서 모델링은 테이블을 만들기 위한 그림 작업에만 한정되지 않는다.

데이터 모델은 현재 또는 원하는 시스템 모습을 가시화하고 구조와 행동을 명세화하며, 구축을 위한 구조화된 틀을 제공한다. 구축 과정에서 결정한 사항을 문서화하고, 관심 영역에 따라 다른 영역의 세부를 숨기는 관점과 목적에 맞는 상세 수준을 제공한다. SQL을 기능과 성능에 맞게 작성하려면 먼저 이러한 논리 구조를 이해해야 한다.

중요성은 파급효과, 정보 요구의 간결한 표현, 데이터 품질에서 나온다. 구축 후반에 데이터 구조를 바꾸면 표준·프로그램·이행 데이터와 테스트까지 광범위한 영향 분석과 수정이 필요하다. 반대로 잘 만든 모델은 많은 기능 명세보다 업무의 요구와 한계를 간결하게 보여준다. 중복의 미정의, 불충분한 업무 규칙, 같은 성격의 데이터 분리로 인한 불일치는 장기간 축적한 데이터를 활용하는 시점에 품질 문제와 기회 손실로 나타날 수 있다.

- 중복을 경계한다. 같은 정보를 여러 장소에 저장하면 일관성을 유지하기 위한 부담이 커진다.
- 비유연성을 경계한다. 데이터의 정의를 사용 프로세스와 분리하여 작은 업무 변화가 구조 전체의 변경으로 번지는 가능성을 줄인다.
- 비일관성을 경계한다. 중복이 없어도 관련 데이터의 갱신 규칙이 불명확하면 모순이 생기므로 데이터 사이의 연관 규칙을 정의한다.

단계	중점과 산출물
개념적 모델링	추상화 수준이 높고 업무 중심이며 포괄적이다. 핵심 엔터티와 관계를 발견하여 조직의 요구를 공식화하고 사용자·개발자의 논의 기반을 만든다.

단계	중점과 산출물
논리적 모델링	업무 데이터의 논리 구조와 규칙을 구체화한다. 키·속성·관계를 정확하게 표현하고 식별자 확정, 정규화, M:M 해소, 참조 무결성 규칙 및 이력 관리 전략을 정한다.
물리적 모델링	실제 저장소에 구현할 구조를 정한다. 테이블·칼럼, 저장 장치와 저장 구조, 접근 방법, 성능 등 물리 조건을 고려한다.

논리 모델링의 핵심은 업무 데이터에 존재하는 사실을 접근 주체나 처리 방법과 별개로 인식하여 기록하는 것이다. 정규화는 중복을 제거하고 속성을 적절한 엔터티에 배치하여 일관성과 신뢰성을 높인다. 프로젝트에서는 보통 계획·분석 단계에 개념 모델, 분석 단계에 논리 모델, 설계 단계에 물리 모델을 수행한다. 실제 프로젝트에서는 개념과 논리 작업을 함께 진행하기도 하고, 반복 개발에서는 분석·설계 양쪽에서 논리·물리 모델을 발전시킨다. 데이터 축과 애플리케이션 축의 산출물은 지속적으로 상호 검증한다.

데이터 독립성은 응용 프로그램과 데이터 구조가 서로 과도하게 종속되지 않게 하는 것이다. 접근 방식에 따라 파일 구조까지 바꾸던 방식의 한계를 줄이고 유지보수 비용·복잡도·중복을 낮추며 요구 변화에 대응하기 위해 필요하다. 이를 이해하는 기준이 외부·개념·내부의 3단계 구조와 각 단계 사이의 사상이다.

스키마	역할
외부 스키마	개별 사용자 또는 프로그램이 보는 개인적 데이터베이스 관점이다. 여러 외부 스키마가 존재할 수 있다.
개념 스키마	모든 사용자 관점을 통합한 조직 전체의 데이터와 관계를 표현한다. 하나의 통합된 개념 스키마를 구성한다.
내부 스키마	물리적 장치에 데이터가 실제로 저장되는 형식과 방법을 표현한다.

논리적 독립성은 개념 스키마의 변화가 외부 스키마와 응용에 영향을 주지 않도록 지원하는 성질이다. 물리적 독립성은 내부 저장 구조가 바뀌어도 개념·외부 스키마와 응용에 영향을 주지 않게 하는 성질이다. 단계가 나뉘어 있다는 사실만으로 자동 보장되는 것은 아니다. 외부와 개념을 연결하는 논리적 사상, 개념과 내부를 연결하는 물리적 사상을 변경에 맞추어 관리해야 독립성이 유지된다.

모델링의 기본 요소는 업무가 관여하는 어떤 것, 그것의 성격, 그것 사이의 관계이다. 집합 개념으로는 엔터티·속성·관계이고, 개별 개념으로는 인스턴스·속성값·관계 패어링이다. 이 책은 실무 용례에 따라 엔터티를 집합, 인스턴스를 단일 요소로 사용한다. 예컨대 과목은 엔터티, 수학과 영어는 그 인스턴스이다.

모델링은 DBA만의 작업이 아니다. 업무를 분석하는 개발자와 분석가가 모델을 작성하고, 모델러와 DBA는 방법·표준·검증을 지원한다. 모든 구축 참여자는 모델을 해석할 수 있어야 하고 현업 담당자도 의사소통에 필요한 수준으로 이해해야 한다. 업무를 잘못 해석한 모델은 프로그램과 모든 트랜잭션, 축적되는 데이터 전반에 영향을 미치기 때문이다.

ERD는 분석한 엔터티와 관계를 도식화하여 업무 흐름과 프로세스의 연관성을 공유하는 핵심 산출물이다. 원문은 Chen, IDEF1X, IE/Crow's Foot, Min-Max/ISO, UML, Barker 표기법을 소개하고 IE와 Barker를 함께 사용한다. 표기법은 달라도 엔터티·속성·관계를 표현한다는 의미를 먼저 이해하면 상호 전환이 가능하다.

- ERD 작성 흐름: 엔터티를 그린다 → 적절히 배치한다 → 관계를 연결한다 → 관계명을 적는다 → 관계차수를 표시한다 → 필수·선택 여부를 표시한다.
- 배치할 때 중요한 엔터티를 왼쪽 상단에, 업무 중심 엔터티를 중앙에 두고 관련 엔터티를 주변에 배치하면 읽기와 연결이 편리하다. 원문의 배치 원칙은 프로젝트 상황에 따라 조정할 수 있는 가이드이다.

- 관계를 중복 연결하거나 불필요한 순환 관계를 만들지 않도록 확인하고, 관계명은 현재형과 구체적인 동사를 사용한다. 관계선에는 하나·다수와 필수·선택을 별도로 표현한다.

좋은 데이터 모델은 필요한 데이터가 모두 정의된 완전성, 같은 사실을 한 번만 기록하는 중복 배제, 업무 규칙의 명확한 표현을 갖추어야 한다. 또 애플리케이션과 독립되고 통합된 데이터를 재사용할 수 있어야 하며, 관련자가 같은 의미로 이해하는 의사소통 도구가 되어야 한다. 동일한 성격의 데이터를 조직 전체에서 한 번 정의하여 여러 영역에서 참조하는 통합성이 중요하다. 원문은 성능 등 부가 목적에 따른 의도적 중복은 존재할 수 있다고 구분한다. 잘 정돈된 통합은 업무 변화에 대한 확장성과 단순성에도 연결된다.

제2절 엔터티

원문 PDF 33-41쪽

엔터티는 업무에 필요하고 유용한 정보를 저장·관리하기 위한 집합적 대상이다. 사람·장소·물건뿐 아니라 사건과 개념도 포함한다. 동질적인 인스턴스들이나 그 행위의 집합이며, 집합의 특성은 속성으로 설명한다. 눈에 보이는 사물만 찾으면 보험상품·사건 같은 중요한 관리 대상을 놓칠 수 있다.

엔터티는 인스턴스의 집합이고 인스턴스는 그 집합에 속하는 한 요소다. 과목에 수학·영어가, 강사에 개별 강사가, 사건에 사건번호로 구분되는 개별 사건이 속한다. ERD에서는 보통 사각형으로 표현하되 표기법에 따라 속성·식별자 표시가 달라진다.

성립 조건	검토 이유
해당 업무에서 필요한 정보	환자는 병원에서는 핵심 대상이지만 일반 회사 인사 업무에서 반드시 별도로 관리하는 대상은 아니다.
유일하게 식별 가능	사원 이름은 동명이인이 있을 수 있으므로 개별 사원을 유일하게 구분하는 식별자가 필요하다.
영속적인 인스턴스 집합	한 개의 대상만을 가리키는 것이 아니라 둘 이상의 인스턴스를 갖는 집합인지 검증한다.
업무 프로세스에서 이용	입력·조회·수정·삭제가 전혀 없다면 잘못 도출한 엔터티 또는 누락한 프로세스인지 확인한다.
속성 포함	이름만 있고 속성이 없거나 주식별자만 있는 경우 분석 누락을 검토한다. 관계 엔터티는 주식별자 속성만으로 성립할 수 있다.
다른 엔터티와 관계	고립되어 있다면 연관성을 놓쳤는지 검토한다. 통계·코드·시스템 내부용 엔터티는 관계 표현을 생략할 수 있다.

관계 생략은 업무 관계가 전혀 없다는 뜻과 구분한다. 통계 엔터티는 원천 업무 데이터에서 조회 목적의 별도 집합을 만들기 때문에 관계를 생략하기도 한다. 코드 엔터티는 과도한 관계선이 가독성을 해치고 참조 규칙을 DB 외부에서 관리하는 경우가 있다. 트랜잭션 로그처럼 시스템 내부 필요로 생성되는 엔터티도 업무 관계를 생략해 표현할 수 있다.

분류 기준	유형과 의미
유무형	유형엔터티는 사원·물품처럼 물리 형태가 있고 안정적으로 활용된다. 개념엔터티는 조직·보험상품처럼 물리 형태 없이 관리할 정보이다. 사건엔터티는 주문·청구·미납처럼 업무 수행에 따라 발생하며 발생량이 많을 수 있다.
발생 시점	기본엔터티는 업무에 원래 존재하여 독립 생성되고 자기 고유 식별자를 갖는다. 중심엔터티는 기본엔터티로부터 발생하여 계약·주문처럼 업무의 중심이 된다. 행위엔터티는 둘 이상 부모로부터 발생하는 주문목록·사원변경이력 등으로 상세 분석 중 드러나는 경우가 많다.

분류 기준	유형과 의미
생성 의존성	스스로 생성할 수 있는지에 따라 독립엔터티와 의존엔터티로도 구분한다.

명명할 때는 현업 용어를 사용하고 약어를 피하며 단수 명사로 표현한다. 엔터티명은 전체에서 유일해야 하고 생성 목적을 드러내야 한다. 고객이 주문한 제품을 관리하는 대상을 막연히 고객제품이라 부르면 고객 소유 제품인지 주문 상품인지 불명확해진다. 주문목록처럼 업무 발생 의미가 드러나는 이름을 선택해야 의사소통 오류를 줄일 수 있다.

제3절 속성

원문 PDF 42-48쪽

속성은 업무에서 필요한 인스턴스에 대해 관리하는, 의미상 더 이상 분리되지 않는 최소 데이터 단위이다. 엔터티를 설명하고 인스턴스의 구성요소가 된다. 생년월일을 생년·생월·생일로 나누는 것은 관리 목적에 따른 구분이지만 이름과 주소를 이름주소라는 하나의 기본속성으로 합치면 서로 다른 두 의미를 섞게 된다. 최소 단위의 판단은 단순한 글자 수가 아니라 업무상 의미를 기준으로 한다.

엔터티에는 둘 이상의 인스턴스가 있고 인스턴스의 공통 성격을 속성으로 정의한다. 이름·주소·전화번호는 속성이며 특정 사원의 실제 이름·주소·전화번호는 속성값이다. 하나의 인스턴스가 특정 속성에 갖는 값은 하나여야 한다. 동일 성질의 값이 여러 개라면 별도 엔터티로 분리한다. 속성은 해당 업무에 필요하고 정해진 주식별자에 함수적으로 종속되어야 한다.

분류	정의와 주의점
기본속성	업무에서 직접 추출한 원래 정보이다. 코드로 변형한 값·식별용 일련번호·계산값은 제외한다.
설계속성	모델링과 업무 규칙화를 위해 새로 만들거나 변형한 속성이다. 코드와 유일성을 부여할 일련번호가 해당한다.
파생속성	다른 속성의 계산·변형으로 만들어진다. 이자는 원금·기간·이율 변화에 영향을 받는 사례이다. 원인 속성과 계산 로직을 명시하고 관련 생성·수정·삭제에서 함께 관리해야 한다.

파생속성은 원인 데이터가 바뀔 때 정합성을 유지할 책임이 추가되므로 꼭 필요한 경우에 한해 적게 정의하는 것이 좋다. 정의했다면 어떤 엔터티의 어떤 속성으로 계산하는지 속성정의서에 기록하여 검증에 사용한다. 원문은 일반 엔터티보다 통계나 배치 처리에서 많이 활용한다고 설명한다.

다른 분류 기준	구분
엔터티 구성 역할	PK속성은 인스턴스를 식별하고, FK속성은 다른 엔터티와의 관계로 포함되며, 나머지는 일반속성이다.
분해 가능성	주소처럼 세부 의미로 나눌 수 있으면 복합속성, 나이·성별처럼 더 나누지 않는 속성은 단순속성이다.
값의 수	한 값이면 단일값속성, 여러 전화번호처럼 동일 성질의 값이 여럿이면 다중값속성이다. 다중값은 제1정규화 또는 별도 엔터티와 관계로 처리한다.

도메인은 속성이 가질 수 있는 값의 범위이다. 데이터 유형·크기·제약사항을 정하는 것으로 이해할 수 있다. 원문 사례에서 학점은 0.0~4.0 사이 실수, 주소는 20자리 이내 문자열이다. 범위를 벗어난 값은 허용하지 않는다. 용어사전과 도메인 정의를 함께 사용하면 용어와 데이터 유형의 일관성을 확보할 수 있다.

속성명은 현업에서 사용하는 명사형 이름을 사용하고 서술식·소유격·과도한 수식어를 피한다. 통용되지 않는 약어는 분석자 사이의 대화와 운영을 어렵게 한다. 가능한 전체 모델에서 이름의 유일성을 확보하면 데이터 흐름과 정합성을 파악하고 테이블 통합·분리나 칼럼 중복 같은 반정규화 때 이름 충돌을 해결하는 데 도움이 된다.

제4절 관계

원문 PDF 49-56쪽

관계는 엔터티의 인스턴스 사이에 존재 또는 행위로 부여된 논리적 연관성이다. 개별 인스턴스 사이의 연결을 관계 패어링이라 하고, 그 연결들의 집합을 관계로 표현한다. 같은 두 엔터티 사이에도 서로 다른 종류의 연관이 있으면 관계가 둘 이상 생길 수 있다. 관계는 엔터티와 속성을 어떻게 정의하는지에 따라서도 달라진다.

사원이 부서에 소속되어 있는 것은 존재에 의한 관계이고, 고객이 주문하여 주문이 생기는 것은 행위에 의한 관계이다. 원문의 ERD에서는 둘을 구분하지 않고 표현한다. UML 클래스 다이어그램에서는 항상 이용하는 존재적 연결을 연관관계로, 상대 클래스의 행위에 따른 연결을 의존관계로 구분하며 각각 실선·점선 등으로 표현한다.

표현 요소	학습 내용
관계명	양쪽 관점에서 각각 이름을 가진다. 시작·끝점의 주체에 따라 포함한다·소속된다처럼 능동형과 수동형이 된다. 모호한 동사를 피하고 현재형으로 쓴다.
관계차수	한 인스턴스가 상대편 몇 인스턴스와 연결되는지 나타낸다. 1:1, 1:M, M:N을 구분한다. 다수는 Crow's Foot의 까마귀발 모양으로 표시한다.
관계선택사양	반드시 관계가 있어야 하는 필수참여와 있을 수도 없는 수도 있는 선택참여를 구분한다. 참여 개수와 필수 여부는 서로 다른 정보이다.

1:1은 양쪽에서 상대 하나와 연결되는 관계이다. 1:M은 한쪽 인스턴스가 상대 여러 인스턴스와 연결되고 반대 방향에서는 하나와 연결된다. M:N은 양쪽 모두 여러 인스턴스와 연결되며 이후 두 식별자를 상속하는 관계 엔터티를 두어 세 엔터티로 풀어 표현한다. 단순히 엔터티 상자의 수를 세는 것이 아니라 개별 인스턴스의 참여 수를 확인해야 한다.

필수참여에서는 모든 인스턴스가 상대 인스턴스와 연결되어야 한다. 선택참여는 관계가 없을 수도 있으며 물리 구현에서 연결용 FK가 NULL을 허용할 수 있다. 선택이어야 할 관계를 필수로 잘못 지정하면 데이터 발생을 한 트랜잭션으로 제어해야 하는 불필요한 제약이 생긴다. 반대로 실제로 함께 있어야 할 데이터의 필수성을 놓쳐도 업무 규칙을 제대로 표현하지 못한다. 원문은 양쪽 모두 선택인 관계는 설정을 다시 검토할 필요가 있다고 설명한다.

- 관계 정의 시 확인: 관심 있는 연관 규칙이 있는가, 정보의 조합이 발생하는가, 업무기술서·장표에 연결 규칙이 적혀 있는가, 연결을 가능하게 하는 동사가 있는가.
- 읽는 순서: 기준 엔터티를 하나 또는 각각으로 읽는다 → 대상 엔터티의 개수인 하나·하나 이상을 읽는다 → 필수·선택 여부와 관계명을 읽는다.
- 관계는 양쪽 방향으로 읽어야 한다. 한 주문에 제품을 하나만 주문하는지와 한 제품이 하나의 주문에서만 주문되는지는 별개의 업무 확인이다.

제5절 식별자

원문 PDF 57-73쪽

식별자는 엔터티에 속하는 여러 인스턴스를 구분하는 논리적 이름으로, 엔터티를 대표하는 속성 또는 속성 조합이다. 원문은 업무적으로 구분하는 식별자를 논리 모델링 용어로, 테이블에 접근하는 매개체인 키를 물리 모델링 용어로 구분한다. 주식별자는 모든 인스턴스를 유일하게 구분하고 필요한 최소 속성만으로 구성되며 값이 안정적이고 반드시 존재해야 한다.

특징	확인 기준
유일성	모든 인스턴스를 하나씩 구분한다.
최소성	유일성을 충족하는 최소 속성 수만 포함한다. 사원번호만으로 충분한데 사원분류코드를 덧붙이지 않는다.
불변성	정해진 식별자의 값은 변하지 않는 것을 원칙으로 한다. 변경은 기존 기록과 새 기록의 구분에 영향을 미친다.
존재성	값이 반드시 있어야 하며 NULL을 허용하지 않는다.

보조식별자도 유일하게 구분할 수 있지만 대표 식별자로 선택되지 않은 것이다. 외부식별자는 다른 엔터티에서 관계를 통해 받아오는 속성이므로 주식별자의 네 특성이 그대로 적용되는 것이 아니라 참조 무결성 규칙에 따른다. 주식별자·보조식별자는 대표성 기준, 내부·외부는 생성 위치 기준, 단일·복합은 속성 수 기준, 본질·인조는 대체 여부 기준이므로 분류 기준을 섞지 않아야 한다.

기준	구분과 의미
대표성	주식별자는 엔터티 대표 구분자이자 다른 엔터티와 참조 관계를 연결하는 식별자다. 보조식별자는 구분은 가능하지만 대표성이 없다.
생성 위치	내부식별자는 자체 생성되고 외부식별자는 다른 엔터티와의 관계에서 받아온다.
속성 수	단일식별자는 한 속성, 복합식별자는 둘 이상의 속성으로 구성된다.
대체 여부	본질식별자는 업무에서 만들어진 구분자, 인조식별자는 복잡한 본질 구성을 대신하여 인위적으로 만든 구분자이다.

주식별자는 업무에서 자주 사용하는 속성을 우선한다. 사원번호와 주민등록번호 모두 유일해도 사원 관리에 흔히 쓰는 사원번호를 주식별자로 정하고 다른 것은 보조로 둘 수 있다. 명칭·내역처럼 길고 입력하기 어려운 속성은 가능하면 피한다. 다른 구분자가 없다면 일련번호나 코드를 검토한다. 복합 속성이 지나치게 많으면 조건절과 자식 테이블의 조인도 길어지므로 인조식별자를 고려하되, 상속 속성을 자식에서 직접 조회하는 이점도 함께 판단한다.

관계를 연결하면 부모의 주식별자를 자식 속성으로 상속한다. 이를 자식의 주식별자 구성에 포함하면 식별자관계, 일반속성으로만 포함하면 비식별자관계이다. 식별자관계에서는 부모가 먼저 존재해야 하며 상속된 식별자에 NULL이 올 수 없다. 부모의 식별자만으로 자식의 주식별자를 구성하면 1:1이 되고, 다른 식별 속성을 더해 구성하면 1:M이 된다. 사원번호와 발령일자를 함께 식별자로 갖는 발령이 1:M 사례이다.

비식별자관계는 부모 없는 자식의 생성이 허용되는 경우, 부모와 자식의 생명주기를 별개로 관리하는 경우, 여러 엔터티 통합으로 각기 다른 선택적 관계를 받아야 하는 경우, 자식이 독립 식별자를 갖는 편이 업무·성능상 유리한 경우에 선택한다. 계약이 반드시 사원에 의해 체결되더라도 계약번호만으로 유일하게 식별된다면 계약사원번호를 일반 FK로 받을 수 있다.

식별자관계만 연속 적용하면 세대가 내려갈수록 부모 식별자와 자식의 추가 속성이 누적되어 PK가 커진다. 여러 칼럼을 모두 조인 조건으로 써야 하므로 SQL이 길어지고 조건 누락에 따른 오류가 생기기 쉽다. 반대로 비식별자관계만 적용하면 점검번호·주문번호 같은 자주 쓰는 상위 기준 속성이 하위에 전달되지 않아 단순 조회도 상위 테이블까지 여러 번 조인해야 할 수 있다. 원문 점검 사례에서는 점검번호를 상속받은 하위 테이블이 단독 조건으로 조회할 수 있음을 보여준다.

선택의 핵심은 업무 의존성, 자식의 독립 주식별자 필요성, SQL 조회 전략의 균형이다. 강한 종속으로 부모 식별자 전체가 자식의 식별에 필요하고 더 아래에도 전달해야 한다면 식별자관계가 적합하다. 약한 종속, 독립 PK 구성, 부모 식별자 일부만 필요하거나 전파를 막아야 하는 상황, 부모 쪽 선택관계라면 비식별자관계를 검토한다. 관계 형식을 기계적으로 하나로 통일하지 말고 업무와 성능의 근거를 함께 사용한다.

반드시 기억할 핵심 포인트

- 모델링의 세 특징은 추상화·단순화·명확화이며, 데이터·프로세스·상관의 세 관점과 구분한다.
- 데이터 독립성은 외부·개념·내부 스키마와 단계 사이의 사상을 함께 이해해야 한다.
- 엔터티는 집합, 인스턴스는 개별 요소, 속성은 의미 단위, 속성값은 실제 값이다.
- 속성은 주식별자에 함수적으로 종속되고 인스턴스별로 한 값을 갖는다.
- 관계는 관계명·관계차수·선택사양을 모두 읽어야 업무 규칙을 알 수 있다.
- 주식별자의 유일성·최소성·불변성·존재성을 확인한다.
- 식별자관계의 과도한 누적과 비식별자관계의 불필요한 조인 증가를 함께 경계한다.

챕터 최종 요약

- 업무를 이해한 뒤 관리할 집합·성격·연관성을 각각 엔터티·속성·관계로 표현하고 식별자로 개별 인스턴스를 구분한다. 개념 모델에서 핵심 구조를 잡고 논리 모델에서 규칙을 구체화한 뒤 물리 모델로 구현한다.
- 좋은 모델은 필요한 정보와 업무 규칙을 빠짐없이 표현하면서 중복을 줄이고, 독립성·통합성·재사용성·의사소통을 지원한다. 모델의 품질은 이후 SQL의 정확성과 변경 범위, 데이터 정합성에 직접 영향을 준다.
- ERD는 관계의 개수와 필수 여부를 양방향으로 읽는다. 부모 식별자가 자식 PK에 들어가는지 일반 FK로 들어가는지에 따라 식별자관계와 비식별자관계가 갈리며, 업무 의존성과 실제 조회 조건을 근거로 균형 있게 선택한다.

제2장 데이터 모델과 SQL

과목 I. 데이터 모델링의 이해

원문 PDF 78-118쪽

제1절 정규화

원문 PDF 79-90쪽

정규화는 데이터 모델링에서 기본적으로 먼저 수행해야 하는 작업이다. 엔터티를 언제 분리해야 하는지를 경험이나 직감만으로 결정하지 않고, 속성의 값과 함수적 종속성을 기준으로 판단하게 한다. 원문은 제1·2·3정규형을 고객·주문·상품 사례로 설명한 뒤 성능에 따른 반정규화의 필요와 비용을 검토한다.

제1정규형은 모든 속성이 반드시 하나의 값을 갖도록 하는 것이다. 고객의 연락처 한 칸에 집전화와 휴대전화 여러 개를 넣으면 번호 종류와 필요한 값을 구분하기 어렵고, 다른 종류의 연락 정보가 섞이면서 속성 의미도 흐려진다. 고객연락처 엔터티를 분리하여 고객번호·순번으로 개별 연락처를 관리하면 개수 증가를 행 추가로 수용할 수 있다. 원문은 연락처구분코드를 더하면 집전화·휴대전화·이메일 같은 유형도 명확히 구분할 수 있다고 설명한다.

반복되는 속성도 제1정규형의 검토 대상이다. 주문에 상품번호1·상품명1·상품번호2·상품명2를 두면 세 번째 상품부터 칼럼을 계속 추가해야 한다. 물리 테이블 변경은 Lock과 서비스 운영상의 제약을 만들 수 있고, 각 상품번호를 빠르게 조회하려면 인덱스도 반복해서 늘어난다. 주문상세를 분리하여 주문번호와 상품번호로 관리하면 상품 수를 구조 변경 없이 늘릴 수 있다. 이는 개발 복잡성뿐 아니라 서비스 안정성과도 연결된다.

제2정규형은 일반속성이 주식별자 전체에 종속되어야 한다는 기준이다. 결정자는 다른 값을 정하는 기준값이고 종속자는 그 기준에 의해 값이 정해지는 속성이다. 주문상세의 식별자가 주문번호+상품번호인데 상품명이 상품번호에만 의존한다면 주식별자의 일부에만 종속된 부분 종속이다. 상품번호의 반복은 주문과 상품의 매핑 정보라는 의미가 있지만 상품명의 반복은 같은 상품 사실의 중복이다.

부분 종속 상태에서 상품명이 바뀌면 팔린 주문상세 모두를 고쳐야 한다. 변경 대상이 많으면 부하가 커지고 변경 도중 아직 갱신되지 않은 행을 읽으면 일관성이 깨질 수 있다. 상품 엔터티로 상품번호·상품명을 옮기고 주문상세는 상품번호만 매핑키로 보유하면 상품 정보를 한곳에서 관리한다. 필요한 상품명은 조인으로 가져온다. 이렇게 부분 종속을 제거하여 일반속성이 식별자 전체에 종속되도록 한다.

제3정규형은 일반속성끼리 서로 종속되지 않게 하는 것이다. 주문번호가 고객번호를 결정하고 고객번호가 고객명을 결정하면 주문번호에서 고객명으로 이행적 종속이 생긴다. 고객명이 바뀌었다고 주문의 모든 고객명을 고치는 것은 주문과 무관한 변경 작업을 주문 엔터티에 발생시키는 것이다. 고객 엔터티를 분리하고 주문에는 고객번호만 두면 고객 정보 변화가 주문 데이터 변경을 요구하지 않는다. 원문의 고객번호 NULL 허용은 비회원 주문도 가능한 업무를 표현한다.

정규형	제거하는 문제	원문 조정 사례
제1정규형	한 속성의 다중값, 반복 속성으로 인한 개수 제약	고객연락처·주문상세 분리
제2정규형	복합 식별자 일부에만 일반속성이 종속되는 부분 종속	상품번호에만 종속하는 상품명을 상품으로 분리
제3정규형	일반속성을 경유하는 이행적 종속	고객번호에 종속하는 고객 정보를 고객으로 분리

반정규화는 성능을 위해 데이터 중복을 허용하는 것이다. 먼저 정규화를 수행하고 성능 문제가 실제로 있으며 다른 방법이 없는지 검토한 뒤 필요한 대상에 적용한다. 조회가 빨라질 수 있더라도 입력·수정·삭제 성능과 정합성 유지 부담이 늘 수 있으므로 항상 성능이 좋아진다고 일반화하면 안 된다.

성능 향상 사례는 고객의 최근 신용카드 결제수단번호를 미리 채워 주는 조회이다. 정규화된 모델에서는 주문에서 고객 주문들을 찾고 결제와 조인한 뒤 결제일시 내림차순으로 최근 한 건을 구한다. 주문이 많아질수록 최종 출력은 한 건이어도 읽고 조인하고 정렬할 데이터가 늘어난다. 결제에 고객번호를 중복 보유시키고 고객번호+결제수단구분코드+결제일시 인덱스를 구성하면 원문은 Index Range Scan Descending으로 최근 한 건만 읽는 접근이 가능하다고 설명한다.

성능 저하 가능 사례는 주문에 송장번호를 중복 보유시키는 경우다. 주문·배송 조인을 줄일 수 있지만 주문 시점에는 송장번호가 아직 없고 판매자가 포장·배송 준비를 마친 후에야 부여된다. 따라서 주문 입력 때 NULL을 넣고 나중에 주문의 송장번호를 갱신하는 작업이 새로 생긴다. 적절한 인덱스를 갖춘 정규 모델과 비교하여 조회 이득이 작다면 추가 UPDATE, 불일치 위험, 운영 비용을 감수할 근거가 부족하다. 반정규화는 이러한 비용에도 적용해야 할 명확한 근거가 있을 때 선택한다.

제2절 관계와 조인의 이해

원문 PDF 91-98쪽

관계를 맺는다는 것은 부모의 식별자를 자식에 상속하고 이를 매핑키로 사용하여 데이터를 연결할 수 있게 한다는 뜻이다. 자식 식별자에 포함되면 식별관계, 일반속성으로 받으면 비식별관계이다. SQL의 조인은 이 매핑키를 통해 분리된 데이터의 관련 값을 결합하는 것이다. 관계선의 의미를 먼저 파악하면 조인 조건을 임의로 외우는 부담이 줄어든다.

고객과 주문의 원문 사례에서 고객 한 명은 여러 번 주문할 수 있고 각 주문은 한 고객에 의해 생긴다. 주문번호 1100001을 찾으면 그 행의 고객번호가 100이고, 고객에서 100을 찾아 고객명 정우진을 확인한다. 이 연결의 매핑키가 고객번호이고 SQL의 A.고객번호 = B.고객번호가 같은 의미를 표현한다. 주문을 제한하는 조건과 두 집합을 연결하는 조건을 구분해서 읽는다.

계층형 데이터 모델은 같은 집합의 데이터 사이에 상하 계층이 존재하는 모델이다. EMP에서 MGR은 관리자의 EMPNO를 담는, 자신에게서 상속된 식별자다. SMITH의 MGR을 통해 FORD를 찾고 다시 그 관리자를 찾을 수 있다. 같은 EMP에 서로 다른 별칭을 주어 자기 자신과 연결하는 것이 셀프조인이다. 쇼핑몰의 식품→수산, 가전→냉장고 같은 카테고리 계층도 같은 종류의 관리 구조다. 계층형 쿼리를 이해하기 전에 자기참조 관계를 이해해야 한다.

상호배타적 관계는 연결 가능한 여러 관계 중 하나만 성립하는 경우다. 원문의 주문은 개인고객 또는 법인고객 중 하나로부터만 식별자를 받는다. 개인/법인번호에 실제 구분자의 값이 들어가고 고객구분코드가 어느 집합의 번호인지 나타낸다. 개인 주문과 법인 주문을 구분해 각각 조인한 결과를 UNION ALL로 합치는 SQL이나, 번호가 중복되지 않는 전제에서 양쪽을 외부 조인한 후 COALESCE로 고객명을 선택하는 SQL을 제시한다.

배타적 관계의 SQL은 번호의 중복 가능성과 데이터 품질을 함께 고려해야 한다. 원문은 개인번호·법인번호의 중복 여부를 확인하고 필요한 경우 고객구분코드를 입력받아 대상을 확정해야 한다고 강조한다. 또 연결 대상이 전혀 없는 품질 오류 상황에서 내부 조인과 UNION ALL 방식은 행이 없는 공집합이 되고, 외부 조인 방식은 주문 행이 남아 고객명이 NULL인 행을 출력한다. 같은 목적처럼 보여도 누락 데이터에 대한 결과 의미가 다르다.

제3절 모델이 표현하는 트랜잭션의 이해

원문 PDF 99-101쪽

트랜잭션은 데이터베이스의 논리적 연산 단위이다. 계좌이체에서 보내는 계좌의 차감과 받는 계좌의 가산은 모두 실행되거나 모두 취소되어야 한다. 하나의 업무 단위로 묶어 All or Nothing의 원자성을 보장하며 커밋 단위도 함께 구성한다.

데이터 모델의 필수 관계도 함께 발생해야 할 데이터를 표현한다. 고객이 상품을 구매함으로써 주문이 생기는 일반적인 쇼핑몰이라면 주문만 생기고 주문상세가 없는 것은 업무 의미에 맞지 않는다. IE 표기에서는 관계선 끝의 원이 없고, Barker 표기에서는 해당 필수 관계를 실선으로 표현한다. 원문은 도구의 선택관계 기본 설정을 그대로 두어 필수성을 놓치지 않도록 주의를 준다.

함께 태어나는 주문과 주문상세는 한 트랜잭션에서 입력하고 모두 성공한 뒤 커밋해야 한다. 각각을 별도 API와 커밋으로 나누면 배터리 소진·앱 종료·연결 장애 등으로 하나만 처리되어 불완전한 데이터가 남을 수 있다. 이 사례에서는 원래 따로 발생할 일이 없으므로 재사용성을 이유로 분리해도 실질적인 이점을 얻기 어렵다. 프레임워크의 자동 커밋을 사용하더라도 모델이 요구하는 논리적 단위를 개발자가 확인해야 한다.

- 계좌이체 처리 관계: 차감 → 가산 → 두 작업 모두 성공 → 커밋.
- 주문 처리 관계: 주문 입력 + 주문상세 입력 → 하나의 커밋 단위.
- 필수·선택 관계의 의미 → 데이터 발생 시점과 의존성 → 트랜잭션 범위 → 데이터 정합성으로 연결하여 이해한다.

제4절 Null 속성의 이해

원문 PDF 102-107쪽

NULL은 값이 아직 정의되지 않았거나 현재 입력할 수 없는, 값의 부재를 의미한다. 공백이나 숫자 0과 다르다. 원문 주문 모델은 주문금액과 주문취소금액에 NULL을 허용하며 이 선택이 계산 결과에 어떤 영향을 주는지 설명한다. NULL 여부는 IS NULL 또는 IS NOT NULL로 판단하고 수치 연산에 그대로 포함하면 결과도 NULL이 된다.

첫 사례의 최종주문금액은 주문금액에서 취소금액을 빼는 것이다. 주문금액 100,000과 취소금액 20,000은 어느 방식이나 80,000이다. 그러나 주문금액 15,000, 취소금액 NULL인 행에서 단순 뺄셈은 NULL, 뺄셈 결과 전체를 NVL로 0 처리하면 0이 된다. 각 속성의 NULL을 먼저 0으로 대체한 뒤 빼야 15,000이라는 사례의 의도에 맞는 결과가 나온다. NULL을 고려한다는 사실만으로 충분하지 않고 처리 위치가 중요하다.

집계함수는 NULL 값을 제외하고 계산한다. 취소금액이 모두 NULL인 다섯 주문 사례에서 주문금액 합계는 300,000이지만 취소금액 합계는 NULL이다. SUM(주문금액)-SUM(주문취소금액)은 NULL이 되고, 각 행의 차액을 계산한 뒤 SUM하면 모든 차액이 NULL이므로 그 합계도 NULL이다. 이를 마지막에 NVL로 바꾸면 0이 된다. 반면 각 속성의 합계를 구하고 그 합계가 NULL일 때 각각 0으로 바꾸어 빼면 300,000을 얻는다.

평균에서는 분모의 업무 정의를 먼저 정해야 한다. 원문은 취소금액 20,000·NULL·10,000·10,000·10,000의 다섯 행을 제시한다. SUM(취소금액)/COUNT(*)는 50,000을 전체 다섯 행으로 나누어 10,000이다. AVG(취소금액)는 NULL을 제외한 네 행의 평균이므로 12,500이다. 전체 주문의 평균 취소금액인지 취소가 발생한 주문의 평균인지에 따라 적합한 계산이 달라진다.

NULL 허용은 모델 설계 때 업무 의미를 파악하여 결정해야 한다. 쉽게 허용하면 모든 관련 SQL에서 계산·집계·분모에 대한 판단이 필요해지고 오류 및 관리 비용이 늘어난다. SQL의 NULL 특성과 속성의 업무 의미를 함께 이해해야 정확한 결과를 얻을 수 있다.

제5절 본질식별자 vs. 인조식별자

원문 PDF 108-115쪽

본질식별자는 업무에 의해 만들어진 구분자이고, 인조식별자는 복잡한 본질 구성을 대체하려고 인위적으로 만든 구분자이다. 유일성과 NULL 금지만 충족하면 충분하다고 생각하여 업무의 본질을 검토하지 않은 채 새 일련번호를 만드는 것은 경계해야 한다. 원문은 인조식별자의 편의와 함께 중복 데이터 품질, 추가 인덱스 문제를 다룬다.

주문상품을 주문번호+상품번호로 식별하면 한 주문에서 특정 상품은 한 번만 나타난다. 수량은 일반속성이다. 이 구성을 단일 주문상품번호로 바꾸어 시퀀스 NEXTVAL로 채번하면, 원문의 단순 사례에서는 추가적인 업무 이득 없이 시퀀스 객체만 필요해진다. 식별자 선택은 단순히 입력 오류를 피하기 위한 기술적 유일성 이상의 업무 규칙을 포함한다.

하나의 주문으로 같은 상품을 우리집·부모님집·친구집에 각각 보내야 한다면 주문번호+상품번호는 중복되어 사용할 수 없다. 원문은 주문번호+주문순번을 식별자로 삼고 상품과 배송지를 일반속성으로 두는 모델을 제시한다. 이를 단일 주문상세번호로 바꾸면 주문별 순번을 따로 관리하지 않고 시퀀스 객체로 채번할 수 있어 개발 작업이 줄어드는 이점이 있다. 다만 이 편의가 다른 비용을 없애지는 않는다.

중복 입력 사례에서는 같은 입력 로직이 두 번 실행되어도 매번 다른 시퀀스 번호가 부여되면 PK 제약에 위반되지 않는다. 업무상 같은 주문상세가 두 행으로 저장될 수 있다. 반면 주문번호+주문순번이 동일한 원문 입력은 PK 제약으로 두 번째 행이 차단된다. 인조식별자를 선택했을 때 DBMS의 그 PK만으로 업무 중복을 막을 수 없으므로 애플리케이션에서 방어할 책임을 고려해야 한다.

추가 인덱스 비용도 있다. 가장 기본적인 접근이 주문번호만으로 찾거나 주문번호와 상품번호로 찾는 것이라면, 본질식별자의 주문번호+상품번호 PK 인덱스를 활용할 수 있다. PK가 주문상품번호로 바뀌면 실제 접근을 위해 주문번호+상품번호 인덱스를 추가해야 한다. 추가 인덱스는 저장 용량과 DML 성능에 영향을 준다. 식별 속성이 과도하게 많은 경우처럼 필요가 명확하면 인조식별자를 고려하되, 본질식별자와 장단점을 비교하여 남용하지 않는 것이 원문의 결론이다.

반드시 기억할 핵심 포인트

- 제1정규형은 한 속성 한 값, 제2정규형은 주식별자 전체에 종속, 제3정규형은 일반속성 사이 이행 종속 배제다.
- 정규화 후 반정규화를 검토하며 조회 이득과 추가 DML·정합성 비용을 함께 본다.
- 관계의 식별자 상속이 조인의 매핑키를 만든다. 자기참조 관계도 조인할 수 있다.
- 필수 관계로 함께 발생하는 데이터는 하나의 논리적 트랜잭션과 커밋으로 묶는다.
- NULL 연산 결과와 집계의 NULL 제외 성질, 처리 위치와 평균 분모를 구분한다.
- 인조식별자의 유일성은 업무상 중복 방지와 다르며 실제 접근 인덱스도 따로 필요할 수 있다.

챕터 최종 요약

- 정규화는 속성 의미와 함수 종속성을 기준으로 엔터티를 나누어 중복 변경과 정합성 문제를 줄인다. 분리한 정보는 관계가 상속한 식별자를 매핑키로 조인한다. 성능을 위해 중복을 허용할 때도 조회 이득보다 추가 트랜잭션 비용이 커지지 않는지 검증한다.
- 모델은 테이블 구조뿐 아니라 데이터의 동시 발생과 트랜잭션 범위도 표현한다. 주문과 주문상세처럼 함께 생겨야 하는 정보는 한 커밋 단위로 처리한다. 속성에 NULL을 허용했다면 관련 계산·집계·평균의 업무 의미를 정확히 정의한다.
- 식별자는 업무 규칙과 조회 구조에 영향을 준다. 본질식별자의 중복 제약과 인덱스 활용 이점, 인조식별자의 채번 편의와 추가 품질·인덱스 비용을 함께 비교한다.

제1장 SQL 기본

과목 II. SQL 기본과 활용

원문 PDF 122-270쪽

제1절 관계형 데이터베이스 개요

원문 PDF 123-138쪽

데이터베이스는 조직이나 개인이 필요로 하는 데이터를 일정한 형태로 저장한 것이다. 데이터량이 많고 여러 사용자가 동시에 다양한 목적으로 사용하면 개인용 파일 관리만으로는 정합성·동시 변경·장애 복구를 다루기 어렵다. DBMS는 데이터를 효율적으로 관리하고 손상이나 유실을 방지하며 필요한 데이터를 복구하는 기능을 제공한다. 원문은 파일 관리에서 계층형·망형, 관계형, 객체 지원을 포함한 객체 관계형 구조로 발전한 흐름을 소개한다.

파일을 여러 개 복사해 공유하면 변경을 모든 복사본에 반영하지 못하여 서로 다른 정보가 존재할 수 있다. 관계형 데이터베이스는 정규화한 테이블 모델링으로 중복과 이상현상을 줄이고 동시성 관리·병행 제어로 여러 사용자가 데이터를 공유·조작하도록 한다. 메타데이터와 표준을 관리하여 데이터 성격을 체계화하고, 보안·제약조건·장애 회복으로 무결성과 복구를 지원한다. 원문은 단일 사용자·단일 애플리케이션의 특정 업무에서는 파일 방식이 유용할 수도 있다고 구분한다.

SQL은 관계형 데이터베이스의 데이터를 정의·조작·제어하기 위한 언어이다. 데이터를 개별 절차의 나열보다 집합으로 다룬다. 선수 전체에서 포지션이 MF인 집합을 추출하듯이 조건에 맞는 데이터를 조회하고 입력·수정·삭제한다. SQL 교육의 일차 목표는 정확한 데이터 출력이며 튜닝은 응답시간과 자원 사용 측면에서 효과적인 SQL을 작성하는 것이다. 문장은 세미콜론으로 구분되는 독립적인 처리 단위다.

분류	원문 명령어	역할
DML	SELECT, INSERT, UPDATE, DELETE	데이터 조회 및 행 입력·수정·삭제
DDL	CREATE, ALTER, DROP, RENAME	테이블 등 데이터 구조 생성·변경·삭제·이름 변경
DCL	GRANT, REVOKE	접근 및 객체 사용 권한 부여·회수
TCL	COMMIT, ROLLBACK	논리 작업 단위의 결과 제어

ANSI/ISO 표준화는 벤더별 차이로 낮아진 호환성을 높이는 역할을 한다. 원문은 초기 표준에서 SQL-92, SQL-99와 이후 개정으로 기능이 확장된 흐름을 설명한다. 표준 조인, 스칼라 서브쿼리·Top N, ROLLUP·CUBE·GROUPING SETS, 윈도우 함수 등이 확장된 기능이다. 벤더 고유 기능도 존재하므로 사용 편의와 특성을 판단하되 가능한 표준을 기준으로 사용할 것을 권고한다. 데이터 유형과 내장 함수는 벤더별 차이가 특히 크다.

관계 대수	원문의 SQL 대응	학습 의미
UNION	UNION	합집합. 중복을 제거하는 처리 부담이 있으므로 같은 결과를 얻는 상황이면 UNION ALL을 검토한다.
INTERSECTION	INTERSECT	두 집합의 공통 부분
DIFFERENCE	EXCEPT / Oracle MINUS	첫 집합에서 두 번째 집합의 공통 부분을 제외
PRODUCT	CROSS JOIN	조건 없는 모든 조합, M×N건

관계 대수	원문의 SQL 대응	학습 의미
SELECT	WHERE 절	조건을 만족하는 행을 선택. SQL의 SELECT 절과 이름을 혼동하지 않는다.
PROJECT	SELECT 절	필요한 칼럼을 선택
JOIN	여러 JOIN 기능	분리한 데이터의 연관성을 연결
DIVIDE	원문은 현재 사용되지 않는다고 설명	관계 대수의 나눗셈에 해당하는 개념

테이블은 데이터를 저장하는 관계형 데이터베이스의 기본 객체이며 행과 칼럼의 2차원 구조이다. 칼럼은 특정 속성, 행은 연결된 한 건의 데이터, 필드는 그 교차 지점이다. 테이블에 아직 행이 없어도 정의한 속성과 구조를 갖고 존재한다. 새로운 선수를 추가할 때는 새 테이블이 아니라 같은 구조에 행을 추가한다.

정규화는 불필요한 중복과 입력·수정·삭제 이상현상을 줄이기 위해 테이블을 분할하고 데이터 정합성을 확보하는 과정이다. 각 행을 한 의미로 식별하는 한 개 이상 칼럼이 기본키이며, 다른 테이블의 키와 연결하는 칼럼이 외부키이다. 선수의 팀코드가 팀의 키를 참조하는 것이 원문 사례다. 정규화로 나눈 데이터를 다시 필요한 정보로 결합하는 핵심 기능이 조인이다.

ERD는 엔터티·관계·속성으로 테이블 사이의 업무 의미를 직관적으로 표현한다. 케이리그 예에서는 한 팀이 여러 선수를 포함할 수 있고 각 선수는 한 팀에 속하며, 팀과 전용구장, 구장과 경기 일정이 연결된다. EMP·DEPT 예에서는 부서 하나에 여러 사원이 속할 수 있고 각 사원은 한 부서에 속한다. 조회하기 전에 어느 테이블에 필요한 속성이 있으며 어떤 키로 연결되는지 확인한다.

유형	특징과 원문 기준
CHARACTER(s)	고정 길이 문자. Oracle·SQL Server에서 CHAR. 길이가 짧으면 차이만큼 공백을 채운다. 원문 최대 길이는 Oracle 2,000바이트, SQL Server 8,000바이트.
VARCHAR(s)	가변 길이 문자. Oracle은 VARCHAR2, SQL Server는 VARCHAR. 선언 길이는 최대 한계이며 실제 입력 바이트에 맞추어 저장한다. 원문 최대 길이는 각각 4,000·8,000바이트.
NUMERIC	숫자. 원문은 Oracle NUMBER와 SQL Server의 여러 숫자 유형을 비교한다. NUMBER(8,2)는 전체 8자리 중 소수 2자리다.
DATETIME	날짜·시각. 원문은 Oracle DATE의 1초 단위, SQL Server DATETIME의 3.33ms 단위 관리를 설명한다.

데이터 유형은 받아들일 값의 종류를, 크기와 정밀도는 한계를 정한다. 맞지 않는 유형이나 허용 크기를 넘는 값을 입력하면 오류가 날 수 있다. CHAR 비교는 짧은 쪽을 공백으로 채워 같은 길이로 비교하므로 끝 공백만 다른 값은 같게 판단한다. VARCHAR가 포함된 비교에서는 끝 공백도 문자로 취급한다. 길이가 다양한 데이터에는 가변 길이가 저장 공간 측면에서 유리할 수 있다. 잘못 정한 유형은 입력 후 변경하기 어려우므로 설계 때부터 검토한다. 이 표의 수치와 제품 설명은 원문의 설명 기준이다.

제2절 SELECT 문

원문 PDF 139-146쪽

SELECT 뒤에 조회할 칼럼을 쉼표로 나열하고 FROM 뒤에 그 칼럼이 있는 테이블을 지정한다. ALL은 기본값으로 중복을 포함하여 출력하며 생략할 수 있다. DISTINCT는 선택한 데이터의 중복을 제거한다. 선수 POSITION 조회에서 ALL은 480행을 출력하고 DISTINCT는 네 포지션과 미정인 NULL을 포함한 다섯 종류를 출력한다. 모든 칼럼을 확인할 때는 별표(*)를 사용할 수 있다.

별칭 ALIAS는 조회 결과의 칼럼 레이블에 의미 있는 이름을 부여하는 기능이다. 칼럼이나 표현식 바로 뒤에 적으며 AS는 생략할 수 있지만 가독성을 위해 쓰는 편이 좋다. 공백·특수문자를 포함하거나 대소문자 구분이 필요하면 이중 인용부호로 감싼다. 원문은 SQL Server의 별칭 인용 방식도 함께 제시한다. 별칭은 원본 데이터 구조를 바꾸는 것이 아니라 결과를 설명하는 이름이다.

산술 연산은 NUMBER와 DATE 유형에 적용하며 괄호, 곱셈·나눗셈, 덧셈·뺄셈 순으로 우선한다. 선수의 키에서 몸무게를 빼거나 몸무게를 키의 제곱으로 나누는 BMI 사례를 통해 표현식을 작성한다. 산술식이나 함수 결과는 기존 칼럼과 다른 의미이므로 적절한 별칭을 붙인다. 원문 BMI 예는 ROUND로 소수 두 자리까지 표시한다.

합성 연산은 문자와 문자, 칼럼과 문자, 칼럼과 칼럼을 연결하여 새로운 출력 칼럼을 만든다. Oracle은 ||, SQL Server는 +를 사용하며 CONCAT도 제시된다. 원문은 선수명·키·몸무게와 단위 문구를 연결해 체격정보를 출력한다. 단순한 리포트라면 별도 프로그램에서 이어 붙이지 않고 SQL 표현식으로 만들 수 있다. 문자 상수의 작은따옴표와 별칭의 인용부호는 역할이 다르다.

제3절 함수

원문 PDF 147-178쪽

내장 함수는 DBMS가 제공하여 SQL에서 값을 간편하게 조작하는 기능이고 사용자 정의 함수와 구분한다. 단일행 함수는 각 행을 개별 처리하여 한 결과를 돌려주며, 다중행 함수는 여러 행을 입력으로 사용한다. 다중행에는 집계·그룹·윈도우 함수가 포함된다. 인수는 여러 개여도 한 번의 함수 적용 결과는 하나라는 점이 공통이다. 단일행 함수는 SELECT·WHERE·ORDER BY 등에 사용할 수 있고 상수·변수·표현식·다른 함수를 인수로 받을 수 있다.

중첩 함수는 안쪽 함수를 먼저 계산하고 그 결과를 바깥 함수의 인수로 전달한다. 원문은 지역번호와 전화번호를 연결한 문자열을 LENGTH 또는 LEN에 넘겨 길이를 구한다. 함수를 외울 때는 이름뿐 아니라 입력 유형, 반환 유형, 인수의 순서와 생략 가능한 부분을 확인한다. 문자 함수도 길이·코드 번호처럼 숫자를 반환할 수 있다.

문자 함수	역할과 사용 조건
LOWER(문자열), UPPER(문자열)	알파벳을 각각 소문자·대문자로 변환한다.
ASCII(문자), CHR/CHAR(번호)	문자의 ASCII 번호 또는 번호에 대응하는 문자를 얻는다. CHR은 Oracle, CHAR는 SQL Server 표현이다.
CONCAT(문자열1, 문자열2)	두 문자열을 연결한다. 원문은 || 또는 + 합성과 같은 목적을 설명한다.
SUBSTR/SUBSTRING(문자열,m,n)	m 위치부터 n개 문자를 취한다. 원문의 Oracle SUBSTR은 길이를 생략하면 끝까지 반환한다.
LENGTH/LEN(문자열)	문자열 길이를 숫자로 반환한다.
LTRIM, RTRIM	왼쪽·오른쪽부터 지정 문자를 제거하며 기본은 공백이다. 원문 SQL Server 설명에서는 지정 문자 없이 공백 제거만 허용한다.
TRIM	머리말·꼬리말·양쪽의 지정 문자를 제거한다. Oracle의 LEADING·TRAILING·BOTH 중 기본은 BOTH이며 원문 SQL Server 설명은 해당 방향 옵션을 지원하지 않는다.

Oracle의 DUAL은 사용자 테이블이 필요 없는 표현식에도 SELECT와 FROM의 형태를 갖추기 위한 더미 테이블이다. SYS 소유이며 모든 사용자가 접근할 수 있고 DUMMY 칼럼에 X인 한 행을 갖는다. 원문은 문자열 길

이를 구하는 SELECT에 FROM DUAL을 사용한다. SQL Server는 테이블 칼럼 없이 상수나 표현식만 선택할 때 FROM 없이 실행할 수 있으나 사용자 테이블 칼럼을 쓰면 FROM이 필요하다.

숫자 함수	역할
ABS(수)	절댓값
SIGN(수)	음수·0·양수에 따라 -1·0·1
MOD(수1,수2), %	나머지
CEIL/CEILING(수)	해당 수 이상인 최소 정수
FLOOR(수)	해당 수 이하인 최대 정수
ROUND(수,m)	지정 소수 자릿수로 반올림. m 생략 기본 0
TRUNC(수,m)	지정 소수 자릿수 아래를 버림. m 생략 기본 0. 원문은 SQL Server에서 제공하지 않는다고 설명
SIN, COS, TAN	삼각함수
EXP(수)	e의 거듭제곱
POWER(수1,수2)	수1의 수2제곱
SQRT(수)	제곱근
LOG	지정 밑에 대한 로그. Oracle과 SQL Server의 인수 순서 차이를 확인
LN(수)	자연로그. 원문은 SQL Server에서 제공하지 않는다고 설명

음수에서는 올림과 내림을 단순히 소수점 삭제로 생각하면 안 된다. 원문에서 CEILING(-38.123)은 -38, FLOOR(-38.123)은 -39이다. ROUND와 TRUNC도 다르다. 38.5235를 소수 셋째 자리까지 표시하면 반올림은 38.524, 버림은 38.523이다. EMP의 SAL/12를 반올림·버림하거나 정수로 올리는 예제로 함수 결과 차이를 확인한다.

날짜 함수는 DATE 값을 처리한다. 현재 날짜·시각은 Oracle SYSDATE, SQL Server GETDATE()로 구한다. EXTRACT(YEAR·MONTH·DAY FROM 날짜)와 DATEPART는 해당 구성값을 추출한다. 원문은 SQL Server YEAR·MONTH·DAY 함수와 Oracle TO_NUMBER(TO_CHAR(날짜,서식))도 대응시킨다. TO_CHAR만 사용하면 01 같은 문자, TO_NUMBER까지 사용하면 1 같은 숫자가 된다는 반환 유형 차이를 확인한다.

원문 날짜 연산	결과 의미
날짜 + 숫자	날짜에 해당 일수를 더한다.
날짜 - 숫자	날짜에서 해당 일수를 뺀다.
날짜1 - 날짜2	두 날짜 차이의 일수다.
날짜 + 숫자/24	날짜에 시간 수를 더한다.

날짜의 출력은 시스템 구성이나 서식에 따라 달라도 내부 날짜 값의 의미와 구분해야 한다. 원문은 날짜를 세기·연·월·일·시·분·초에 해당하는 값으로 저장하며 계산에 사용할 수 있다고 설명한다. 날짜 일부를 잘라 표시할지 숫자로 비교할지에 따라 추출 함수 또는 변환 함수를 선택한다.

명시적 변환은 TO_NUMBER·TO_CHAR·TO_DATE·CAST·CONVERT처럼 함수로 변환 의도를 적는 것이다. 암시적 변환은 데이터베이스가 자동으로 유형을 바꾸는 것이다. 원문은 암시적 변환이 성능 저하나 변환 불가 오류를 유발할 수 있으므로 명시적으로 변환할 것을 권고한다. TO_NUMBER는 변환 가능한 문자를 숫자로, TO_CHAR는 숫자·날짜를 서식에 맞는 문자로, TO_DATE는 서식에 맞는 문자열을 날짜로 바꾼다. CAST는 AS 뒤의 목표 유형으로, CONVERT는 목표 유형과 선택적 스타일을 사용한다.

변환 예에서는 날짜를 YYYY/MM/DD 등의 문자 형식으로 출력하고, 숫자를 달러와 원화 서식으로 표시한다. 로컬 통화 서식의 L과 같은 형식 문자는 출력 목적을 정한다. 또 문자로 관리한 ZIP_CODE1·ZIP_CODE2를 TO_NUMBER 또는 CAST AS INT로 변환한 뒤 합산하여 문자 합성과 숫자 덧셈의 차이를 보여준다. 변환 결과의 유형과 화면상 형식을 함께 확인한다.

CASE는 IF-THEN-ELSE와 유사한 비교·분기 표현이다. SIMPLE CASE는 CASE 다음의 칼럼 또는 표현식을 각 WHEN 기준값과 동등 비교하여 해당 THEN 값을 반환한다. SEARCHED CASE는 CASE 뒤에 공통 비교 대상을 두지 않고 WHEN마다 비교·범위 조건을 작성하므로 더 다양한 조건을 처리한다. 조건이 맞지 않으면 ELSE 값을 사용하고 ELSE를 생략하면 NULL이다. 함수처럼 중첩할 수 있다. 원문은 부서 위치의 지역 분류, 급여 등급, 보너스 계산을 제시한다.

Oracle DECODE는 표현식을 기준값들과 비교하여 대응값을 반환하고 일치하는 기준이 없으면 기본값을 반환한다. SIMPLE CASE의 동등 비교와 같은 목적을 수행한다. 원문은 CASE와 DECODE 각각 표현상의 장단점이 있으므로 요구에 맞추어 선택하도록 설명한다. 여러 범위 조건을 포함하는 SEARCHED CASE와 단순한 기준값 대응을 구분한다.

NULL 함수	반환 규칙
NVL(표현식1,표현식2) / ISNULL	첫 값이 NULL이면 둘째 값으로 대체한다. 원문은 결과 데이터 유형을 맞추어야 한다고 설명한다.
NULLIF(표현식1,표현식2)	두 값이 같으면 NULL, 다르면 첫 값을 반환한다.
COALESCE(표현식1,표현식2,...)	왼쪽부터 NULL이 아닌 최초 값을 반환한다. 모두 NULL이면 NULL이다.

NULL은 아직 정의되지 않은 값으로 0이나 공백과 다르다. NOT NULL 또는 PK로 제한하지 않은 속성은 NULL을 포함할 수 있다. 수치 연산에 NULL이 포함되면 결과도 NULL이므로 원문의 연봉 계산에서 (SAL*12)+COMM은 커미션이 NULL인 사원의 연봉도 NULL이 된다. 커미션을 NVL(COMM,0)으로 대체한 뒤 더하면 원문의 의도에 맞는 연봉을 구할 수 있다. 대체값은 연산 목적에 따라 판단한다.

NULL인 값이 있는 한 행과 검색 결과 자체가 없는 공집합은 다르다. KING의 MGR은 NULL이지만 행이 존재하므로 NVL로 9999를 출력할 수 있다. 반면 EMP에 없는 JSC를 검색하면 행이 없어서 NVL(MGR,9999)도 호출할 대상 행을 만들지 못한다. 원문은 그룹을 지정하지 않은 MAX(MGR)가 그 공집합에서 NULL 한 결과를 돌려주는 성질을 이용해 NVL(MAX(MGR),9999)로 9999를 출력한다. 스칼라 서브쿼리도 공집합에서 NULL을 내는 경우를 함께 언급한다.

NULLIF는 MGR가 7698이면 NULL로 바꾸고 아니면 MGR를 유지하는 예로 설명하며 CASE로도 같은 의도를 표현한다. COALESCE는 COMM을 우선 선택하고 NULL이면 SAL을 사용한다. COMM이 0인 행은 NULL이 아니므로 0을 그대로 선택한다. 대체·특정값의 NULL 변환·여러 대안 중 최초값 선택의 세 목적을 구분해야 한다. 집계함수 안에서 NULL을 무조건 0으로 바꾸는 것은 평균의 대상이나 불필요한 연산에 영향을 주므로 집계의 NULL 규칙과 함께 판단한다.

제4절 WHERE 절

원문 PDF 179-198쪽

WHERE는 FROM 뒤에서 조회할 개별 행을 조건으로 제한한다. 전통적 조인 문법에서는 테이블을 연결하는 조인 조건도 여기에 적는다. 불필요한 모든 행을 요청하면 서버의 CPU·메모리와 네트워크 자원을 많이 쓰므로 필요한 조건을 명확히 지정해야 한다. 원문은 조건 없는 전체 스캔을 튜닝 검토 대상으로 보면서도 병렬 처리 등의 유용한 전체 스캔까지 나쁘다고 단정하지 않는다.

종류	문법	의미
비교	=, >, >=, <, <=	같음·대소·이상·이하 비교
범위	BETWEEN a AND b	a와 b를 모두 포함하는 범위
목록	IN (list)	목록의 하나와 일치
형태	LIKE '비교문자열'	문자열 패턴과 일치
NULL 판정	IS NULL / IS NOT NULL	값의 부재 여부
논리	AND, OR, NOT	동시 만족, 하나 이상 만족, 조건 부정
부정 비교	!=, ^=, <>	같지 않음. 원문은 <>를 표준 표현으로 제시
부정 SQL	NOT BETWEEN, NOT IN	범위 밖 또는 목록과 불일치

처리 우선순위는 괄호 → 비교·SQL 연산자 → NOT → AND → OR이다. 특히 OR와 AND를 함께 적을 때 주의한다. 팀이 K02 또는 K07이면서 MF이고 키가 170~180이어야 하는 조건에서 팀 OR 조건을 괄호로 묶지 않으면 K02 조건만으로도 행이 선택될 수 있다. 원문은 괄호로 팀 조건을 묶거나 TEAM_ID IN ('K02','K07')로 표현하여 의도를 분명히 한다.

문자형 값을 비교할 때는 'K02'처럼 작은따옴표로 감싼다. 인용부호 없이 K02를 쓰면 식별자로 해석되어 원문에서 부적합한 식별자 오류가 난다. 숫자형 상수 170은 인용부호가 필요 없다. 숫자 칼럼과 숫자로 변환 가능한 문자열을 비교하면 자동 변환될 수 있지만, 변환 가능성을 기대하기보다 유형을 맞추는 원칙과 명시적 변환의 필요를 함께 기억한다.

문자 비교는 양쪽 유형을 확인한다. 양쪽 모두 CHAR이면 짧은 쪽을 공백으로 채워 비교하고 끝 공백만 다른 것은 같게 본다. 한쪽이라도 VARCHAR이면 끝 공백도 문자로 취급하며 서로 다른 문자나 길이를 기준으로 비교한다. 상수와 비교할 때는 변수 유형에 맞춘 비교 규칙이 적용된다는 원문 설명을 확인한다.

IN은 동일 칼럼의 여러 동등 조건을 OR로 연결한 목적을 간결하게 표현한다. 원문은 다중 리스트도 제시한다. (JOB,DEPTNO) 조합으로 ('MANAGER',20), ('CLERK',30)을 지정하는 것과 JOB IN ('MANAGER','CLERK') AND DEPTNO IN (20,30)은 다르다. 앞의 형태는 지정한 두 쌍만 허용하고 뒤는 각 목록 조건을 독립적으로 만족하는 조합을 허용하므로 결과가 달라진다.

LIKE의 %는 0개 이상 문자, 밑줄(_)은 정확히 한 문자를 대신한다. '장%'는 장으로 시작하는 이름을, '장_호'는 세 글자이며 장으로 시작하고 호로 끝나는 이름을 찾는 원문 사례다. 와일드카드 없이 정확한 값을 쓰면 해당 사례에서는 = 비교와 같은 결과를 얻는다. 패턴에서 한 글자와 임의 길이를 구분해야 한다.

BETWEEN은 양 끝값을 포함한다. HEIGHT BETWEEN 170 AND 180은 HEIGHT>=170 AND HEIGHT<=180과 같은 범위이다. NOT BETWEEN은 그 양 끝과 내부를 제외한다. NOT의 위치를 읽을 때도 부정 대상이 어느 조건 전체인지 확인한다. 원문은 포지션이 MF가 아니고 키가 175~185 밖인 조건을 비교한다.

NULL은 확정된 비교값이 아니므로 = NULL로 검사하지 않는다. 원문 POSITION = NULL 예는 오류 메시지 대신 선택된 행이 없는 결과를 보이며, POSITION IS NULL로 바꾸면 NULL인 세 선수를 찾는다. 수치 연산의 NULL 전파와 NULL 판정 구문은 서로 다른 용도다. WHERE가 원하는 행을 선택했는지는 문법 성공뿐 아니라 조건의 의미와 결과로 확인한다.

제5절 GROUP BY, HAVING 절

원문 PDF 199-213쪽

집계함수는 여러 행을 받아 그룹마다 하나의 결과를 반환한다. 보통 GROUP BY와 함께 사용하지만 전체 테이블을 한 그룹으로 집계하면 GROUP BY가 없어도 된다. 집계식의 ALL은 기본값이고 DISTINCT는 같은 표현식 값을 하나로 취급한다. COUNT·MAX·MIN은 문자와 날짜에도 사용할 수 있으며 집계는 SELECT·HAVING·ORDER BY 등에 사용한다.

집계함수	역할과 NULL
COUNT(*)	NULL 여부와 관계없이 행 수를 센다.
COUNT(표현식)	표현식이 NULL이 아닌 행을 센다.
SUM(표현식)	NULL을 제외한 합계
AVG(표현식)	NULL을 제외한 평균
MAX, MIN	최댓값·최솟값. 문자·날짜에도 적용
STDDEV	표준편차
VARIANCE / VAR	분산

원문 전체 선수 집계에서는 COUNT(*)가 480, COUNT(HEIGHT)가 447이다. 키가 NULL인 33행은 키 건수와 키의 최대·최소·평균 대상에서 제외된다. 결과는 최대 196, 최소 165, 평균 179.31이다. NULL을 제외한다는 것은 그 행 전체가 테이블에서 사라진다는 뜻이 아니라 해당 표현식의 집계 대상에서 제외된다는 뜻이다.

GROUP BY는 WHERE를 통과한 행을 기준 칼럼이나 표현식의 값별로 소그룹화한다. 포지션별 평균키를 선택하려면 POSITION을 그룹 기준으로 정해야 한다. 일반 칼럼과 집계함수를 기준 없이 함께 SELECT하면 원문의 단일 그룹 오류가 난다. 원문 환경에서 GROUP BY에는 SELECT 별칭을 사용할 수 없으며 원래 칼럼·표현식으로 작성해야 한다. GROUP BY 자체는 출력 순서를 보장하지 않으므로 정렬이 필요하면 ORDER BY를 사용한다.

WHERE는 그룹을 만들기 전에 개별 행을 제한하고 HAVING은 만든 그룹의 집계 결과를 제한한다. 따라서 AVG(HEIGHT)>=180 같은 집계 조건을 WHERE에 적으면 원문에서 그룹 함수가 허가되지 않는다는 오류가 난다. GROUP BY POSITION 뒤의 HAVING에 쓰면 평균이 180 이상인 포지션만 남는다. SELECT에 출력하지 않는 MAX 등의 집계함수도 HAVING 조건으로 사용할 수 있다.

그룹 기준 칼럼의 조건은 WHERE와 HAVING 양쪽에 쓸 수 있는 경우가 있다. 원문은 특정 두 팀의 인원을 집계하는 예에서 같은 결과라면 WHERE로 먼저 필요한 행만 남겨 집계 대상을 줄이는 것을 권한다. 반면 원본 행을 제한하는 WHERE 변경은 집계값 자체를 바꿀 수 있고, 이미 계산된 그룹을 고르는 HAVING은 남아 있는 그룹의 집계값을 다시 계산하지 않는다. 두 조건의 처리 대상을 구별해야 한다.

Oracle 예에서는 GROUP BY와 HAVING을 바꾸어 적어도 동작하는 경우를 보이지만 SQL Server는 문법 오류가 발생한다. 원문은 논리 흐름과 호환성을 고려하여 GROUP BY 뒤에 HAVING을 적도록 권고한다. 문법이 허용한다는 것과 읽기 좋은 표준적인 순서를 따르는 것은 별개의 판단이다.

CASE와 집계를 조합하면 정규화로 행에 저장된 구분값을 정해진 보고서 칼럼으로 펼칠 수 있다. 원문 월별 평균급여는 먼저 HIREDATE에서 입사월을 추출하고, CASE로 월에 해당하는 급여만 각 월 칼럼에 남긴 뒤, DEPTNO로 그룹화하여 AVG를 구한다. 개인 이름은 중간 흐름을 설명할 때만 보여 주고 최종 부서 리포트에서 제외한다. Oracle DECODE로도 같은 목적을 표현한다. 여러 절차의 프로그램으로 처리할 내용을 한 SQL로 구성하는 사례다.

집계 안의 NULL 처리와 집계 결과의 NULL 처리는 다르다. SUM은 일부 값이 NULL이면 이미 그 값을 제외하므로 SUM(NVL(SAL,0))처럼 매 행을 0으로 바꾸는 처리는 같은 결과라면 불필요할 수 있다. 출력 결과가 NULL일 때 0을 보여 주려면 NVL(SUM(SAL),0)처럼 최종 집계 뒤에 적용한다. CASE에서 해당하지 않는 행을 NULL로 둘 수 있고 ELSE 생략 기본값도 NULL이다. 같은 결과를 얻는 범위에서 불필요한 0 처리와 연산을 줄이는 원칙이다.

팀별 FW·MF·DF·GK 인원수는 각 포지션일 때 1을 남기는 CASE를 SUM하고 결과 NULL을 0으로 표시한다. 전체 COUNT(*)와 네 포지션 합계는 포지션 NULL 행 때문에 다를 수 있다. 원문 K08 사례는 네 분류 합계 42와 전체 45의 차이가 미정 포지션 세 명임을 보여 준다. GROUP BY 없이 CASE를 AVG에 적용하면 각 포지션 평균과 전체 평균을 한 행에 나란히 출력할 수도 있다.

제6절 ORDER BY 절

원문 PDF 214-223쪽

ORDER BY는 조회 결과를 지정 칼럼이나 표현식 순으로 정렬하며 SQL 끝에 둔다. ASC는 오름차순이며 기본값, DESC는 내림차순이다. 칼럼명뿐 아니라 SELECT 별칭과 선택 목록의 정수 위치도 사용할 수 있다. 다만 위치 번호는 선택 목록 변경 때 의미가 바뀌고 유지보수성이 낮으므로 원문은 칼럼명이나 별칭을 권한다.

여러 정렬 기준은 왼쪽부터 우선한다. 원문은 키 내림차순, 키가 같으면 백넘버 오름차순으로 정렬한다. 각 항목의 방향을 각각 정하므로 DESC 하나가 목록 전체에 적용된다고 읽지 않는다. 숫자의 오름차순은 작은 값부터, 날짜는 빠른 날짜부터다. 원문 환경에서 Oracle은 NULL을 큰 값처럼 배치하여 ASC 때 마지막, DESC 때 처음에 두고 SQL Server는 반대다.

논리 단계	역할
1 FROM	대상 테이블·집합을 참조한다.
2 WHERE	대상이 아닌 개별 행을 제거한다.
3 GROUP BY	남은 행들을 소그룹으로 만든다.
4 HAVING	그룹 조건을 만족하는 결과만 남긴다.
5 SELECT	출력 값과 표현식을 계산·선택한다.
6 ORDER BY	최종 데이터를 정렬한다.

이 순서는 SQL의 의미를 해석하는 논리 순서이며 실제 물리 실행 순서는 실행계획으로 결정된다. FROM에 정의하지 않은 테이블 칼럼을 뒤 절에서 쓸 수 없고, SELECT에서 정한 별칭은 그보다 앞서 해석하는 GROUP BY에서 재사용할 수 없다는 원문의 사례를 이 흐름과 연결해 이해한다.

일반 조회에서는 SELECT로 출력하지 않은 칼럼도 ORDER BY에 사용할 수 있다. 원문은 EMPNO·ENAME만 출력하면서 MGR로 정렬한다. 그러나 서브쿼리·인라인 뷰 바깥에서는 안쪽 SELECT에 선택하지 않은 칼럼을 더 이상 사용할 수 없다. 내부에서 정렬에 썼다는 이유만으로 그 칼럼이 외부에 공개되지는 않는다. DISTINCT나 GROUP BY, UNION 등이 있는 경우의 ORDER BY는 해당 결과 집합에서 사용할 수 있는 열의 범위를 확인해야 한다.

GROUP BY 뒤에서는 개별 행의 SAL 같은 값을 그대로 선택하거나 정렬할 수 없다. 그룹 기준과 집계 결과의 수준으로 데이터가 바뀌었기 때문이다. 원문 JOB별 조회에서 SAL로 선택·정렬하면 GROUP BY 표현식 오류가 나지만, SUM(SAL)로 선택하고 ORDER BY SUM(SAL)로 정렬하면 된다. SELECT에 출력하지 않는 집계함수도 그룹 결과에 맞는 식이면 ORDER BY에 사용할 수 있다.

제7절 조인

원문 PDF 224-240쪽

조인은 둘 이상의 테이블을 논리적인 값의 연관성으로 연결하여 필요한 데이터를 출력하는 기능이다. 보통 PK·FK 관계를 사용하지만 반드시 그러한 제약이 선언되어 있어야만 조인할 수 있는 것은 아니다. 정규화로 분리한 사실을 결합하기 때문에 관계형 데이터베이스의 핵심 기능이 된다. 테이블이 여러 개여도 내부적으로는 두 집합씩 조인하고 그 중간 결과와 다음 집합을 연결하며 순서는 옵티마이저가 정한다.

EQUI JOIN은 칼럼 값이 같은 경우를 =로 연결하는 등가 조인이다. 전통적 표현은 FROM에 테이블을 나열하고 WHERE에 연결 조건을 적으며, 표준 표현은 JOIN ... ON에 적는다. 원문 PLAYER.TEAM_ID와 TEAM.TEAM_ID를 연결하면 선수와 소속팀 정보를 함께 읽을 수 있다. 같은 값의 논리적 관련성만으로도 가능하며 PK·FK 전용 문법이 아니다.

여러 테이블에 같은 칼럼명이 있으면 테이블명이나 별칭으로 소속을 명시해야 모호하지 않다. 지금은 중복 이름이 없어도 나중에 구조가 바뀔 수 있어 원문은 조인 SQL의 칼럼 앞에 소속을 표시할 것을 권고한다. 긴 테이블명 대신 짧은 별칭을 쓰면 읽기와 작성이 쉬워진다. FROM에서 별칭을 정했다면 SELECT·WHERE 등에서도 그 별칭을 사용하며 원래 테이블명을 섞으면 오류가 난다.

여러 테이블을 하나의 연결된 내부 조인 집합으로 결합하려면 원문의 기준으로 N개 테이블에 N-1개 이상 연결 조건이 필요하다. 단순히 FROM에 나열만 하면 관계가 자동 설정되는 것이 아니다. 선수→팀→경기장에서는 TEAM_ID와 STADIUM_ID의 두 연결이 필요하다. 골키퍼만 조회한다는 검색 조건은 이 연결 조건에 더하는 행 제한이며 서로를 대체하지 않는다.

Non EQUI JOIN은 = 이외의 범위·대소 연산으로 연결하는 비등가 조인이다. 원문의 EMP 급여와 SALGRADE는 동일 번호가 아니라 SAL이 LOSAL과 HISAL 사이에 있는지로 급여 등급을 찾는다. BETWEEN은 이를 설명하는 한 사례이며 >·>=·<·<= 등도 사용할 수 있다. 서로 논리적 관련성이 있고 모델에 맞는 조건이어야 하므로 아무 테이블이나 비등가 비교로 연결할 수 있다고 생각하지 않는다.

INNER JOIN은 연결 조건이 참인 행만 남기고 OUTER JOIN은 연결 실패 행도 기준 집합에서 보존한다. 조인 성공 행은 양쪽 값이 보이고, 실패 행은 기준 테이블 값은 남으며 연결 상대에서 가져올 칼럼은 NULL이 된다. 원문은 홈팀이 없는 경기장과 사원이 없는 부서도 조회하는 사례를 제시한다.

Oracle 전통 문법의 (+)는 보존할 기준 테이블의 반대편, 즉 데이터가 없을 때 NULL로 채울 쪽 칼럼에 붙인다. 경기장 A를 모두 남기려면 B.TEAM_ID(+) = A.HOMETEAM_ID와 같이 표현한다. 원문에서 내부 조인은 홈팀이 있는 15경기장만, 외부 조인은 없는 다섯 경기장도 더한 20경기장을 출력한다. 사원·부서 외부 조인은 사원이 없는 40번 부서를 포함하여 15행을 출력한다.

한 테이블에 모든 정보를 집중하면 조회가 단순해 보일 수 있지만 중복 정합성과 변경 작업 비용이 커지고 큰 테이블에서 단순 데이터를 찾는 비용도 생긴다. 정규화와 조인은 이러한 균형을 이루는 구조다. 조인 조건을 빠뜨리거나 잘못 연결하면 불필요한 데이터 조합과 자원 사용, 긴 응답시간을 만들므로 모델의 논리 관계를 정확히 표현해야 한다.

제8절 표준 조인

원문 PDF 241-266쪽

표준 조인은 FROM 절에서 연결 형태와 조건을 명시하여 WHERE의 행 검색 조건과 구분한다. INNER JOIN이 기본이며 INNER를 생략할 수 있다. 일반 INNER JOIN은 ON 또는 USING으로 조건을 작성한다. NATURAL JOIN은 같은 이름의 모든 칼럼으로 동등 연결하는 방식이다. OUTER와 CROSS는 결과에 포함하는 행·조합의 의미가 다르므로 각각 별도로 이해한다.

NATURAL JOIN은 두 테이블에 같은 이름으로 존재하는 모든 칼럼을 자동으로 = 비교한다. EMP와 DEPT는 DEPTNO가 공통 칼럼이므로 별도 연결 조건 없이 조인된다. 조인 칼럼은 같은 유형이어야 하며 SELECT에서 그 공통 칼럼 앞에 테이블명·별칭을 붙일 수 없다. 추가 ON·USING으로 같은 조인을 다시 지정하지 않는다. 원문은 SQL Server에서 지원하지 않는다고 설명한다.

같은 이름이 같은 의미를 보장하지는 않는다. 원문은 DEPT를 복사한 DEPT_TEMP의 20·30번 부서명을 바꾼 뒤 NATURAL JOIN한다. DEPTNO뿐 아니라 DNAME·LOC까지 모두 같아야 하므로 두 부서가 제외되어 두 행만 나온다. SELECT *에서는 같은 이름의 조인 칼럼을 하나로 합치고 앞에 출력한다. 일반 ON 내부 조인은 공통 칼럼을 별도 열로 표시하므로 같은 행을 연결해도 출력 열 모양은 달라질 수 있다.

USING(칼럼명,...)은 동일 이름의 칼럼 가운데 어떤 것만 조인에 사용할지 직접 고른다. DEPTNO만 선택하면 부서명이 달라도 연결되어 네 행이 나오지만, DEPTNO와 DNAME을 같이 지정하면 이름이 바뀐 부서는 제외된다. SELECT *에서 USING 칼럼은 하나로 합쳐 먼저 보이며 나머지 칼럼은 양쪽에서 각각 출력된다. USING에 참여하는 칼럼은 SELECT에서도 접두사 없이 써야 하며 원문은 SQL Server 미지원으로 구분한다.

ON은 임의의 연결 조건을 명시하므로 칼럼 이름이 달라도 연결할 수 있다. TEAM.TEAM_ID와 STADIUM.HOMETEAM_ID를 비교하는 원문 예는 이름이 달라 USING 대신 ON을 쓴다. 동일 이름 칼럼을 사용하면 어느 테이블 것인지 접두사를 명확히 붙인다. ON의 괄호는 선택 사항이다. 여러 테이블은 JOIN ... ON을 연속하여 연결할 수 있고, EMP와 DEPT·DEPT_TEMP, 선수·팀·경기장, 일정·경기장·홈팀·원정팀 사례로 확장한다.

ON 연결 조건과 WHERE 검색 조건은 함께 쓸 수 있다. 내부 조인에서는 조건을 ON에 추가하거나 WHERE에 두어 같은 결과를 얻는 사례가 있지만 검색 목적은 WHERE에 쓰는 것을 원문은 권한다. 특히 외부 조인에서 상대의 조인 대상을 제한하는 조건은 ON에 있어야 한다는 예외를 강조한다. 외부 조인의 보존 범위를 정하는 조건과 결과 행을 제거하는 검색 조건을 구분해야 한다.

다중 조인에서도 역할이 다른 같은 테이블을 별칭으로 구분한다. 원문 경기 일정 조회는 TEAM을 홈팀용 C와 원정팀용 D로 각각 조인하고, HOME_SCORE가 AWAY_SCORE보다 3 이상 큰 조건으로 홈팀의 큰 점수 차 승리를 찾는다. 이처럼 조인 횟수는 물리적 테이블 종류 수만이 아니라 해당 SQL에서 사용하는 각 역할을 기준으로 이해한다.

CROSS JOIN은 연결 조건 없이 양쪽의 모든 조합을 만드는 곱집합이다. EMP 14행과 DEPT 4행이면 56행이고 사원마다 모든 부서가 결합된다. WHERE에 동등 연결 조건을 더하면 내부 조인과 같은 결과가 될 수 있지만 원문은 그럴 목적이면 INNER JOIN을 명시할 것을 권고한다. 일반적인 업무 모델에서 필요 빈도가 높지는 않아도 보고서·튜닝·차원 조합 등의 목적으로 의도적으로 사용할 수 있다.

표준 OUTER JOIN은 전통적 (+) 표현의 조건 구분 불명확, IN·OR 사용 제약, 표시 누락, FULL OUTER 미지원 같은 불편을 개선한다. LEFT OUTER JOIN은 왼쪽, RIGHT OUTER JOIN은 오른쪽 집합을 보존한다. 상대에 연결될 값이 없으면 상대 칼럼을 NULL로 채운다. OUTER 키워드는 생략할 수 있지만 보존 방향인 LEFT·RIGHT·FULL의 의미는 유지된다. 조건은 ON 또는 USING으로 명시한다.

FULL OUTER JOIN은 양쪽의 연결 성공 행과 양쪽 모두의 연결 실패 행을 보존한다. LEFT와 RIGHT 외부 조인 결과를 합친 의미이되 성공 행은 한 번만 표시한다. 원문은 DEPT_TEMP의 부서번호를 20 증가시켜 일부만 일

치하게 만든 뒤 FULL JOIN으로 여섯 행을 출력한다. 내부 조인이었다면 양쪽에 공통인 두 부서만 나왔을 것이다.

원문 비교 집합	결과 키 조합	건수
TABI: B,C,D,E / TAB2: A,B,C의 INNER	B-B, C-C	2
LEFT OUTER	B-B, C-C, D-NULL, E-NULL	4
RIGHT OUTER	NULL-A, B-B, C-C	3
FULL OUTER	NULL-A, B-B, C-C, D-NULL, E-NULL	5
CROSS	왼쪽 4개와 오른쪽 3개의 모든 조합	12

이 비교의 건수는 원문에 제시한 키 집합을 전제로 한다. INNER는 조건을 만족하는 연결, OUTER는 여기에 기준 집합의 연결 실패 행 보존, CROSS는 조건 없는 전체 조합이라는 결과 의미를 먼저 파악한다. 문법 이름만 암기하지 말고 어떤 행이 남으며 어느 칼럼이 NULL이 되는지까지 확인한다.

반드시 기억할 핵심 포인트

- SQL은 집합을 다루며 테이블의 행·칼럼과 PK·FK 의미를 먼저 이해한다.
- ALL은 기본값, DISTINCT는 중복 제거, ALIAS는 출력 이름이다.
- 함수는 입력·반환 유형과 인수 순서, 중첩 평가 방향을 함께 익힌다.
- NULL과 공집합은 다르며 NVL만으로 없는 행을 만들 수 없다.
- 조건 우선순위는 괄호→비교·SQL 연산→NOT→AND→OR이다.
- WHERE는 개별 행, HAVING은 그룹 결과를 제한한다. GROUP BY는 정렬을 보장하지 않는다.
- FROM→WHERE→GROUP BY→HAVING→SELECT→ORDER BY는 논리 순서이며 물리 순서는 실행계획에 따른다.
- 조인은 PK·FK 선언에만 한정되지 않는다. 테이블 역할별 별칭과 누락 없는 연결 조건이 중요하다.
- NATURAL은 모든 공통 이름, USING은 지정한 공통 이름, ON은 명시한 임의 조건으로 연결한다.
- OUTER JOIN은 기준 집합의 연결 실패 행을 보존하고 상대 칼럼을 NULL로 채운다.

챕터 최종 요약

- SELECT는 테이블의 필요한 칼럼과 표현식을 선택하고 WHERE로 행을 제한한다. 문자·숫자·날짜·변환 함수와 CASE는 각 행의 값을 가공한다. NULL 대체는 업무 의미와 적용 위치를 확인하고, 행이 없는 공집합과 구분한다.
- GROUP BY는 조건을 통과한 행을 그룹으로 바꾸며 집계함수는 그룹마다 값을 반환한다. COUNT(*)와 COUNT(표현식), SUM·AVG의 NULL 제외 규칙을 구분한다. HAVING은 집계 후 그룹을 고르고 ORDER BY는 최종 순서를 정한다.
- SQL을 해석할 때는 논리 순서와 칼럼의 유효 범위를 따른다. 인라인 뷰의 선택 밖 칼럼과 그룹의 개별 행 값은 밖에서 임의로 사용할 수 없다. 실제 접근·조인·정렬의 수행 순서는 실행계획으로 확인한다.
- 정규화한 정보를 다시 연결하는 조인은 등가·비등가의 조건과 INNER·OUTER·CROSS의 결과 의미를 함께 이해한다. 표준 조인은 ON·USING으로 연결 조건을 드러내며 NATURAL의 자동 공통 칼럼과 외부 조인의 보존 방향을 특히 주의한다.

제2장 SQL 활용

과목 II. SQL 기본과 활용

원문 PDF 272-388쪽

제1절 서브 쿼리

원문 PDF 273-287쪽

서브쿼리는 다른 SQL 문 안에 포함된 SQL 문이다. 메인 쿼리가 서브쿼리를 포함하므로 두 쿼리는 종속 관계다. 조인은 참여 테이블이 대등하여 각 테이블의 칼럼을 조회에 사용할 수 있지만, 중첩 서브쿼리의 칼럼을 메인 쿼리에서 직접 출력할 수는 없다. 출력이 필요하면 조인, 함수 또는 스칼라 서브쿼리 등 원문이 제시하는 방식을 선택한다.

결과 단위가 선택 기준이다. 조직 1건과 사원 여러 건을 조인하면 결과는 사원 단위가 된다. 조직별로 조건을 만족하는 사원이 있는지만 확인하려는 경우에는 조직을 메인 쿼리로 두고 사원을 서브쿼리로 검사한다. 조인으로 늘어난 행을 DISTINCT로 다시 조직 단위로 줄이는 방법보다 요구되는 결과 단위를 먼저 판단해야 한다.

비연관 서브쿼리는 메인 쿼리의 칼럼을 참조하지 않고 비교할 값을 제공하는 데 주로 쓰인다. 연관 서브쿼리는 메인 쿼리의 칼럼을 참조한다. 원문은 메인 행에 대해 서브 조건을 확인하는 논리 관계와 실제 실행 순서를 구별하며, 실제 수행 순서는 상황에 따라 달라질 수 있다고 설명한다. 따라서 설명상의 선후 관계를 모든 실행계획에 고정해서 적용하지 않는다.

반환 형태	사용 조건	주의점
단일 행	결과가 1건 이하일 때 =, <, <=, >, >=, <> 등과 비교	2건 이상이면 실행 시 오류. 컴파일 때는 알 수 없다.
다중 행	IN, ALL, ANY/SOME, EXISTS 등과 사용	IN은 일치 값, ALL은 모든 값, ANY는 하나 이상의 값, EXISTS는 존재 여부를 검사한다.
다중 칼럼	메인 쿼리의 여러 칼럼을 동시에 비교	칼럼 개수와 대응 위치가 같아야 한다. 원문은 SQL Server 미지원으로 설명한다.

정남일 선수의 소속팀을 구한 뒤 같은 팀 선수를 찾는 예는 단일 행 비교다. 동명이인 정현수의 소속팀은 2건이므로 = 비교가 오류를 내고 IN 비교가 두 팀을 반환한다. 팀별 최소 키를 (TEAM_ID, HEIGHT)로 함께 비교하는 예에서는 같은 팀의 최소 키가 동률이면 여러 선수가 출력된다. 팀별 평균 키와 개인 키를 비교하는 연관 예는 개인마다 자기 팀의 평균을 기준으로 삼는다.

EXISTS는 조건에 맞는 행의 존재만 필요하며, 만족하는 한 건을 찾으면 더 검색하지 않는 특성이 있다. SELECT 절의 스칼라 서브쿼리는 한 행·한 칼럼의 값을 반환하고 2건 이상이면 오류다. FROM 절의 인라인 뷰는 서브쿼리 결과를 테이블처럼 다루며, 그 안에서 정렬한 결과에 바깥 쿼리의 행 제한을 적용할 수 있다. HAVING 절에서는 집계된 그룹을 서브쿼리 결과와 비교한다.

서브쿼리는 괄호로 감싼다. 원문은 중첩·스칼라 서브쿼리의 ORDER BY 사용 제한과 인라인 뷰의 정렬 가능 여부를 구분한다. 뷰는 실제 데이터를 별도로 갖는 대신 SELECT 정의를 저장한다. 사용 시 DBMS가 정의를 참조해 질의를 재작성한다. 테이블 구조 변경의 영향 완화, 복잡한 질의의 단순화, 일부 칼럼을 숨기는 보안성이 장점이다. 기존 뷰로 다른 뷰를 정의할 수도 있으나 중첩에 따른 성능 문제를 주의한다. CREATE VIEW로 정의하고 DROP VIEW로 제거한다.

제2절 집합 연산자

원문 PDF 288-300쪽

조인이 테이블의 행과 행을 연결한다면 집합 연산자는 SELECT 결과 집합끼리 연산한다. 서로 다른 테이블의 유사한 결과를 합치거나 같은 테이블의 다른 조건 결과를 결합할 때 사용한다. 원문은 실행계획을 분리하는 튜닝 목적도 설명한다. 각 SELECT의 칼럼 수와 같은 위치의 데이터 타입이 일치해야 한다.

연산자	집합 관계	중복 처리
UNION	합집합	최종 중복 행을 하나로 만든다.
UNION ALL	결과를 그대로 결합	중복을 유지한다.
INTERSECT	교집합	최종 중복을 제거한다.
MINUS / EXCEPT	앞 집합에서 뒤 집합을 뺀 차집합	최종 중복을 제거한다. Oracle은 MINUS, SQL Server는 EXCEPT.

ORDER BY는 최종 결과에 대해 마지막에 한 번 기술한다. 최종 칼럼 제목은 첫 번째 SELECT의 별칭을 따른다. 원문의 포지션별 평균 키와 팀별 평균 키를 결합하는 예에서는 첫 번째 SELECT의 별칭이 표시되고, 상수 구분코드로 두 결과의 성격을 구별한다. 차집합은 두 집합의 순서를 바꾸면 결과가 달라진다.

삼성블루윙즈 소속 선수와 골키퍼 집합은 서로 겹친다. 원문 결과는 UNION이 88행, UNION ALL이 92행이며, 중복 검사 쿼리는 두 조건에 동시에 속하는 4명을 보여 준다. 결과가 상호 배타적인 경우 UNION과 UNION ALL의 데이터는 같을 수 있으나 정렬 순서까지 같다고 전제하지 않는다.

원문은 같은 테이블의 조건을 OR·IN으로 결합하거나, 차집합을 NOT EXISTS·NOT IN, 교집합을 EXISTS·IN으로 표현하는 비교 예를 제시한다. 이때 집합 연산의 중복 제거 의미를 유지하려고 SELECT DISTINCT를 사용한다. 구문만 바꾸기 전에 동일한 결과 단위와 중복 처리인지 확인한다.

제3절 그룹 함수

원문 PDF 301-316쪽

COUNT·SUM·AVG·MAX·MIN 같은 집계함수에 더해 ROLLUP, CUBE, GROUPING SETS를 사용하면 여러 수준의 소계·총계를 하나의 SQL로 표현할 수 있다. 같은 테이블을 집계 수준마다 반복해서 읽고 UNION ALL로 합치던 방식보다 SQL이 단순해지고 반복 액세스를 줄일 수 있다는 것이 원문의 연결 설명이다. 결과를 일정한 순서로 보려면 ORDER BY를 명시한다.

ROLLUP의 인수는 계층 관계다. N개의 그룹 항목이면 N+1 수준의 집계가 생기며 순서를 바꾸면 집계 내용도 바뀐다. 부서명·업무명 예에서는 부서·업무별 9행에 부서 소계 3행과 총계 1행이 더해져 13행이 된다. GROUP BY DNAME, ROLLUP(JOB)처럼 일부에만 적용하면 부서별 업무 소계는 있지만 전체 부서 총계는 생기지 않는다.

ROLLUP(DNAME, (JOB, MGR))처럼 괄호로 묶은 항목은 하나의 복합 항목으로 취급한다. 괄호 안 각 칼럼에 대한 소계를 따로 만들지 않는다는 점이 중요하다. 집계로 생성된 행의 구분은 표시되는 NULL 자체에 맡기지 않고 GROUPING(EXPR)을 이용한다. 소계 계산으로 묶인 항목이면 1, 그 밖에는 0이며 CASE 또는 Oracle DECODE로 원하는 소계 명칭을 표시할 수 있다.

CUBE는 그룹 칼럼의 모든 조합을 집계한다. N개 항목이면 2의 N승 수준이며 인수 간에는 ROLLUP과 같은 상하 관계가 없다. 따라서 인수 순서가 달라도 집계 데이터는 같고 표시 순서는 달라질 수 있다. 원문의 부서·업무 예는 ROLLUP에 업무별 소계 5행을 추가한 18행이다. 필요한 모든 조합을 얻지만 ROLLUP보다 연산 대상이 많아 부하를 고려해야 한다.

GROUPING SETS는 필요한 집계 집합을 직접 열거한다. 부서별 집계와 업무별 집계만 필요한 경우 두 집합을 지정하여 상세 조합이나 총계까지 불필요하게 계산하지 않는다. 인수들의 순서는 집계 의미를 바꾸지 않는다. 괄호로 묶은 각 집합을 하나의 기준으로 취급하며, 원문의 복합 집계는 (부서, 업무, 관리자), (부서, 업무), (업무, 관리자)의 결과를 결합한다.

제4절 윈도우 함수

원문 PDF 317-338쪽

윈도우 함수는 행과 행 사이의 관계를 SQL로 표현한다. GROUP BY로 행을 그룹별 한 행으로 줄이는 집계와 달리, 개별 행을 유지한 채 파티션별 합·순위·앞뒤 행·비율을 함께 표시할 수 있다. OVER가 필수이고, 함수 인수와 PARTITION BY, ORDER BY, WINDOWING의 역할을 분리해서 이해한다. 원문은 윈도우 함수의 직접 중첩을 허용하지 않되 서브쿼리에서 사용할 수 있다고 설명한다.

구성	역할	확인할 점
PARTITION BY	전체 집합을 분석할 소그룹으로 분리	생략 시 전체가 대상이다.
OVER 안 ORDER BY	순위·누적·행 순서의 기준	동률을 세부 구분하려면 추가 정렬 칼럼이 필요하다.
ROWS	현재 행 앞뒤의 물리적 행 수로 범위 지정	파티션 경계에서는 존재하는 행만 포함한다.
RANGE	정렬 값에 대한 논리적 범위 지정	같은 정렬 값을 가진 행이 함께 범위에 들어갈 수 있다.

RANK는 같은 값에 같은 순위를 주고 다음 순위를 건너뛴다. DENSE_RANK는 동률 다음 등수를 연속해서 부여한다. ROW_NUMBER는 같은 값에도 서로 다른 번호를 준다. 급여가 같은 사람 중 누가 먼저인지까지 결정하려면 급여 다음에 이름 등 정렬 기준을 추가한다. 원문의 FORD·SCOTT 다음 JONES는 RANK 4, DENSE_RANK 3이다.

일반 집계함수도 OVER와 결합한다. SUM은 파티션별 합계 또는 정렬 순서에 따른 누계를, MAX·MIN은 정해진 범위의 최댓값·최솟값을 제공한다. 같은 급여를 RANGE 누계로 묶으면 동률 행이 같은 누계를 갖는다. 파티션별 최고 급여 행 추출에는 인라인 뷰에서 RANK를 구한 뒤 순위 1을 선택하는 방식도 제시한다.

ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING은 현재 행과 바로 앞·뒤 행을 대상으로 한다. 원문의 AVG 예는 HIREDATE 정렬을 사용하므로 앞뒤 행은 입사일 정렬의 이웃 행이다. 첫 행과 마지막 행에는 각각 한쪽 이웃이 없으므로 존재하는 두 행으로 평균을 구한다. RANGE의 급여 -50부터 +150까지 조건은 행 수가 아니라 급여 값 범위에 드는 사람 수를 센다.

FIRST_VALUE와 LAST_VALUE는 정해진 윈도우의 첫 값과 마지막 값을 반환한다. 동률 중 어느 행을 택하는지 정해지지 않으면 ORDER BY의 세부 기준을 추가한다. LAST_VALUE 예는 CURRENT ROW부터 UNBOUNDED FOLLOWING까지 범위를 지정하므로 현재 행 이후 파티션 끝의 값을 본다. LAG와 LEAD는 각각 앞·뒤 몇 번째 행을 읽는다. 두 번째 인수는 이동 행 수(기본 1), 세 번째 인수는 해당 행이 없을 때 사용할 값이다.

비율 함수도 계산 기준이 다르다. RATIO_TO_REPORT는 행의 값이 파티션 합계에서 차지하는 비율을 소수로 반환하며 전체 비율 합은 1이다. PERCENT_RANK는 (RANK-1)/(n-1)이며, 단일행은 0이다. CUME_DIST는 정렬 순서상 현재 동률 행까지 포함하는 누적 비율이며 동률이면 뒤쪽 행까지 누적한 결과를 함께 사용한다. NTILE(N)은 행을 N개 그룹으로 나누고 나머지는 앞 그룹부터 한 명씩 배분한다. 원문의 14행 을 4등분하면 4·4·3·3행이다.

원문은 함수·프레임별 DBMS 지원 차이를 설명한다. 본 노트는 그 설명 범위를 요약하며 원문에 없는 최신 지원 범위를 덧붙이지 않는다. 통계·회귀 함수는 원문에서도 이름만 참조로 제시하고 상세 설명을 생략한다.

제5절 Top N 쿼리

원문 PDF 339-343쪽

Top N은 정해진 기준에서 상위 N개 행을 얻는 조회다. Oracle ROWNUM은 처리 결과 행에 임시로 붙는 번호이며 WHERE에서 행 수 제한에 사용한다. 한 행은 ROWNUM <= 1 등으로 제한하지만 두 번째 이후의 행을 ROWNUM = N으로 직접 얻는 방식은 사용할 수 없다. 여러 행은 <= N 또는 < N+1 형태로 제한한다.

같은 SELECT에서 ROWNUM 제한과 ORDER BY를 함께 쓰면 먼저 선택된 일부 행만 정렬하게 되어 전체 급여 상위 행을 보장하지 않는다. 인라인 뷰에서 전체 후보를 급여 내림차순으로 정렬한 다음, 바깥 쿼리에서 ROWNUM <= N을 적용해야 원문의 의도인 급여 상위 N건이 된다.

SQL Server TOP(Expression)은 출력 행 수를, PERCENT는 결과 중 처음 몇 퍼센트를 제한한다. WITH TIES는 마지막 선택 행과 같은 정렬 값을 가진 행도 포함하며 ORDER BY와 함께 사용한다. 원문 TOP(2) WITH TIES 예는 공동 2위까지 포함하여 3행을 출력한다. 단순 N건 제한과 동순위를 포함한 제한의 차이를 구별한다.

ROW LIMITING 절은 ORDER BY 다음에 기술한다. OFFSET은 건너뛸 행 수, FETCH는 반환할 행 수 또는 비율, ONLY는 지정 수만, WITH TIES는 마지막 행의 동순위까지 포함한다. OFFSET만 있으면 건너뛴 이후 전체를 반환한다. 원문은 Oracle 12.1, SQL Server 2012부터의 문법으로 소개하며 ROW와 ROWS를 구별하지 않아도 된다고 설명한다.

제6절 계층형 질의와 셀프 조인

원문 PDF 344-357쪽

사원과 관리자처럼 같은 엔터티의 상하 관계를 순환 관계로 모델링하면 동일 테이블 안에 계층이 들어간다. 셀프 조인은 동일 테이블을 서로 다른 역할의 테이블처럼 조인한다. 테이블 별칭과 칼럼의 별칭 한정이 필수다. 자식의 관리자 번호를 부모의 사원 번호와 연결하면 부모에서 자식으로, 그 반대 조건이면 자식에서 부모로 전개한다. 더 깊은 레벨을 셀프 조인으로 찾으려면 조인을 추가해야 하므로 전용 계층 질의가 필요해진다.

Oracle START WITH는 전개의 시작 행을 정한다. CONNECT BY는 다음 연결 대상의 조건이고 PRIOR는 이전에 읽은 행의 칼럼을 가리킨다. MGR = PRIOR EMPNO는 관리자에서 부하로 내려가고, EMPNO = PRIOR MGR는 부하에서 관리자로 올라간다. 루트·레벨은 전개 방향과 시작점에 따라 결정되므로 업무상의 최상위 사원만 항상 LEVEL 1인 것은 아니다.

LEVEL은 시작 행 1에서 한 단계마다 증가한다. CONNECT_BY_ISLEAF는 더 전개할 자식이 없는 행을 1로 표시한다. 순환이 있으면 실행 시 오류가 발생할 수 있으며 NOCYCLE은 순환 이후의 전개를 멈추게 한다. CONNECT_BY_ISCYCLE은 조상에 다시 연결되는 상황을 표시한다. 원문 표의 사용 옵션 표기는 NOCYCLE 관련 설명과 함께 읽는다.

CONNECT BY 조건은 계층을 어떻게 이어 갈지 정하는 반면 WHERE는 전개 후 결과를 필터한다. ORDER SIBLINGS BY는 형제 노드끼리 정렬한다. SYS_CONNECT_BY_PATH(칼럼, 구분자)는 루트부터 현재 행까지의 경로를, CONNECT_BY_ROOT 칼럼은 현재 행이 속한 루트의 값을 보여 준다. LPAD를 LEVEL과 결합해 들여쓰면 계층이 눈에 보이게 출력된다.

- SQL Server 재귀 CTE 처리: ① 앵커 멤버와 재귀 멤버를 나눈다. ② 앵커를 실행해 T0를 만든다. ③ Ti를 입력으로 재귀 멤버를 실행해 Ti+1을 만든다. ④ 빈 집합이 될 때까지 반복한다. ⑤ T0부터 Tn까지 UNION ALL 결과를 반환한다.
- 재귀 CTE의 앵커가 시작 집합이고 이전 단계 결과가 다음 단계의 바깥 집합이 된다. 자동으로 원하는 조직도 표시 순서까지 보장하지 않으므로 원문은 경로 형태의 SORT 칼럼과 마지막 ORDER BY를 추가한다. 앵커·재귀 멤버의 데이터 형식도 일치시킨다.

제7절 PIVOT 절과 UNPIVOT 절

원문 PDF 358-367쪽

PIVOT은 행 값을 열로 전환하고 집계한다. 집계함수는 계산할 값, FOR는 열로 펼칠 기준 칼럼, IN은 실제로 펼칠 값들을 지정한다. 집계함수와 FOR에 들어 있지 않은 나머지 칼럼이 집계 기준이 되므로, 인라인 뷰에서 필요한 열만 먼저 선택해야 의도한 단위로 집계된다. 원문의 JOB·DEPTNO·SAL 입력은 JOB별로 부서 급여를 열에 펼친다. 입사연도 표현식을 입력에 더하면 연도도 집계 기준에 추가된다.

집계함수와 IN 값에 별칭을 지정하면 결과 열 이름을 제어할 수 있다. 원문은 두 곳에 모두 별칭을 주는 방식을 권한다. 여러 집계함수와 여러 FOR 칼럼도 사용할 수 있고, 복수 FOR 칼럼이면 IN에서도 대응하는 값의 쌍을 지정한다. 결과에 붙은 열 이름으로 SELECT·WHERE를 작성할 수 있다. PIVOT을 사용할 수 없으면 SUM(CASE ...)와 GROUP BY로 같은 변환을 표현하는 예를 제시한다.

UNPIVOT은 여러 열을 행으로 전환한다. 값이 들어갈 칼럼, 원래 어느 열에서 왔는지 설명할 FOR 칼럼, 대상 열과 그 설명 값을 IN에서 정한다. 원문의 D10_SAL·D20_SAL은 SAL로 모이고 DEPTNO에는 10·20이 들어간다. 기본 결과에서 값이 NULL인 행은 빠지며 INCLUDE NULLS를 지정하면 포함된다. 원문 예는 3행에서 4행으로 늘어난다.

급여와 인원수처럼 서로 대응하는 여러 열을 함께 UNPIVOT할 수도 있고 설명용 열을 여러 개 둘 수도 있다. 전용 구문이 없는 경우 원문은 대상 열 수만큼 카티션 곱으로 행을 복제한 뒤 CASE로 해당 열을 선택하는 방식을 설명한다. PIVOT은 집계가 포함된 변환이므로 단순히 모양만 반대로 돌린다고 원래 개별 행이 되살아난다고 생각해서는 안 된다.

제8절 정규 표현식

원문 PDF 368-383쪽

정규 표현식은 문자열의 규칙을 나타내는 검색 패턴이며 검색과 치환에 사용한다. 점(.)은 기본적으로 개행을 제외한 문자 하나, |는 대체, 역슬래시는 다음 문자를 일반 문자로 해석하게 한다. ^와 $는 시작·끝을 고정한다. 대체 패턴은 기술 순서에 따라 일치하므로 a|aa와 aa|a의 결과를 동일하게 보지 않는다.

종류	원문의 표현	의미
반복 횟수	?, *, +	각각 0~1회, 0회 이상, 1회 이상
횟수 지정	{m}, {m,}, {,n}, {m,n}	정확히 m회, 최소 m회, 최대 n회, m~n회
묶기·역참조	(expr), ₩n	표현식을 한 단위로 묶고 1~9번째 일치한 부분을 다시 참조
문자 리스트	[char...], [^char...], [0-9]	목록의 한 문자, 목록 밖 한 문자, 범위
POSIX 클래스	[:digit:], [:lower:], [:upper:], [:alpha:], [:alnum:], [:xdigit:], [:punct:], [:blank:], [:space:]	숫자·대소문자·영문숫자·16진수·기호·공백 종류
PERL 계열	₩d/₩D, ₩w/₩W, ₩s/₩S	숫자/비숫자, 영문숫자·밑줄/그 밖, 공백/비공백

일반 수량사는 가능한 많이 일치시키는 탐욕적 방식이다. 수량사 뒤에 ?를 붙인 ??, *?, +?, {m,n}? 등은 최소로 일치시키는 비탐욕적 방식이다. 정확한 횟수 {m}처럼 범위에 여지가 없으면 탐욕성에 따른 결과 차이가 없다. 괄호의 묶음과 역참조는 반복된 문자열을 찾거나 치환할 때 부분 문자열의 위치를 바꾸는 데 사용한다.

REGEXP_LIKE는 패턴 일치 여부를 조건으로 검사한다. REGEXP_REPLACE는 일치한 부분을 바꾼 문자열을 반환하고 일치가 없으면 원본을 돌려준다. REGEXP_SUBSTR는 일치한 문자열, REGEXP_INSTR는 위치,

REGEXP_COUNT는 횟수를 반환한다. SUBSTR·INSTR의 subexpr은 0이면 전체 패턴이고 1 이상이면 왼쪽 괄호 순서의 부분 표현식이다. INSTR의 return_opt는 0이면 시작 위치, 1이면 일치 뒤 위치다.

공통 인수는 검색 문자열, 패턴, 시작 위치, 일치 차례, 일치 옵션이며 함수마다 지원하는 인수가 다르다. match_param의 i는 대소문자 무시, c는 구분, n은 점이 개행에도 일치, m은 다중 행, x는 패턴 공백 무시다. 여러 옵션을 함께 지정할 수 있다. 원문 REGEXP_REPLACE의 occurrence 기본값 표기는 그대로 원문 설명에 속하므로 여기에서는 기본값을 새로 일반화하지 않고 인수의 역할과 원문 사례만 정리한다.

반드시 기억할 핵심 포인트

- 서브쿼리의 반환 행 수와 비교 연산자는 함께 결정한다. 다중 칼럼은 위치까지 대응한다.
- UNION ALL만 중복을 그대로 유지한다. 집합의 정렬은 마지막 ORDER BY로 지정한다.
- ROLLUP은 인수 순서가 중요하고 CUBE·GROUPING SETS는 집계 의미가 인수 순서에 좌우되지 않는다.
- 파티션, 정렬, 프레임은 다른 기준이다. ROWS는 행 수, RANGE는 값 범위다.
- 상위 N건은 먼저 올바른 정렬 집합을 만든 뒤 제한한다. 동률 포함 여부도 별도 결정한다.
- 계층의 연결 조건과 전개 뒤 필터를 구별한다. PIVOT의 나머지 열은 집계 기준이 된다.
- 정규 표현식 함수는 일치 여부·문자열·위치·개수·치환 결과를 구별한다.

챕터 최종 요약

- 서브쿼리는 메인 집합의 행을 유지하며 조건이나 값을 제공한다. 조인으로 행이 늘어나는 관계와 구별하고 단일·다중 행, 다중 칼럼, 연관 여부를 각각 판단한다.
- 집합 연산은 같은 모양의 결과를 합·교·차로 결합한다. 여러 집계 수준은 ROLLUP·CUBE·GROUPING SETS로, 개별 행의 분석은 OVER를 갖는 윈도우 함수로 표현한다.
- 순위와 비율의 정의, 프레임의 포함 범위, 정렬 후 Top N, 시작점과 방향을 갖는 계층 전개가 결과를 결정한다.
- 행·열 전환에서는 집계 기준과 NULL 포함 여부를, 패턴 검색에서는 반복·묶기·앵커·반환값의 종류를 확인한다.

제3장 관리 구문

과목 II. SQL 기본과 활용

원문 PDF 390-461쪽

제1절 DML

원문 PDF 391-402쪽

DML은 테이블의 데이터를 조작한다. INSERT는 입력, UPDATE는 수정, DELETE는 삭제이며 MERGE는 소스와 타깃을 비교해 입력·수정을 한 문장에 표현한다. 원문은 각 예제 뒤에 SELECT로 변경 내용을 확인하고 ROLLBACK으로 취소한 다음 다음 예제를 실행하는 흐름으로 설명한다. 따라서 서로 다른 예제의 행 수를 하나의 연속 실행 상태로 섞지 않는다.

단일 행 INSERT는 INTO의 칼럼과 VALUES의 값을 1:1로 대응시킨다. 칼럼을 나열하는 순서는 테이블 정의 순서와 달라도 되지만 값은 나열한 칼럼의 순서와 맞아야 한다. 문자 값은 작은따옴표로 감싸고 수치는 수치로 기술한다. 생략된 칼럼은 DEFAULT가 있으면 그 값, 그렇지 않으면 NULL이 적용되며 필수 칼럼에 값이 없으면 제약조건 오류가 생긴다.

INTO의 칼럼 목록을 생략하면 정의된 모든 칼럼의 순서와 개수에 맞춰 값을 제공해야 한다. 이후 칼럼 추가·삭제 시 값 개수 불일치가 발생할 수 있어 원문은 명시적인 칼럼 목록을 권한다. VALUES 안에 서브쿼리를 둘 수도 있고, INSERT INTO ... SELECT ...는 조회된 여러 행을 한 번에 입력한다. 이때 대상 칼럼 수와 SELECT의 칼럼 수가 같아야 한다. 원문은 TEAM_ID의 K를 A로 바꾸고 지역명에 올스타를 붙인 팀 자료를 입력하는 예를 제시한다.

UPDATE의 SET은 바꿀 칼럼과 새 값을, WHERE는 수정할 행을 정한다. WHERE가 없으면 전체 행이 수정된다. SET의 서브쿼리가 값을 반환하지 않아 NULL이 되면 대상 칼럼에 NULL이 들어갈 수 있다. 원문은 경기장의 홈팀 정보가 존재하는 행만 EXISTS로 골라 수정하는 방식과, 같은 소스를 두 번 조회하는 비효율을 MERGE로 줄이는 방식을 연결해 설명한다. 여러 칼럼을 함께 대입하는 예도 있다.

DELETE는 WHERE에 맞는 행을 제거하며 FROM은 원문의 기본 구문에서 생략 가능하다. 조건이 없으면 모든 행을 삭제하지만 테이블 구조는 남는다. 다른 테이블을 EXISTS로 참조하거나 GROUP BY·HAVING 서브쿼리를 이용해 삭제 대상을 정할 수 있다. 원문의 팀별 인원 10명 이하 조건은 해당 팀의 모든 선수 행을 삭제하는 조건이지 각 팀에서 10명만 지우는 제한이 아니다.

MERGE의 INTO는 타깃, USING은 소스, ON은 두 집합을 대응시키는 조건이다. 일치한 행에는 WHEN MATCHED THEN UPDATE를, 일치하지 않은 소스 행에는 WHEN NOT MATCHED THEN INSERT를 수행한다. 소스는 테이블뿐 아니라 서브쿼리일 수 있고, 원문은 수정 부분만 또는 입력 부분만 사용하는 예도 제시한다. ON의 일치 판단과 UPDATE·INSERT의 실제 칼럼 대응을 나누어 읽으면 긴 문장이 분명해진다.

DML 변경의 확정은 트랜잭션 처리 방식과 연결된다. Oracle 예는 COMMIT 또는 ROLLBACK으로 종료하며 SQL Server는 기본 AUTO COMMIT을 설명한다. 전체 행 제거에서 TRUNCATE가 DELETE보다 부하가 적다는 원문의 설명은 DDL·트랜잭션 차이와 함께 읽어야 한다. 원문은 SQL Server에서 명시적으로 시작한 트랜잭션 안의 TRUNCATE를 롤백할 수 있다는 예외도 별도로 밝힌다.

제2절 TCL

원문 PDF 403-417쪽

트랜잭션은 서로 분리할 수 없는 한 개 이상의 데이터베이스 연산으로 이루어진 논리적 작업 단위다. 계좌이체의 출금과 입금처럼 전부 적용하거나 전부 취소해야 한다. 데이터 변경 DML이 주된 대상이고 SELECT FOR UPDATE처럼 배타적 잠금을 요구하는 조회도 대상이 될 수 있다. COMMIT은 확정, ROLLBACK은 취소, SAVEPOINT는 부분 취소의 기준을 정한다.

특성	원문의 의미
원자성	모든 연산이 성공하거나 전혀 수행되지 않은 상태로 남아야 한다.

특성	원문의 의미
일관성	수행 전 데이터가 올바르면 수행 후에도 모순된 상태가 없어야 한다.
고립성·격리성	수행 중 다른 트랜잭션의 영향으로 잘못된 결과를 만들지 않아야 한다.
지속성·영속성	성공한 변경 내용은 영구적으로 저장된다.

COMMIT 전에는 자신의 변경을 조회하고 취소할 수 있다. 다른 사용자는 아직 확정되지 않은 변경을 보지 못하며, 변경 행에는 잠금이 걸려 다른 변경이 제한된다. COMMIT하면 변경이 반영되고 다른 사용자가 결과를 볼 수 있으며 관련 잠금이 해제된다. ROLLBACK은 반영되지 않은 변경을 취소하여 시작 이전 상태로 돌리고 잠금을 해제한다. 이로써 영구 확정 전에 내용을 확인하고 논리적으로 관련된 작업을 묶어 무결성을 보장한다.

원문의 SQL Server 트랜잭션 방식은 세 가지다. AUTO COMMIT은 각 명령의 성공·오류에 따라 DBMS가 확정·취소한다. 암시적 트랜잭션은 시작을 DBMS가 처리하고 종료는 사용자가 명시한다. 명시적 트랜잭션은 BEGIN TRANSACTION 또는 BEGIN TRAN으로 시작하고 COMMIT·ROLLBACK으로 끝낸다. 전체 ROLLBACK은 최초 BEGIN TRANSACTION까지 되돌아간다는 설명이다. 자동 커밋된 변경을 뒤늦게 ROLLBACK하는 것으로 생각하지 않는다.

SAVEPOINT는 트랜잭션 안의 특정 시점을 이름으로 저장한다. Oracle은 SAVEPOINT 이름, ROLLBACK TO 이름을 사용하고 SQL Server는 SAVE TRANSACTION 이름, ROLLBACK TRANSACTION 이름을 사용한다. 같은 이름을 다시 정하면 마지막 저장점이 유효하다. A 이후 B를 만든 뒤 A까지 롤백하면 B는 무효가 되므로 미래의 B로 다시 돌아갈 수 없다. 저장점 없이 ROLLBACK하면 모든 미확정 변경을 취소하고 트랜잭션을 종료한다.

원문의 단계 예시는 INSERT 후 A 저장점, UPDATE 후 B 저장점, DELETE의 순서다. B로 되돌리면 DELETE만 취소되고 UPDATE와 INSERT는 남는다. A로 되돌리면 UPDATE도 취소되고 INSERT는 남는다. 전체 ROLLBACK하면 INSERT까지 취소된다. 부분 롤백은 그 이전의 작업을 자동으로 커밋한다는 뜻이 아니다.

Oracle 설명에서는 대상 SQL 수행 시 트랜잭션이 시작되고 COMMIT·ROLLBACK으로 종료한다. DDL 전후의 자동 커밋, 정상 접속 종료 시 커밋, 비정상 단절 시 롤백도 설명한다. SQL Server는 기본 AUTO COMMIT과 비정상 접속 단절 시 롤백을 구분한다. 접속 종료·지원 범위에 대한 기술은 원문의 설명 기준으로 정리한 것이다.

제3절 DDL

원문 PDF 418-442쪽

DDL은 테이블 등의 객체 구조를 정의한다. 테이블 생성 전 저장할 칼럼, 데이터 유형, 후보키 중 기본키, 다른 테이블을 참조할 외래키를 정한다. 팀 이름을 선수마다 반복 저장하면 팀명 변경이나 팀 삭제 때 수정·삭제 이상이 생길 수 있으므로 팀을 분리하고 TEAM_ID로 참조하는 관계가 모델링·정규화와 DDL을 이어 준다.

CREATE TABLE은 테이블 이름 다음 괄호 안에 칼럼명과 유형을 쉼표로 구분한다. 마지막 항목 뒤에는 쉼표를 두지 않는다. 같은 테이블에서 칼럼명이 중복되지 않아야 하고 이름·예약어·길이 제한을 따른다. 원문은 문자로 시작하는 식별자와 허용 문자, CHAR·VARCHAR 계열의 길이 지정, DATE 유형의 크기 미지정 등을 설명한다. 다른 테이블에 같은 칼럼 이름이 있는 것은 허용되며 조인 관계의 연결고리가 된다.

제약조건은 허용되는 데이터만 유지해 무결성을 보장한다. 칼럼 정의 뒤에 쓰는 칼럼 수준 방식과 마지막에 CONSTRAINT로 쓰는 테이블 수준 방식을 혼합할 수 있다. 뒤늦게 제약을 추가하면 이미 저장된 자료 때문에 처리하기 어려울 수 있어 생성 시 검토해야 한다. PK는 단일 또는 복합 칼럼이고 한 테이블에 한 기본키 제약을 둔다.

제약·기본값	역할	원문에서 구분할 점
PRIMARY KEY	행을 고유하게 식별	고유성+NOT NULL. 자동 UNIQUE 인덱스 생성으로 설명한다.

제약·기본값	역할	원문에서 구분할 점
UNIQUE	고유 값 유지	원문은 NULL을 고유키 제약 대상에서 제외한다고 설명한다.
NOT NULL	필수 입력	NULL 값 입력을 금지한다.
CHECK	값의 범위·조건 제한	조건식으로 허용 데이터 제한
FOREIGN KEY	다른 테이블의 키 참조	참조 무결성 옵션과 연결
DEFAULT	값 생략 시 기본값 제공	NULL·공백·숫자 0·공집합을 혼동하지 않는다.

생성 후 Oracle DESCRIBE/DESC, SQL Server sp_help로 구조를 확인한다. CREATE TABLE ... AS SELECT(CTAS)는 SELECT 결과로 테이블을 만들고 SQL Server는 SELECT ... INTO ...를 사용한다. 원문은 CTAS에서 NOT NULL 외 기본키·고유키·외래키·CHECK 제약은 사라지므로 필요한 제약을 다시 추가해야 한다고 강조한다. SQL Server의 Identity 속성 관련 복사 차이도 함께 언급한다.

ALTER TABLE ADD는 칼럼을 추가하며 원문 기준 새 칼럼은 마지막에 위치한다. DROP COLUMN은 데이터 유무와 관계없이 칼럼을 제거하되 적어도 하나의 칼럼은 남아야 하고 삭제한 칼럼은 복구할 수 없다는 설명이다. Oracle은 MODIFY, SQL Server는 ALTER COLUMN으로 유형·크기·NOT NULL 등을 바꾼다. 데이터가 있으면 크기 축소에 제한이 있고, NULL만 있거나 빈 테이블인 경우의 변경 가능 조건을 별도로 설명한다.

DEFAULT 변경은 이후 입력에 영향을 주며 기존 행 전체를 바꾸는 UPDATE가 아니다. NOT NULL 추가는 기존 NULL이 없어야 한다. 칼럼 이름은 Oracle RENAME COLUMN, SQL Server sp_rename으로 바꾼다. 원문은 관련 제약의 반영과 이름 변경으로 기존 스크립트·저장 프로시저가 영향받을 수 있다는 점을 함께 제시한다. ADD CONSTRAINT·DROP CONSTRAINT로 외래키 등의 제약을 추가·제거한다.

외래키가 참조하는 부모 테이블 또는 부모 행은 참조 조건에 의해 삭제가 제한될 수 있다. 원문에서 TEAM을 DROP하거나 선수가 속한 팀 행을 DELETE하면 참조 제약 오류가 발생하고 제거되지 않는다. 이것은 실수로 필요한 데이터를 지우지 않게 하는 참조 무결성의 실제 효과다. 테이블 이름은 Oracle RENAME ... TO, SQL Server sp_rename으로 바꾼다.

DROP TABLE은 구조와 데이터를 모두 제거한다. Oracle의 CASCADE CONSTRAINTS는 참조 제약까지 제거하는 옵션이며 SQL Server에는 해당 옵션이 없어 참조 외래키 또는 참조 테이블을 먼저 처리하는 설명이다. TRUNCATE TABLE은 구조를 유지하며 모든 행을 제거하고 저장 공간을 재사용 가능하게 한다. 원문은 내부 처리와 자동 커밋 특성으로 DDL로 분류한다. DELETE와 TRUNCATE를 단지 결과 행 수가 0이라는 이유로 같은 작업으로 보지 않는다. TRUNCATE의 복구 관련 일반 설명은 앞 절의 SQL Server 명시적 트랜잭션 예외와 함께 읽는다.

제4절 DCL

원문 PDF 443-459쪽

사용자를 생성하고 필요한 권한을 부여하면 운영 계정을 공유하지 않고 특정 데이터만 공개할 수 있다. 사용자 생성, 로그인, 테이블 생성, 다른 사용자의 객체 조회는 각각 별도 권한의 문제다. 접속에 성공했다고 모든 DDL·DML이 허용되는 것은 아니다. 원문은 Oracle의 SYS·SYSTEM·SCOTT 계정을 각각 관리용·관리용·샘플용 역할로 소개한다.

Oracle은 사용자로 인스턴스에 접속하고 대응 스키마에 객체를 생성하는 형태다. SQL Server는 인스턴스에 접속할 로그인과 데이터베이스별 사용자를 만들고 둘을 매핑한다. 사용자는 스키마에 대한 권한을 가지며 객체는 스키마에 속한다. 따라서 로그인 → 인스턴스 → 데이터베이스 사용자 → 스키마 → 테이블의 계층을 구별해야 한다.

SQL Server는 Windows 인증과 혼합 모드를 설명한다. Windows 인증은 운영체제가 확인한 보안 토큰을 신뢰하는 연결이다. 혼합 모드에서는 Windows 인증과 SQL 인증을 사용할 수 있다. 원문은 Windows 인증의 정책·

계정 잠금·만료 지원과 SQL 인증의 강력한 암호 사용을 설명한다. SQL Server는 로그인을 먼저 만든 뒤 대상 데이터베이스로 이동하여 사용자를 생성하는 흐름이다.

시스템 권한은 CREATE USER·CREATE SESSION·CREATE TABLE처럼 특정 종류의 작업을 할 수 있는 권한이다. 원문의 SQLD 사용자는 생성 후에도 CREATE SESSION이 없어 접속하지 못하고, 접속 권한을 받은 후에도 CREATE TABLE이 없어 테이블 생성에 실패한다. 각각 GRANT로 필요한 권한을 받아야 다음 작업이 가능하다. SQL Server 사례는 스키마 권한도 별도로 부여한다.

객체 권한은 특정 테이블·뷰 등의 SELECT·INSERT·UPDATE·DELETE 같은 작업 권한이다. 다른 사용자의 객체는 소유자/스키마 이름을 한정하고 적절한 권한을 받아야 한다. MENU에 SELECT만 부여받은 SCOTT은 조회할 수 있지만 UPDATE는 권한 부족 오류가 난다. 객체별·작업별로 권한을 따로 관리한다는 의미다. GRANT는 부여, REVOKE는 회수다.

ROLE은 사용자와 여러 권한 사이의 중개 단위다. ROLE을 만들고 시스템·객체 권한을 넣은 다음 사용자나 다른 ROLE에 부여하면 사용자마다 권한을 일일이 반복 설정하는 부담을 줄인다. 원문의 LOGIN_TABLE은 CREATE SESSION·CREATE TABLE을 묶는다. CONNECT·RESOURCE의 권한 목록과 DROP USER ... CASCADE로 사용자 객체를 함께 삭제하는 사례도 제시한다.

SQL Server는 원문에서 제공 역할의 멤버로 참여하는 방식을 설명한다. 서버 수준 역할은 인스턴스 작업용 로그인에, 데이터베이스 수준 역할은 개별 데이터베이스 작업용 사용자에 대응한다. 서버 수준의 public·bulkadmin·dbcreator·diskadmin·processadmin·securityadmin·serveradmin·setupadmin·sysadmin은 기본 권한 상속, 대량 입력, 데이터베이스·파일·프로세스·보안·서버 구성·연결 서버·전체 작업 등 역할을 나눈다. 데이터베이스 역할은 액세스·백업·읽기·쓰기·DDL·읽기/쓰기 거부·소유자·보안 관리로 나뉜다. 원문이 제시한 역할별 범위를 필요한 작업 수준과 연결해 복습한다.

반드시 기억할 핵심 포인트

- DML의 대상은 행, DDL의 대상은 구조, TCL의 대상은 변경의 확정·취소 단위, DCL의 대상은 사용자·권한이다.
- UPDATE·DELETE에서 WHERE를 생략하면 전체 행에 적용된다.
- MERGE의 ON 결과가 MATCHED·NOT MATCHED 분기의 기준이다.
- 저장점 A로 되돌리면 A 이후 저장점은 무효다. 부분 롤백을 커밋으로 오해하지 않는다.
- DEFAULT 변경은 이후 입력에 영향을 주고 기존 자료를 일괄 수정하지 않는다.
- 참조 무결성은 부모 삭제를 제한할 수 있다. CTAS가 모든 제약을 복사하지 않는다.
- Oracle과 SQL Server의 자동 커밋·로그인·스키마 구조를 원문 기준으로 구분한다.

챕터 최종 요약

- INSERT·UPDATE·DELETE·MERGE로 행을 바꾸고, COMMIT·ROLLBACK·SAVEPOINT로 논리적 작업 단위의 무결성을 유지한다.
- 구조는 CREATE·ALTER·RENAME·DROP·TRUNCATE로 관리한다. 제약조건과 데이터 유형이 허용되는 값과 테이블 간 참조를 정한다.
- 권한은 접속, 종류별 작업, 특정 객체 작업으로 구분하고 ROLE로 묶는다. SQL Server의 로그인·사용자·스키마와 Oracle 사용자 체계를 구별한다.
- 전체 행 삭제, 구조 삭제, 변경 취소 가능성은 명령 이름만으로 일반화하지 않고 DBMS·트랜잭션 문맥까지 함께 확인한다.

제1장 SQL 수행 구조

과목 III. SQL 고급 활용 및 튜닝

원문 PDF 465-503쪽

제1절 데이터베이스 아키텍처

원문 PDF 466-478쪽

데이터베이스와 인스턴스. Oracle에서 데이터베이스는 데이터 파일·Redo 로그 파일·컨트롤 파일 등 디스크에 저장된 데이터 집합이다. 인스턴스는 SGA 공유 메모리와 이를 사용하는 프로세스 집합이다. 기본적으로 한 인스턴스는 한 데이터베이스에 접근하며, RAC에서는 여러 인스턴스가 한 데이터베이스를 공유한다. 한 인스턴스가 여러 데이터베이스에 접근하는 구조는 아니다. SQL Server는 한 인스턴스 아래에 시스템 데이터베이스와 사용자 데이터베이스를 함께 두는 구조다.

서버 프로세스와 연결. 서버 프로세스는 사용자 명령을 받아 파싱·최적화·블록 읽기·정렬·결과 전송을 수행한다. 직접 수행하도록 구현되지 않은 작업은 운영체제, I/O 서브시스템 또는 백그라운드 프로세스에 요청한다. 전용 서버 방식에서는 서버 프로세스가 한 사용자 프로세스를 전담한다. SQL마다 연결과 해제를 반복하면 서버 프로세스 생성 비용이 커지므로 OLTP 애플리케이션에서 연결 풀을 이용해 연결을 재사용한다.

공유 서버 방식. 여러 세션이 미리 생성한 서버 프로세스를 공유한다. 사용자 요청은 Dispatcher를 통해 SGA의 요청 큐에 들어간다. 가용 서버 프로세스가 요청을 꺼내 처리하고 응답 큐에 결과를 넣으면 Dispatcher가 사용자에게 전달한다. 전용 서버의 연결 풀과 공유 서버는 모두 재사용이라는 관점에서 이해할 수 있으나, 요청을 전달하는 내부 구조는 구분해야 한다. 원문은 SQL Server가 스레드 기반임을 밝히고, 역할 설명에서는 프로세스로 통칭한다.

백그라운드 프로세스	주요 역할
SMON	인스턴스 복구, 임시 세그먼트·익스텐트 정리
PMON	비정상 종료 프로세스가 사용하던 자원 복구
DBWn	Dirty 버퍼를 데이터 파일에 기록
LGWR	로그 버퍼의 엔트리를 Redo 로그 파일에 기록
ARCn	재사용할 Online Redo 로그를 별도 위치에 보관
CKPT	체크포인트를 유도하고 기록 완료 지점을 컨트롤 파일·데이터 파일 헤더에 반영
RECO	분산 트랜잭션 과정에서 생긴 문제 해결

저장공간의 계층. 블록은 읽고 쓰는 단위이고 익스텐트는 공간을 할당받는 단위다. 세그먼트는 테이블·인덱스·Undo처럼 저장공간을 필요로 하는 오브젝트이며 한 개 이상의 익스텐트를 사용한다. 테이블스페이스는 세그먼트를 담는 논리적 컨테이너이고 여러 데이터 파일로 구성된다. 사용자는 세그먼트의 테이블스페이스를 지정하며, 실제 파일 선택과 익스텐트 할당은 DBMS가 수행한다. 세그먼트는 한 테이블스페이스에만 속하지만 그 안의 여러 데이터 파일에 걸쳐 저장될 수 있다. 파티션 테이블·인덱스는 내부적으로 여러 세그먼트를 갖는다.

블록과 익스텐트의 구분. 한 칼럼만 필요해도 그 레코드가 속한 블록 전체를 읽는다. 따라서 성능 판단에서 레코드 수보다 접근 블록 수가 중요하다. 한 익스텐트 안의 블록은 논리적으로 인접하지만 서로 다른 익스텐트가 파일 안에서 인접한다고 보장할 수는 없다. 원문에서 SQL Server의 페이지는 8KB, 익스텐트는 8페이지인

64KB로 설명한다. 균일 익스텐트는 한 오브젝트가 사용하고, 혼합 익스텐트는 여러 오브젝트가 페이지를 나누어 사용한다. SQL Server의 파일 그룹은 Oracle의 테이블스페이스에 대응한다.

임시 파일과 로그 파일. 임시 데이터 파일은 정렬·해시 작업 중 메모리가 부족할 때 중간 결과를 담는다. 임시 오브젝트는 작업 후 제거되며 원문은 Redo에 의한 복구와 백업 대상에서 제외되는 성격을 설명한다. Oracle은 사용자별 임시 테이블스페이스를 지정할 수 있고 SQL Server는 tempdb를 공유한다. Redo 로그와 트랜잭션 로그는 버퍼 캐시에 가한 변경을 기록한다. 데이터 파일의 랜덤 쓰기를 매 변경마다 기다리는 대신 로그에 빠르게 추가 기록하고, 데이터 파일 동기화는 나중에 일괄 수행하는 것이 Fast Commit의 바탕이다.

복구와 체크포인트. Oracle의 Online Redo 로그는 최소 두 파일을 번갈아 재사용하며, 장애 시 마지막 체크포인트 이후 변경을 재현하는 캐시 복구에 사용한다. Archived Redo 로그는 Online 로그를 덮어쓰기 전에 별도 위치에 보관한 것으로 물리적 저장 매체 장애 시 데이터베이스 복구에 사용한다. 체크포인트 정보는 어디까지 데이터 파일에 반영했는지를 나타낸다. 갱신 주기가 길면 장애 시 다시 적용할 로그가 늘어 인스턴스 복구 시간도 길어진다. SQL Server의 트랜잭션 로그는 내부적으로 가상 로그 파일로 나뉘므로 과도한 자동 증가로 조각화되지 않도록 원문은 충분한 초기 크기와 증가 단위 설정을 설명한다.

공유 메모리와 전용 메모리. Oracle의 SGA와 SQL Server의 Memory Pool은 여러 프로세스가 공유하는 메모리다. 동시 접근을 조정하려고 래치, 버퍼 Lock, 라이브러리 캐시 Lock/Pin 등의 직렬화 기법을 사용한다. Oracle의 PGA는 서버 프로세스에 종속된 전용 영역으로 정렬 및 세션·커서 상태를 저장하며, 공유 접근을 위한 래치가 필요하지 않다. 원문은 SQL Server에서 정렬과 세션 상태가 Memory Pool 내부 영역을 사용한다고 구분한다.

DB 버퍼 캐시. 일반적인 블록 읽기는 먼저 캐시에서 찾고 없으면 디스크에서 캐시에 적재한 뒤 읽는다. 일부 Direct Path Read는 예외다. 변경도 캐시에서 이루어지고 Dirty 블록은 DBWR가 기록한다. Free 버퍼는 비었거나 데이터 파일과 동기화돼 재사용 가능한 버퍼이고, Dirty 버퍼는 변경이 아직 파일에 반영되지 않은 버퍼이며, Pinned 버퍼는 현재 읽기·쓰기가 진행 중인 버퍼다. 새 블록을 올리려면 Free 버퍼가 필요하고 Dirty 버퍼는 먼저 기록해야 재사용할 수 있다. LRU는 사용빈도가 낮은 블록부터 밀어내 한정된 캐시를 관리한다.

공유 풀과 재사용. 공유 풀의 딕셔너리 캐시는 오브젝트·사용자·제약조건 등의 메타 정보를 저장한다. 라이브러리 캐시는 SQL 문, 실행계획, 저장 프로시저를 저장한다. 문법·의미·권한 확인, 최적화, 실행 가능한 형태 생성까지 반복하는 하드 파싱은 비용이 크다. 같은 SQL과 실행계획의 재사용성을 높이면 하드 파싱을 줄이고 DBMS 부하를 낮출 수 있다. SQL Server에서는 프로시저 캐시가 대응 역할을 한다.

로그 버퍼와 WAL. 서버 프로세스는 버퍼 블록을 바꾸기 전에 변경 내용을 로그 버퍼에 기록하고 LGWR는 이를 로그 파일에 기록한다. Dirty 블록을 데이터 파일에 쓰기 전에 관련 로그를 먼저 파일에 기록해야 한다. 이것이 Write Ahead Logging이다. 로그 버퍼도 메모리이므로 장애로 유실될 수 있어 늦어도 커밋 시점에는 로그 파일 기록을 보장한다. 이를 Log Force at Commit이라고 하며, 데이터 블록을 즉시 기록하지 않아도 Fast Commit을 안전하게 완료할 수 있는 이유다.

PGA의 세부 영역. UGA는 세션별 상태를 담는다. 전용 서버에서는 PGA에, 공유 서버에서는 SGA에 할당하며 Large Pool이 있으면 이를 사용하고 없으면 Shared Pool을 사용한다. CGA는 Parse·Execute·Fetch 각 호출이 진행되는 동안만 필요한 정보를 담고 호출 종료 시 반환한다. 여러 호출에 걸쳐 필요한 정보는 UGA에 남는다. Sort Area는 정렬 과정에서 청크 단위로 늘어난다. 원문은 DML의 정렬 및 SELECT 중간 단계는 CGA, SELECT의 최종 출력 단계 정렬은 UGA에 할당하는 차이와 자동 작업 영역 크기 관리도 설명한다.

제2절 SQL 처리 과정

원문 PDF 479-486쪽

선언에서 절차로. SQL은 구조적·집합적·선언적 질의 언어다. 사용자는 원하는 결과 집합을 정의하고, 그 결과를 만드는 실제 절차는 DBMS 내부에서 만들어진다. SQL 옵티마이저가 이 절차를 대신 구성한다. 따라서 같은 결과를 내는 SQL도 내부 접근 경로에 따라 수행량과 성능이 달라진다.

단계·엔진	역할
Parser	구성요소를 분석해 파싱 트리를 만들고 문법·의미 오류 확인
Query Transformer	파싱된 SQL을 일반적이고 표준적인 형태로 변환
Estimator	통계정보를 이용해 선택도·카디널리티·비용 계산
Plan Generator	실행계획 후보 생성
Row-Source Generator	선택한 계획을 SQL 엔진이 실행할 수 있는 코드·프로시저 형태로 구성
SQL Engine	실제 SQL 실행

최적화와 비용. 옵티마이저는 실행계획 후보를 찾고, 오브젝트·시스템 통계정보로 각 후보의 예상비용을 계산한 뒤 최저비용 계획을 선택한다. 실행계획은 이 처리 절차를 트리로 표현한 것이다. 비용은 예상 I/O 횟수나 예상 소요시간을 표현한 값으로 실측값이 아니다. 원문의 두 인덱스와 Full Scan 비교도 비용 차이로 선택 근거를 설명한다. 통계정보 및 계산의 한계 때문에 낮은 예상비용이 실제 최저 수행시간을 보장하지는 않는다.

힌트의 역할과 주의점. 통계정보가 부정확하거나 데이터·프로그램 특성을 충분히 반영하지 못하면 개발자가 힌트로 인덱스·조인 순서·조인 방식 등을 유도할 수 있다. Oracle의 힌트는 주석 안의 /*+ ... */ 형태로 기술한다. 문법·의미 오류, 잘못된 별칭·인덱스 참조, 논리적으로 불가능한 접근 경로, 버그 등으로 무시될 수 있으며, 그 밖에는 우선적으로 따른다. 잘못 작성해도 SQL 오류가 나지 않을 수 있으므로 실제 계획이 의도와 같은지 확인해야 한다.

힌트 분류. 원문은 ALL_ROWS·FIRST_ROWS의 최적화 목표, FULL·INDEX·INDEX_FFS·INDEX_SS 등의 접근 경로, UNNEST·USE_CONCAT 등의 쿼리 변환, ORDERED·LEADING의 조인 순서, USE_NL·USE_MERGE·USE_HASH의 조인 방식, PARALLEL 계열의 병렬 처리, PUSH_PRED·PUSH_SUBQ·QB_NAME 등의 기타 제어를 구분한다. SQL Server는 테이블명 뒤 WITH로 주는 테이블 힌트, FROM의 조인 힌트, 문장 끝 OPTION의 쿼리 힌트로 구분한다. 원문은 잘못된 힌트나 참조에 대한 SQL Server의 오류 처리와 Oracle의 무시 동작을 대비한다.

제3절 데이터베이스 I/O 메커니즘

원문 PDF 487-499쪽

블록 단위 I/O와 캐시. 데이터 파일에서 캐시로 적재할 때, 캐시 블록을 읽고 바꿀 때, Dirty 블록을 파일에 쓸 때 모두 블록 단위로 처리한다. 메모리 읽기는 디스크 읽기보다 빠르지만 메모리는 유한하다. 따라서 물리적 읽기를 줄이고 캐시 효율을 높이되, 가장 먼저 동일 결과에 필요한 논리적 블록 요청 수를 줄여야 한다. 디스크 I/O가 필요하면 서버 프로세스는 I/O를 요청하고 완료를 기다리므로 경합이 심해질수록 대기도 길어진다.

버퍼 캐시 히트율의 계산과 한계. BCHR은 전체 논리적 읽기 중 디스크 읽기 없이 캐시에서 바로 찾은 비율이다. SQL 트레이스에서는 논리적 읽기 = Query + Current, BCHR = (Query + Current - Disk) / (Query + Current) × 100으로 계산한다. 원문의 822블록 중 Disk 18블록은 97.8%다. Disk는 이미 논리적 읽기에 포함되므로 822에 18을 더하지 않는다. 작은 테이블을 반복 조회하면 캐시 히트율은 높아도 총 읽기와 래치·버퍼 Lock 경합이 커질 수 있다. 히트율만으로 효율을 판단하지 않는 이유다.

시퀀셜과 랜덤 접근. 시퀀셜 접근은 레코드 간 논리적 또는 물리적 순서를 따라 읽는다. 인덱스 리프의 연결을 따라 읽기와 테이블의 물리적 순서대로 읽기가 해당한다. 랜덤 접근은 순서를 따르지 않고 특정 레코드를 찾으려고 한 블록씩 접근한다. I/O 튜닝은 읽은 레코드 중 실제 선택되는 비중을 높이고 랜덤 접근 횟수를 줄이는 방향으로 수행한다. 테이블뿐 아니라 인덱스를 넓게 읽고 대부분 버리면 비효율적이다. 원문은 인덱스 칼럼 순서를 바꿔 같은 결과를 얻는 데 필요한 인덱스 읽기를 줄이고, 조건 칼럼 추가로 테이블 랜덤 접근을 줄이는 사례를 제시한다.

Single Block과 MultiBlock. Single Block I/O는 한 요청에서 한 블록을, MultiBlock I/O는 인접한 여러 블록을 함께 적재한다. MultiBlock은 I/O 호출 수를 줄여 대량 스캔에 유리하며 원문에서는 익스텐트 범위를 넘지 않는다고 설명한다. 인덱스의 논리적 순서는 물리적 위치와 다르므로 일반적인 인덱스·테이블 랜덤 접근은 Single Block 방식이 맞는다. 다만 원문은 Oracle 10g 이후 테이블 접근 없이 인덱스만 읽는 경우 Range Scan·Full Scan에도 MultiBlock이 적용될 수 있다고 조건을 명시한다. 원문의 9i 실험에서는 64블록을 읽을 때 Range Scan은 64회, Fast Full Scan은 9회 I/O 요청을 보인다.

네트워크와 캐시의 영향. NAS·SAN을 통해 대량 데이터를 읽거나 RAC 인스턴스 사이에 블록을 공유하면 네트워크도 성능에 영향을 준다. 같은 디스크 읽기 수라도 디스크 경합, 파일시스템 버퍼 캐시, SAN 캐시 상태에 따라 대기시간이 달라진다. 원문은 데이터베이스 자체 캐시가 있으므로 파일시스템 버퍼 캐시는 최소화하는 방향을 설명한다. 이런 환경 차이가 있어도 논리적 블록 요청을 줄이는 것이 가장 근본적인 해결 원리라는 결론은 같다.

I/O 효율화의 세 축. 첫째, SQL이 필요한 최소 블록만 읽도록 논리적 집합을 구성한다. 원문은 카드 거래 실적을 기간별로 여러 번 읽고 조인하는 대신 1년치 자료를 한 번 읽어 조건부 집계하는 사례를 든다. 둘째, 전략적 인덱스 구성, 파티션·클러스터·윈도우 함수 등 기능, 최적화 목표, 정확한 통계정보를 제공한다. 셋째, 그래도 적절한 계획이 나오지 않으면 힌트를 사용하고 의도한 경로를 검증한다. 하드웨어 개선보다 SQL의 I/O 발생량 자체를 줄이는 것이 근본적인 개선이다.

반드시 기억할 핵심 포인트

- 블록은 I/O 단위, 익스텐트는 할당 단위, 세그먼트는 저장공간이 필요한 오브젝트다.
- WAL과 커밋 시 로그 기록 보장이 Fast Commit과 복구의 근거다.
- 공유 풀의 실행계획 재사용은 하드 파싱을 줄인다.
- 비용과 히트율은 각각 예상 및 비율 지표이므로 실제 일량과 함께 해석한다.
- 시퀀셜 접근의 선택 비중을 높이고 테이블 랜덤 접근을 줄인다.
- 힌트를 작성한 뒤에는 실제 선택한 실행계획을 확인한다.

챕터 최종 요약

- SQL은 결과 집합을 선언하며 옵티마이저가 이를 실행 절차로 만든다. 파싱·변환·비용 산정·계획 선택·로우소스 생성 뒤 SQL 엔진이 수행한다.
- 저장공간은 블록·익스텐트·세그먼트·테이블스페이스로 연결되고, 일반 읽기와 변경은 버퍼 캐시를 거친다. 로그 선행 기록과 체크포인트가 장애 복구를 뒷받침한다.
- 성능의 중심은 필요한 결과를 얻기 위해 읽는 블록 수다. 캐시 히트율, 물리적 읽기, I/O 호출 수를 구분하고 논리적 읽기 자체를 최소화한다.

제2장 SQL 분석 도구

과목 III. SQL 고급 활용 및 튜닝

원문 PDF 504-529쪽

제1절 예상 실행계획

원문 PDF 505-511쪽

실행계획을 확인하는 목적. 실행계획은 DBMS가 SQL을 수행하기 위해 수립한 처리 절차다. 실행 전에 계획을 읽으면 불필요한 전체 스캔, 조인 순서, 잘못 예상한 결과 크기 등 성능 문제를 사전에 발견할 수 있다. 다만 예상 계획은 실제 수행 과정 전체를 보여주는 정보가 아니므로 계획만으로 원인을 설명할 수 없을 때는 실행 통계와 트레이스로 넘어간다.

Oracle EXPLAIN PLAN. EXPLAIN PLAN FOR 뒤에 분석할 SQL을 두면 해당 계획을 PLAN_TABLE에 저장한다. 원문은 필요한 테이블 생성 스크립트와 함께 Oracle 10g부터 기본 PLAN_TABLE 및 public synonym이 제공됨을 설명한다. SET STATEMENT_ID는 계획을 구분하는 선택 항목이다. 저장 후 utlxpls.sql 또는 병렬 항목을 보여주는 utlxplp.sql로 출력할 수 있으며, 이 스크립트는 내부적으로 DBMS_XPLAN을 사용한다. 실행계획을 생성한 것이 실제 결과 집합을 실행한 것은 아니다.

AutoTrace 설정	SQL 수행	표시 내용
ON	수행	결과·예상 계획·통계
ON EXPLAIN	수행	결과·예상 계획
ON STATISTICS	수행	결과·통계
TRACEONLY	수행	예상 계획·통계
TRACEONLY EXPLAIN	수행하지 않음	예상 계획
TRACEONLY STATISTICS	수행	통계

AutoTrace 사용 조건. 실행계획만 볼 때는 PLAN_TABLE이 있으면 되지만 실행통계까지 보려면 V_$SESSTAT, V_$STATNAME, V_$MYSTAT 등의 조회 권한이 필요하다. 원문은 일반 사용자에게 필요한 권한을 PLUSTRACE 롤로 묶어 부여하는 절차를 제시한다. TRACEONLY라는 이름이 붙었다고 모두 실행하지 않는 것은 아니다. 통계를 측정하거나 결과를 출력하는 설정은 실제 수행이 필요하고, TRACEONLY EXPLAIN만 계획만 생성한다.

DBMS_XPLAN.DISPLAY. 첫 번째 인자는 계획이 저장된 테이블, 두 번째는 STATEMENT_ID, 세 번째는 출력 형식이다. 두 번째 인자가 NULL이면 마지막 EXPLAIN PLAN의 계획을 출력한다. BASIC은 ID·Operation·Name 중심이고 ROWS·BYTES·COST 등을 추가해 예상 건수·데이터 크기·비용을 볼 수 있다. TYPICAL·SERIAL·ALL 등 형식과 PARTITION·PARALLEL·PREDICATE·PROJECTION·REMOTE·NOTE 등의 항목을 선택한다. 즉 실행계획의 수집과 출력 형식 제어를 구분해서 이해한다.

SQL Server 예상 계획. SET SHOWPLAN_TEXT ON 후 SQL을 입력하면 실행하지 않고 텍스트 예상 계획을 출력한다. SHOWPLAN_ALL ON은 PhysicalOp, LogicalOp, EstimateRows 등 더 자세한 항목을 제공한다. 이후 실제 통계를 측정하려면 이 설정을 OFF로 바꿔야 한다. 실행하지 않고 보여주는 계획과 실제 행 수를 포함하는 실행 프로필은 분석 목적이 다르다.

제2절 SQL 트레이스

원문 PDF 512-518쪽

Oracle 파일 기반 트레이스. 현재 세션에 SQL_TRACE를 TRUE로 설정하고 분석할 SQL을 수행한 뒤 FALSE로 해제하면 서버의 USER_DUMP_DEST가 가리키는 디렉터리에 .trc 파일이 생긴다. 원문은 프로세스·세션·파라미터·인스턴스 뷰를 연결해 파일 경로를 찾는 쿼리도 제시한다. 원시 트레이스에는 PARSE·EXEC·FETCH 등이 기록되지만 직접 읽기 어려우므로 TKPROF로 읽기 쉬운 보고서를 만든다. TKPROF의 SYS=NO 옵션은 파싱 과정에서 내부적으로 수행한 SQL을 제외한다.

Call 통계	의미
Parse / Execute / Fetch	파싱·실행·결과 인출 단계별 통계
count	각 단계 호출 횟수
cpu / elapsed	CPU 사용시간 / 해당 단계 소요시간
disk	디스크에서 읽은 블록 수
query / current	Consistent 모드 / Current 모드 블록 읽기 수
rows	각 단계가 읽거나 갱신한 처리 건수

통계 항목 사이의 관계. AutoTrace의 DB BLOCK GETS는 트레이스 Current에, CONSISTENT GETS는 Query에, PHYSICAL READS는 Disk에 대응한다. 결과 인출의 왕복과 행 수는 Fetch 호출 수·Fetch 행 수와 연결된다. Call 통계는 SQL 전체의 단계별 일량을 보여주고, Row Source Operation은 실행계획 내부의 어느 단계에서 많은 작업이 일어났는지를 보여준다. 둘을 함께 봐야 문제의 위치와 양을 구분할 수 있다.

Row Source 해석. 왼쪽 Rows는 각 단계가 출력한 행 수다. 오른쪽 cr은 Consistent 블록 읽기, pr은 디스크 읽기, pw는 디스크 쓰기, time은 소요시간이며 마이크로초로 표시된다. 원문은 9i에서는 pr·pw 대신 r·w를 사용했다고 구분한다. 부모 단계 수치는 자식의 수행량을 포함한 누적값이므로 모두 더하면 중복된다. 예를 들어 테이블 접근 cr=2, 자식 인덱스 접근 cr=1이면 테이블 단계 자체의 순수 읽기는 1이다.

SGA의 실행통계와 DISPLAY_CURSOR. 서버의 트레이스 파일에 접근하기 어려운 경우, STATISTICS_LEVEL을 ALL로 하거나 해당 SQL에 GATHER_PLAN_STATISTICS 힌트를 사용해 단계별 수행통계를 SGA에 수집한다. DBMS_XPLAN.DISPLAY_CURSOR는 SQL_ID·자식 커서 번호·출력 형식으로 그 정보를 출력한다. IOSTATS, MEMSTATS, ALLSTATS로 I/O·메모리·전체 정보를 선택하고, 인자를 NULL로 주면 직전에 수행한 커서를 확인한다. 기본적으로 누적값이며 ALLSTATS LAST처럼 LAST를 더하면 마지막 수행의 일량을 본다.

DISPLAY_CURSOR 항목	뜻·해석
Starts	해당 실행 단계가 호출된 횟수
E-Rows	옵티마이저가 예상한 행 수
A-Rows	실제로 처리·출력한 행 수
A-Time	실제 소요시간
Buffers	캐시에서 읽은 버퍼 블록 수
Reads	디스크에서 읽은 블록 수

예상과 실제를 연결해 읽기. E-Rows는 예상 실행계획의 Rows 또는 Card와 대응하고 A-Rows는 실제 결과를 보여준다. 원문의 NL 조인 예에서 선행 EMP 집합이 12행을 내면 후행 PK_DEPT 인덱스 단계의 Starts도 12가 된다. 따라서 단순한 최종 행 수만 보지 않고 선행 단계 출력이 후행 단계 호출 횟수를 어떻게 늘리는지 살핀다. A-Time·Buffers 등도 누적 특성을 고려해 해석해야 한다.

SQL Server 실제 수행 분석. STATISTICS PROFILE, STATISTICS IO, STATISTICS TIME을 ON으로 하고 SQL을 수행한다. PROFILE은 결과 뒤에 연산자별 실제 행 수와 실행 횟수를 포함한 실행 프로필을 출력한다. IO는 페이지 읽기 정보를, TIME은 구문 분석·컴파일·실행에 걸린 CPU 및 경과시간을 밀리초 단위로 보여준다. SHOWPLAN_TEXT·SHOWPLAN_ALL이 켜져 있다면 먼저 꺼야 실제 실행된다.

SQL Server 읽기 지표. 검색 수는 해당 테이블 또는 인덱스를 읽은 접근 횟수이며, 논리적 읽기는 데이터 캐시에서 읽은 페이지 수, 물리적 읽기는 디스크에서 읽은 페이지 수다. 미리 읽기는 필요할 것으로 예상한 데이터·인덱스 페이지를 실제 사용 전에 캐시에 가져온 수다. 실행 프로필의 Rows는 연산자의 실제 출력 행 수, Executes는 실행 횟수다. 따라서 인덱스를 사용했다는 사실만으로 효율을 판단하지 않고 실제 페이지 읽기와 반복 실행을 확인한다.

제3절 응답 시간 분석

원문 PDF 519-526쪽

대기 이벤트란. DBMS 프로세스는 자원이나 다른 프로세스의 작업 완료가 필요하면 수행을 멈추고 대기한다. 대기 종류·횟수·시간을 기록한 것이 Oracle의 Wait Event, SQL Server의 Wait Type이다. 공유 메모리의 누적 정보와 필요 시 활성화하는 이벤트 트레이스를 통해 현재 병목과 원인을 분석한다. 대기 이름은 관찰한 현상을 나타내므로 이름과 누적 시간만으로 원인을 단정하지 말고 실제 작업량·환경을 함께 살핀다.

분류	원문 주요 이벤트·해석
라이브러리 캐시	latch: shared pool, latch: library cache → 커서 검색·최적화 경합; library cache lock/pin → 참조 오브젝트 DDL 관련
클라이언트·네트워크	SQL*Net message from client는 다음 요청을 기다리는 Idle 대기; to client·more data 계열은 전송·수신 지연과 연결
디스크 I/O	db file sequential read는 Single Block, db file scattered read는 MultiBlock 읽기; direct path read/write·temp 등도 구분
버퍼 캐시	cache buffers chains·cache buffers lru chain 래치, buffer busy waits, free buffer waits
Lock	enq: TM - contention, enq: TX 계열의 행·인덱스·ITL 경합
로그·체크포인트	log file sync, log file switch completion, log buffer space, checkpoint completed

Lock과 래치. 원문은 사용자 데이터를 보호하는 Lock과 SGA의 내부 자료구조를 보호하는 가벼운 래치를 구분한다. 래치는 큐로 먼저 요청한 순서를 보장하는 구조가 아니므로 획득에 실패하면 반복 시도하고 필요 시 대기한다. latch free는 자원을 여러 번 요청했으나 계속 사용 중이어서 대기한 경우다. 캐시에서만 읽어도 이런 경합이 커지면 동시성이 낮아질 수 있어 I/O 양과 반복 접근을 줄이는 튜닝이 필요하다.

응답 시간 분해. 원문은 Response Time = Service Time + Wait Time = CPU Time + Queue Time으로 정의한다. 서비스 시간은 정상적으로 일을 한 CPU 시간이며, 대기 시간은 수행을 멈춘 시간이다. CPU는 파싱에 쓴 시간인지 본연의 연산에 쓴 시간인지 나누고, 대기는 소비시간이 큰 이벤트부터 분석한다. 측정·분석·개선을 반복해 병목을 줄여 나가는 방법이다. CPU 사용이 있다는 사실과 유효한 업무를 효율적으로 수행한다는 사실은 별도로 검토한다.

AWR의 목적과 수집. AWR은 동적 성능 뷰의 정보를 주기적으로 저장하고 분석하여 시스템 상태, 병목 원인, 튜닝 대상을 찾는 Oracle 표준 도구다. 원문은 Statspack의 SQL 기반 딕셔너리 조회와 대비해 AWR의 공유 메모리 직접 접근 방식을 설명한다. DBA_HIST_로 시작하는 뷰 또는 제공 스크립트로 보고서를 출력한다. 보고서 분석에서 가장 중요한 것은 시작·종료 스냅샷으로 정하는 측정 구간이다.

측정 구간 선택. 일별 부하 패턴 비교에는 업무시간 전체를 사용할 수 있지만 성능 문제의 원인을 찾으려면 장애 또는 피크 시간 전후의 가능한 짧은 구간을 선택한다. 긴 구간 평균은 짧고 심한 장애를 희석할 수 있다. 사용자 경험 시간과 운영체제의 CPU·메모리·I/O 관찰자료를 연결해 실제 문제가 있던 시간을 찾은 뒤 분석한다. 원문은 보고서 첫 장을 먼저 읽고 필요한 상세 항목으로 들어가는 순서를 권한다.

AWR 요약의 네 부분. Load Profile은 초당 부하와 트랜잭션당 평균 부하를 보여준다. 트랜잭션 수는 Commit과 Rollback 횟수의 합이다. Instance Efficiency는 버퍼·라이브러리·래치·파싱 관련 비율을 보여주며, 비정상 수치는 당시 부하와 함께 해석한다. Shared Pool Statistics는 구간 시작·종료의 공유 풀 상태를 비교한다. Top 5 Timed Events는 Idle을 제외한 큰 대기를 보여주고 비교를 위해 대기 이벤트가 아닌 CPU Time도 함께 싣는다.

Top 5의 올바른 해석. 상위 이벤트는 상대적 순위이므로 정상 시스템에도 반드시 존재한다. CPU Time 비중이 높다는 사실은 서비스 수행이 대기보다 많다는 신호로 볼 수 있지만 CPU 사용률과 실제 SQL 일량을 더 확인한다. 래치 경합은 CPU 사용률과 함께 분석하고, 트랜잭션 중심 시스템의 log file sync가 상위라고 불필요한 커밋을 단정하지 않는다. 데이터베이스는 I/O 집약적이므로 sequential/scattered read가 상위여도 정상일 수 있다. 원문은 I/O 대기 비중과 CPU 사용률이 함께 높은 상태가 지속되는지 등 상세 근거로 튜닝 필요성을 판단한다.

반드시 기억할 핵심 포인트

- 예상 계획, SQL 전체 Call 통계, 실행 단계별 Row Source 통계는 서로 보완한다.
- TRACEONLY EXPLAIN은 실제 수행 없이 계획만 확인한다.
- 부모 실행 단계의 통계에는 자식 단계의 일량이 누적된다.
- E-Rows와 A-Rows, Starts를 연결해 반복 수행과 추정 오차를 파악한다.
- LAST 없는 커서 통계는 기본적으로 누적값이다.
- 응답시간을 CPU와 대기로 나누고 문제 시간대의 짧은 구간을 분석한다.

챕터 최종 요약

- 실행 전에는 EXPLAIN PLAN·SHOWPLAN으로 계획을 확인하고, 실행 후에는 트레이스와 단계별 실제 통계로 수행량을 검증한다.
- Query·Current·Disk, Starts·E-Rows·A-Rows, 부모·자식 누적값을 구분하면 어디서 읽기와 반복 호출이 늘어나는지 추적할 수 있다.
- 시스템 분석은 CPU 시간과 대기시간을 분리하고 AWR의 측정 구간·부하·효율·이벤트를 함께 해석한다. 순위와 비율만으로 원인을 단정하지 않는다.

제3장 인덱스 튜닝

과목 III. SQL 고급 활용 및 튜닝

원문 PDF 530-579쪽

제1절 인덱스 기본 원리

원문 PDF 531-557쪽

B*Tree의 구조. 루트·브랜치에는 하위 노드의 값 범위와 주소가 있고, 리프에는 정렬된 키와 테이블 레코드의 ROWID가 있다. 같은 키는 ROWID 순으로 정렬된다. 리프끼리는 양방향 연결돼 정방향·역방향 범위 스캔이 가능하다. 루트에서 리프까지 시작점을 찾는 수직 탐색과, 리프에서 조건 범위를 읽는 수평 탐색을 구분한다. 비유일 인덱스에서 한 행을 찾았더라도 같은 키가 더 있을 수 있으므로 다음 행·다음 리프까지 확인한다.

NULL 저장의 DBMS 차이. Oracle B*Tree는 구성 칼럼이 모두 NULL인 레코드를 저장하지 않고, 하나라도 NULL이 아니면 저장한다. SQL Server는 모든 키가 NULL이어도 저장한다. 원문은 정렬 시 Oracle의 NULL은 뒤, SQL Server의 NULL은 앞에 위치한다고 대비한다. 이 차이 때문에 Oracle에서 IS NULL 조건만으로 필요한 모든 행을 인덱스에서 찾을 수 있는지는 다른 구성 칼럼의 NOT NULL 조건 등을 함께 확인해야 한다.

스캔	방식·조건	핵심 주의점
Index Range Scan	수직 탐색 후 필요한 리프 범위 읽기; 선두 칼럼 조건 필요	범위가 넓거나 테이블 접근이 많으면 느리다
Index Full Scan	인덱스 키 순서로 리프 전체 읽기	처음 리프를 찾는 수직 탐색은 실제로 수행; 필터·정렬 대체 목적
Index Unique Scan	유일 인덱스의 전체 키를 등치로 탐색	한 키를 찾고 수평 범위 탐색 없이 종료
Index Skip Scan	조건에 맞을 가능성이 있는 하위 블록만 골라 접근	선두 값 종류가 적고 후행 값 종류가 많을 때 유용
Index Fast Full Scan	트리를 따르지 않고 세그먼트 전체를 MultiBlock 스캔	키 순서 미보장; 조회 칼럼이 인덱스에 모두 있어야 함
Range Scan Descending	범위를 뒤에서 앞으로 읽기	내림차순·최댓값 접근에 활용

스캔 방식 선택의 의미. Range Scan은 일반적이지만 그 이름만으로 성능을 보장하지 않는다. 선두 조건이 없을 때 Full Scan은 테이블보다 작은 인덱스를 먼저 읽어 대부분 걸러내는 차선책이 될 수 있다. 정렬을 생략해 첫 일부 결과를 빨리 주려는 목적의 선택일 수도 있다. FIRST_ROWS를 사용한 뒤 결과를 끝까지 읽으면 원래 목적과 달라져 더 큰 I/O를 일으킬 수 있다. SQL Server는 Oracle의 Range·Unique를 Index Seek로, Full Scan을 Index Scan으로 표현한다.

Full Scan과 Fast Full Scan. Full Scan은 인덱스 구조를 따라 정렬 순서를 유지하며 테이블 칼럼을 추가로 읽는 데도 사용할 수 있다. Fast Full Scan은 순서를 보장하지 않고 MultiBlock 및 병렬 스캔을 활용하지만 인덱스 안에 필요한 모든 칼럼이 있어야 한다. 정렬된 인덱스의 역방향 탐색은 ORDER BY DESC나 MAX를 위한 첫 행 접근에도 사용된다. 다만 최종 결과 순서가 필요하면 ORDER BY를 명시해야 한다는 배치 I/O의 조건도 함께 기억한다.

Skip Scan과 IN-List. Skip Scan은 선두 조건이 없거나 범위 조건인 경우에도 브랜치 정보를 활용해 필요한 하위 블록만 고른다. 선두 칼럼의 가능한 값을 IN-List로 직접 제공하면 각 값에 대해 수직 탐색하는 INLIST ITERATOR가 될 수 있지만, 이는 Skip Scan의 내부 원리와 같지 않다. IN-List에 쓴 값만 실제 전체 종류라고 보장돼야 결과가 달라지지 않으며 종류가 너무 많으면 반복 탐색 비용이 커진다.

인덱스 조각화. B*Tree의 B는 Balanced이므로 삭제 때문에 어떤 리프만 더 깊어지는 불균형은 발생하지 않는다. 다만 한쪽 리프가 비는 Skew와 전체 블록 밀도가 낮아지는 Sparse가 생겨 스캔 효율이 나빠질 수 있다. Oracle에서 완전히 빈 리프는 재사용 가능하지만 트리 엔트리가 곧바로 모두 제거되는 것은 아니며, 원문은 SQL Server의 Ghost 레코드 비동기 정리와 대비한다. 부분적으로 빈 블록은 그 키 범위에 새 값이 들어오지 않으면 오래 남을 수 있다.

재생성의 판단. 조각화로 크기·스캔 비용이 늘면 재생성이나 빈 공간 제거가 도움이 될 수 있지만, 여유 공간을 없애면 이후 분할과 DML 부하가 증가할 수 있다. 재생성 자체의 시간·부하도 크므로 단순한 주기 작업으로 보지 않는다. 원문은 분할 경합이 매우 높을 때, 자주 읽는 인덱스의 스캔 효율·높이를 개선할 때, 대량 삭제 후 재입력까지 오래 걸릴 때, 행 수가 일정한데 크기만 계속 늘 때처럼 기대 효과가 분명한 경우를 제시한다.

비트맵 인덱스. 각 값에 해당하는 행을 비트로 표현하며 NULL과 부정형 조건도 처리할 수 있다. 값 종류가 적으면 저장 효율이 좋지만 단일 인덱스로 많은 행을 찾으면 테이블 랜덤 접근은 여전히 많다. 여러 비트맵의 Bitwise 연산으로 먼저 교집합·합집합을 좁혀 테이블 방문을 줄일 때 장점이 커진다. 다양한 임의 질의가 있는 읽기 중심 DW·OLAP에 적합하며, 한 행 변경도 비트맵 범위의 Lock에 영향을 주므로 DML이 많은 OLTP에 부적합하다는 것이 원문의 핵심 조건이다.

함수기반·리버스 키. 함수기반 인덱스는 원래 칼럼이 아니라 NVL·UPPER 등 함수를 적용한 값을 저장한다. 쿼리 조건과 같은 표현식이 있어야 가공 조건에도 인덱스를 활용할 수 있지만 입력·수정 때 함수 수행비용이 추가되므로 남용하지 않는다. 리버스 키는 순차 증가 키를 거꾸로 저장해 오른쪽 끝 리프에 집중되는 동시 INSERT 경합을 분산한다. 원래 순서가 사라지므로 등치 검색에 쓰고 부등호·LIKE 범위검색에는 사용할 수 없다. 원문은 reverse 함수 인덱스에 빗대어 원리를 설명한다.

Oracle 클러스터 인덱스. 같은 클러스터 키의 레코드를 같은 블록에 모으고 넘치면 체인으로 연결한다. 여러 테이블을 함께 저장하는 다중 테이블 클러스터도 가능하다. 클러스터 인덱스의 한 키는 같은 키를 담는 첫 데이터 블록을 가리키며 테이블 행과 1:M 관계다. 따라서 키마다 한 번 클러스터에 도달한 뒤 연속 스캔할 수 있어 넓은 범위에 유리하다. 새 키 입력·수정이 빈번하면 클러스터 할당·이동 부담이 있으므로 키 선택에 주의한다.

클러스터형 인덱스와 IOT. SQL Server 클러스터형 인덱스는 리프 자체가 전체 행을 담는 데이터 페이지다. 정렬된 테이블 저장 구조이므로 테이블당 하나만 가능하며, 넓은 범위 검색·작은 반복 조회 테이블·적은 칼럼과 많은 행·입력 순서와 조회 순서가 다른 테이블에 유리하다. 정렬 유지와 분할 때문에 입력 부하가 커질 수 있다. Oracle에서는 IOT가 이에 가깝고 PK 기준이라는 제약이 있다. SQL Server 클러스터형 인덱스는 중복 키도 허용하며 내부 uniquifier로 구분한다. Oracle 클러스터 인덱스와 이름만 보고 같은 것으로 보면 안 된다.

2차 인덱스의 참조. SQL Server 비클러스터형 인덱스가 클러스터형 인덱스의 키를 보유하면 행 이동 때 모든 물리 주소를 갱신하지 않아도 되지만, 조회마다 클러스터형 인덱스를 다시 수직 탐색하는 비용이 든다. Oracle IOT의 Secondary 인덱스는 Logical ROWID = PK + Physical Guess를 사용한다. Guess는 최초 생성·재생성 시의 블록 주소로 이동 뒤 부정확할 수 있다. Guess 위치에 행이 없으면 PK로 다시 탐색하여 주소 갱신 부하와 탐색 비용을 절충한다.

범위 스캔을 막는 조건. 선두 칼럼에 함수를 적용하거나 부정형 비교를 쓰면 일반적인 Range Scan의 시작점을 정하기 어렵다. 그렇다고 인덱스 Full Scan까지 항상 불가능한 것은 아니다. Oracle의 IS NULL은 모든 NULL 키를 저장하지 않는 특성을 확인해야 하고, 다른 구성 칼럼이 NULL이 아니거나 NOT NULL이면 Range Scan이 가능한 경우가 있다. SQL Server는 NULL도 저장한다. 가공 조건과 DBMS의 저장 특성을 나누어 판단한다.

가공을 비교값 쪽으로 옮기기. 원문은 SUBSTR(업체명) 비교를 접두 LIKE로, 월급여 곱셈을 상수 나눗셈으로, 날짜 TO_CHAR 비교를 하루 시작 이상·다음 날 미만 범위로 바꾸는 방식을 보여준다. 연결한 두 칼럼은 개별 조건으로 나누고, 문자열을 나눌 필요가 있으면 칼럼보다 바인드값을 가공한다. 묵시적 형변환도 칼럼 가공을

일으킬 수 있다. 숫자형과 문자형 비교에서 문자 칼럼이 숫자로 변환되면 인덱스가 비효율적일 뿐 아니라 변환 오류·잘못된 결과 위험도 있으므로 비교값의 타입을 칼럼에 맞춘다.

제2절 테이블 액세스 최소화

원문 PDF 558-566쪽

ROWID는 메모리 포인터가 아니다. ROWID에는 데이터 파일·블록·행 위치 등이 담기지만 이것으로 곧바로 캐시 행을 읽는 구조는 아니다. 블록 주소를 해시해 버킷·체인을 찾고 버퍼 헤더의 포인터로 캐시 블록에 접근한다. 없으면 Free 버퍼를 확보하고 필요 시 Dirty 블록을 기록한 뒤 디스크 블록을 적재한다. 그 과정의 래치·버퍼 Lock 및 선행 작업 대기까지 고려하면 인덱스에서 행마다 테이블을 찾는 비용이 크다. Oracle의 TABLE ACCESS BY INDEX ROWID, SQL Server의 RID 또는 Bookmark Lookup이 이 접근이다.

클러스터링 팩터와 손익분기점. 인덱스 순서대로 읽을 때 테이블의 같은 키 자료가 물리적으로 모여 있으면 적은 블록을 방문한다. 흩어져 있으면 같은 행 수를 읽어도 블록 접근량이 커진다. 인덱스 접근이 Table Full Scan보다 느려지는 선택 비율을 손익분기점이라고 한다. 원문은 보통 낮은 비율에서 결정되지만 군집성·캐시 상태에 따라 크게 달라짐을 설명한다. 인덱스는 랜덤·Single Block 접근, 전체 스캔은 시퀀셜·MultiBlock 접근이라는 차이가 핵심 원인이다. 고정된 비율 하나를 모든 테이블에 적용하지 않는다.

테이블 방문을 줄이는 인덱스 칼럼 추가. 테이블에 도달한 후 대부분 버리는 조건이 있다면 그 조건 칼럼을 기존 인덱스에 추가해 인덱스에서 먼저 걸러낼 수 있다. 이때 인덱스 자체 스캔량은 그대로여도 테이블 랜덤 접근은 줄어든다. 기존 인덱스 칼럼 순서를 함부로 바꾸면 그 인덱스를 이용하던 다른 SQL이 손해 볼 수 있으므로 새 인덱스 추가와 기존 확장의 영향을 함께 본다. 필요한 모든 칼럼을 인덱스에 담으면 테이블 방문 자체가 없는 Covered 쿼리도 가능하지만 인덱스 크기·DML 비용이 증가한다.

INCLUDE와 저장 구조 활용. SQL Server INCLUDE 칼럼은 리프에만 저장돼 수직 탐색 키가 되지는 못하지만 리프의 필터나 출력 자료로 사용할 수 있다. 일반 복합 키에 넣으면 루트·브랜치에도 포함되는 차이가 있다. IOT·클러스터형 인덱스는 테이블 랜덤 접근을 없애고, Oracle 클러스터는 같은 키의 접근을 모은다. 해시 클러스터는 해시값이 같은 자료를 모아 별도 인덱스 없이 위치를 찾지만 등치 검색만 가능하다. 넓은 범위의 자주 쓰는 조건으로 파티션하면 전체 테이블 대신 일부 파티션만 스캔할 수 있다. 적절한 인덱스와 부분범위처리도 손익분기점 한계를 줄이는 방법이다.

수동 재정렬. 특정 인덱스 순서로 테이블을 재생성하면 해당 인덱스의 클러스터링 팩터가 좋아질 수 있다. 그러나 다른 인덱스의 군집성은 나빠질 수 있고 주기적인 재구성에는 관리 비용·가용성 부담이 있다. 자주 사용하는 핵심 인덱스를 기준으로 실제 효과와 다른 중요 SQL의 손해를 확인한 경우에만 선택한다. 여러 인덱스 키가 강하게 연관된 예외를 제외하면 모든 인덱스의 군집성을 동시에 좋게 만들 수는 없다.

배치 I/O와 출력 순서. 배치 I/O는 캐시에 없는 테이블 블록을 만날 때마다 즉시 디스크 요청하지 않고 여러 요청을 모아 처리한다. 원문은 11g에서는 NL 조인 Inner 접근, 12c부터는 일반적인 인덱스 ROWID 접근에도 작동할 수 있다고 설명한다. 실행계획에 TABLE ACCESS BY INDEX ROWID BATCHED가 나타나며 실제 I/O 순서에 따라 결과 순서가 인덱스 키 순서와 달라질 수 있다. 따라서 인덱스 정렬만 믿고 ORDER BY를 생략한 ROWNUM 상위 조회 패턴은 수정해야 한다. 원문은 기능 자체를 끄는 것보다 필요한 ORDER BY를 추가하는 방식을 권한다.

제3절 인덱스 스캔 효율화

원문 PDF 567-573쪽

선행 범위 조건의 비효율. 복합 인덱스는 앞 칼럼부터 정렬된다. 선행 칼럼이 모두 등치 조건이면 후행 범위에 맞는 레코드가 한곳에 모여 필요한 구간만 읽고 멈출 수 있다. 반면 앞 칼럼이 누락되거나 부등호·BETWEEN·LIKE 범위 조건이면 뒤 등치 조건을 만족하는 행이 앞 값별로 흩어져 넓게 읽고 버려야 한다. 원문의 아파트 매물 사례는 같은 조건이라도 인터넷매물 범위 칼럼을 앞으로 옮기면 스캔 구간이 넓어지는 과정을 보여준다.

범위를 IN-List로 바꾸기. 범위 안의 가능한 값이 적고 모두 정확히 열거할 수 있으면 IN-List로 바꿔 값마다 등치 탐색할 수 있다. 원문에서는 인터넷매물 '1'·'2'를 각각 탐색하여 넓은 중간 범위를 읽는 비용을 줄인다. 다만 수직 탐색도 값의 개수만큼 늘기 때문에 IN-List가 많거나 인덱스 높이가 크면 오히려 손해다. 필요 없는 리프 스캔량과 반복 브랜치 탐색량의 차이를 비교한다. 원문은 이런 상황에서 Oracle Skip Scan도 대안이 될 수 있다고 설명한다.

둘 이상의 범위 조건과 선택적 입력. 두 범위 조건을 쓰면 첫 범위가 스캔량을 거의 결정하고 뒤 조건은 주로 필터 역할을 하므로 대량 자료에서 차이가 커진다. 검색값이 없을 때도 사용하려고 지역을 LIKE로 표현한 경우, 입력 유무별 SQL을 나누고 값이 있으면 지역 등치 조건을 쓰는 방법을 제시한다. UNION ALL의 상·하단을 바인드 NULL 여부로 나눌 수도 있다. 지역이 없는 분기까지 최적화하려면 별도 인덱스나 칼럼 순서 변경을 검토하되, 다른 분기의 효율과 상품명 조건의 선택도를 함께 고려한다.

제4절 인덱스 설계

원문 PDF 574-575쪽

기본 구성 순서. 먼저 조건절에 항상 또는 자주 쓰는 칼럼을 고른다. 그중 등치 조건으로 자주 조회되는 칼럼을 앞에 둔다. 다음으로 정렬을 생략할 수 있도록 ORDER BY·GROUP BY 칼럼을 검토한다. 정렬을 인덱스로 대체하려면 앞부분 칼럼을 빠뜨리지 않고 같은 순서를 사용해야 한다. 단 등치로 고정된 칼럼은 ORDER BY에서 빠지거나 다른 순서에 있어도 되는 예외가 있다. 뒤쪽 칼럼이 빠지는 것은 가능하다.

선택도와 전체 시스템 균형. 인덱스 생성 여부에는 결합 선택도가 중요하지만 칼럼 순서는 개별 선택도만으로 결정하지 않는다. 실제 조건 형태·빈도·효용이 우선 판단자료다. 쿼리 수행빈도, 업무 중요도, 데이터량, 클러스터링 팩터, DML 빈도와 갱신 칼럼, 저장공간, 관리 비용을 종합한다. 개별 SQL만 빠르게 하려고 인덱스를 계속 추가하면 전체 DML 부하가 커진다. 원문은 액세스 경로·조건·정렬·값 종류와 사용량을 함께 볼 수 있는 설계도를 작성해 중복을 줄이고 시스템 전체 대안을 선택할 것을 강조한다.

반드시 기억할 핵심 포인트

- 인덱스 튜닝은 인덱스 스캔 범위 최소화와 테이블 랜덤 접근 최소화의 두 축이다.
- Range Scan에도 넓은 스캔·잦은 테이블 방문 비효율이 있다.
- 선두 등치 조건 뒤 범위 조건을 배치하면 필요한 구간을 좁히기 쉽다.
- B*Tree는 높이가 균형 잡혀도 Skew·Sparse로 읽기 효율이 나빠질 수 있다.
- INCLUDE는 리프 전용 자료이고, Covered 쿼리는 테이블 접근 없이 처리된다.
- 배치 I/O 때문에 인덱스 순서만 믿고 ORDER BY를 생략하지 않는다.
- 조회 개선과 DML·공간·관리 비용을 함께 평가한다.

챕터 최종 요약

- B*Tree는 수직 탐색으로 시작점을 찾고 정렬된 리프를 수평 스캔한다. 스캔 방식은 선두 조건·유일성·필요 칼럼·정렬 요구에 따라 달라진다.
- 인덱스에서 적게 읽는 것과 테이블을 적게 방문하는 것은 다른 목표다. 칼럼 순서는 스캔 범위에, 필터 칼럼 추가·Covered 구조·INCLUDE·군집 저장은 테이블 접근에 영향을 준다.
- 선행 범위 조건은 후행 조건의 집약성을 떨어뜨릴 수 있다. IN-List는 반복 탐색 비용과 비교하며, 인덱스 설계는 실제 쿼리 패턴과 전체 시스템 비용으로 결정한다.

제4장 조인 튜닝

과목 III. SQL 고급 활용 및 튜닝

원문 PDF 580-617쪽

제1절 NL 조인

원문 PDF 581-588쪽

중첩 루프 구조. NL 조인은 Outer 집합의 한 행을 읽고 그 행의 조인 키로 Inner 집합을 찾아 처리하는 과정을 반복한다. 원문은 중첩 반복문과 PL/SQL로 이를 설명하지만, SQL 엔진이 조인할 때 내부적으로 매 행 별도 SQL을 재귀 수행하는 것은 아니라고 구분한다. Outer의 전체 결과를 모두 만든 뒤 한꺼번에 조인하는 방식이 아니라 한 행씩 다음 단계로 이어가는 구조다. 이 때문에 적은 결과를 빨리 가져오는 부분범위처리에 유리하다.

조건 비교와 실행 경로. 원문의 부서·사원 예시는 부서 위치 인덱스 → 부서 테이블의 gb 필터 → 사원 부서번호 인덱스 → 사원 테이블의 급여 필터 순으로 진행하고 마지막에 급여 역순 정렬한다. 단 각 스캔을 모두 끝낸 뒤 다음 단계로 가는 것이 아니라 조건을 통과한 행마다 후행 접근을 반복한다. ORDER BY의 전체 집합 정렬은 이 흐름과 달리 자료를 모아야 한다. 실행계획을 자식부터 읽는 규칙은 이 예시 이해에는 도움이 되지만 모든 조인·서브쿼리에 기계적으로 적용하지 말고 실제 메커니즘을 우선한다.

세 가지 부하지점. 첫째 Outer 인덱스에서 읽어 테이블을 방문한 뒤 많이 걸러지면 필터 칼럼을 인덱스에 추가하는 방안을 본다. 둘째 Outer에서 살아남은 행마다 Inner 인덱스를 수직 탐색하므로 선행 결과 건수와 인덱스 높이가 비용을 늘린다. 셋째 Inner 인덱스 후 테이블을 읽고 대부분 버리면 그 필터 칼럼 추가를 검토한다. 조인 순서 변경, 인덱스 구성·접근 경로 개선으로 랜덤 접근을 줄인 뒤에도 비효율적이면 다른 조인을 검토한다.

적용 조건. NL은 인덱스 기반의 랜덤 접근이 많아 대량 결과를 끝까지 처리하면 비효율적이다. 반면 한 행씩 진행하므로 작은 집합 또는 일부 결과만 Fetch하는 온라인 트랜잭션에 적합하다. 조인 키 선두 인덱스의 존재와 구성에 민감하며 Outer 처리 범위가 전체 일량을 좌우한다. 원문은 OLTP 튜닝에서 먼저 NL 수행량과 랜덤 접근 지점을 분석하고, 효과적인 구성이 불가능할 때 해시·소트 머지를 고려하는 순서를 설명한다.

Prefetch와 배치 I/O. Prefetch는 디스크 읽기가 필요할 때 곧 읽을 블록까지 미리 캐시에 적재한다. 배치 I/O는 읽을 요청을 모았다가 한꺼번에 처리한다. 둘 다 행별 I/O 호출의 비효율을 줄이는 기능이며, 실행계획에서 Inner 테이블 접근의 위치나 NESTED LOOPS 단계 표현이 달라질 수 있다. 모두 캐시에 있으면 기본 방식과 성능·출력 순서가 같지만 실제 디스크 읽기에서는 차이가 생긴다. 특히 배치 I/O는 결과 순서를 바꿀 수 있어 순서가 필요하면 ORDER BY를 명시한다. 원문은 NLJ_PREFETCH·NLJ_BATCHING과 각각의 NO_ 힌트도 제시한다.

제2절 소트 머지 조인

원문 PDF 589-592쪽

두 단계의 처리. 양쪽 집합을 조인 키로 정렬하는 소트 단계와 정렬된 집합을 맞춰 읽는 머지 단계로 나뉜다. Inner가 정렬돼 있으므로 같은 키 범위를 읽다가 다음 키를 만나면 멈출 수 있다. Outer도 같은 순서로 진행하므로 다음 키를 찾을 때 처음부터 다시 탐색하지 않고 멈춘 위치를 활용한다. 원문은 인덱스로 미리 정렬된 집합을 이용해 소트를 생략할 수 있으며 Oracle의 경우 Outer에 적용하는 조건을 명시한다.

일량과 부분범위처리. NL은 Outer 행마다 반복하는 Inner 접근이 핵심이지만 소트 머지는 각 테이블의 검색조건을 만족한 집합의 크기가 전체 정렬·스캔량을 결정한다. 대량 랜덤 접근을 줄일 수 있어도 초대용량 정렬 자체가 비싸면 이점이 줄어든다. Outer가 미리 정렬돼 있고 사용자가 일부만 Fetch하면 Outer를 끝까지 읽지 않아도 되는 제한적 부분범위처리가 가능하다. 조인에 사용할 집합을 만드는 과정에서 인덱스 랜덤 접근이 많으면 머지 단계의 이점이 사라질 수도 있다.

연산자 조건. 원문은 Oracle은 부등호 또는 조인 조건이 없는 경우에도 소트 머지를 사용할 수 있지만 SQL Server는 등치 조인에서 사용한다고 구분한다. 따라서 SQL 문에 같은 조인 기호가 있다고 양 DBMS에서 같은 방식이 적용된다고 가정하지 않는다. 소트 머지는 정렬한 중간 집합을 순차적으로 읽는다는 원리와 정렬 비용을 함께 기억한다.

제3절 해시 조인

원문 PDF 593-596쪽

Build와 Probe. 작은 쪽 집합을 Build Input으로 선택해 해시 테이블을 만들고, 반대쪽 Probe Input의 키를 같은 해시 함수에 넣어 버킷을 찾은 뒤 일치하는 행을 확인한다. 같은 입력은 같은 해시값을 얻지만 서로 다른 입력이 같은 값을 얻는 충돌도 가능하다. 버킷 안의 체인에서 실제 키를 비교해야 하므로 버킷을 찾는 것만으로 조인이 끝나는 것은 아니다. 해시 조인은 NL의 반복 랜덤 접근과 소트 머지의 양쪽 정렬을 피하는 대신 해시 테이블 생성 비용을 지불한다.

메모리와 중복 키의 조건. Build가 가용 메모리에 담길 만큼 작고 해시 키 중복이 적어야 효과가 크다. 같은 키가 많으면 버킷을 늘려도 그 키의 엔트리가 한 체인에 모이므로 체인 탐색이 길어진다. 전체 테이블 크기보다 검색 조건을 적용한 Build 집합의 크기를 봐야 한다. Build 생성은 전체범위처리가 필요하지만 생성 후 Probe를 읽는 단계는 필요한 일부만 처리하는 것이 가능하다.

메모리 초과와 Grace 해시 조인. In-Memory 구축이 어렵다면 양쪽 집합에 해시 함수를 적용해 독립적인 작은 파티션 짝으로 나누고 Temp에 저장한다. 이후 짝마다 작은 파티션을 Build로 선택해 조인하며 모든 짝을 처리한다. 원래 어느 테이블이 작았는지와 별개로 짝별 Build를 다시 정한다. 작은 파티션도 메모리를 넘으면 추가 분할하는 Recursive 해시 조인이 필요하다. 디스크 기록·재읽기가 추가되므로 메모리 안에서 끝나는 경우보다 느려진다.

선택 기준과 동시성. 조인 인덱스가 없거나, 인덱스가 있어도 NL의 조인 접근량이 크거나, 양쪽 정렬 부하가 심할 때 해시 조인을 검토한다. 원문은 수행빈도가 낮고 수행시간이 긴 대량 배치·DW·OLAP 조인에 특히 적합하다고 설명한다. 인덱스는 여러 SQL이 재사용하는 영구 구조인 반면 해시 테이블은 한 SQL을 위해 생성 후 소멸한다. 짧고 매우 빈번한 SQL에 무조건 해시를 적용하면 CPU·메모리·메모리 확보 경합으로 동시성이 낮아질 수 있다. OLTP에서도 조건을 충족하는지는 개별적으로 판단한다.

제4절 스칼라 서브 쿼리

원문 PDF 597-600쪽

값을 반환하는 쿼리. 스칼라 서브쿼리는 한 레코드에 대해 하나의 값을 반환하며 SELECT 목록 등 칼럼이 올 수 있는 여러 위치에서 사용한다. 원문의 부서명 조회는 사원 행의 부서번호를 입력으로 받아 이름을 구하고 대응 부서가 없으면 NULL을 출력한다. 행마다 값을 얻는 처리 경로를 NL 조인과 연결해 이해하되, 캐시 및 옵티마이저 변환이 실제 수행을 바꿀 수 있다는 점을 함께 본다.

입·출력 캐시. 실행 중 입력과 출력을 캐시에 저장하고 같은 입력이 다시 오면 재수행 대신 저장값을 사용한다. 일반적으로 해싱을 이용하므로 입력값 종류가 적고 충돌 가능성이 낮을 때 효과적이다. 입력값 종류가 많으면 캐시 확인비용 때문에 CPU 사용과 시간이 늘 수 있다. 원문은 SQL Server가 2000까지 제공하던 Cache 기능을 2005부터 제거했다고 구분하므로 Oracle의 캐시 효과를 모든 DBMS에 그대로 적용하지 않는다.

여러 값을 얻는 경우. 평균·최소·최대 급여를 각각 스칼라 서브쿼리로 쓰면 같은 부서의 사원 범위를 반복 읽는다. 한 스칼라 쿼리에서 여러 칼럼을 직접 반환할 수 없으므로 원문은 고정 길이로 결합한 문자열 하나를 반환하고 바깥에서 SUBSTR·TO_NUMBER 또는 SQL Server의 SUBSTRING·CAST로 분리하는 기법을 제시한다. LPAD·STR로 구간 길이를 맞추는 것이 이 예시의 전제다. 단 원문은 조인 조건 Pushdown을 사용할 수 있는 경우에는 관련 쿼리 변환과 함께 검토하라고 연결한다.

Unnesting. 원문은 Oracle 12c부터 스칼라 서브쿼리도 Unnesting돼 일반 조인 형태로 변환될 수 있다고 설명한다. 예시에서는 UNNEST 힌트로 집계 뷰와 Hash Outer Join이 나타나고 MERGE를 함께 사용하면 조인·집계 위치도 달라진다. 따라서 SELECT 목록의 스칼라 서브쿼리라는 작성 형태만 보고 항상 같은 NL 경로·캐시 동작이라고 단정하지 말고 실제 실행계획을 확인한다.

제5절 고급 조인 기법

원문 PDF 601-614쪽

인라인 뷰로 먼저 집계. 1:M 조인 뒤 결과를 다시 1쪽 단위로 집계한다면 M쪽을 먼저 같은 단위로 묶고 조인해 반복 횟수를 줄일 수 있다. 원문의 연간 상품판매는 일별 365000행마다 상품을 찾는 대신 상품코드별 1000행으로 집계한 뒤 1000번만 조인한다. 조인 단계의 블록 읽기도 730002에서 2101로 감소한다. 핵심은 어떤 단위로 결과가 필요한지 먼저 정해 불필요하게 커진 조인 집합을 만들지 않는 것이다.

배타적 관계의 구현. 작업지시는 개통신청 또는 장애접수 중 하나와 관계를 맺는다. 한쪽 번호만 채우는 두 FK 칼럼으로 구현하면 두 테이블을 각각 Outer 조인하고 NVL·ISNULL로 값을 고를 수 있다. 작업구분과 공통 접수번호로 구현하면 구분값별 UNION ALL 또는 조건부 조인을 사용할 수 있다. UNION ALL이 항상 같은 자료를 두 번 읽는 것은 아니다. [작업구분 + 방문예정일시] 인덱스면 두 구간이 분리되지만 날짜가 앞이거나 날짜만 있으면 스캔·테이블 접근이 중복될 수 있다.

중복 접근을 줄이는 조건부 조인. 작업지시를 한 번 읽고 DECODE 또는 CASE로 작업구분에 맞는 접수번호만 조인 키로 내보내면 각 원본 테이블에 조건부 Outer 조인할 수 있다. 두 방식의 결과 의미를 유지하면서 인덱스 구성이 만드는 스캔 중복을 줄이는 것이 원문의 튜닝 관점이다. 어떤 형태가 유리한지는 작업구분·기간 조건과 실제 인덱스 순서를 함께 보아 결정한다.

부등호 조인과 누적. 같은 지점에서 상대 판매월이 기준 월 이하인 행을 모두 연결하고 합산하면 지점별 누적 매출을 구할 수 있다. 기준행 매출은 조인으로 반복되므로 원문은 MIN으로 한 값을 취하고 상대 매출을 SUM 한다. 윈도우 함수가 지원되면 PARTITION BY 지점과 판매월 순 프레임으로 같은 누적을 직접 표현한다. 원문은 등치뿐 아니라 부등호·BETWEEN 등 업무 관계에 맞는 연산자로 조인할 수 있음을 보여준다.

점이력과 선분이력. 점이력은 변경 시작 시점만 쌓고, 선분이력은 시작·종료 시점을 함께 관리한다. 선분이력은 조회일이 두 경계 사이에 속하는지로 그 시점의 행을 간단히 찾는다. 원문은 최종 종료일을 99991231로 관리하는 약속을 두고, 시각까지 관리하면 99991231235959를 사용한다. 대신 새 이력 입력 때 기존 마지막 행의 종료일도 갱신해야 하고, 종료일이 PK에 포함되면 키 갱신·무결성 관리 및 DML 부담을 검토해야 한다.

현재 이력의 예외. 미래에 유효해질 이력을 미리 넣지 않는다면 종료일자='99991231' 등치 조건으로 최종 행을 읽는 것이 유리하다. 예약 입력이 있으면 마지막 행이 현재 유효하다고 볼 수 없으므로 오늘 날짜가 시작·종료 사이에 있는지 검사해야 한다. 여러 선분이력을 함께 조회할 때도 각 이력에 동일한 기준 시점을 적용한다. Oracle의 TO_CHAR(SYSDATE, 'yyyymmdd')와 SQL Server의 CONVERT(VARCHAR(8), GETDATE(), 112)는 원문에서 문자열 일자에 맞추는 예시다.

거래 시점과 현재 시점 조인. 일별 거래와 종목이력을 조인하면서 거래일자를 이력의 시작·종료 사이에 놓으면 거래 당시 종목명·상장주식수를 얻는다. 현재 날짜를 조건에 넣으면 과거 거래라도 현재 이력 정보를 붙이는 의미다. 둘은 결과가 필요한 기준 시점이 다르다. 종목 마스터와 이력이 분리된 모델이면 최종 정보는 마스터와 조인하는 방식도 원문에서 구분한다. 단순히 '최신'이라는 표현으로 두 요구를 섞지 않는다.

ROWID를 이용한 점이력 조회. 일반적으로 고객의 서비스만료일 이전 변경일 중 MAX를 찾고 같은 이력을 다시 조인한다. 원문은 인덱스 MIN/MAX 최적화가 되면 이것만으로 충분히 빠를 수 있다고 먼저 설명한다. 극도로 빈번하고 한 블록까지 줄여야 하는 경우에는 서브쿼리에서 해당 행의 ROWID를 얻어 본문의 이력 테이블을 직접 접근하는 기법을 제시한다. 쿼리에 이력이 두 번 적혀도 일반 NL의 인덱스→행 접근과 같은 일량으로 만들려는 것이다.

ROWID 기법의 강한 전제. 원문의 ROWID·ROWNUM 기법은 고객번호와 변경일자 순의 특정 인덱스 및 의도한 탐색 순서에 의존한다. 인덱스 구성이 바뀌면 결과도 달라질 수 있으므로 관련 프로그램을 관리하고 인덱스 변경 시 검증하는 절차가 필요하다. 원문은 보통 MIN/MAX 최적화만으로 충분하면 이 기법을 굳이 적용하지 말라고 권한다. SQL Server는 Oracle처럼 사용자가 직접 ROWID를 지정하는 테이블 접근을 지원하지 않는다고 구분한다.

반드시 기억할 핵심 포인트

- NL은 Outer 결과와 Inner 반복 탐색·테이블 방문의 곱을 줄이는 것이 핵심이다.
- 소트 머지는 양쪽 정렬 비용, 해시는 Build의 메모리 크기와 키 중복을 평가한다.
- 해시 테이블은 한 SQL을 위한 일시 구조이므로 고빈도 적용 시 전체 자원과 동시성을 본다.
- 스칼라 캐시는 입력 종류가 적을 때 효과적이며 Unnesting 여부를 확인한다.
- M쪽을 결과 단위로 먼저 집계하면 조인 횟수를 줄일 수 있다.
- 선분이력의 현재 조회는 미래 예약 입력 유무에 따라 조건이 달라진다.
- 인덱스 순서에 의존하는 ROWID·ROWNUM 기법은 변경 관리가 필수다.

챕터 최종 요약

- NL은 행별 반복 탐색, 소트 머지는 정렬된 집합의 머지, 해시는 Build 해시 테이블의 Probe라는 차이가 있다. 작은 반복 조회·대량 스캔·정렬·메모리의 비용을 비교한다.
- 쿼리 구조를 바꿀 때는 집계 단위와 조인 횟수, 캐시 입력 종류, 배타적 관계의 분기·중복 접근을 함께 검토한다.
- 시간 이력은 기준 시점과 시작·종료의 의미가 우선이다. 거래 당시·현재·최종 이력을 구분하고, 성능 기법의 인덱스 및 예약 입력 조건을 지켜야 한다.

제5장 SQL 옵티마이저

과목 III. SQL 고급 활용 및 튜닝

원문 PDF 618-665쪽

제1절 SQL 옵티마이징 원리

원문 PDF 619-627쪽

옵티마이저와 실행계획. SQL은 원하는 결과 집합을 정의하고, 그 결과를 얻는 처리 절차는 DBMS가 만든다. 옵티마이저가 내부적으로 생성한 처리 절차가 실행계획이다. 따라서 SQL 성능을 이해하려면 문장만 읽는 데서 멈추지 않고, 실제로 어떤 액세스와 조인 경로가 선택되는지 해석해야 한다.

규칙기반과 비용기반. 규칙기반 옵티마이저(RBO)는 인덱스 구조, 연산자, 조건절 형태 등에 대해 미리 정한 액세스 경로 우선순위로 계획을 고른다. 비용기반 옵티마이저(CBO)는 후보 계획별 예상 일량 또는 시간을 계산하고 합산 비용이 가장 낮은 계획을 선택한다. 원문은 Oracle 10g부터 RBO 지원이 중단되었다는 배경 아래 CBO를 중심으로 설명한다.

최적화 목표. 전체 처리속도 최적화는 최종 결과를 끝까지 읽는 전제에서 I/O·CPU·메모리 사용을 줄이는 데 초점을 둔다. 최초 응답속도 최적화는 일부 결과를 빨리 읽고 멈추는 전제에서 첫 응답을 앞당긴다. 후자의 계획으로 결과를 끝까지 읽으면 오히려 자원 소모와 전체 수행시간이 늘 수 있다. 원문의 ALL_ROWS, FIRST_ROWS, FIRST_ROWS(n), SQL Server FAST 관련 설명은 이 목표 차이를 보여 준다.

옵티마이저 행동의 입력. 같은 결과를 내는 SQL이라도 문장의 구조와 연산자에 따라 다른 계획을 선택할 수 있다. 인덱스·IOT·클러스터링·파티셔닝·MV 같은 물리적 구성은 선택 가능한 경로 자체를 바꾼다. PK·FK·CHECK·NOT NULL은 무결성뿐 아니라 결과의 성질을 증명하는 정보다. 예를 들어 인덱스 칼럼의 NOT NULL 제약은 전체 건수를 구하는 COUNT 쿼리에서 해당 인덱스를 사용할 근거가 된다.

통계·힌트·환경. CBO의 판단 기준은 통계정보이며, 사용자가 지정한 힌트는 옵티마이저의 일반 판단보다 우선한다. 관련 파라미터, DBMS 종류와 버전도 행동에 영향을 준다. SQL·데이터·통계·하드웨어가 같아 보여도 업그레이드 후 계획이 바뀔 수 있으므로 실행계획을 고정된 문장 속성처럼 생각하면 안 된다.

한계의 첫 원인은 제공된 환경이다. 적절한 인덱스나 저장 구조가 없으면 옵티마이저가 좋은 경로를 새로 만들어 낼 수는 없다. 모든 분포를 정확하게 수집·유지하기도 현실적으로 어렵다. 히스토그램의 수집 비용이 있고, 칼럼 수가 늘수록 모든 조합의 결합 분포를 보관하는 것은 더욱 어려워진다.

칼럼 상관관계의 문제. 원문은 직급과 연봉이 서로 관련된 경우를 든다. 각 조건을 독립적으로 보고 선택도를 곱하면 '부장'이면서 일정 연봉 이상인 사원 수를 실제보다 작게 추정할 수 있다. 개별 칼럼 통계가 정확하다는 사실만으로 여러 조건을 결합한 추정까지 정확해지는 것은 아니다. 이 오차는 액세스 방식과 조인 순서 선택에 이어진다.

그 밖의 한계. 바인드 변수에는 평균적인 균등분포를 가정하는 문제가 있고, 비용 모델 자체에도 현실을 단순화한 가정이 들어간다. CBO도 일부 판단은 규칙에 의존한다. 실제 하드웨어의 I/O 특성과 애플리케이션 부하가 모델의 전제와 달라도 비용 오차가 생긴다. 원문은 이를 근거로 옵티마이저를 무조건 불신하기보다 원리와 제약을 알고 부족한 부분을 보완하도록 강조한다.

통계 종류	주요 항목	추정에 제공하는 정보
테이블	레코드 수, 블록 수, 빈 블록 수, 평균 행 길이	처리할 전체 데이터 규모
인덱스	높이, 리프 블록 수, 클러스터링 팩터, 인덱스 레코드 수	탐색·스캔·테이블 방문 비용
칼럼	값의 수, 최저·최고 값, 밀도, NULL 수, 히스토그램	조건별 분포와 선택도

통계 종류	주요 항목	추정에 제공하는 정보
시스템	CPU 속도, 평균 I/O 속도, 초당 I/O 처리량	운영 하드웨어의 비용 특성

선택도에서 계획까지. 선택도는 전체 대상 중 조건을 만족할 것으로 예상되는 비율이다. 히스토그램을 이용할 수 없고 '=' 조건의 값이 균등하게 분포한다고 보면 선택도는 1/Distinct Value 개수다. 카디널리티는 한 액세스 단계의 예상 출력 건수이며 '총 로우 수 × 선택도'로 계산한다. 여러 독립 조건이면 선택도를 곱한다. 원문의 사원 1,000명, 부서 값 10개, 직급 값 4개 사례에서는 부서 조건의 예상 건수가 100명이고 두 조건을 결합하면 25명이다.

히스토그램. 값별 분포를 보관하면 균등분포 가정보다 정확한 카디널리티를 구할 수 있다. 도수분포 히스토그램은 값의 수가 적을 때 각 값에 버킷을 할당하고 빈도를 저장한다. 높이균형 히스토그램은 값이 많을 때 여러 값을 한 버킷으로 대표시키되 각 버킷에 같은 비중의 데이터가 들어가게 한다. 빈도가 높은 값은 여러 버킷을 차지할 수 있다. 원문은 12c 기준 상위도수분포와 하이브리드도 언급하지만, 원리를 자세히 설명하는 대상은 앞의 두 유형이다.

바인딩 시점과 분포. 원문의 기본 설명에서는 최적화한 계획을 캐시에 저장한 뒤 실행 시 값을 바인딩하므로, 최적화 시점에 특정 입력값의 분포를 알 수 없다. 히스토그램이 있어도 평균 분포에 의존하는 이유다. 분포가 불균일한 칼럼에서 이 가정은 입력값별 성능 차이를 만들 수 있다. 뒤의 Peeking과 적응적 커서 공유는 이 부작용을 완화하려는 기능으로 연결해 읽어야 한다.

I/O 비용 모델. 원문은 I/O 비용 모델과 CPU 비용 모델을 구분한 뒤 전자를 자세히 설명한다. 여기서 비용의 단위는 논리적·물리적 블록 읽기 수 자체가 아니라 예상 I/O Call 횟수다. 인덱스를 통한 테이블 액세스는 Single Block I/O이므로 한 블록을 읽는 호출 단위로 계산한다. Full Scan은 Multiblock I/O를 사용하므로 총 블록 수를 한 번에 읽는 단위로 나누어 호출 수를 추정한다.

인덱스 경유 비용의 구성. 원문의 식은 'blevel + 리프 블록 수 × 유효 인덱스 선택도 + 클러스터링 팩터 × 유효 테이블 선택도'다. blevel은 리프에 도달하기 전 브랜치 탐색 비용, 둘째 항은 필요한 범위를 수평 탐색하는 비용, 셋째 항은 테이블 랜덤 액세스 비용을 뜻한다. 유효 인덱스 선택도는 조건을 찾기 위해 스캔할 인덱스 비율이고, 유효 테이블 선택도는 인덱스 필터까지 거친 뒤 최종적으로 테이블을 방문할 비율이다. 같은 선택도라는 말이라도 어느 단계의 대상을 뜻하는지 구분해야 한다.

클러스터링 팩터와 Full Scan. 클러스터링 팩터는 인덱스 순서로 전체 로우를 방문할 때 예상되는 테이블 블록 접근량으로, 같은 값의 데이터가 얼마나 모여 있는지를 비용에 반영한다. Full Scan에서는 Multiblock I/O 단위가 커질수록 예상 호출 수가 줄어 비용도 작아진다. 다만 비용은 모델의 예상치이므로 특정 경로의 비용이 낮다고 실제 경과시간까지 언제나 가장 짧다고 단정하지 않는다.

제2절 SQL 공유 및 재사용

원문 PDF 628-638쪽

소프트 파싱과 하드 파싱. SQL 파서는 문법 오류를 검사하고, 객체·칼럼 존재와 접근 권한 등 의미 오류를 확인한다. 이어 공유 메모리에서 같은 SQL과 실행계획을 찾는다. 캐시의 계획을 찾아 실행하면 소프트 파싱이고, 찾지 못해 최적화를 거쳐 계획을 만들면 하드 파싱이다. 원문은 Oracle 라이브러리 캐시와 SQL Server 프로시저 캐시를 설명의 편의상 라이브러리 캐시로 통칭한다.

공유가 필요한 이유. 테이블 다섯 개만 조인해도 순서 후보가 5!=120개이며, 여기에 조인 방식과 각 테이블의 액세스 방식까지 조합된다. 옵티마이저는 구조 정보, 오브젝트·시스템 통계, 파라미터를 읽어 후보를 평가해야 한다. 이 무거운 작업으로 만든 절차를 한 번 쓰고 버리지 않고 여러 세션이 재사용하도록 캐시에 보관한다.

SQL 문자열 자체가 식별자다. SQL 문장을 해시 함수에 넣어 해당 버킷을 찾고, 연결된 엔트리에서 같은 문자열을 탐색한다. 찾으면 저장된 계획을 실행하고, 못 찾으면 최적화한 SQL과 계획을 연결한다. 별도 SQL ID가

있어도 원문 설명의 핵심은 전체 문장과 식별자가 대응한다는 점이다. 공백·줄바꿈·대소문자·주석·Owner 표기·힌트·리터럴 값의 차이가 별도 SQL을 만들 수 있다.

작성 표준과 리터럴 SQL. 개발 초기에 SQL 작성 규칙을 정하고 지키면 불필요한 차이를 줄일 수 있다. 원문이 특히 문제 삼는 패턴은 사용자가 입력한 값을 조건절 문자열에 붙여 매번 다른 SQL을 만드는 경우다. 동일한 로그인 처리라도 사용자 ID마다 별도 최적화와 메모리 공간이 필요해진다. 개별 실행이 빨라 보여도 접속이 몰리면 라이브러리 캐시 경합과 CPU·메모리 부하가 시스템 확장성을 해친다.

바인드 변수의 역할. SQL 본문은 같게 두고 입력값만 매개변수로 전달하면 처음 만든 계획을 여러 세션이 공유한다. 최초 세션은 하드 파싱하지만 이후 세션은 같은 계획에 다른 값만 바인딩한다. 파싱 시간과 메모리 사용을 줄이는 효과는 동시 사용자가 많은 OLTP에서 특히 중요하다. 원문의 2만 회 수행 트레이스는 Parse Call이 반복되어도 라이브러리 캐시 미스는 최초 한 번이라는 사실을 보여 준다.

항상 같은 선택은 아니다. 수행빈도가 낮고 실행시간이 긴 배치·DW·OLAP 쿼리에서는 전체 시간 중 파싱 비중이 작다. 칼럼 값 종류가 적고 분포가 치우친 경우도 특정 상수의 히스토그램을 활용하는 것이 유리할 수 있다. 원문은 이런 조건을 제외한 OLTP에서는 바인드 변수를 사용할 것을 권고한다. '실행계획 공유'와 '특정 값에 가장 알맞은 최적화'가 때로 충돌하는 관계를 이해해야 한다.

자동 매개변수화의 위치. 원문 기준 SQL Server의 단순 매개변수화와 Oracle CURSOR_SHARING의 FORCE·SIMILAR는 리터럴을 자동 변수화해 파싱 부하를 완화하는 기능이다. 그러나 사용자가 의도한 상수까지 바뀌고 실행계획이 변하는 부작용이 있다. 원문은 Oracle에서는 이를 응급처방으로 다루며 영구 대체책으로 삼지 않도록 하고, SQL Server에서도 직접 바인드 변수를 사용하는 편을 강조한다. 이 설정과 버전 설명을 현재 모든 제품의 기본값으로 일반화하지 않는다.

Peeking과 Parameter Sniffing. 최초 실행의 변수 값을 보고 그 분포로 계획을 정하는 기능은 Oracle의 Bind Peeking, SQL Server의 Parameter Sniffing으로 설명된다. 처음 값에 맞는 계획이 이후 다른 분포의 값에는 맞지 않을 수 있고, 캐시에서 밀려난 뒤 다시 만들어지면 수행계획도 달라질 수 있다. 사전 실행계획 확인 때는 값이 바인딩되지 않아 평균 분포 계획을 보았더라도 실제 첫 실행에서는 다른 계획이 선택될 수 있다는 점을 원문이 경고한다.

분포에 따른 계획 분기. 원문은 11g의 Adaptive Cursor Sharing을 입력값 분포별 계획을 사용하는 보완책으로 소개한다. 개발자가 보완하는 사례로는 아파트매물 도시 조건을 UNION ALL로 나눈다. 데이터 비중이 큰 서울시·경기도는 Full Scan, 나머지는 인덱스 경로를 유도하고 입력값 조건으로 해당 분기만 수행하도록 구성한다. 핵심은 단순히 UNION ALL을 쓰는 것이 아니라 입력값의 분포와 각 경로의 비용 차이를 반영하는 것이다.

애플리케이션 커서 캐싱. 바인드 변수를 써도 매번 문법·의미 검사, 캐시 검색, 수행 메모리 할당을 반복하면 소프트 파싱 비용은 남는다. 이미 연 커서를 해제하지 않고 재사용하는 애플리케이션 커서 캐싱은 이 반복을 줄인다. 원문의 반복 실행 트레이스에서는 Parse Call이 한 번만 발생한다. 바인드 변수는 하드 파싱을 줄이는 축이고, 커서 캐싱은 반복 소프트 파싱까지 줄이는 축이다.

개발 언어에 따른 구현. 원문은 Pro*C의 커서 유지 옵션, Java의 Implicit Caching 설정, PreparedStatement를 루프 바깥에 두고 마지막에 닫는 방식으로 같은 효과를 설명한다. PL/SQL은 Static SQL에서 별도 옵션 없이 커서를 캐시하지만 Dynamic SQL이나 Ref Cursor에는 같은 자동 효과가 없다고 설명한다. 이처럼 커서 유지 기능과 API 동작은 사용 환경을 전제로 읽어야 한다.

Static SQL과 Dynamic SQL. Static SQL은 문자열 변수에 담지 않고 프로그램 코드 사이에 직접 기술하는 Embedded SQL이다. PreCompiler가 문법·객체·권한을 미리 확인하고 런타임 라이브러리 호출로 변환할 수 있다. Dynamic SQL은 문자열 변수에 담기 때문에 조건에 따라 SQL 구조를 바꾸거나 실행 시 입력받을 수 있지만, 사전 컴파일 단계에서 그 내용을 확인할 수 없다.

분류를 혼동하지 않기. 두 방식 모두 DBMS에 전달될 때는 SQL 문장이며 옵티마이저는 그 문장 자체를 인식한다. 따라서 Dynamic SQL이라는 이유로 라이브러리 캐시 효율이 나빠지는 것이 아니다. Dynamic SQL도 바

인드 변수를 쓰면 공유할 수 있다. 반대로 매번 다른 리터럴을 결합하면 하드 파싱이 늘어난다. 다만 언어에따라 Static SQL에만 커서 캐싱이 작동하는 차이는 별도로 고려해야 한다.

제3절 쿼리 변환

원문 PDF 639-663쪽

쿼리 변환의 목적과 방식. Query Transformer는 본격적인 비용 계산과 계획 생성 전에 SQL을 의미적으로 같은 결과를 내는 다른 형태로 재작성한다. 휴리스틱 변환은 결과가 보장되면 경험상 유리하다는 규칙에 따라 적용한다. 비용기반 변환은 변환 후 비용이 더 낮을 때 적용하고 그렇지 않으면 원래 구조를 유지한다. 모든 변환의 전제는 결과 집합의 의미를 보존하는 것이다.

서브쿼리 Unnesting. 중첩 서브쿼리는 논리적으로 메인 쿼리가 읽은 각 로우를 넘겨 필터링하는 부모·자식 구조다. Unnesting은 이 계층을 같은 레벨의 조인 구조로 풀어 다양한 액세스 경로와 조인 방식을 평가하게 한다. 변환하지 않으면 메인과 서브쿼리를 별도 Subplan으로 최적화하며 실행계획에 FILTER가 나타난다. 각 블록의 최적이 전체 쿼리의 최적을 보장하지 않는 것이 구조를 풀려는 이유다.

Unnesting과 결과 건수 보장. 서브쿼리 쪽 조인 키가 PK·UNIQUE 등으로 유일하면 일반 조인으로 바꾸어도 행이 불필요하게 늘지 않는다는 근거가 된다. 반대로 유일성을 확신할 수 없는 집합을 일반 조인하면 원래 IN·EXISTS의 결과가 중복 확대될 수 있다. 서브쿼리 쪽을 먼저 읽을 때는 UNIQUE 처리를 거친 뒤 조인하고, 메인 쪽을 먼저 읽으면 세미 조인을 적용하는 식으로 결과를 보장한다.

세미 조인. 메인 테이블 한 로우에 대해 내부 집합에서 첫 일치 로우를 찾는 순간 탐색을 멈추고 메인의 다음 로우로 넘어간다. 따라서 내부의 여러 로우가 일치하더라도 메인 로우가 여러 번 결과에 나타나지 않는다. 이는 단순한 일반 NL 조인과 구분해야 하는 핵심이다. 원문은 UNNEST·NO_UNNEST 힌트도 소개하지만, 변환이 언제나 빠르지는 않으므로 기본적으로 비용과 계획을 확인하도록 한다.

뷰 Merging. 사람이 읽기 좋게 나눈 인라인 뷰의 쿼리 블록을 참조하는 쿼리 블록과 합치면, 옵티마이저가 전체 테이블에 대해 더 다양한 액세스·조인 순서를 검토할 수 있다. 단순 필터 뷰는 이런 변환이 자연스럽지만 GROUP BY나 DISTINCT를 포함한 복잡한 뷰는 합친 뒤 비용이 더 커질 수 있어 비용 판단이 중요하다. MERGE·NO_MERGE 힌트는 원문이 제시하는 제어 수단이다.

뷰 병합의 제약. 원문은 집합 연산, CONNECT BY, ROWNUM, SELECT 목록의 집계함수, 분석함수 등을 뷰 병합이 불가능한 구성으로 정리한다. 이 목록은 원문이 설명하는 DBMS·버전의 동작 범위다. 특히 분석함수 등을 가진 블록은 조건을 어디서 적용하느냐에 따라 의미가 달라질 수 있으므로, 사람이 보기 편하다는 이유만으로 블록을 풀거나 조건을 이동해서는 안 된다.

조건절 Pushdown. 뷰를 참조하는 외부 조건을 내부로 밀어 넣어 가능한 한 일찍 필터링한다. 전체를 GROUP BY한 뒤 부서를 고르는 대신, 해당 부서만 읽어 집계하면 내부 처리량과 다음 단계 출력량을 함께 줄인다. 인덱스를 이용할 조건이 내부에 생겨 액세스 경로가 달라질 수도 있다. 조인으로 연결된 반대쪽 상수 조건은 조건절 이행을 거쳐 뷰 안으로 들어갈 수 있다.

조건절 Pullup과 조인 조건 Pushdown. Pullup은 한 블록 안의 조건을 밖으로 끌어내 다른 블록에 다시 Pushdown하는 데 쓴다. 원문의 두 집계 뷰 사례에서는 한쪽의 부서 10 조건이 다른 뷰에도 적용된다. 조인 조건 Pushdown은 NL 조인 중 외부 테이블에서 읽은 값을 내부 뷰로 건건이 넘기는 것이다. 실행계획의 VIEW PUSHED PREDICATE는 넘겨받은 부서에 대해서만 집계하는 동작을 보여 준다.

부분범위처리와 버전 조건. 원문은 집계 뷰의 조인 조건 Pushdown을 Oracle 11g의 기능으로 소개하며, 지원하지 않는 환경에서는 집계 뷰 전체를 반복 처리하는 비효율이 생길 수 있음을 보인다. 집계가 하나면 스칼라 서브쿼리로 바꾸어 해당 부서만 처리할 수 있다. 여러 집계를 각각 스칼라 서브쿼리로 쓰면 같은 범위를 반복 읽으므로, 원문은 값을 문자열로 결합해 한 번 구한 뒤 밖에서 분리하는 우회법도 제시한다. 이는 해당 기능이 없는 환경의 문맥으로 읽어야 한다.

조건절 이행. A=B이고 B=C이면 A=C라는 관계를 이용해 새로운 필터를 내부 생성한다. 사원 부서가 10이고 사원 부서와 부서 테이블의 부서번호가 같으면 부서 테이블에도 부서번호 10 조건이 생긴다. 이 조건을 각 집합에 먼저 적용하면 조인 대상이 줄고, 원래는 고려하기 어려웠던 인덱스도 사용할 수 있다. 원문은 같은 테이블의 칼럼 간 관계와 부등호 관계도 이행의 근거가 될 수 있다고 설명한다.

불필요한 조인 제거. 1:M 조인에서 1쪽 테이블을 조인식 이외에 참조하지 않고 결과에 영향을 주지 않는다는 근거가 있으면 M쪽만 읽도록 바꿀 수 있다. 내부 조인에서는 PK가 유일성을, FK가 조인 성공을 증명한다. FK 칼럼이 NULL을 허용하면 조인을 제거할 때 원래 실패할 NULL 로우가 포함될 수 있으므로 IS NOT NULL 조건을 내부에 추가한다. 외부 조인은 결과 보존 성질을 별도로 따지며, 실제 지원 여부는 원문이 비교하는 버전에 따라 다르다.

OR-Expansion. 서로 다른 칼럼의 OR 조건을 UNION ALL의 분기로 나누면 각 분기가 각자 유리한 인덱스를 쓸 수 있다. 그러나 같은 테이블을 여러 번 액세스하고 교집합을 중복 읽으므로 항상 유리하지 않다. 원문은 처음부터 비용기반으로 작동하는 변환이라고 설명한다. 중복 결과를 막기 위해 아래 분기에 LNNVL을 적용하며, 이 함수는 조건이 FALSE 또는 UNKNOWN일 때 TRUE가 된다. 단순히 'job <> 값'만 쓰면 NULL의 의미가 달라질 수 있다.

집합 연산의 조인 변환. INTERSECT·MINUS를 조인으로 바꿀 수 있다. 원문의 MINUS 사례는 HASH ANTI JOIN 뒤 HASH UNIQUE로 차집합과 중복 제거를 구현한다. NULL 허용 칼럼에서는 집합 연산의 NULL 처리와 같은 결과를 내도록 내부 비교 처리가 필요하다. 원문에 등장하는 SYS_OP_MAP_NONNULL는 이 문맥의 비공식 내부 함수이며, 일반적인 공개 문법처럼 확대해서 사용법을 추가하지 않는다.

추가 필터와 비교 순서. 조인 칼럼의 NULL은 어차피 내부 조인에 실패하므로 필요한 경우 IS NOT NULL을 먼저 적용해 불필요한 접근을 줄인다. BETWEEN의 하한이 상한보다 큰 경우에는 테이블을 읽기 전 필터가 거짓임을 판단해 하위 액세스를 생략할 수 있다. 따라서 실행계획에 TABLE ACCESS FULL이 보인다고 그 단계가 반드시 수행되는 것은 아니다. 조건을 평가하는 순서도 CPU 일량에 영향을 주므로, 원문은 선택도가 낮아 많은 로우를 먼저 제거하는 조건을 앞서 평가하도록 옵티마이저가 조정하는 원리를 설명한다.

반드시 기억할 핵심 포인트

- 선택도 → 카디널리티 → 예상 비용 → 액세스·조인 방식이라는 연결을 이해한다.
- 정확한 단일 칼럼 통계도 칼럼 간 상관관계와 바인드 입력값 문제까지 해결하지는 못한다.
- 바인드 변수는 계획 공유를, 애플리케이션 커서 캐싱은 반복 파싱 생략을 돕는다.
- UNNESTING·뷰 병합·조건 이동·조인 제거에는 결과 보장의 조건이 있다.
- 원문의 DBMS 버전별 기능·설정 설명을 현재 모든 환경의 사실로 일반화하지 않는다.

챕터 최종 요약

- 옵티마이저는 주어진 구조·제약·통계·파라미터 안에서 가장 낮은 예상 비용의 계획을 찾는다. 목표가 전체 처리인지 첫 응답인지부터 구분해야 한다.
- 비용 추정은 분포와 가정에 의존한다. 히스토그램은 불균일 분포를 반영하지만 바인드 변수·칼럼 상관관계·운영 환경에서 오차가 남는다.
- SQL 문자열을 동일하게 유지하고 입력값을 바인딩하면 무거운 최적화를 공유할 수 있다. 커서를 유지하면 그 계획을 찾는 반복 비용까지 줄일 수 있다.
- 쿼리 변환은 블록 구조를 풀거나 조건을 이동해 선택 가능한 계획을 넓히는 작업이다. 결과 건수·중복·NULL·제약조건을 보존한 뒤 비용을 비교해야 한다.

제6장 고급 SQL 튜닝

과목 III. SQL 고급 활용 및 튜닝

원문 PDF 667-762쪽

제1절 소트 튜닝

원문 PDF 668-690쪽

메모리 소트와 디스크 소트. 정렬은 할당된 Sort Area에서 수행하며 원문 기준 Oracle은 PGA, SQL Server는 버퍼 캐시에 그 영역을 둔다. 정렬이 메모리 안에서 끝나면 Internal 또는 Optimal Sort다. 공간이 부족해 Oracle Temp Tablespace나 SQL Server tempdb를 사용하면 External Sort다. 대상 집합을 디스크에 한 번 기록하면 Onepass, 여러 번 기록하면 Multipass이며, 재기록과 재읽기를 반복할수록 성능이 크게 나빠진다.

소트의 비용과 부분범위처리. 정렬은 메모리뿐 아니라 CPU를 많이 사용하고, 큰 집합이면 디스크 I/O도 유발한다. 전체 대상이 준비돼야 결과를 낼 수 있는 소트는 일부 결과만 빨리 보여 줄 온라인 업무의 부분범위처리를 막기도 한다. 따라서 모델을 먼저 점검하고, 불필요한 연산 제거 → 인덱스로 대체 → 소트 대상·폭 축소 → 메모리 크기 조정 순서로 검토한다.

원문 실행계획 오퍼레이션	나타나는 작업	해석할 점
SORT AGGREGATE	전체 집합의 집계	이름에 SORT가 있어도 실제 정렬은 하지 않음
SORT ORDER BY	요구한 순서로 결과 정렬	인덱스 순서로 대체 가능한지 확인
SORT GROUP BY / HASH GROUP BY	그룹별 집계	집계 알고리즘을 구분
SORT UNIQUE	DISTINCT·UNION 등의 중복 제거	중복 제거가 실제로 필요한지 확인
SORT JOIN	소트 머지 조인	양쪽 조인 집합의 정렬 작업
WINDOW SORT	윈도우 함수 처리	분할·순서에 따른 정렬 비용

모델 때문에 반복하는 집계. 원문은 고객의 가입상품과 고객별상품라인을 통합하면, 가입상품 수준의 조회마다 라인 테이블을 GROUP BY해야 하는 사례를 든다. 별도 가입상품 엔터티가 유지되면 단순 조인으로 해결된다. GROUP BY·UNION·DISTINCT가 자주 쓰이는 SQL에서는 데이터 모델의 중복을 뒤늦게 제거하는 것은 아닌지 살펴야 한다. 단순히 테이블 수를 줄이는 설계가 성능에도 유리하다고 볼 수 없다.

UNION을 UNION ALL로 바꿀 조건. UNION은 두 집합을 합친 뒤 중복을 제거하고, UNION ALL은 그대로 이어 붙인다. 부서 10과 20의 사원을 조회하면서 PK인 사원번호가 결과에 있으면 두 분기 사이에 중복이 없으므로 중복 제거를 생략할 수 있다. 그러나 사원번호 없이 직무와 관리자만 선택하면 두 부서에 같은 값 조합이 있을 수 있으므로 함부로 바꾸면 결과가 달라진다. 연산자 교체의 근거는 원본 행의 구분이 아니라 SELECT 결과의 중복 가능성이다.

DISTINCT 대신 존재 여부 확인. 어떤 월에 해당 지역 과금이 있었는지만 필요하면 모든 과금 행을 읽어 과금연월을 중복 제거할 필요가 없다. 연월 테이블을 먼저 읽고 EXISTS로 첫 과금 행의 존재만 확인할 수 있다. [과금연월+지역] 인덱스가 이를 뒷받침하면 첫 일치에서 탐색을 끝낸다. 원문 사례는 대량 스캔·중복 제거 대신 82개 블록만 읽는 방식으로 바뀌며, 35개 연월 결과를 얻는다.

불필요한 COUNT 제거. 존재만 확인하는 프로그램이 전체 COUNT(*)를 구하면 조건에 맞는 모든 행을 읽는다. 원문은 Oracle에서 ROWNUM<=1로 첫 행에서 멈추고 행이 없을 때 예외를 처리하는 방식, SQL Server의 TOP N 또는 EXISTS를 이용하는 방식을 제시한다. 필요한 것은 0건인지 아닌지라는 사실뿐이므로, 정확한 전체 건수를 만드는 일을 생략하는 것이다.

인덱스로 ORDER BY와 GROUP BY 대체. 인덱스는 키 순서로 정렬돼 있으므로 조건과 정렬이 그 순서에 맞으면 별도 소트를 없앨 수 있다. 원문의 [region+custid] 인덱스에서 region을 '='로 고정하면 custid 순서 결과를 바로 읽을 수 있다. region별 집계도 선두 키 순서로 읽으면 SORT GROUP BY NOSORT가 가능하다. 다만 끝까지 모두 읽는 대량 업무에서는 인덱스 랜덤 액세스가 더 비쌀 수 있고, 원래 대상이 적으면 소트 제거 효과도 작다.

MIN·MAX와 First Row Stopkey. [주문일자+주문번호] 인덱스는 하루의 마지막 주문번호를 찾을 때 전체 범위를 읽지 않고 끝값에 바로 접근할 수 있다. 원문 계획에는 FIRST ROW와 INDEX RANGE SCAN(MIN/MAX)이 나타난다. MAX 안에서 인덱스 칼럼을 가공하면 정렬된 원래 값과 연결되는 최적화가 막힐 수 있다. 원문은 MAX(주문번호)+1과 MAX(주문번호+1)을 구분하고, 제품별 동작 차이도 함께 설명한다.

소트할 데이터의 폭 줄이기. 소트가 불가피해도 문자열 패딩과 날짜 포맷 등 출력 가공을 정렬 전에 하면 레코드 폭이 커져 Sort Area를 많이 쓴다. 원문의 파일 출력 사례는 원래 칼럼을 먼저 정렬하고 마지막 SELECT에서 LPAD·TO_CHAR로 가공하는 구조가 더 적은 메모리를 사용한다. 메모리를 늘리기 전에 정렬 대상의 건수와 한 행 크기부터 줄여야 한다.

Top N Stopkey와 Top N Sort. 정렬 기준에 맞는 인덱스가 있으면 N건을 읽고 멈추는 Top N Stopkey가 작동해 전체 범위 접근을 피한다. 인덱스가 없어 전체 대상은 읽어야 해도 Top N Sort는 N개 후보만 유지한다. 처음 N개를 정렬해 두고 이후 값이 현재 최악의 후보보다 유리할 때만 배열에 넣어 다시 정렬하고 밀려난 값을 버린다. 후자는 모든 후보를 읽는 I/O까지 없애 주는 기능은 아니다.

Top N이 인식되는 SQL 형태. Oracle의 원문 패턴은 정렬하는 인라인 뷰를 감싸 ROWNUM 상한을 적용하고, 페이지 시작 행 필터는 더 바깥에 둔다. ROWNUM을 별칭으로 선택한 뒤 그 별칭에 BETWEEN만 적용하는 식으로 구조를 줄이면 같은 결과라도 Top N Stopkey와 Sort가 작동하지 않을 수 있다고 설명한다. 윈도우 함수로 최신 이력을 고를 때도 MAX 비교 대신 순위 함수와 1위 조건을 사용하는 방식이 Top N Sort에 유리한 사례를 제시한다.

최신 이력의 정확성과 성능. 날짜와 순번을 연결해 MAX를 구하면 인덱스 칼럼을 가공해 Stopkey를 활용하지 못할 수 있다. 반대로 INDEX_DESC 힌트와 ROWNUM만 믿으면 인덱스 구성이 바뀔 때 결과가 틀릴 수 있다. 원문은 원하는 역순을 ORDER BY에 명시하고 한 건을 제한해 결과를 보장하는 형태를 설명한다. 상관 참조를 인라인 뷰 안에 배치할 수 있는 범위는 10g·11g·12c에서 차이가 있으므로 제시된 버전 전제를 함께 확인해야 한다.

Sort Area 크기 조정. 원문 기준 Oracle의 수동 관리는 SORT_AREA_SIZE를 조정하며, 자동 PGA 메모리 관리는 PGA_AGGREGATE_TARGET과 WORKAREA_SIZE_POLICY=AUTO를 사용한다. 자동 모드에서는 SORT_AREA_SIZE가 무시된다. 다른 작업이 거의 없는 대량 배치에서 자동 모드의 프로세스별 제한 때문에 여유 메모리를 충분히 쓰지 못하면 세션을 MANUAL로 전환해 조정하는 방안을 설명한다. 이는 SQL과 대상량을 먼저 개선한 뒤, 메모리 소트 또는 적은 디스크 왕복을 목표로 검토하는 관리 기법이다.

제2절 DML 튜닝

원문 PDF 691-699쪽

인덱스 유지 비용. 테이블 데이터를 변경하면 관련 인덱스 탐색·변경과 Redo·Undo 생성도 필요하다. UPDATE에서 테이블 행은 직접 바꾸지만 정렬된 인덱스는 삭제 후 삽입하는 방식으로 유지하고, 이에 대한 Undo도 기록한다. INSERT·DELETE는 원문이 설명하는 NULL 제외 조건을 고려하면 전체 인덱스 수의 영향을 받는다. 대량 작업에서는 인덱스를 제거하거나 UNUSABLE로 만든 뒤 다시 생성하는 총시간이 유지하면서 작업하는 시간보다 짧을 수 있다.

일반 INSERT와 Direct Path. 원문의 일반 Heap 입력은 HWM 아래 여유 블록 목록인 Freelist에서 공간을 찾고 버퍼 캐시에 올린 뒤 기록한다. 평소 트랜잭션에는 재사용 공간을 채우는 방식이 유리하지만 대량 적재에서는 반복 탐색이 부담이다. Direct Path Insert는 Freelist와 버퍼 캐시를 거치지 않고 HWM 밖의 데이터 파일 영역에 직접 쓰고, 커밋 시 HWM을 올린다. 이 경로에서 데이터 Undo를 쌓지 않는 점도 성능 차이의 원인이다.

Direct Path를 사용하는 원문 방법. INSERT SELECT의 APPEND 힌트, 병렬 INSERT, SQL*Loader의 direct 옵션, CTAS가 제시된다. NOLOGGING 테이블 속성을 함께 사용하면 Direct Path 적재에서 Redo를 최소화할 수 있

다. 원문은 일반 INSERT를 로깅하지 않게 하는 방법과 구분하며, 딕셔너리 변경 등 필요한 로깅까지 전혀 없다는 뜻으로 사용하지 않는다.

Lock과 복구의 조건. 원문은 Direct Path Insert 중 Exclusive 테이블 Lock으로 다른 DML이 막히므로 트랜잭션이 빈번한 시간대에는 적용하지 않도록 강조한다. NOLOGGING으로 적재한 데이터는 로그만으로 장애 복구할 수 없으므로 즉시 백업하거나 언제든 재생 가능한 임시·읽기 전용 적재에 사용해야 한다. 속도 향상만 보고 복구 가능성과 가용성 요건을 생략해서는 안 된다.

SQL Server 최소 로깅. 원문 기준 복구 모델이 Bulk-logged 또는 Simple이어야 하며, BULK INSERT와 TABLOCK, SELECT INTO, Heap 테이블 INSERT의 TABLOCK 등을 설명한다. Heap의 TABLOCK은 X 테이블 Lock으로 여러 트랜잭션의 동시 INSERT를 제한한다. B*Tree 테이블에서는 소스를 클러스터형 인덱스 키 순서로 정렬해야 하며, 다음 세 경우를 구분한다. 원문은 이 경우들에서 TABLOCK이 항상 필요한 것은 아니므로 입력 소스의 값 범위가 중복되지 않으면 동시 INSERT도 가능하다고 설명한다. 최소 로깅 후 차등(Differential) 백업을 수행해야 한다는 조건까지 함께 기억한다.

B*Tree 대상 상태	원문의 추가 조건 (공통: Bulk-logged/Simple + 키 순서 정렬)
비어 있는 구조	TABLOCK 힌트 사용
비어 있는 구조	TF-610 활성화
비어 있지 않은 구조	TF-610 활성화 + 새로운 키 범위만 입력

대량 UPDATE·DELETE의 재구성. 많은 행을 바꾸면 테이블과 인덱스 변경, 캐시 적재, Redo·Undo, 경우에 따른 Row Migration 비용이 쌓인다. 원문은 임시 테이블에 유지할 데이터를 저장 → 제약·인덱스 정리 → TRUNCATE → 변환한 데이터를 INSERT → 제약·인덱스 재생성이라는 대안을 제시한다. DELETE는 남길 행만 임시에 보관한다. 손익분기점은 변경 비율과 인덱스 수·Direct Path 여부 등에 따라 달라지며, 원문의 약 20% 테스트 수치를 보편적인 고정 기준으로 쓰면 안 된다.

조인을 내포한 UPDATE의 병목. 변경값을 구하는 상관 서브쿼리와 대상 존재를 검사하는 EXISTS를 함께 쓰면 참조 테이블을 두 번 접근할 수 있다. 고객과 거래가 많으면 인덱스가 있어도 반복 랜덤 액세스가 커진다. 서브쿼리 Unnesting과 해시 세미 조인으로 일부를 줄일 수 있지만 중복 접근 자체는 남는다. 원문은 제품별 확장 UPDATE를 활용해 조인과 변경을 한 흐름으로 처리하는 방안을 연결한다.

수정 가능 조인 뷰와 키 보존. SQL Server는 FROM JOIN을 포함한 UPDATE를 사용할 수 있고, Oracle은 Updatable Join View를 설명한다. 조인 결과에서도 중복 없이 식별할 수 있는 Key-Preserved Table만 변경할 수 있다. 상대 집합의 UNIQUE 인덱스나 조인 키별 집계가 유일성 근거다. GROUP BY 집합을 키 보존 근거로 인정하는 동작과 BYPASS_UJVC 힌트의 지원 여부는 원문에서 10g·11g·12c로 나누므로 버전 조건을 놓치면 안 된다.

MERGE의 활용. 소스와 대상의 일치 여부를 ON으로 판정해 일치 행 UPDATE와 불일치 행 INSERT를 한 문장으로 처리한다. 원문은 신규·변경분을 DW에 반영하는 용도를 들고, 버전에 따라 UPDATE·INSERT의 선택적 사용과 DELETE 지원을 설명한다. 고객별 거래를 먼저 집계한 소스를 USING에 두고 MATCHED UPDATE만 수행하면 수정 가능 조인 뷰의 대안이 된다. 원문 코드의 대상·소스 집합과 버전별 문법을 유지해서 읽는다.

제3절 데이터베이스 Call 최소화

원문 PDF 700-717쪽

Call의 두 가지 구분. 작업 목적에 따라 Parse Call은 파싱, Execute Call은 실행, Fetch Call은 SELECT 결과 전송 요청이다. 발생 위치에 따라 User Call은 외부 클라이언트 요청이고 Recursive Call은 DBMS 내부 요청이다. 파싱 중 딕셔너리 조회나 함수 안 SQL도 Recursive Call에 포함된다. 외부 왕복은 네트워크와 운영체제 자원 준비 비용까지 포함하므로 호출 수를 줄이는 설계가 개별 쿼리와 전체 시스템 확장성에 영향을 준다.

One SQL의 개선 원리. 원문은 월요금납부실적을 읽어 납입방법별 집계에 건건이 INSERT하는 프로그램을 든다. 행 Fetch와 각 방법의 Parse·Execute가 반복되면 최대 110만 번의 Call이 발생할 수 있다. 같은 일을 데이

터 복제와 조건식을 이용한 INSERT SELECT로 합치면 Parse 1회와 Execute 1회로 줄어든다. 서버 트레이스에 나타나는 실제 SQL 일량이 적은데 프로그램 시간이 길면 네트워크·애플리케이션·자원 할당 대기에서 시간이 소비되는지 살펴야 한다.

집합 처리와 시스템 확장성. 선택한 상품마다 위시리스트 INSERT를 호출하는 대신 여러 상품 번호를 한 번에 받아 처리하면 User Call이 줄어든다. 하나의 화면 기능을 얼마나 잘게 나누어 SQL을 호출하는지, 프레임워크가 Array Processing을 지원하는지, 페이지 처리가 어디에서 이루어지는지가 성능을 좌우한다. 원문은 호출 수를 단순 미세 튜닝 문제가 아니라 애플리케이션 아키텍처의 문제로 다룬다.

Array Processing. 데이터를 배열에 모아 한 번의 INSERT·UPDATE·DELETE 호출에서 여러 레코드를 처리한다. Java 사례는 addBatch로 쌓아 일정 건수마다 executeBatch하고, 조회도 같은 단위로 Fetch한다. PL/SQL 사례는 BULK COLLECT와 LIMIT으로 읽고 FORALL로 입력한다. One SQL로 바꾸기 어려워도 호출 횟수를 줄이면 비슷한 수준의 개선을 기대할 수 있다. 읽기만 배열이고 다음 단계 입력이 한 건씩이면 효과가 크게 줄므로 일련의 처리 단계 전체를 확인해야 한다.

부분범위처리의 원리. DBMS는 결과 전체를 한 번에 보내지 않고 Fetch 요청마다 일정량을 나누어 전송한다. 추가 요청이 없으면 커서를 연 상태로 기다린다. 일부만 보여 주고 멈추는 OLTP 화면에서는 정렬을 피하는 인덱스와 함께 첫 배열을 빠르게 채울 수 있다. 반대로 결과 전체를 파일로 받거나 서버에서 끝까지 가공하는 배치·DW에는 '대상이 많아도 일부만 처리한다'는 이점이 적용되지 않는다.

클라이언트 동작도 확인. 툴이나 프로그램이 첫 출력을 시작하기 전에 전체 결과를 모으면 DBMS의 부분 전송 원리를 활용하지 못한다. 첫 화면 이후에도 백그라운드에서 계속 Fetch하면 사용하지 않는 데이터와 I/O가 발생한다. 원문은 이런 차이가 DBMS의 부분범위처리 지원 여부와 다르다는 점을 강조한다. 커서를 닫고 다음 요청 때 새 쿼리를 실행하는 웹 페이징도 열린 커서의 추가 Fetch와 구분한다.

ArraySize를 크게 할 때와 작게 할 때. 전체 다운로드는 전송 총량을 줄일 수 없으므로 배열을 키워 Fetch Call을 줄인다. 앞 일부만 보고 멈추는 화면에서는 작은 배열로 불필요하게 미리 읽고 전송하는 양을 줄일 수 있다. 배열을 키우면 한 블록을 여러 Fetch가 재방문하는 일도 줄어 논리적 블록 I/O까지 감소할 수 있다. 다만 Fetch 횟수와 I/O가 같은 비율로 계속 감소하는 것은 아니므로 무조건 큰 설정은 메모리 낭비가 된다.

배열 Fetch의 원문 예. 한 블록에 10행씩 세 블록, 총 30행이 있을 때 ArraySize=3이면 블록 경계를 사이에 두고 반복 읽어 Fetch 10회·블록 I/O 12회가 필요하다. ArraySize=10이면 Fetch와 블록 I/O가 각각 3회, 30이면 Fetch가 1회로 줄어든다. Java setFetchSize나 PL/SQL Cursor FOR LOOP의 자동 배열 처리 설명은 원문에 제시된 API·버전 조건을 전제로 한다. rs.next 호출마다 네트워크 Call이 생기는 것은 아니며 배열 버퍼를 소진할 때 추가 요청한다.

서버에서 페이지를 완성하기. 웹 요청마다 앞 행부터 Fetch해 목표 페이지까지 버리는 방식은 뒤쪽으로 갈수록 Call과 전송이 늘어난다. DB 서버에서 필요한 페이지를 제한하고 화면에 보일 행만 가져와야 한다. 페이지 처리는 Fetch Call·네트워크 전송·블록 I/O·AP 및 웹 서버 자원 사용을 함께 줄인다. 실제 사용자가 앞쪽 일부만 볼 때 전체 집합을 모두 보내는 것은 시스템 전체의 낭비다.

분산 쿼리. 원격 조인의 핵심은 네트워크 전송량을 줄이는 것이다. 원문 사례는 원격 sales 50만 건을 로컬에 가져오는 대신, 날짜로 줄인 order 939건을 원격에 보내 그곳에서 조인·집계하고 결과 5건만 받는다. DRIVING_SITE 힌트는 어느 쪽에서 처리할지 유도한다. 작은 원본 테이블만 고르는 판단보다 필터 후 집합과 집계 후 전송량을 함께 보는 것이 중요하다.

사용자 정의 함수의 비용. 원문은 사용자 함수가 별도 실행 엔진을 거치며 Context Switching을 일으켜 내장 함수보다 비싸다고 설명한다. 함수 안에 SQL까지 있으면 읽는 행마다 Execute·Fetch의 Recursive Call이 반복된다. 네트워크를 통과하지 않아도 횟수가 매우 많으면 큰 부하다. 주문일자 검사 함수를 천만 행에 호출하는 대신 일자·휴무일 테이블과 EXISTS·NOT EXISTS로 검사하는 사례가 그 대안이다.

함수·프로시저의 적절한 위치. 함수는 소량 조회나 부분범위처리가 가능한 상황에서 호출을 제한하고, 대량 처리에서는 조인·스칼라 서브쿼리 또는 One SQL로 펼칠 여지를 찾는다. 그렇다고 사용자 정의 프로그램을 무조건 금지한다는 뜻은 아니다. 호출은 적고 내부에 여러 SQL을 수행하는 프로시저는 외부 프로그램의 수많은 User Call을 Recursive Call로 바꿔 성능을 높일 수 있다. 호출 위치와 횟수, 내부 작업을 함께 판단해야 한다.

제4절 파티셔닝

원문 PDF 718-726쪽

파티션의 의미와 목적. 테이블 또는 인덱스의 데이터를 키 기준으로 여러 저장영역에 나누면서 메타정보로 하나의 논리 객체로 관리한다. 관리 측면에서는 파티션별 백업·추가·삭제·변경, 성능 측면에서는 관련 파티션만 조회·변경하고 경합과 부하를 분산하는 이점이 있다. 대량 데이터를 인덱스로 랜덤 접근하는 비용이 큰 경우에도 전체 초대용량 테이블 대신 필요한 세그먼트만 Full Scan할 수 있다.

유형	분할 기준	원문의 용도·조건
Range	키 값의 범위	날짜 기준 월·분기 등 연속 구간
Hash	해시 함수 반환값	고른 분산, 병렬 처리와 DML 경합 분산
List	미리 지정한 불연속 값 목록	지역 등 업무 분류에 따른 분할
Composite	Range·List 안에 하위 파티션 구성	범위 관리와 하위 Hash·List 등의 이점 결합

제품·버전별 구성. 원문은 Oracle의 Range 및 Range+Hash 생성문과 버전별 지원 유형을 제시한다. SQL Server는 원문에서 다루는 2008까지 Range 단일 파티션을 지원하는 것으로 설명한다. SQL Server 생성 절차는 필요에 따라 파일그룹·파일을 구성한 뒤 파티션 함수로 경계를 정의하고, 파티션 구성표로 저장 위치를 지정해 테이블을 연결하는 순서다. 이 지원 범위를 현재 제품 전체에 대한 설명으로 확대하지 않는다.

Partition Pruning. 옵티마이저는 테이블 파티션 구성과 조건절을 분석해 관련 없는 파티션을 대상에서 제외한다. 파티션 성능 개선의 핵심은 단순히 조각이 많다는 데 있지 않고 이 제외가 실제로 일어나는 데 있다. 정적 Pruning은 상수 조건으로 컴파일 때 접근할 파티션을 결정하고, 동적 Pruning은 바인드 값 또는 조인에서 전달된 값으로 실행 때 결정한다. 실행계획의 Pstart·Pstop이 숫자 또는 KEY로 나타나는 차이를 원문에서 확인할 수 있다.

파티션 키를 가공하지 않기. 사용자가 함수를 씌우거나 자료형 불일치로 묵시적 변환이 발생하면 정상적인 Pruning이 막힐 수 있다. 인덱스 키를 가공했을 때 탐색 조건으로 쓰기 어려워지는 것과 같은 관계다. 테이블이 파티셔닝됐다는 사실만 확인하지 말고 실제 조건식과 실행계획에서 어떤 파티션을 읽는지 연결해야 한다.

Local과 Global. Local 파티션 인덱스는 테이블 파티션과 1:1로 대응하고 DBMS가 그 관계를 관리한다. Global은 테이블과 독립적인 파티션 구성을 가진다. 원문은 SQL Server에서 각각 aligned·unaligned라고 부른다고 설명한다. 테이블 파티션 작업에 맞춰 관리하기에는 Local이 유리하며, Global·비파티션 인덱스는 파티션 이동·삭제 등 작업 뒤 UNUSABLE과 재생성·REBUILD 가능성을 고려해야 한다는 것이 원문의 관리 관점이다.

Prefixed와 Nonprefixed. 파티션 키가 인덱스 키의 왼쪽 선두에 있으면 Prefixed, 없거나 선두가 아니면 Nonprefixed다. 이 구분은 Local/Global과 다른 축이다. 원문은 네 조합과 비파티션 인덱스를 구분하고, Oracle은 Global Nonprefixed를 허용하지 않는다고 설명한다. 파티션 인덱스를 논할 때 테이블과의 대응 관계와 인덱스 키 순서라는 두 기준을 섞지 않아야 한다.

성능에 따른 인덱스 선택. 파티션 키 조건이 없으면 여러 인덱스 파티션을 방문해야 하므로 OLTP에서는 비효율이 커질 수 있다. NL 조인의 내부 집합으로 넓은 파티션 범위를 반복 접근하거나, 인덱스 순서로 ORDER BY를 대체해 부분범위처리하려는 경우에는 비파티션 전략을 검토한다. Local Prefixed는 파티션 키 '=' 조건에 유리하고 선두가 범위면 불리할 수 있다. Local Nonprefixed도 파티션 키 조건이 필요하며 좁은 범위검색에서 비교 우위가 있다. 관리 편의성과 조회 경로를 함께 맞춰야 한다.

제5절 대용량 배치 프로그램 튜닝

원문 PDF 727-742쪽

배치의 범위와 성능 목표. 배치는 사용자와 상호작용하지 않고 대량 데이터의 일련 작업을 정기적 또는 규칙에 따라 일괄 수행한다. 정기·이벤트성·On-Demand 배치가 있으며, 원문은 일·월 단위에서 시간·분 단위와 근실시간 요구로 주기가 짧아지는 변화를 설명한다. 온라인과 경계가 흐려져도 배치는 첫 일부 결과가 아니라 전체 처리 완료를 최적화 목표로 삼아야 한다.

전체 배치 윈도우. 개별 SQL을 빠르게 해도 전체 작업 완료시간이 줄지 않으면 목적을 달성한 것이 아니다. 여러 팀의 작업이 동시에 몰리면 CPU·메모리·디스크와 Lock·Latch 경합으로 실제 일한 시간보다 대기가 커질 수 있다. 원문은 DOP 32로 5분 걸리는 프로그램을 직렬 10분으로 수행하는 편이 전체 자원 배분에 유리할 수 있음을 든다. 시작 시각과 동시 작업 수를 배치 윈도우로 조절하는 것도 중요한 튜닝이다.

구현 패턴별 접근. 절차형 루프 프로그램은 반복 Call·Random I/O·동일 데이터 중복 접근이 생기므로 병목 SQL, 인덱스·접근 경로, 범위를 나눈 프로그램 동시 실행, Array Processing을 검토한다. One SQL 위주 구조는 대량에 맞는 Full Scan·해시 조인·임시 테이블·파티션·병렬 처리를 고려한다. 복잡한 업무를 무리하게 통합해 결과를 바꾸면 안 되며, 통합이 어렵다면 기존 구조에서 반복 비용을 줄인다.

병렬 처리의 기본. 작업 범위를 작은 단위로 나누어 여러 프로세스·스레드가 동시에 처리한다. Oracle 사례는 FULL과 PARALLEL, 인덱스 경로에서는 INDEX 또는 INDEX_FFS와 PARALLEL_INDEX를 함께 지정해 의도한 접근과 병렬화가 맞게 한다. SQL Server의 원문 사례는 MAXDOP 힌트를 사용한다. 단순히 병렬 힌트 하나가 모든 접근 경로에 같은 의미로 적용된다고 생각해서는 안 된다.

Query Coordinator와 서버 집합. QC는 병렬 SQL을 발행한 세션으로 필요한 병렬 서버를 확보해 작업을 할당하고 진행을 관리한다. 병렬 지시가 없는 테이블은 직접 처리하기도 하며, 병렬 서버의 집계 결과를 최종 통합하고 사용자에게 전송한다. 원문은 출력 단계의 스칼라 서브쿼리도 QC가 수행한다고 설명한다. 실제 병렬 작업은 한 개 또는 두 개의 서버 집합이 분담한다.

Intra와 Inter-Operation Parallelism. 같은 서버 집합의 프로세스가 배타적인 범위를 독립 처리하는 것은 Intra-Operation이며 그 집합 내부 통신은 없다. 생산자 집합이 읽은 데이터를 소비자 집합에 분배하거나 QC에 보내는 단계는 Inter-Operation이며 프로세스 간 통신이 발생한다. 읽기와 정렬이 각각 병렬 집합을 필요로 하면 지정 DOP의 두 배 서버가 사용될 수 있다. DOP를 실제 총 세션 수와 같다고 보지 않는다.

테이블 큐와 실행계획. Table Queue는 서버 집합 사이 또는 QC와 서버 집합 사이 데이터 파이프라인이며 : TQ 형식 식별자로 표시된다. 생산자는 PX SEND, 소비자는 PX RECEIVE, TQ 칼럼은 각 오퍼레이션을 수행하는 서버 집합을 보여 준다. 원문의 DOP=2 예시는 두 집합의 서버 네 개와 집합 사이 2^2개 파이프라인을 설명한다. 최종 SELECT 결과의 소비자는 QC다.

IN-OUT 표시	의미	통신·해석
S→P	QC의 직렬 결과를 병렬 서버로 전달	프로세스 간 통신; 직렬 오퍼레이션
P→S	병렬 결과를 QC로 전달	프로세스 간 통신
P→P	병렬 서버 집합 사이 전달	두 집합과 데이터 재분배
PCWP	현재 단계와 부모를 같은 서버 집합에서 처리	단계 사이 별도 프로세스 통신 없음
PCWC	현재 단계와 자식을 같은 서버 집합에서 처리	단계 사이 별도 프로세스 통신 없음

데이터 재분배 방식. RANGE는 정렬 키의 구간으로 소비자를 나누어 ORDER BY·GROUP BY를 처리한다. HASH는 조인 키나 그룹 키의 해시값으로 같은 키가 같은 처리자에게 가게 한다. BROADCAST는 읽은 집합을 반대쪽 모든 서버에 보내 작은 집합 조인에 쓴다. KEY는 파티셔닝 기준을 이용하며 ROUND-ROBIN은 정렬·해시 키에 의존하지 않고 분산한다. 이 분배 비용과 균형이 병렬 성능에 중요하다.

PQ_DISTRIBUTE의 역할. 이 힌트는 조인에 앞선 데이터 분배 방식만 지정한다. HASH로 분배했다고 해시 조인으로 수행된다는 뜻은 아니며 원문은 HASH 분배 후 소트 머지 조인하는 계획을 보인다. 통계가 부정확하거나 기존 파티션을 제대로 활용하지 못할 때, 필요하면 사용자가 분배를 조정할 수 있다. 대상은 inner 테이블 별칭과 outer·inner의 분배 방식이다.

Partition-Wise Join. 양쪽이 같은 조인 키와 기준으로 미리 파티셔닝돼 있으면 같은 파티션 짝끼리 독립 조인하는 Full Partition-Wise Join이 가능하며, PQ_DISTRIBUTE(inner, none, none)으로 유도한다. 한쪽만 있다면 반대쪽을 그 기준으로 동적 분할하는 Partial 방식이고 partition/none 또는 none/partition을 사용한다. 양쪽을 hash/hash로 재분할할 수도 있다. broadcast/none과 none/broadcast는 어느 집합을 복제 전송할지 지정한다.

통계 오차와 Broadcast 위험. 원문은 실제 천만 건인 임시 테이블을 통계 부재로 5,248건으로 보아 DOP 32의 모든 서버로 Broadcast한 사례를 든다. 과도한 통신과 Temp 소모로 장시간 후 실패했으며, PQ_DISTRIBUTE로 분배를 바꾸고 DOP를 낮춰 완료했다. 병렬 성능은 프로세스 수보다 데이터 크기 추정과 전송 구조에 좌우될 수 있다는 사례다.

적정 병렬도의 조건. 동시 사용자가 적고 직렬 대비 개선이 확실한 대량 작업에서 유용하며 작은 테이블은 대상에서 제외한다. OLTP라도 수행빈도가 낮고 빨리 끝내 전체 자원을 오히려 줄일 때만 고려한다. 병렬도를 높여도 선형 개선은 보장되지 않으므로 효과가 확실한 최소 DOP를 찾고 전체 배치 목표를 맞춘다. 반대로 다른 업무를 멈춘 데이터 이행은 시스템 자원을 최대 활용하는 목표가 맞을 수 있다. 원문은 병렬 DML의 Exclusive 테이블 Lock도 주간 트랜잭션과 충돌하는 제약으로 강조한다.

제6절 고급 SQL 활용

원문 PDF 743-758쪽

CASE를 이용한 조건부 집계. 납입방법별 행을 지로·자동이체·카드 등의 칼럼으로 바꿀 때 같은 집계 테이블을 방법마다 읽고 외부 조인하면 One SQL이어도 중복 액세스가 많다. CASE로 각 방법의 금액만 선택하고 SUM하면 한 번 읽어 고객·월별로 모을 수 있다. NVL은 해당 방법 데이터가 없을 때 결과를 0으로 바꾼다. 원문은 PIVOT 지원도 언급하지만 복잡한 IF·ELSE 처리를 집합으로 표현하는 기초로 CASE·DECODE를 강조한다.

데이터 복제 기법. 번호를 가진 복제용 테이블과 조인 조건 없이 Cross Join하면 원본 행을 번호 개수만큼 복제할 수 있다. Oracle의 DUAL과 CONNECT BY LEVEL도 작은 번호 집합을 만든다. 원문은 상세 집계에 소계가 필요할 때 두 배 복제 후 번호에 따라 고객등급 또는 '소계'를 선택해 한 번 더 집계하는 방법을 보인다. 복제의 목적과 배수를 통제하지 않으면 카티션 곱은 단순한 중복 폭증이 된다.

M:M 집합의 결합. 부서별 판매계획과 채널별 판매실적을 상품·연월로 직접 조인하면 양쪽 모두 여러 행이라 계획과 실적이 서로 곱해진다. 먼저 각 집합을 상품·연월로 집계하면 1:1이 되어 Full Outer Join이 가능하지만 원문은 일부 제품·버전에서 테이블을 반복 읽는 계획을 보인다. 대안은 계획 분기의 실적 칼럼을 숫자 NULL, 실적 분기의 계획 칼럼을 숫자 NULL로 맞춰 UNION ALL한 뒤 상품·연월별 합계를 구하는 방식이다.

앞쪽 페이지 중심 조회. 원문의 표준 패턴은 정렬된 내부 집합을 ROWNUM으로 페이지 끝+1건까지만 읽고, 바깥에서 페이지 시작~끝 번호를 남긴다. COUNT(*) OVER()는 이 제한된 집합의 건수를 구해 다음 페이지 존재를 판정한다. 페이지 크기 10, 3페이지라면 31행만 읽고 21~30행을 출력하므로 전체 건수를 구할 필요가 없다. [종목코드+거래일시] 인덱스가 있으면 소트를 피하고 첫 페이지를 특히 빠르게 얻는다.

뒤쪽 페이지까지 자주 이동할 때. 매번 처음부터 현재 페이지까지 읽는 패턴은 뒤로 갈수록 이전 행을 반복한다. 원문은 다음 페이지에 이전 화면의 마지막 거래일시를 바인딩해 그 뒤에서 읽고, 이전 페이지는 첫 거래일시 이하를 역순으로 읽어 제한한 뒤 소량만 다시 정렬하는 방법을 설명한다. 페이지 크기보다 한 건 더 읽어 다음 데이터 유무를 판단한다. 정렬을 힌트와 인덱스의 우연한 순서에만 의존하면 인덱스 변경으로 결과가 달라질 수 있어 ORDER BY로 의미를 보장한다.

페이지 처리의 분기 통합. 조회·다음·이전 버튼에 따라 별도 SQL을 호출할 수 있고, UNION ALL의 각 분기에 이동 방향 조건을 두어 하나의 SQL로 만들 수도 있다. 핵심은 서버에서 필요한 작은 결과를 만들고 인덱스의 위치 탐색으로 이미 본 구간을 되읽지 않는 것이다. 원문은 사용자 요구·연결 유지 방식·인덱스 구성에 따라 구현을 응용하도록 한다.

윈도우 함수로 최근 상태 이어 쓰기. 상태가 바뀔 때만 상태코드를 기록한 장비측정 데이터에서 NULL인 행에 가장 최근 상태를 표시해야 한다. 앞쪽 일부만 조회하면 각 행에서 인덱스 역순으로 최근 비NULL을 찾는 스칼라 방식이 유리할 수 있다. 전체를 출력할 때는 LAST_VALUE(상태코드 IGNORE NULLS)와 처음부터 현재 행까지의 ROWS 프레임으로 한 흐름에서 최근 상태를 전달한다. 부분 조회와 전체 조회의 목표 차이에 따라 형태를 선택한다.

WITH의 두 실행 방식. Materialize는 공통 집합을 내부 임시 테이블에 만들어 반복 재사용하고, Inline은 물리적 임시 저장 없이 참조 횟수만큼 수행하는 방식이다. WITH로 이름을 정의했다는 사실만으로 한 번 계산해 재사용한다고 단정할 수 없다. 원문 기준 Oracle은 두 방식과 MATERIALIZE·INLINE 힌트를 설명하고 SQL Server는 Inline 방식을 설명한다. 실제 버전별 동작을 전제로 구분해야 한다.

WITH의 수명과 대량 사례. Materialize의 임시 데이터는 영구 객체가 아니라 해당 SQL 실행 동안만 유지된다. 여러 WITH 집합을 선언하고 다른 WITH 집합을 참조할 수 있다. 원문의 위험고객카드 사례는 2천만 건 이상의 고객과 1억 건 이상의 카드를 반복 읽는 대신, 작은 위험고객카드 집합을 임시로 만들어 여러 거래 집계에서 재사용한다. 실행계획의 TEMP TABLE TRANSFORMATION·LOAD AS SELECT·임시 테이블 재접근으로 이를 확인한다. 임시 저장 자체의 비용과 반복 접근 절감 효과를 함께 보아야 한다.

반드시 기억할 핵심 포인트

- 소트 연산을 없애는 최적화와 소트가 남아도 메모리·비교 횟수를 줄이는 최적화를 구분한다.
- 인덱스 정렬과 명시적 ORDER BY의 역할을 구분하고, 힌트만으로 결과 순서를 보장하지 않는다.
- 대량 적재·변경은 인덱스 유지·버퍼·로그 비용을 함께 줄이되 Lock과 복구 조건을 지킨다.
- Call을 줄이려면 Fetch부터 후속 DML까지 집합 또는 배열 처리로 연결해야 한다.
- 파티션 성능은 Pruning, 병렬 성능은 작업 분할과 통신량에 달려 있다.
- 배치는 개별 첫 응답보다 전체 작업 완료와 자원 배분을 최적화한다.

챕터 최종 요약

- 소트 튜닝은 모델·불필요 연산을 먼저 보고 인덱스 순서, Top N, 행 폭 축소, 메모리 조정을 차례로 연결한다.
- DML은 테이블 한 번 변경으로 끝나지 않는다. 인덱스·Undo·Redo·캐시·조인 비용을 고려해 Direct Path, 재적재, 조인 UPDATE·MERGE를 선택한다.
- One SQL과 Array Processing은 반복 데이터베이스 Call을 줄인다. 부분범위처리·적정 FetchSize·서버 페이징은 사용하지 않을 결과의 접근·전송까지 줄인다.
- 파티션은 관련 저장영역만 고르고, 병렬은 독립 처리 범위를 나눈다. Local/Global·키 순서·Pruning·재분배·QC를 연결해 계획을 읽어야 한다.
- CASE·복제·UNION ALL·윈도우·WITH는 복잡한 절차를 집합으로 바꾸는 수단이다. 같은 SQL 한 문장 안에서도 중복 읽기와 결과 의미를 별도로 검증한다.

제7장 Lock과 트랜잭션 동시성 제어

과목 III. SQL 고급 활용 및 튜닝

원문 PDF 764-786쪽

제1절 Lock

원문 PDF 765-772쪽

Lock의 목적. 같은 좌석을 두 사람이 동시에 예약해도 정확히 한 사람에게만 배정하려면, 여러 트랜잭션의 충돌하는 작업을 순서대로 실행하게 해야 한다. Lock은 이 직렬화 장치로 데이터 일관성과 무결성을 보호한다. DBMS가 자동으로 필요한 Lock을 설정하고 일부는 사용자가 제어할 수 있지만, 구체적인 방식과 기능은 DBMS마다 다르다.

공유와 배타. 공유(S) Lock은 읽기에 사용하며 다른 공유 Lock과 함께 설정할 수 있다. 배타(X) Lock은 변경에 사용하고 트랜잭션이 완료될 때까지 유지하며 같은 자원의 공유·배타 Lock과 호환되지 않는다. 이 설명은 일반적인 Lock 기반 읽기를 먼저 이해하기 위한 것이다. 뒤의 Oracle 일반 SELECT처럼 공유 Lock 없이 과거 버전을 읽는 경우는 별도로 구분한다.

블로킹. 필요한 Lock이 선행 트랜잭션의 Lock과 호환되지 않아 후행 세션이 진행하지 못하는 상태다. 공유끼리는 호환되지만 공유·배타 또는 배타끼리는 충돌할 수 있다. 선행 트랜잭션이 COMMIT·ROLLBACK으로 끝나야 해당 Lock이 해제된다. 실행 중인 SQL 자체를 빠르게 하고, 원자성을 훼손하지 않는 범위에서 트랜잭션을 짧게 하며, 같은 데이터를 동시에 대량 갱신하지 않도록 설계하는 것이 기본이다.

대기와 교착상태의 구분. 블로킹은 앞 작업이 끝나면 풀릴 수 있지만, 교착상태는 세션들이 서로 상대가 잡은 자원을 기다려 스스로 진행할 수 없는 순환 대기다. DBMS는 한쪽에 오류를 발생시켜 순환을 끊는다. 여러 테이블을 변경할 때 모든 프로그램이 같은 순서로 접근하게 하면 관련 교착을 예방할 수 있다. 불필요한 격리 수준 상승은 Lock 유지와 충돌을 늘리므로 해법으로 단정하지 않는다.

대기시간을 통제하는 원문 방법. SQL Server는 LOCK_TIMEOUT으로 대기 상한을 설정할 수 있다. Oracle은 UPDATE·DELETE 전에 SELECT FOR UPDATE NOWAIT 또는 WAIT n을 수행해 잠금을 시도하고, 실패·시간초과 예외에 따라 처리할 수 있다. 이는 원자성을 잘못 나누는 대안이 아니라 사용자가 무한정 기다리지 않도록 하는 제어다. 원문은 SQL 성능과 트랜잭션 설계를 먼저 개선할 것을 강조한다.

SQL Server의 공유 Lock 유지. 원문에서 설명하는 기본 Read Committed의 Lock 기반 읽기는 다음 레코드로 이동하면 공유 Lock을 해제한다. 나중에 계산 결과로 변경할 데이터는 읽은 뒤 값이 바뀌면 비일관한 갱신이 될 수 있으므로 HOLDLOCK으로 유지하는 사례를 제시한다. 단, 두 세션이 동시에 S를 얻고 X로 바꾸려 하면 서로의 S 해제를 기다리는 전환 교착이 생길 수 있다.

Update Lock. SQL Server의 UPDLOCK은 나중에 변경할 목적으로 읽을 때 사용하는 U 모드다. 같은 자원에 U는 한 트랜잭션만 얻을 수 있어 두 변경자가 S를 함께 얻는 전환 교착 가능성을 줄인다. U는 다른 U와는 호환되지 않지만 S와는 호환되므로 단순 읽기는 가능하다. 변경 시 배타 모드로 전환하며 트랜잭션 완료까지 유지하는 흐름을 원문 고객 적립포인트 사례에서 확인할 수 있다.

Intent·Schema·Bulk Update Lock. 하위 행에 Lock을 설정하면 상위 개체에 Intent Lock을 두어 아래에서 어떤 작업이 진행 중인지 알린다. 상위 개체를 잠글 때 모든 하위 Lock을 일일이 검사하지 않게 하는 표지다. Sch-S는 컴파일에서 스키마 안정성을, Sch-M은 DDL의 스키마 변경을 보호한다. 원문은 Sch-S가 Sch-M 이외와 호환되고 Sch-M은 다른 Lock과 호환되지 않는다고 설명한다. Bulk Update는 병렬 Bulk 로딩을 허용하면서 일반 트랜잭션은 제한하는 테이블 Lock이다.

Lock 레벨과 Escalation. 원문은 행·페이지·익스텐트·테이블·데이터베이스 및 인덱스 키 단위를 구분한다. 낮은 레벨은 동시성에 유리하지만 관리할 Lock 수와 자원이 늘고, 높은 레벨은 적은 관리 자원으로 넓은 범위를 잠가 동시성을 낮춘다. 원문은 한정된 메모리의 Lock 매니저가 임계치를 넘을 때 더 큰 범위의 Lock으로 확장하는 현상을 Lock Escalation으로 설명한다. 이 서술은 원문의 DBMS·버전 설명 범위로 읽는다.

Oracle의 로우 Lock. INSERT·UPDATE·DELETE와 SELECT FOR UPDATE로 얻는 로우 Lock은 배타적이며 COMMIT·ROLLBACK까지 다른 변경을 막는다. 일반 SELECT는 읽은 행에 공유 Lock을 설정하지 않는다. Undo를 이용해 일관된 과거 버전을 읽으므로 변경 중인 행을 읽는 쪽은 기다리지 않고, 일반 읽기 때문에 변경이 기다리지도 않는다. 단, SELECT FOR UPDATE의 명시적 잠금과 변경자끼리의 경합은 예외로 구분해야 한다.

Oracle의 테이블 Lock. 로우 Lock을 얻을 때 테이블 구조 변경을 막는 테이블 Lock도 함께 얻는다. 원문은 RS·RX·S·SRX·X 모드를 제시하며 SELECT FOR UPDATE는 RS, 일반 DML은 RX를 얻는 대표 사례다. RX끼리 호환되므로 테이블 Lock이 있다는 이유만으로 다른 세션의 모든 DML이 막히는 것은 아니다. 현재 작업을 알리는 표지라는 의미와 배타적으로 테이블 전체를 잠그는 모드를 구분해야 한다.

구현 차이의 결과. 원문은 Oracle의 로우 Lock을 별도 Lock 매니저의 메모리 엔트리 대신 레코드 속성으로 구현하므로 많은 행을 변경해도 Lock Escalation이 일어나지 않는다고 설명한다. 그러나 변경자끼리의 Lock은 여전히 존재하므로 트랜잭션을 불필요하게 길게 유지해도 된다는 뜻은 아니다. 제품별 차이를 성능·동시성 설계와 연결해서 이해해야 한다.

제2절 트랜잭션

원문 PDF 773-777쪽

논리적 작업 단위. 트랜잭션은 업무 처리를 위한 최소 논리 단위이며 하나의 연산으로만 이루어지지 않을 수 있다. 계좌이체는 출금과 입금이 함께 완료되거나 모두 취소돼야 한다. DBMS는 여러 갱신을 물리적으로 같은 순간에 실행하는 대신, 전체 성공 또는 전체 취소인 All or Nothing으로 이 업무 의미를 보장한다.

ACID	의미	구분할 핵심
Atomicity 원자성	전부 처리되거나 하나도 처리되지 않음	업무의 분해 불가능한 단위
Consistency 일관성	성공 후에도 모순 없는 상태 유지	업무 규칙·무결성 결과
Isolation 격리성	실행 중간 결과에 다른 트랜잭션이 접근하지 못함	동시 작업 간 간섭 제어
Durability 영속성	성공 완료 결과를 영속 저장	완료한 결과의 보존

Dirty Read. 다른 트랜잭션이 바꿨지만 아직 커밋하지 않은 값을 읽는 현상이다. 그 변경이 롤백되면 실제 확정되지 않은 값에 근거해 처리한 트랜잭션이 비일관한 상태가 된다. Read Committed는 커밋된 데이터만 읽게 하므로 이를 막지만, 그것만으로 같은 트랜잭션의 모든 읽기가 같은 결과라는 뜻은 아니다.

Non-Repeatable Read. 한 트랜잭션이 같은 쿼리를 두 번 수행하는 사이 다른 트랜잭션이 기존 행을 수정·삭제해 결과가 달라진다. 원문의 잔고 확인 후 인출 사례는 앞서 충분했던 잔액이 다른 거래로 줄어 인출 시 부족해지는 상황을 보여 준다. 확인 시점의 행을 이후 갱신에서도 전제로 써야 한다면 그 사이 변경 가능성을 제어해야 한다.

Phantom Read. 처음 없던 행이 다른 트랜잭션의 INSERT 때문에 다음 조회에 새로 나타나는 현상이다. 원문은 지역별 고객과 연령별 고객을 차례로 집계하는 사이 신규 고객이 등록돼 두 집계의 총수가 달라지는 사례를 든다. 기존에 읽은 행의 값을 유지하는 것과, 검색 조건에 해당하는 새 행까지 막는 것은 다른 수준의 요구다.

원문의 격리 수준	Dirty Read	Non-Repeatable Read	Phantom Read
Read Uncommitted	가능	가능	가능
Read Committed	불가	가능	가능
Repeatable Read	불가	불가	가능

원문의 격리 수준	Dirty Read	Non-Repeatable Read	Phantom Read
Serializable Read	불가	불가	불가

격리 수준은 허용 현상의 기준이다. 원문은 SQL92의 네 수준을 비교하고 모든 DBMS가 모두 지원하는 것은 아니라고 설명한다. 원문 기준 SQL Server·DB2는 네 수준, Oracle은 Read Committed와 Serializable을 지원하며, 반복 읽기 보호에는 FOR UPDATE를 활용한다. 각 수준의 이름을 특정 Lock 구현과 무조건 같은 뜻으로 읽지 말고, 어떤 읽기 현상이 허용되는지를 먼저 확인한다.

격리성과 동시성. 원문의 일반 Lock 방식은 Repeatable Read 등에서 공유 Lock을 트랜잭션 끝까지 유지해 이후 변경을 대기시킨다. 대량 집합에서 이를 오래 유지하면 동시성이 떨어지고 교착 가능성도 높아져 심하면 테이블 수준 잠금이 필요할 수 있다. 반면 Snapshot 방식은 필요한 시점의 과거 버전을 읽어 변경자와 읽기 간 간섭을 줄인다. 이 관계가 다음 절의 MVCC로 이어진다.

제3절 동시성 제어

원문 PDF 778-784쪽

목표와 업무 경계. 동시성 제어는 다중 트랜잭션의 상호 간섭에서 데이터베이스를 보호하는 것이다. Lock을 적게 쓰면 일관성 유지가 어려워지고, 강하게 오래 쓰면 동시에 진행할 작업이 줄어드는 관계가 있다. 목표는 무결성을 지키면서 동시에 실행할 트랜잭션 수를 늘리는 것이다. SET TRANSACTION만으로 모든 업무 문제를 해결할 수는 없다. n-Tier 환경에서는 조회와 갱신이 서로 다른 연결로 실행돼 논리적 업무가 DB 트랜잭션 하나와 일치하지 않을 수 있다.

비관적 동시성 제어. 같은 데이터를 동시에 변경할 수 있다고 가정하고 읽을 때 미리 Lock을 걸어 완료까지 유지한다. Oracle SELECT FOR UPDATE, SQL Server의 HOLDLOCK·UPDLOCK이 원문 수단이다. 읽고 계산하는 동안 다른 변경을 막아 일관된 갱신을 할 수 있지만 사용자 대기와 긴 잠금이 생길 수 있다. 원문은 WAIT·NOWAIT 등과 예외 처리를 함께 써 불필요한 무한 대기를 피하도록 한다.

낙관적 동시성 제어. 동시에 같은 데이터를 바꾸는 일이 적다고 가정해 읽을 때는 잠그지 않는다. 대신 UPDATE 시 처음 읽은 값들이 그대로인지 조건으로 비교하고, 바뀌어 변경 행 수가 0이면 충돌을 알린다. 변경일시를 관리한다면 그 값을 함께 읽고 UPDATE 조건에서 같음을 비교하는 형태로 단순화할 수 있다. '읽을 때 Lock이 없다'는 특징만 기억하고 변경 검사까지 생략하면 원문의 낙관적 제어가 아니다.

일반 Lock 기반 읽기의 한계. 원문은 각 잔고 1,000원인 열 계좌를 합산하는 동안 다른 트랜잭션이 계좌 3에서 100원을 빼 계좌 7에 더하는 사례를 든다. 합산 쿼리가 이미 읽은 계좌 3의 과거 값과 나중에 읽는 계좌 7의 변경 후 값을 섞으면, 실제 전후 총액은 모두 10,000원인데 10,100원으로 계산할 수 있다. 커밋된 값만 읽더라도 단일 시점의 합계가 보장되지 않는 읽기 방식의 문제다.

잠금을 강화할 때의 대가. 공유 Lock을 끝까지 유지하면 앞서 읽은 값은 보호되지만, 합산자는 계좌 3을 잠그고 이체자는 계좌 7을 잠근 상태에서 서로 상대 행을 기다릴 수 있다. 원문은 같은 사례를 통해 격리성 상승이 일관성을 높이면서 동시성 저하와 교착 위험을 만든다는 점을 설명한다. MVCC는 이 읽기·쓰기 간섭을 과거 버전으로 해소하는 다른 축이다.

MVCC의 처리 과정. 변경할 때 이전 상태를 복원할 정보를 Undo에 남긴다. 읽는 중 쿼리 또는 트랜잭션 시작 이후 바뀐 값을 만나면, 아직 미완료인 변경뿐 아니라 이미 커밋한 변경도 기준 시점에 맞게 되돌려 CR Copy를 생성하고 그 버전을 읽는다. 배타 Lock의 해제를 기다릴 필요 없이 기준 시점 값을 볼 수 있어 읽기와 쓰기의 간섭을 줄이면서 일관성을 높인다.

버전 저장 비용. MVCC에도 Undo 블록 I/O, CR Copy 생성, CR 블록 캐싱이라는 추가 작업이 있다. 원문은 Oracle의 Undo 세그먼트와 SQL Server의 tempdb를 버전 정보 저장 위치로 설명한다. 이러한 방식은 변경자끼리의 충돌까지 없애 주는 것은 아니며, 일반 SELECT와 실제 갱신을 구분해서 이해해야 한다.

문장 수준 읽기 일관성. 다른 트랜잭션의 삽입·수정·삭제가 진행돼도 하나의 SQL 안에서는 쿼리 시작 시점을 기준으로 읽는 것이다. 기본 Read Committed의 MVCC 설명은 이 범위의 일관성을 제공한다. 같은 트랜잭션

에서 다음 SQL을 실행하면 새 문장 기준이 될 수 있으므로, 문장 수준이 곧 트랜잭션 전체의 동일 스냅샷은아니다. 원문은 SQL Server의 READ_COMMITTED_SNAPSHOT 설정을 관련 수단으로 제시한다.

트랜잭션 수준 읽기 일관성. 여러 SQL을 포함한 트랜잭션 전체가 시작 시점을 공통 기준으로 읽는 것이다. 원문은 Oracle의 Serializable, SQL Server의 ALLOW_SNAPSHOT_ISOLATION과 SNAPSHOT 격리 설정을 설명한다. 자신이 해당 트랜잭션에서 만든 변경은 그대로 읽는다는 예외도 함께 제시한다. 현재 값만 읽는지, 문장 시작인지, 트랜잭션 시작인지 기준 시점을 명확히 구분해야 한다.

Snapshot too old. 필요한 Undo 정보가 다른 트랜잭션에 의해 재사용돼 과거 시점의 CR Copy를 만들지 못하면 발생한다. 원문은 블록 클린아웃 실패라는 추가 조건도 언급하되 세부 설명은 생략한다. 오래 실행하는 쿼리와 잦은 변경이 겹치면 필요한 버전을 오랫동안 보관해야 하는 관계가 있으므로, 단순히 커밋을 더 자주 하는 것을 해결책으로 삼으면 안 된다.

- 원문의 완화책 1: Undo 영역을 늘리고 불필요하게 잦은 커밋을 줄인다. 열린 커서를 커밋 전후로 계속 Fetch하는 fetch across commit 구조를 피하거나 커밋 횟수를 줄인다.
- 원문의 완화책 2: 트랜잭션 집중 시간과 긴 쿼리 수행 시간을 분리한다. 큰 테이블을 범위별 단계로 처리하는 방법은 읽기 일관성에 문제가 없는 경우에만 적용한다.
- 원문의 완화책 3: 오랜 시간 같은 블록을 재방문하는 NL 조인·인덱스 접근을 점검하고 조인 방식 변경이나 Full Scan을 검토한다.
- 원문의 완화책 4: 경우에 따라 ORDER BY로 소트를 유도하거나, 대량 UPDATE 직후 테이블·인덱스를 Full Scan하는 방법을 고려한다. 원문은 이를 상황별 대안으로 제시한다.

반드시 기억할 핵심 포인트

- 공유·배타 호환성, 대기와 순환 대기를 구분한다.
- 잠금 시간을 줄여도 업무 원자성을 훼손하면 안 된다.
- SQL Server U는 S와 호환되고 U와는 호환되지 않아 변경자 간 선점을 조정한다.
- Oracle의 일반 SELECT는 공유 로우 Lock 없이 과거 버전을 읽지만 변경자끼리는 기다린다.
- 격리 수준·비관/낙관 제어·MVCC 기준 시점은 서로 다른 구분이다.
- 낙관적 제어는 변경 검사가 필수이고, MVCC는 필요한 Undo가 유지돼야 한다.

챕터 최종 요약

- Lock은 다중 트랜잭션의 충돌 작업을 직렬화해 무결성을 지킨다. 빠른 SQL·짧은 트랜잭션·일정한 접근 순서가 대기와 교착을 줄이는 기반이다.
- 트랜잭션은 ACID를 가진 업무 단위다. Dirty·Non-Repeatable·Phantom Read를 구분하고 실제 요구에 필요한 격리 수준을 선택해야 한다.
- 비관적 제어는 먼저 잠그고, 낙관적 제어는 변경 직전에 처음 읽은 상태와 비교한다. 둘 다 논리적 업무의 충돌을 처리하는 방법이다.
- MVCC는 Undo로 기준 시점의 값을 만들어 읽는다. 문장 수준은 쿼리 시작, 트랜잭션 수준은 트랜잭션 시작을 기준으로 하며, Undo가 사라지면 Snapshot too old가 발생할 수 있다.

3. SQL 문법과 원문 코드 해설

다음 SQL은 원문에서 옮긴 코드입니다. 긴 줄의 시각적 줄바꿈과 들여쓰기를 정리했으며, 원문에 확인되지 않은 실행 결과는 덧붙이지 않았습니다.

제1장 데이터 모델링의 이해

과목 I. 데이터 모델링의 이해

원문 PDF 9-76쪽

식별자 상속으로 하위 테이블에서 직접 조회

원문 PDF 71쪽

SQL 원문

```
SELECT A.기본가중치, A.조정가중치
FROM REP007T A
WHERE A.점검번호 = '301'
```

수행하는 작업

점검번호가 하위 테이블까지 상속된 모델에서 해당 점검의 가중치를 조회한다.

주요 키워드와 구문의 역할

키워드·구문	역할
SELECT	기본가중치와 조정가중치를 출력한다.
WHERE A.점검번호	상위 테이블을 거치지 않고 하위 테이블의 상속 속성에 조건을 준다.

원문에 따른 처리 과정

1. 비식별자관계만 사용한 원문 비교 모델은 점검번호를 얻기 위해 REP005T와 REP001T까지 연결한다.
2. 식별자관계로 점검번호가 전달된 비교 모델은 REP007T에 조건을 바로 적용한다.

원문 결과·설명

원문은 결과 행·수치를 제시하지 않고 SQL의 길이, 불필요한 조인, 성능과 개발 용이성 차이를 설명한다.

반드시 이해할 부분

- 관계 선택은 ERD 모양뿐 아니라 조회 SQL 구조를 바꾼다.
- 상위 기준 속성을 자식에서 직접 쓸 필요가 있는지 고려한다.

제2장 데이터 모델과 SQL

과목 I. 데이터 모델링의 이해

원문 PDF 78-118쪽

정규화 모델에서 고객의 최근 신용카드 조회

원문 PDF 87쪽

SQL 원문

```
SELECT A.결제수단번호
FROM (
    SELECT B.결제수단번호
    FROM 주문 A, 결제 B
    WHERE A.주문번호 = B.주문번호
      AND A.고객번호 = 1234
      AND B.결제수단구분코드 = '신용카드'
    ORDER BY B.결제일시 DESC
) A
WHERE ROWNUM = 1;
```

수행하는 작업

고객 1234의 신용카드 결제 중 최근 한 건의 결제수단번호를 구한다.

주요 키워드와 구문의 역할

키워드·구문	역할
주문번호 조인	고객의 주문과 결제를 연결한다.
ORDER BY … DESC	결제일시가 최근인 순서로 정렬한다.
ROWNUM = 1	정렬한 인라인 뷰에서 한 건을 선택한다.

원문에 따른 처리 과정

1. 고객의 주문을 찾고 결제 데이터와 조인한다.
2. 신용카드 결제를 결제일시 내림차순으로 정렬하고 바깥에서 최근 한 건을 가져온다.
3. 원문은 주문이 많을수록 최종 출력이 한 건이어도 조인·읽기·정렬 부하가 증가한다고 설명한다.

원문 결과·설명

원문은 실제 결제수단번호 결과값을 제시하지 않고 처리 구조의 성능 문제를 설명한다.

반드시 이해할 부분

- 최종 출력 건수와 내부에서 처리하는 건수는 다를 수 있다.
- 정규화 모델에서 고객번호가 어느 엔터티에 존재하는지 확인한다.

고객번호 반정규화 후 같은 조회

원문 PDF 88쪽

SQL 원문

```
SELECT A.결제수단번호
FROM (
    SELECT A.결제수단번호
    FROM 결제 A
    WHERE A.고객번호 = 1234
      AND A.결제수단구분코드 = '신용카드'
    ORDER BY A.결제일시 DESC
) A
WHERE ROWNUM = 1;
```

수행하는 작업

고객번호를 결제에 중복 보유시킨 모델에서 주문 조인을 제거한다.

주요 키워드와 구문의 역할

키워드·구문	역할
결제 A	고객 조건을 결제에 직접 적용한다.
고객번호+결제수단구분코드+결제일시	원문이 제시한 인덱스 구성이다.

원문에 따른 처리 과정

1. 결제에 고객번호가 있으므로 주문과 연결할 필요가 없다.

2. 제시한 인덱스에서 Index Range Scan Descending으로 최근 한 건만 읽는 접근을 설명한다.

원문 결과·설명

구체적인 번호나 측정 시간은 제시되지 않는다. 원문은 해당 인덱스를 이용하여 조회 부하를 크게 줄일 수 있다고 설명한다.

반드시 이해할 부분

- 중복 속성이 조인을 줄이고 접근 경로를 바꿀 수 있다.

- 원문의 향상 사례를 반정규화가 항상 유리하다는 결론으로 확대하지 않는다.

상속된 고객번호를 매핑키로 조인

원문 PDF 92쪽

SQL 원문

```
SELECT B.고객명
FROM 주문 A, 고객 B
WHERE A.주문번호 = '1100001'
  AND A.고객번호 = B.고객번호
```

수행하는 작업

특정 주문의 고객명을 고객 엔터티에서 찾아온다.

주요 키워드와 구문의 역할

키워드·구문	역할
A.주문번호	조회할 주문을 정한다.
A.고객번호 = B.고객번호	상속된 고객번호로 두 집합을 연결한다.

원문에 따른 처리 과정

1. 주문번호 1100001을 찾는다.
2. 해당 행의 고객번호 100을 확인한다.
3. 고객에서 고객번호 100을 찾아 고객명을 확인한다.

원문 결과·설명

원문 표의 고객명은 정우진이다.

반드시 이해할 부분

- 검색 조건과 조인 조건을 구분한다.
- 관계는 연결된 집합을 조회할 수 있도록 하는 매핑 정보다.

NULL 대체 위치에 따른 최종주문금액 차이

원문 PDF 103, 104쪽

SQL 원문

```
SELECT 주문금액 - 주문취소금액 COL1
     , NVL(주문금액-주문취소금액,0) COL2
     , NVL(주문금액,0)-NVL(주문취소금액,0) COL3
FROM 주문
```

수행하는 작업

NULL이 있는 두 속성의 뺄셈에서 계산 후 대체와 계산 전 대체를 비교한다.

주요 키워드와 구문의 역할

키워드·구문	역할
NVL	첫 값이 NULL이면 두 번째 인자로 바꾼다.
COL2와 COL3	전체 식의 결과를 바꾸는지 각각의 피연산자를 바꾸는지 비교한다.

원문에 따른 처리 과정

1. NULL이 없는 100,000-20,000은 세 식 모두 80,000이다.
2. 15,000-NULL은 먼저 NULL이 되므로 COL1은 NULL, COL2는 0이다.
3. COL3은 취소금액을 먼저 0으로 바꾸어 15,000-0을 계산한다.

원문 결과·설명

원문 첫 행은 80,000/80,000/80,000, 둘째 행은 NULL/0/15,000이다.

반드시 이해할 부분

- NULL 대체 함수를 사용했는지보다 어느 단계에 적용했는지가 중요하다.
- NULL의 업무 의미를 정한 뒤 대체값을 결정한다.

전체 주문과 취소금액 보유 주문의 평균

원문 PDF 106, 107쪽

SQL 원문

```
SELECT SUM(주문취소금액)/COUNT(*) COL1
     , AVG(주문취소금액) COL2
FROM 주문
```

수행하는 작업

평균에서 전체 행 수와 NULL을 제외한 행 수의 차이를 보여준다.

주요 키워드와 구문의 역할

키워드·구문	역할
COUNT(*)	NULL 여부와 무관하게 전체 다섯 행을 센다.
AVG	NULL을 제외한 네 취소금액의 평균을 계산한다.

원문에 따른 처리 과정

1. 취소금액의 합계 50,000을 구한다.

2. COL1은 다섯 행으로, COL2는 값이 있는 네 행으로 나눈다.

원문 결과·설명

원문 결과는 COL1=10,000, COL2=12,500이다.

반드시 이해할 부분

- 평균의 분모는 업무 요건으로 먼저 정의한다.

- 집계함수의 NULL 제외 규칙을 이해한다.

본질식별자의 인덱스와 실제 접근 조건

원문 PDF 114, 115쪽

SQL 원문

```
SELECT *
FROM 주문상품
WHERE 주문번호 = :B1;

SELECT *
FROM 주문상품
WHERE 주문번호 = :B1
  AND 상품번호 = :B2;
```

수행하는 작업

주문상품의 일반적인 두 접근 패턴을 보여주어 식별자별 인덱스 비용을 비교한다.

주요 키워드와 구문의 역할

키워드·구문	역할
:B1, :B2	원문이 제시한 조회 입력값 자리이다.
주문번호+상품번호	본질식별자 모델의 PK 인덱스 구성이다.

원문에 따른 처리 과정

1. 주문번호만 또는 주문번호와 상품번호로 원하는 주문상품을 찾는다.
2. 본질식별자 모델은 이 접근에 PK 인덱스를 활용할 수 있다.
3. 주문상품번호를 PK로 삼는 인조식별자 모델은 원문 비교에서 별도 IX1 인덱스가 필요하다.

원문 결과·설명

실제 결과 행은 제시하지 않는다. 원문은 추가 인덱스의 용량과 DML 성능 영향을 설명한다.

반드시 이해할 부분

- 식별자 단순화가 필요한 접근 인덱스까지 없애지는 않는다.
- 채번 편의·중복 제약·조회 패턴을 함께 검토한다.

제1장 SQL 기본

과목 II. SQL 기본과 활용

원문 PDF 122-270쪽

ALL과 DISTINCT로 포지션 조회

원문 PDF 140, 141쪽

SQL 원문

```
SELECT ALL POSITION
FROM PLAYER;

SELECT POSITION
FROM PLAYER;

SELECT DISTINCT POSITION
FROM PLAYER;
```

수행하는 작업

중복 포함 조회의 기본값과 포지션 종류만 남기는 조회를 비교한다.

주요 키워드와 구문의 역할

키워드·구문	역할
ALL	중복을 포함하며 생략할 수 있다.
DISTINCT	같은 출력값의 중복을 제거한다.

원문에 따른 처리 과정

1. PLAYER의 POSITION을 선택한다.
2. 앞의 두 문장은 모든 행의 값을 출력하고 세 번째는 서로 다른 값만 남긴다.

원문 결과·설명

원문 ALL과 생략형은 480행, DISTINCT는 DF·FW·MF·GK와 미정인 NULL을 포함하여 5행이다.

반드시 이해할 부분

- ALL은 기본 동작이다.
- 서로 다른 값의 종류 수와 원본 행 수를 구분한다.

급여를 12로 나눈 값의 반올림과 버림

원문 PDF 155쪽

SQL 원문

```
SELECT ENAME
     , ROUND(SAL / 12, 1) AS SAL_ROUND
     , TRUNC(SAL / 12, 1) AS SAL_TRUNC
FROM EMP;
```

수행하는 작업

같은 수식을 소수 한 자리로 표현할 때 ROUND와 TRUNC의 차이를 확인한다.

주요 키워드와 구문의 역할

키워드·구문	역할
ROUND(...,1)	소수 한 자리까지 반올림한다.
TRUNC(...,1)	소수 한 자리 아래를 버린다.
AS	각 계산의 의미를 구분하는 출력 별칭이다.

원문에 따른 처리 과정

1. 행마다 SAL을 12로 나눈다.

2. 동일 계산값에 ROUND와 TRUNC를 각각 적용한다.

원문 결과·설명

원문은 14행을 출력한다. SMITH는 66.7과 66.6, WARD는 104.2와 104.1로 차이가 나며 SCOTT는 두 결과 모두 250이다.

반드시 이해할 부분

- 원래 수식이 같아도 끝자리 처리 방식에 따라 값이 달라진다.

- 원문은 TRUNC를 Oracle 함수로 제시한다.

입사일의 연·월·일 추출

원문 PDF 158, 159, 160쪽

SQL 원문

```
SELECT ENAME AS 사원명, HIREDATE AS 입사일자
     , EXTRACT(YEAR FROM HIREDATE) AS 입사년도
     , EXTRACT(MONTH FROM HIREDATE) AS 입사월
     , EXTRACT(DAY FROM HIREDATE) AS 입사일
FROM EMP;
```

수행하는 작업

날짜 값에서 필요한 구성값을 나누어 출력한다.

주요 키워드와 구문의 역할

키워드·구문	역할
EXTRACT	날짜의 지정 구성요소를 추출한다.
YEAR / MONTH / DAY	추출할 연·월·일을 정한다.

원문에 따른 처리 과정

1. EMP 각 행의 HIREDATE를 참조한다.
2. 같은 날짜에 세 EXTRACT를 적용하여 연·월·일을 각각 출력한다.

원문 결과·설명

원문 결과에서 SMITH의 입사일자 1980-12-17은 입사년도 1980, 입사월 12, 입사일 17로 표시된다. 전체 14행이다.

반드시 이해할 부분

- 날짜 원본과 그 구성값을 함께 출력할 수 있다.
- 원문은 SQL Server DATEPART 및 YEAR·MONTH·DAY, Oracle TO_NUMBER(TO_CHAR(...))와 대응시킨다.

SEARCHED CASE로 급여 등급 분류

원문 PDF 166쪽

SQL 원문

```
SELECT ENAME
     , CASE
           WHEN SAL >= 3000 THEN 'HIGH'
           WHEN SAL >= 1000 THEN 'MID'
           ELSE 'LOW'
       END AS SALARY_GRADE
FROM EMP;
```

수행하는 작업

사원 급여를 범위 조건에 따라 세 등급으로 바꿔 출력한다.

주요 키워드와 구문의 역할

키워드·구문	역할
CASE WHEN	각 WHEN에 비교 조건을 직접 적는다.
THEN	조건이 맞을 때의 반환값이다.
ELSE	앞 조건에 해당하지 않을 때 반환한다.
END	CASE 표현을 끝낸다.

원문에 따른 처리 과정

1. 급여 3000 이상이면 HIGH로 분류한다.
2. 그다음 1000 이상인 경우 MID, 나머지는 LOW로 분류한다.

원문 결과·설명

원문 14행에서 SMITH는 LOW, ALLEN은 MID, SCOTT·KING·FORD는 HIGH이다.

반드시 이해할 부분

- 범위 조건은 SEARCHED CASE로 표현한다.
- 여러 조건의 순서와 반환값을 함께 읽는다.

커미션 NULL을 고려한 연봉 계산

원문 PDF 171, 172쪽

SQL 원문

```
SELECT ENAME AS 사원명, SAL AS 월급, COMM AS 커미션
     , (SAL * 12) + COMM AS 연봉A
     , (SAL * 12) + NVL(COMM, 0) AS 연봉B
FROM EMP;
```

수행하는 작업

NULL이 포함된 산술식과 연산 전 대체한 산술식의 결과를 비교한다.

주요 키워드와 구문의 역할

키워드·구문	역할
NVL(COMM,0)	커미션 NULL을 0으로 바꾼 뒤 더한다.
SAL * 12	원문 연봉 계산의 급여 부분이다.

원문에 따른 처리 과정

1. 각 행의 SAL에 12를 곱한다.
2. 연봉A는 COMM을 그대로, 연봉B는 NULL이면 0으로 바꾸어 더한다.

원문 결과·설명

원문 SMITH는 COMM이 NULL이어서 연봉A도 NULL이고 연봉B는 9600이다. COMM=300인 ALLEN은 둘 다 19500이다. 전체 14행이다.

반드시 이해할 부분

- NULL은 0과 다르다.
- NULL이 들어간 뒤의 결과가 아니라 연산 목적에 맞는 피연산자를 처리한다.

공집합을 집계한 뒤 대체값 출력

원문 PDF 175쪽

SQL 원문

```
SELECT NVL(MAX(MGR), 9999) AS MGR
FROM EMP
WHERE ENAME = 'JSC';
```

수행하는 작업

검색한 사원이 없어도 원문에서 9999 한 결과를 얻는 과정을 보여준다.

주요 키워드와 구문의 역할

키워드·구문	역할
WHERE ENAME = 'JSC'	원문 데이터에 존재하지 않는 사원을 검색한다.
MAX(MGR)	그룹을 지정하지 않은 집계로 공집합에서 NULL 결과를 만든다.
NVL	그 집계 결과 NULL을 9999로 바꾼다.

원문에 따른 처리 과정

1. WHERE를 통과하는 원본 행은 없다.
2. MAX(MGR)가 NULL 한 결과를 반환한다.
3. 그 NULL에 NVL을 적용한다.

원문 결과·설명

원문은 MGR=9999인 한 행을 출력한다. 단순 NVL(MGR,9999) 조회는 행이 없는 공집합으로 남는다.

반드시 이해할 부분

- NULL인 필드와 행 자체가 없는 상황을 구분한다.
- 집계 적용 전후의 데이터 수준을 확인한다.

IN과 BETWEEN으로 복합 검색 조건 표현

원문 PDF 195, 196쪽

SQL 원문

```
SELECT TEAM_ID AS 팀ID, PLAYER_NAME AS 선수명, POSITION AS 포지션
     , BACK_NO AS 백넘버, HEIGHT AS 키
FROM PLAYER
WHERE TEAM_ID IN ('K02', 'K07')
  AND POSITION = 'MF'
  AND HEIGHT BETWEEN 170 AND 180;
```

수행하는 작업

두 팀 소속이면서 MF이고 키가 지정 범위 안인 선수를 선택한다.

주요 키워드와 구문의 역할

키워드·구문	역할
IN	K02 또는 K07에 속하는지 확인한다.
AND	팀·포지션·키 조건을 모두 만족시킨다.
BETWEEN	170과 180을 포함하는 범위를 확인한다.

원문에 따른 처리 과정

1. 팀의 두 대안을 하나의 IN 조건으로 만든다.
2. MF 조건 및 양 끝을 포함한 키 범위를 동시에 적용한다.

원문 결과·설명

원문은 괄호를 사용한 OR 동등조건과 같은 결과 33행을 설명한다.

반드시 이해할 부분

- OR와 AND를 함께 쓰면 팀 대안을 괄호로 묶어야 한다.
- 목록·범위 구문은 조건의 논리 구조를 간결하게 표현한다.

전체 선수 집계와 NULL 제외 건수

원문 PDF 200쪽

SQL 원문

```
SELECT COUNT(*) AS 전체행수, COUNT(HEIGHT) AS 키건수
     , MAX(HEIGHT) AS 최대키, MIN(HEIGHT) AS 최소키
     , ROUND(AVG(HEIGHT), 2) AS 평균키
FROM PLAYER;
```

수행하는 작업

전체 테이블을 한 그룹으로 보고 건수와 키 통계를 구한다.

주요 키워드와 구문의 역할

키워드·구문	역할
COUNT(*)	전체 행 수를 센다.
COUNT(HEIGHT)	키가 NULL이 아닌 행만 센다.
MAX / MIN / AVG	NULL을 제외한 키의 최대·최소·평균을 구한다.

원문에 따른 처리 과정

1. GROUP BY 없이 PLAYER 전체를 하나의 집계 대상으로 사용한다.
2. 행 수는 전부 세고 각 키 집계는 값이 있는 대상을 사용한다.
3. 평균을 ROUND로 두 자리로 출력한다.

원문 결과·설명

원문 결과는 전체행수 480, 키건수 447, 최대키 196, 최소키 165, 평균키 179.31이다.

반드시 이해할 부분

- COUNT(*)와 COUNT(칼럼)의 차이는 NULL 포함 여부다.
- 집계 결과 형식과 원본 집계 대상을 구분한다.

포지션별 평균키와 HAVING

원문 PDF 204쪽

SQL 원문

```
SELECT POSITION AS 포지션, ROUND(AVG(HEIGHT), 2) AS 평균키
FROM PLAYER
GROUP BY POSITION
HAVING AVG(HEIGHT) >= 180;
```

수행하는 작업

포지션 그룹의 평균키가 180 이상인 결과만 표시한다.

주요 키워드와 구문의 역할

키워드·구문	역할
GROUP BY POSITION	포지션 값별로 집계 대상을 나눈다.
HAVING AVG(HEIGHT)	집계한 그룹의 평균으로 그룹을 제한한다.

원문에 따른 처리 과정

1. POSITION 값별로 그룹을 만든다.
2. 각 그룹의 키 평균을 구한다.
3. 평균이 180 이상인 그룹만 남기고 출력용 평균은 두 자리로 반올림한다.

원문 결과·설명

원문은 GK 186.26, DF 180.21의 두 행을 출력한다.

반드시 이해할 부분

- 그룹의 AVG 조건은 WHERE가 아니라 HAVING에 둔다.
- GROUP BY 자체는 정렬 보장이 아니다.

급여 범위로 급여등급을 찾는 비등가 조인

원문 PDF 233, 234, 235쪽

SQL 원문

```
SELECT A.ENAME, A.JOB, A.SAL, B.GRADE
FROM EMP A, SALGRADE B
WHERE A.SAL BETWEEN B.LOSAL AND B.HISAL;
```

수행하는 작업

사원 급여가 속한 등급 범위를 연결한다.

주요 키워드와 구문의 역할

키워드·구문	역할
BETWEEN B.LOSAL AND B.HISAL	급여가 등급의 하한·상한에 포함되는지로 조인한다.
A, B	사원과 급여등급의 소속을 구분하는 별칭이다.

원문에 따른 처리 과정

1. EMP의 급여와 SALGRADE 각 등급의 범위를 비교한다.
2. 범위 안에 들어가는 등급을 연결한다.

원문 결과·설명

원문 범위 설명에서 SCOTT의 급여 3000은 2001~3000인 4등급에 해당한다. 이어지는 같은 목적의 별칭 출력 예는 사원 14명을 보여 준다.

반드시 이해할 부분

- 조인은 같은 번호의 = 비교에 한정되지 않는다.
- BETWEEN은 비등가 조인 설명을 위한 한 사례다.

USING의 공통 조인 칼럼과 접두사

원문 PDF 249, 250쪽

SQL 원문

```
SELECT DEPTNO, A.DNAME, A.LOC, B.DNAME, B.LOC
FROM DEPT A JOIN DEPT_TEMP B
USING (DEPTNO);
```

수행하는 작업

같은 이름의 DEPTNO만 선택하여 두 부서 집합을 연결한다.

주요 키워드와 구문의 역할

키워드·구문	역할
USING(DEPTNO)	공통 칼럼 중 DEPTNO를 연결 기준으로 지정한다.
SELECT DEPTNO	USING 칼럼은 테이블 접두사를 붙이지 않고 선택한다.

원문에 따른 처리 과정

1. 원문 DEPT_TEMP는 20·30번 부서명만 바뀐 상태다.

2. DEPTNO만 비교하므로 부서명이 달라도 같은 부서번호로 연결한다.

원문 결과·설명

원문은 부서번호 10·20·30·40의 네 행을 출력한다. 공통 DEPTNO는 한 열, 양쪽 DNAME·LOC는 각각 출력된다. A.DEPTNO로 고르면 원문에서 ORA-25154 오류가 난다.

반드시 이해할 부분

- NATURAL처럼 모든 공통 칼럼이 아니라 지정한 칼럼만 비교한다.

- USING에 참여한 공통 칼럼의 접두사 제한을 기억한다.

홈팀이 없는 경기장도 보존하는 LEFT JOIN

원문 PDF 261, 262쪽

SQL 원문

```
SELECT A.STADIUM_NAME, A.STADIUM_ID, A.SEAT_COUNT, A.HOMETEAM_ID
     , B.TEAM_NAME
FROM STADIUM A LEFT OUTER JOIN TEAM B
  ON B.TEAM_ID = A.HOMETEAM_ID
ORDER BY A.HOMETEAM_ID;
```

수행하는 작업

등록된 모든 경기장을 남기고 연결되는 홈팀 이름을 함께 출력한다.

주요 키워드와 구문의 역할

키워드·구문	역할
LEFT OUTER JOIN	왼쪽 STADIUM의 행을 보존한다.
ON	서로 이름이 다른 홈팀 식별자와 팀 식별자의 연결 조건이다.
ORDER BY	홈팀 ID 기준으로 결과를 정렬한다.

원문에 따른 처리 과정

1. 경기장의 HOMETEAM_ID와 TEAM_ID를 비교하여 팀을 연결한다.
2. 연결할 팀이 없어도 경기장 행은 유지하며 TEAM에서 가져올 값은 NULL로 채운다.
3. 홈팀 ID로 정렬한다.

원문 결과·설명

원문은 내부 조인의 15행에 홈팀 없는 안양·마산·일산·부산시민·대구시민경기장 다섯 행을 더한 20행을 출력한다.

반드시 이해할 부분

- 외부 조인에서 보존 방향과 NULL을 채울 상대를 구분한다.
- OUTER는 생략할 수 있고 LEFT의 의미는 그대로다.

제2장 SQL 활용

과목 II. SQL 기본과 활용

원문 PDF 272-388쪽

단일 행 서브쿼리로 같은 팀 선수 조회

원문 PDF 275, 276쪽

SQL 원문

```
SELECT PLAYER_NAME AS 선수명, POSITION AS 포지션, BACK_NO AS 백넘버
FROM PLAYER
WHERE TEAM_ID = (SELECT TEAM_ID
                 FROM PLAYER
                 WHERE PLAYER_NAME = '정남일')
ORDER BY PLAYER_NAME;
```

수행하는 작업

정남일의 팀을 기준으로 같은 팀 선수 정보를 조회한다.

주요 키워드와 구문의 역할

키워드·구문	역할
=	서브쿼리가 반환한 단일 값과 비교
ORDER BY	선수명 정렬

원문에 따른 처리 과정

1. 서브쿼리로 정남일의 TEAM_ID를 얻는다.

2. 메인 PLAYER에서 같은 팀을 찾아 정렬한다.

원문 결과·설명

원문은 51행을 표시한다. 같은 이름이 둘 이상이면 단일 행 비교 오류가 생길 수 있다.

반드시 이해할 부분

- 단일 행 조건은 이름의 업무적 유일성까지 고려한다.

다중 행은 IN으로 비교

원문 PDF 278쪽

SQL 원문

```
SELECT REGION_NAME AS 연고지명, TEAM_NAME AS 팀명, E_TEAM_NAME AS 영문팀명
FROM TEAM
WHERE TEAM_ID IN (SELECT TEAM_ID
                  FROM PLAYER
                  WHERE PLAYER_NAME = '정현수')
ORDER BY TEAM_NAME;
```

수행하는 작업

동명이인의 소속팀을 모두 조회한다.

주요 키워드와 구문의 역할

키워드·구문	역할
IN	반환 집합 중 일치하는 팀 선택

원문에 따른 처리 과정

1. 동명이인 선수의 팀 ID 집합을 얻는다.
2. 그 집합에 속하는 TEAM 행을 조회한다.

원문 결과·설명

원문은 전남 드래곤즈·성남 일화천마 2행을 제시한다. 같은 예에서 =를 쓰면 ORA-01427이다.

반드시 이해할 부분

- 결과가 2건 이상일 수 있으면 단일 값 비교로 가정하지 않는다.

팀별 최소 키를 다중 칼럼으로 비교

원문 PDF 279쪽

SQL 원문

```
SELECT TEAM_ID AS 팀코드, PLAYER_NAME AS 선수명, POSITION AS 포지션,
       BACK_NO AS 백넘버, HEIGHT AS 키
FROM PLAYER
WHERE (TEAM_ID, HEIGHT) IN (SELECT TEAM_ID, MIN(HEIGHT)
                            FROM PLAYER
                            GROUP BY TEAM_ID)
ORDER BY TEAM_ID, PLAYER_NAME;
```

수행하는 작업

팀과 최소 키가 동시에 일치하는 선수를 찾는다.

주요 키워드와 구문의 역할

키워드·구문	역할
(TEAM_ID, HEIGHT)	팀별 값의 쌍을 비교
MIN	팀별 최소 키

원문에 따른 처리 과정

1. 서브쿼리에서 팀별 최소 키를 집계한다.
2. 메인 행의 팀·키 쌍을 집계 결과와 비교한다.

원문 결과·설명

원문은 동률을 포함한 19행을 제시한다.

반드시 이해할 부분

- 한 팀 한 명으로 제한하는 쿼리가 아니다.

UNION으로 두 팀 결과 결합

원문 PDF 291, 292쪽

SQL 원문

```
SELECT TEAM_ID AS 팀코드, PLAYER_NAME AS 선수명, POSITION AS 포지션,
       BACK_NO AS 백넘버, HEIGHT AS 키
FROM PLAYER
WHERE TEAM_ID = 'K02'
UNION
SELECT TEAM_ID AS 팀코드, PLAYER_NAME AS 선수명, POSITION AS 포지션,
       BACK_NO AS 백넘버, HEIGHT AS 키
FROM PLAYER
WHERE TEAM_ID = 'K07';
```

수행하는 작업

두 팀 선수 결과를 합집합으로 결합한다.

주요 키워드와 구문의 역할

키워드·구문	역할
UNION	두 SELECT의 결과 결합 및 중복 제거

원문에 따른 처리 과정

1. 각 조건의 선수 집합을 만든다.
2. 전체 출력 칼럼 기준으로 중복을 제거한다.

원문 결과·설명

원문은 100행을 제시하며 DISTINCT와 OR 또는 IN으로 표현하는 비교 SQL도 설명한다.

반드시 이해할 부분

- 두 SELECT의 칼럼 수·위치별 데이터 타입이 맞아야 한다.

ROLLUP으로 부서 소계와 총계 생성

원문 PDF 303, 304쪽

SQL 원문

```
SELECT B.DNAME, A.JOB, COUNT(*) AS EMP_CNT, SUM(A.SAL) AS SAL_SUM
FROM EMP A, DEPT B
WHERE B.DEPTNO = A.DEPTNO
GROUP BY ROLLUP(B.DNAME, A.JOB);
```

수행하는 작업

부서·업무별 집계에 부서 소계와 전체 합계를 더한다.

주요 키워드와 구문의 역할

키워드·구문	역할
ROLLUP	왼쪽 항목부터 유지하는 계층 집계
COUNT / SUM	인원수와 급여 합계

원문에 따른 처리 과정

1. 사원과 부서를 조인한다.
2. 부서·업무, 부서, 전체 수준의 집계를 만든다.

원문 결과·설명

원문은 상세 9행+부서 소계 3행+전체 1행, 총 13행을 설명한다.

반드시 이해할 부분

- 항목 순서를 바꾸면 소계의 기준도 달라진다.

RANK와 DENSE_RANK의 동률 처리

원문 PDF 321쪽

SQL 원문

```
SELECT JOB, ENAME, SAL,
       RANK() OVER (ORDER BY SAL DESC) AS RK,
       DENSE_RANK() OVER (ORDER BY SAL DESC) AS DR
FROM EMP;
```

수행하는 작업

급여 순위에서 동률 뒤 등수의 차이를 비교한다.

주요 키워드와 구문의 역할

키워드·구문	역할
RANK	동률 다음 등수를 건너뜀
DENSE_RANK	등수를 연속 부여

원문에 따른 처리 과정

1. 급여 내림차순을 기준으로 각 함수의 순위를 계산한다.

원문 결과·설명

FORD와 SCOTT는 모두 2위. 다음 JONES는 RK=4, DR=3이다.

반드시 이해할 부분

- 정렬 값의 동률과 다음 등수 부여 방식을 구별한다.

ROWS 프레임으로 이웃 행 평균

원문 PDF 327, 328쪽

SQL 원문

```
SELECT MGR, ENAME, HIREDATE, SAL,
       ROUND(AVG(SAL) OVER (PARTITION BY MGR ORDER BY HIREDATE
             ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING)) AS AVG_SAL
FROM EMP;
```

수행하는 작업

같은 관리자 아래에서 입사일 순 이웃 행과 자신의 평균 급여를 구한다.

주요 키워드와 구문의 역할

키워드·구문	역할
PARTITION BY MGR	관리자별 분석 범위
ROWS BETWEEN	앞 한 행~뒤 한 행

원문에 따른 처리 과정

1. 관리자별로 나눈 후 입사일로 순서를 정한다.
2. 현재 행과 존재하는 앞뒤 행의 급여를 평균 내고 반올림한다.

원문 결과·설명

원문은 첫 행 ALLEN이 본인과 다음 행의 평균 1425를 갖는다고 설명한다.

반드시 이해할 부분

- 프레임의 이웃은 ORDER BY HIREDATE의 이웃이다.

RANGE로 급여 값 범위의 인원 계산

원문 PDF 329쪽

SQL 원문

```
SELECT ENAME, SAL,
       COUNT(*) OVER (ORDER BY SAL
           RANGE BETWEEN 50 PRECEDING AND 150 FOLLOWING) AS EMP_CNT
FROM EMP;
```

수행하는 작업

현재 급여보다 50 적은 값부터 150 큰 값까지의 인원수를 센다.

주요 키워드와 구문의 역할

키워드·구문	역할
RANGE	정렬 값 기준의 논리적 범위
COUNT(*) OVER	각 행에 해당 범위 인원수 표시

원문에 따른 처리 과정

1. 전체 행을 급여 기준으로 분석한다.
2. 각 행의 급여 -50~+150에 포함되는 행 수를 계산한다.

원문 결과·설명

원문 ADAMS의 급여 1100에 대해 1050~1250 범위의 ADAMS·WARD·MARTIN 3행이 포함된다.

반드시 이해할 부분

- ROWS의 앞뒤 행 수와 달리 값의 범위다.

LAG의 이동 거리와 대체값

원문 PDF 333, 334쪽

SQL 원문

```
SELECT ENAME, HIREDATE, SAL,
       LAG(SAL, 2, 0) OVER (ORDER BY HIREDATE) AS LAG_SAL
FROM EMP
WHERE JOB = 'SALESMAN';
```

수행하는 작업

판매사원을 입사일로 정렬하여 두 행 앞의 급여를 표시한다.

주요 키워드와 구문의 역할

키워드·구문	역할
LAG(SAL, 2, 0)	두 행 앞 값이 없으면 0
ORDER BY HIREDATE	앞뒤를 판단하는 기준

원문에 따른 처리 과정

1. SALESMAN 행을 대상으로 입사일 순서를 정한다.
2. 두 행 앞 SAL을 읽고 해당 행이 없으면 0을 쓴다.

원문 결과·설명

원문 첫 두 행의 LAG_SAL은 0이며 이후 두 행은 1600과 1250이다.

반드시 이해할 부분

- 행이 없는 경우의 대체값과 이동 행 수를 구분한다.

Oracle 정렬 후 상위 3건

원문 PDF 340쪽

SQL 원문

```
SELECT ENAME, SAL
FROM (SELECT ENAME, SAL
      FROM EMP
      ORDER BY SAL DESC)
WHERE ROWNUM <= 3;
```

수행하는 작업

급여 내림차순으로 정렬한 후보에서 3행을 추출한다.

주요 키워드와 구문의 역할

키워드·구문	역할
인라인 뷰	급여 정렬
ROWNUM <= 3	정렬된 결과의 행 수 제한

원문에 따른 처리 과정

1. 인라인 뷰에서 급여 내림차순 집합을 만든다.
2. 바깥 쿼리에서 처음 3건을 취한다.

원문 결과·설명

원문 결과는 KING 5000, SCOTT 3000, FORD 3000이다.

반드시 이해할 부분

- 정렬 전 행 제한과 정렬 후 행 제한을 구별한다.

SQL Server TOP WITH TIES

원문 PDF 342쪽

SQL 원문

```
SELECT TOP(2) WITH TIES
       ENAME, SAL
FROM EMP
ORDER BY SAL DESC;
```

수행하는 작업

급여 상위 2개 위치에 마지막 동순위까지 포함한다.

주요 키워드와 구문의 역할

키워드·구문	역할
TOP(2)	상위 행 제한
WITH TIES	마지막 정렬 값의 동순위 포함

원문에 따른 처리 과정

1. 급여 내림차순을 기준으로 상위 행을 선택한다.

2. 마지막 선택 급여와 같은 행을 추가로 포함한다.

원문 결과·설명

원문은 KING 5000, SCOTT 3000, FORD 3000의 3행을 제시한다.

반드시 이해할 부분

- N이라는 수가 최종 행 수의 상한은 아닐 수 있다.

ROW LIMITING의 FETCH

원문 PDF 343쪽

SQL 원문

```
SELECT EMPNO, SAL
FROM EMP
ORDER BY SAL, EMPNO
FETCH FIRST 5 ROWS ONLY;
```

수행하는 작업

급여·사번 정렬에서 처음 5행을 반환한다.

주요 키워드와 구문의 역할

키워드·구문	역할
FETCH FIRST 5 ROWS ONLY	정렬 결과의 처음 5행

원문에 따른 처리 과정

1. 급여, 사번 순서로 결과를 정렬한다.
2. 처음 5행만 반환한다.

원문 결과·설명

원문은 급여 800, 950, 1100, 1250, 1250의 5행을 표시한다.

반드시 이해할 부분

- 같은 쪽의 OFFSET 5 ROWS 예는 첫 5행을 건너뛰고 나머지 9행을 반환한다.

Oracle 계층형 질의의 순방향 전개

원문 PDF 349, 350쪽

SQL 원문

```
SELECT LEVEL AS LV, LPAD(' ', (LEVEL - 1) * 2) || EMPNO AS EMPNO, MGR,
       CONNECT_BY_ISLEAF AS ISLEAF
FROM EMP
START WITH MGR IS NULL
CONNECT BY MGR = PRIOR EMPNO;
```

수행하는 작업

관리자가 없는 사원부터 부하 방향으로 계층을 조회한다.

주요 키워드와 구문의 역할

키워드·구문	역할
START WITH	시작 루트 선택
PRIOR EMPNO	직전 부모의 사원번호
LEVEL / ISLEAF	깊이와 끝 노드 표시

원문에 따른 처리 과정

1. MGR가 NULL인 행에서 시작한다.

2. 부모 EMPNO와 다음 행 MGR를 연결해 하위로 전개한다.

원문 결과·설명

원문은 14행을 제시한다. 7839가 LEVEL 1이며 리프 행의 ISLEAF는 1이다.

반드시 이해할 부분

- 루트는 START WITH와 전개 방향에 의해 정해진다.

SQL Server 재귀 CTE

원문 PDF 353, 354쪽

SQL 원문

```
WITH EMPLOYEES_ANCHOR AS (
    SELECT EMPLOYEEID, LASTNAME, FIRSTNAME, REPORTSTO, 0 AS LEVEL
    FROM EMPLOYEES
    WHERE REPORTSTO IS NULL /* 재귀 호출의 시작점 */
    UNION ALL
    SELECT R.EMPLOYEEID, R.LASTNAME, R.FIRSTNAME, R.REPORTSTO, A.LEVEL + 1
    FROM EMPLOYEES_ANCHOR A, EMPLOYEES R
    WHERE R.REPORTSTO = A.EMPLOYEEID
)
SELECT LEVEL, EMPLOYEEID, LASTNAME, FIRSTNAME, REPORTSTO
FROM EMPLOYEES_ANCHOR
GO
```

수행하는 작업

앵커 집합과 재귀 연결을 반복하여 사원 계층을 만든다.

주요 키워드와 구문의 역할

키워드·구문	역할
앵커 SELECT	REPORTSTO가 NULL인 시작 행
UNION ALL	시작과 단계별 결과 결합
재귀 참조	직전 단계에서 다음 부하 연결

원문에 따른 처리 과정

1. 앵커를 T0로 얻는다.

2. 현재 단계의 사원번호와 부하의 REPORTSTO를 연결한다.

3. 새 결과가 없을 때까지 반복하고 모든 단계를 합한다.

원문 결과·설명

원문은 9행을 제시한다. 이 코드의 LEVEL 시작값은 0이다.

반드시 이해할 부분

- 조직도와 같은 출력 순서는 추가 정렬이 필요하다.

PIVOT으로 부서 값을 열로 전환

원문 PDF 358, 359쪽

SQL 원문

```
SELECT *
FROM (SELECT JOB, DEPTNO, SAL FROM EMP)
PIVOT (SUM(SAL) FOR DEPTNO IN (10, 20, 30))
ORDER BY 1;
```

수행하는 작업

업무별 부서 급여 합계를 열로 펼친다.

주요 키워드와 구문의 역할

키워드·구문	역할
SUM(SAL)	집계값
FOR DEPTNO	열로 펼칠 기준
IN	열로 만들 부서 값

원문에 따른 처리 과정

1. 필요한 입력 열만 선택한다.

2. JOB을 나머지 집계 기준으로 하여 부서별 합계를 열에 배치한다.

원문 결과·설명

원문은 JOB별 5행을 표시한다.

반드시 이해할 부분

- 입력에 불필요한 열이 있으면 집계 단위가 달라진다.

UNPIVOT에서 NULL 포함

원문 PDF 364, 365쪽

SQL 원문

```
SELECT JOB, DEPTNO, SAL
FROM T1
UNPIVOT INCLUDE NULLS (SAL FOR DEPTNO IN (D10_SAL AS 10, D20_SAL AS 20))
ORDER BY 1, 2;
```

수행하는 작업

부서별 급여 열을 행으로 바꾸면서 NULL 값도 포함한다.

주요 키워드와 구문의 역할

키워드·구문	역할
SAL	변환된 값 열
DEPTNO	원래 열을 설명하는 값
INCLUDE NULLS	NULL인 변환 행도 포함

원문에 따른 처리 과정

1. D10_SAL·D20_SAL을 각각 SAL 행으로 전환한다.
2. 설명 값 10·20을 DEPTNO에 놓는다.

원문 결과·설명

기본 UNPIVOT은 3행, INCLUDE NULLS를 지정한 원문 예는 4행이다.

반드시 이해할 부분

- T1은 원문 PDF363쪽에서 PIVOT 결과로 만든 예제 테이블이다.

REGEXP_LIKE로 이름 패턴 검색

원문 PDF 380쪽

SQL 원문

```
SELECT FIRST_NAME, LAST_NAME
FROM HR.EMPLOYEES
WHERE REGEXP_LIKE(FIRST_NAME, '^Ste(v|ph)en$');
```

수행하는 작업

Ste로 시작하고 v 또는 ph 다음 en으로 끝나는 이름을 찾는다.

주요 키워드와 구문의 역할

키워드·구문	역할
^ / $	문자열의 시작과 끝 고정
(v|ph)	v 또는 ph 선택

원문에 따른 처리 과정

1. FIRST_NAME 전체가 패턴과 일치하는지 검사한다.

원문 결과·설명

원문은 Stephen Stiles, Steven King, Steven Markle의 3행을 제시한다.

반드시 이해할 부분

- 부분 일치와 문자열 전체 일치의 차이를 앵커로 확인한다.

제3장 관리 구문

과목 II. SQL 기본과 활용

원문 PDF 390-461쪽

INSERT SELECT로 여러 팀 자료 입력

원문 PDF 393쪽

SQL 원문

```
INSERT
INTO TEAM (TEAM_ID, REGION_NAME, TEAM_NAME, ORIG_YYYY, STADIUM_ID)
SELECT REPLACE(TEAM_ID, 'K', 'A') AS TEAM_ID,
       REGION_NAME, REGION_NAME || ' 올스타' AS TEAM_NAME,
       2019 AS ORIG_YYYY, STADIUM_ID
FROM TEAM
WHERE REGION_NAME IN ('성남', '인천');
```

수행하는 작업

두 지역의 팀을 조회하여 변환된 자료를 입력한다.

주요 키워드와 구문의 역할

키워드·구문	역할
INSERT INTO	입력 대상 칼럼
SELECT	값을 공급할 다중 행 조회

원문에 따른 처리 과정

1. 지역 조건으로 원본 행을 고른다.

2. 팀 ID·이름·연도를 원문 표현식으로 변환해 대상 칼럼 순서대로 입력한다.

원문 결과·설명

원문은 2행 입력을 표시한다.

반드시 이해할 부분

- 칼럼 개수·순서 대응을 확인한다.

NULL인 포지션만 수정

원문 PDF 394쪽

SQL 원문

```
UPDATE PLAYER
SET POSITION = 'MF'
WHERE POSITION IS NULL;
```

수행하는 작업

포지션이 정의되지 않은 선수만 MF로 수정한다.

주요 키워드와 구문의 역할

키워드·구문	역할
SET	새 값 대입
IS NULL	수정할 행 제한

원문에 따른 처리 과정

1. NULL 조건에 맞는 행을 식별한다.
2. 선택된 행의 POSITION을 수정한다.

원문 결과·설명

원문은 3행 갱신을 표시한다.

반드시 이해할 부분

- WHERE가 없으면 전체 행을 바꾼다.

다른 테이블 조건으로 삭제 대상 선정

원문 PDF 397쪽

SQL 원문

```
DELETE PLAYER A
WHERE EXISTS (SELECT 1
              FROM TEAM X
              WHERE X.TEAM_ID = A.TEAM_ID
                AND X.ORIG_YYYY < 1980);
```

수행하는 작업

1980년 이전에 창단한 팀의 선수 행을 삭제한다.

주요 키워드와 구문의 역할

키워드·구문	역할
DELETE	PLAYER 행 제거
EXISTS	같은 팀의 창단 조건 존재 확인

원문에 따른 처리 과정

1. 각 선수의 TEAM_ID와 TEAM을 연관시킨다.
2. 조건에 맞는 팀이 있으면 해당 선수 행을 삭제한다.

원문 결과·설명

원문은 49행 삭제를 표시한다.

반드시 이해할 부분

- 팀 행을 지우는 코드가 아니라 선수 행을 지우는 코드다.

MERGE의 일치·불일치 분기

원문 PDF 400쪽

SQL 원문

```
MERGE
INTO TEAM T
USING TEAM_TMP S
ON (T.TEAM_ID = S.TEAM_ID)
WHEN MATCHED THEN
    UPDATE
    SET T.REGION_NAME = S.REGION_NAME,
        T.TEAM_NAME = S.TEAM_NAME,
        T.DDD = S.DDD,
        T.TEL = S.TEL
WHEN NOT MATCHED THEN
    INSERT (T.TEAM_ID, T.REGION_NAME, T.TEAM_NAME, T.STADIUM_ID, T.DDD, T.TEL)
    VALUES (S.TEAM_ID, S.REGION_NAME, S.TEAM_NAME, S.STADIUM_ID, S.DDD, S.TEL);
```

수행하는 작업

원문의 임시 소스 TEAM_TMP로 TEAM 자료를 입력·수정한다.

주요 키워드와 구문의 역할

키워드·구문	역할
USING	소스 집합
ON	팀 ID 일치 판단
MATCHED / NOT MATCHED	수정·입력 분기

원문에 따른 처리 과정

1. 팀 ID로 타깃과 소스를 대응시킨다.
2. 이미 있으면 지정 칼럼을 수정하고 없으면 소스 값으로 삽입한다.

원문 결과·설명

원문은 20행 병합을 표시한다. TEAM_TMP는 원문 PDF398-399쪽에서 준비한 테이블이다.

반드시 이해할 부분

- 일치 판단 조건과 실제 변경 칼럼을 분리해 읽는다.

Oracle 저장점까지 부분 롤백

원문 PDF 412쪽

SQL 원문

```
SAVEPOINT SVPT2;
UPDATE PLAYER SET HEIGHT = 100;
ROLLBACK TO SVPT2;
```

수행하는 작업

키 전체 수정 작업을 저장점 이후 범위에서 취소한다.

주요 키워드와 구문의 역할

키워드·구문	역할
SAVEPOINT	복귀 기준 지정
ROLLBACK TO	해당 저장점 이후 변경 취소

원문에 따른 처리 과정

1. 저장점을 정한다.
2. 480행을 수정한 뒤 저장점까지 되돌린다.

원문 결과·설명

원문은 480행 갱신과 롤백 완료 메시지를 표시한다.

반드시 이해할 부분

- 부분 롤백과 전체 트랜잭션 종료는 구별한다.

SQL Server 저장점과 부분 롤백

원문 PDF 412쪽

SQL 원문

```
BEGIN TRAN
SAVE TRAN SVTR2;
UPDATE PLAYER SET HEIGHT = 100;
ROLLBACK TRAN SVTR2;
```

수행하는 작업

명시적 트랜잭션 안에서 저장점 이후 수정을 취소한다.

주요 키워드와 구문의 역할

키워드·구문	역할
BEGIN TRAN	명시적 트랜잭션 시작
SAVE TRAN	저장점 정의
ROLLBACK TRAN 이름	저장점까지 취소

원문에 따른 처리 과정

1. 기본 AUTO COMMIT과 구별하여 트랜잭션을 명시한다.
2. 저장점 이후 UPDATE를 수행하고 부분 취소한다.

원문 결과·설명

원문은 480행 영향과 명령 완료 메시지를 표시한다.

반드시 이해할 부분

- 자동 커밋된 문장을 나중에 되돌리는 방식이 아니다.

Oracle 테이블 정의와 기본키·외래키

원문 PDF 422쪽

SQL 원문

```
CREATE TABLE PLAYER (
    PLAYER_ID CHAR(7) NOT NULL,
    PLAYER_NAME VARCHAR2(20) NOT NULL,
    TEAM_ID CHAR(3) NOT NULL,
    E_PLAYER_NAME VARCHAR2(40),
    NICKNAME VARCHAR2(30),
    JOIN_YYYY CHAR(4),
    POSITION VARCHAR2(10),
    BACK_NO NUMBER(2),
    NATION VARCHAR2(20),
    BIRTH_DATE DATE,
    SOLAR CHAR(1),
    HEIGHT NUMBER(3),
    WEIGHT NUMBER(3),
    CONSTRAINT PLAYER_PK PRIMARY KEY (PLAYER_ID),
    CONSTRAINT PLAYER_FK FOREIGN KEY (TEAM_ID) REFERENCES TEAM (TEAM_ID)
);
```

수행하는 작업

선수 속성과 필수값·키·참조 관계를 정의한다.

주요 키워드와 구문의 역할

키워드·구문	역할
유형과 NOT NULL	값의 저장 형태·필수 여부
PRIMARY KEY	선수 ID 유일 식별
REFERENCES	TEAM의 팀 ID 참조

원문에 따른 처리 과정

1. 칼럼과 유형을 정의한다.
2. 테이블 수준에서 기본키와 외래키를 추가한다.

원문 결과·설명

원문은 테이블 생성 완료를 표시한다. TEAM은 앞서 정의된 부모 테이블이다.

반드시 이해할 부분

- 칼럼 수준 NOT NULL과 테이블 수준 키 정의가 함께 쓰인다.

생성 뒤 외래키 추가

원문 PDF 435, 436쪽

SQL 원문

```
ALTER TABLE PLAYER
ADD CONSTRAINT PLAYER_FK
FOREIGN KEY (TEAM_ID) REFERENCES TEAM (TEAM_ID);
```

수행하는 작업

선수의 팀 ID가 팀 테이블을 참조하도록 제약한다.

주요 키워드와 구문의 역할

키워드·구문	역할
ADD CONSTRAINT	제약조건 추가
FOREIGN KEY / REFERENCES	자식·부모 키 지정

원문에 따른 처리 과정

1. 기존 PLAYER 테이블에 참조 제약을 더한다.

원문 결과·설명

원문은 변경 완료를 표시하고 이후 부모 테이블·부모 행 삭제 시 참조 제약 오류 사례를 설명한다.

반드시 이해할 부분

- 이미 저장된 자료와 제약이 양립해야 한다.

입력 후 확정과 객체 조회 권한 부여

원문 PDF 451쪽

SQL 원문

```
INSERT INTO MENU VALUES (1, '화이팅');
COMMIT;
GRANT SELECT ON MENU TO SCOTT;
```

수행하는 작업

SQLD 소유 MENU에 값을 넣고 SCOTT에 조회만 허용한다.

주요 키워드와 구문의 역할

키워드·구문	역할
COMMIT	입력 확정
GRANT SELECT ON	특정 객체의 조회 권한 부여

원문에 따른 처리 과정

1. 원문은 SQLD로 접속한 상태에서 실행한다.
2. 자료를 확정하고 조회 권한을 준다.

원문 결과·설명

원문은 입력 1행·커밋·권한 부여 완료를 표시한다. 이후 SCOTT의 SELECT는 가능하지만 UPDATE는 권한 부족으로 실패한다.

반드시 이해할 부분

- 사용자 생성·접속 권한과 객체별 작업 권한은 다르다.

ROLE로 권한을 묶어 부여

원문 PDF 456쪽

SQL 원문

```
CREATE ROLE LOGIN_TABLE;
GRANT CREATE SESSION, CREATE TABLE TO LOGIN_TABLE;
GRANT LOGIN_TABLE TO SQLD;
```

수행하는 작업

접속·테이블 생성 권한을 ROLE에 묶어 SQLD에 전달한다.

주요 키워드와 구문의 역할

키워드·구문	역할
CREATE ROLE	권한 묶음 생성
GRANT ... TO ROLE	권한을 ROLE에 포함
GRANT ROLE TO USER	사용자에 묶음 부여

원문에 따른 처리 과정

1. LOGIN_TABLE을 만든다.
2. 두 시스템 권한을 넣고 ROLE을 SQLD에 준다.

원문 결과·설명

원문은 각 작업 완료 뒤 SQLD의 접속과 MENU 생성 성공을 보여 준다.

반드시 이해할 부분

- 권한 → ROLE → 사용자라는 중개 관계를 이해한다.

제1장 SQL 수행 구조

과목 III. SQL 고급 활용 및 튜닝

원문 PDF 465-503쪽

Single Block 인덱스 읽기 실험

원문 PDF 494쪽

SQL 원문

```
create table t
as
select * from all_objects;

alter table t add
constraint t_pk primary key(object_id);

select /*+ index(t) */ count(*)
from t where object_id > 0
```

수행하는 작업

테스트 테이블과 PK 인덱스를 만들고 인덱스 스캔의 I/O 요청을 관찰한다.

주요 키워드와 구문의 역할

키워드·구문	역할
CREATE TABLE AS SELECT	ALL_OBJECTS를 이용한 테스트 자료 생성
PRIMARY KEY	실험에 사용할 PK 인덱스 구성
INDEX(t)	인덱스 접근 유도

원문에 따른 처리 과정

1. 원문은 Oracle 9i 실험으로 Range Scan의 읽기 통계와 대기 이벤트를 확인한다.

원문 결과·설명

원문에서 디스크 64블록, db file sequential read 64회가 관찰된다.

반드시 이해할 부분

- 블록 수와 I/O 호출 수를 구분한다.

MultiBlock 방식과 I/O 호출 수 비교

원문 PDF 495쪽

SQL 원문

```
-- 디스크 I/O가 발생하도록 버퍼 캐시 Flushing
alter system flush buffer_cache;

-- Multiblock I/O 방식으로 인덱스 스캔
select /*+ index_ffs(t) */ count(*)
from t where object_id > 0
```

수행하는 작업

앞 실험과 같은 양의 인덱스 블록을 Fast Full Scan으로 읽어 비교한다.

주요 키워드와 구문의 역할

키워드·구문	역할
FLUSH BUFFER_CACHE	원문 실험에서 디스크 읽기를 일으키기 위한 캐시 비우기
INDEX_FFS(t)	Index Fast Full Scan 유도

원문에 따른 처리 과정

1. 인접한 여러 블록을 함께 읽는 방식으로 호출 횟수가 줄어드는지 통계를 확인한다.

원문 결과·설명

같은 64블록을 읽지만 db file scattered read는 9회다. 원문의 결과는 9i 실험 조건에 해당한다.

반드시 이해할 부분

- 두 코드는 블록량이 같아도 I/O 호출 수가 달라질 수 있음을 보인다.
- 이 캐시 조작문은 원문의 실험 절차다.

Oracle 힌트로 조인과 인덱스 접근 유도

원문 PDF 498쪽

SQL 원문

```
select /*+ leading(d) use_nl(e) index(d dept_loc_idx) */ *
from emp e, dept d
where e.deptno = d.deptno
  and d.loc = 'CHICAGO'
```

수행하는 작업

CHICAGO 부서와 사원을 조인하면서 처리 순서·조인 방식·인덱스 경로를 지정한다.

주요 키워드와 구문의 역할

키워드·구문	역할
LEADING(d)	d를 선행 집합으로 유도
USE_NL(e)	e에 NL 조인 사용 유도
INDEX(d dept_loc_idx)	d의 지정 인덱스 사용 유도

원문에 따른 처리 과정

1. 부서의 loc 조건과 사원·부서 조인 조건을 함께 처리한다.
2. 힌트가 실제로 반영됐는지 실행계획으로 확인해야 한다.

원문 결과·설명

원문은 결과 행을 제시하지 않고 실행계획 제어 예시로 사용한다.

반드시 이해할 부분

- Oracle 힌트는 /*+ ... */에 적으며 별칭과 인덱스명이 정확해야 한다.

SQL Server의 테이블·쿼리 힌트

원문 PDF 498쪽

SQL 원문

```
select *
from dept d with (index(dept_loc_idx)), emp e
where e.deptno = d.deptno
  and d.loc = 'CHICAGO'
option (force order, loop join)
```

수행하는 작업

Oracle 예시와 대응하는 접근 제어를 SQL Server의 힌트 위치로 표현한다.

주요 키워드와 구문의 역할

키워드·구문	역할
WITH (INDEX(...))	테이블 인덱스 지정
OPTION	문장 끝에서 쿼리 힌트 지정
FORCE ORDER, LOOP JOIN	기술된 조인 순서와 루프 조인 유도

원문에 따른 처리 과정

1. FROM에 부서를 먼저 적고 FORCE ORDER로 순서를 유도한다.
2. 테이블 힌트와 쿼리 힌트의 적용 위치를 구분한다.

원문 결과·설명

원문에 실행 결과 행은 제시되지 않는다.

반드시 이해할 부분

- DBMS마다 힌트 문법·오류 처리가 다르다.

제2장 SQL 분석 도구

과목 III. SQL 고급 활용 및 튜닝

원문 PDF 504-529쪽

예상 계획을 저장하고 출력

원문 PDF 505쪽

SQL 원문

```
explain plan set statement_id = 'query1' for
select * from emp where empno = 7900;

@?/rdbms/admin/utlxpls
```

수행하는 작업

사원 조회를 실제 수행하기 전에 예상 실행계획을 저장·출력한다.

주요 키워드와 구문의 역할

키워드·구문	역할
EXPLAIN PLAN	예상 계획 생성
STATEMENT_ID	저장 계획 식별
utlxpls	PLAN_TABLE의 계획 출력 스크립트

원문에 따른 처리 과정

1. 계획을 PLAN_TABLE에 저장한다.
2. 출력 스크립트로 트리와 예상 정보를 읽는다.

원문 결과·설명

원문은 EMP의 ROWID 접근과 PK의 INDEX UNIQUE SCAN 계획을 제시한다.

반드시 이해할 부분

- 계획 생성과 실제 SQL 수행은 구분한다.

세션 SQL 트레이스 활성화와 해제

원문 PDF 512쪽

SQL 원문

```
alter session set sql_trace = true;
select * from emp where empno = 7900;
select * from dual;
alter session set sql_trace = false;
```

수행하는 작업

현재 세션에서 수행한 SQL의 실제 처리정보를 파일에 수집한다.

주요 키워드와 구문의 역할

키워드·구문	역할
ALTER SESSION	현재 세션 설정 변경
SQL_TRACE	파일 기반 트레이스 수집 여부

원문에 따른 처리 과정

1. 수집을 켠다.
2. 분석 대상 SQL을 수행한다.
3. 수집을 끄고 .trc 파일을 TKPROF로 분석한다.

원문 결과·설명

원문은 USER_DUMP_DEST 위치에 트레이스 파일이 생성된다고 설명한다.

반드시 이해할 부분

- 원시 파일과 포맷팅 보고서를 구분한다.

SGA에 수집된 커서 수행통계 조회

원문 PDF 515쪽

SQL 원문

```
select *
from table(dbms_xplan.display_cursor(null, null, 'ALLSTATS'));
```

수행하는 작업

직전에 수행한 커서의 실행 단계별 통계를 출력한다.

주요 키워드와 구문의 역할

키워드·구문	역할
DISPLAY_CURSOR	실제 수행한 커서 계획·통계 출력
ALLSTATS	전체 수행통계 표시
NULL 인자	직전에 수행한 커서 지정

원문에 따른 처리 과정

1. 원문 설명대로 STATISTICS_LEVEL=ALL 또는 GATHER_PLAN_STATISTICS로 먼저 통계를 수집한다.
2. Starts·예상/실제 행 수·시간·버퍼 읽기를 확인한다.

원문 결과·설명

원문은 NL 조인의 선행 12행과 후행 인덱스 Starts=12의 관계를 보여준다.

반드시 이해할 부분

- 기본 누적값과 LAST 옵션을 구분한다.

SQL Server 실제 프로필·I/O·시간 측정

원문 PDF 517, 518쪽

SQL 원문

```
use Northwind
go
set statistics profile on
set statistics io on
set statistics time on
go

select count(*)
from dbo.Orders a, dbo.[Order Details] b
where a.OrderID = b.OrderID
go
```

수행하는 작업

주문과 주문상세 조인을 실제 수행하고 연산자·페이지 읽기·시간을 함께 관찰한다.

주요 키워드와 구문의 역할

키워드·구문	역할
STATISTICS PROFILE	실제 행 수·실행 횟수 출력
STATISTICS IO	논리·물리·미리 읽기 출력
STATISTICS TIME	분석·컴파일·실행 시간 출력

원문에 따른 처리 과정

1. SHOWPLAN 설정이 켜져 있다면 먼저 OFF로 한다.
2. 측정 옵션을 켠 후 쿼리를 수행하고 결과 뒤의 통계를 확인한다.

원문 결과·설명

원문의 조인 건수는 2155이며, 예시에서는 논리적 읽기와 실제 계획이 함께 출력된다.

반드시 이해할 부분

- 검색 수·페이지 읽기 수·Rows·Executes를 구분한다.

제3장 인덱스 튜닝

과목 III. SQL 고급 활용 및 튜닝

원문 PDF 530-579쪽

선두 조건에 의한 Range Scan

원문 PDF 533쪽

SQL 원문

```
create index emp_deptno_idx on emp(deptno);

set autotrace traceonly explain
select * from emp where deptno = 20;
```

수행하는 작업

부서번호를 선두 키로 만든 인덱스의 범위 접근을 확인한다.

주요 키워드와 구문의 역할

키워드·구문	역할
CREATE INDEX	deptno 순으로 키·ROWID 저장
TRACEONLY EXPLAIN	실제 조회 없이 예상 계획 확인

원문에 따른 처리 과정

1. 20의 시작 위치를 수직 탐색한다.
2. 같은 부서의 리프 범위를 읽고 필요한 테이블 행에 접근한다.

원문 결과·설명

원문 계획은 INDEX RANGE SCAN과 TABLE ACCESS BY INDEX ROWID다.

반드시 이해할 부분

- SQL Server는 대응 접근을 Index Seek로 표현한다.

함수기반 인덱스의 표현식 일치

원문 PDF 548쪽

SQL 원문

```
select *
from 주문
where nvl(주문수량, 0) < 100

create index emp_x01 on emp( nvl(주문수량, 0) );
```

수행하는 작업

NULL을 0으로 바꾼 표현식을 조건과 인덱스에 동일하게 적용하는 원리를 보인다.

주요 키워드와 구문의 역할

키워드·구문	역할
NVL	주문수량 NULL을 0으로 처리
함수기반 인덱스	가공 결과 자체를 키로 저장

원문에 따른 처리 과정

1. 일반 주문수량 인덱스와 표현식 인덱스의 차이를 설명한다.

원문 결과·설명

원문 설명상 NULL 주문수량은 인덱스에 0으로 저장된다. 원문 생성문의 대상은 emp, 조회 대상은 주문으로 서로 다르다. 원문 표기를 그대로 보존했으며 연속 실행 예제로 해석하지 않는다.

반드시 이해할 부분

- 원문에 있는 테이블명 불일치를 임의로 수정하지 않았다.
- 함수 수행에 따른 DML 부하를 고려한다.

날짜 칼럼을 가공하지 않는 범위 조건

원문 PDF 556쪽

SQL 원문

```
select *
from 주문
where 일시 >= to_date(:dt, 'yyyymmdd')
  and 일시 < to_date(:dt, 'yyyymmdd') + 1
```

수행하는 작업

TO_CHAR(일시) 비교 대신 조회값을 날짜로 변환해 하루 범위를 지정한다.

주요 키워드와 구문의 역할

키워드·구문	역할
TO_DATE(:dt, 'yyyymmdd')	비교값을 날짜로 변환
>= 및 <	당일 시작 포함·다음 날 시작 제외

원문에 따른 처리 과정

1. 인덱스 칼럼을 원형으로 두고 두 경계로 범위를 탐색한다.

원문 결과·설명

원문은 가공 조건에 대한 튜닝 형태를 제시하며 결과 행은 제공하지 않는다.

반드시 이해할 부분

- 날짜·숫자·문자 비교값의 타입을 인덱스 칼럼에 맞춘다.

SQL Server INCLUDE 칼럼

원문 PDF 564쪽

SQL 원문

```
create index emp_x01 on emp (deptno) include (sal)
```

수행하는 작업

deptno를 탐색 키로 두고 sal은 리프에 추가해 테이블 접근을 줄인다.

주요 키워드와 구문의 역할

키워드·구문	역할
INCLUDE (sal)	sal을 리프에만 저장

원문에 따른 처리 과정

1. deptno로 수직 탐색한다.
2. 리프의 sal로 필터하거나 출력하여 테이블 방문을 줄인다.

원문 결과·설명

원문은 생성 구조의 차이를 설명하며 실행 결과는 제시하지 않는다.

반드시 이해할 부분

- INCLUDE 칼럼은 수직 탐색 키가 되지 않는다.

배치 ROWID 접근과 명시적 정렬

원문 PDF 565, 566쪽

SQL 원문

```
create index emp_x01 on emp(deptno, job, empno);

set autotrace traceonly exp;
select /*+ batch_table_access_by_rowid(e) */ *
from emp e
where deptno = 20
order by job, empno;
```

수행하는 작업

배치 I/O를 적용할 수 있는 계획과 결과 정렬의 관계를 확인한다.

주요 키워드와 구문의 역할

키워드·구문	역할
BATCH_TABLE_ACCESS_BY_ROWID	테이블 블록 요청을 모아 처리하는 경로 유도
ORDER BY	결과 순서를 명시

원문에 따른 처리 과정

1. 원문은 ROWID BATCHED 접근 때문에 인덱스 구성상 정렬 대체가 가능해 보여도 SORT ORDER BY가 나타남을 설명한다.

원문 결과·설명

원문 계획에 TABLE ACCESS BY INDEX ROWID BATCHED와 SORT ORDER BY가 함께 나타난다.

반드시 이해할 부분

- 배치 I/O 시 출력 순서가 인덱스 키 순서와 다를 수 있다.

선행 범위를 작은 IN-List로 분리

원문 PDF 569, 570쪽

SQL 원문

```
select 해당층, 평당가, 입력일, 해당동, 매물구분, 연사용일수, 중개업소코드
from 매물아파트매매
where 인터넷매물 in ('1', '2')
  and 아파트시세코드 = 'A01011350900056'
  and 평형 = '59'
  and 평형타입 = 'A'
order by 입력일 desc
```

수행하는 작업

선행 인터넷매물 범위를 각 등치 구간으로 나누어 불필요한 리프 스캔을 줄인다.

주요 키워드와 구문의 역할

키워드·구문	역할
IN ('1', '2')	두 키에 대해 각각 수직 탐색
INLIST ITERATOR	원문 계획의 반복 탐색 표시

원문에 따른 처리 과정

1. 각 인터넷매물 값에서 나머지 등치 조건까지 만족하는 구간을 찾는다.
2. 두 탐색의 결과를 입력일 역순으로 정렬한다.

원문 결과·설명

원문 계획에 INLIST ITERATOR가 나타나며 SQL Server 예시도 스캔 수 2를 보여준다.

반드시 이해할 부분

- 값이 많으면 반복 수직 탐색이 더 비쌀 수 있다.
- 원문에서 열거한 값과 조건을 그대로 유지한다.

제4장 조인 튜닝

과목 III. SQL 고급 활용 및 튜닝

원문 PDF 580-617쪽

NL 조인의 선행 집합과 후행 탐색

원문 PDF 581쪽

SQL 원문

```
select /*+ ordered use_nl(d) */ e.empno, e.ename, d.dname
from emp e, dept d
where d.deptno = e.deptno
```

수행하는 작업

사원 한 행마다 해당 부서를 찾아 조인하는 순서를 유도한다.

주요 키워드와 구문의 역할

키워드·구문	역할
ORDERED	FROM의 테이블 순서 유도
USE_NL(d)	d를 NL의 후행 집합으로 유도

원문에 따른 처리 과정

1. 사원을 읽는다.
2. 사원의 부서번호로 부서를 찾는다.
3. 각 사원에 대해 반복한다.

원문 결과·설명

원문은 중첩 PL/SQL과 같은 접근 순서를 설명하며 결과 행 목록은 제시하지 않는다.

반드시 이해할 부분

- 실제 엔진이 행마다 독립 SQL을 재귀 호출하는 것과는 구분한다.

소트 머지 조인 유도

원문 PDF 589쪽

SQL 원문

```
select /*+ ordered use_merge(e) */ d.deptno, d.dname, e.empno, e.ename
from dept d, emp e
where d.deptno = e.deptno
```

수행하는 작업

부서와 사원을 조인 키로 정렬한 뒤 머지한다.

주요 키워드와 구문의 역할

키워드·구문	역할
USE_MERGE(e)	소트 머지 방식 유도

원문에 따른 처리 과정

1. 양쪽 집합을 deptno로 정렬한다.
2. 같은 키 범위를 맞춰 읽고 다음 키로 진행한다.

원문 결과·설명

원문 계획은 MERGE JOIN 아래 양쪽 SORT(JOIN)과 테이블 Full Scan을 표시한다.

반드시 이해할 부분

- 정렬 비용과 머지 시 반복 랜덤 접근 절감을 함께 본다.

해시 조인 유도

원문 PDF 593쪽

SQL 원문

```
select /*+ ordered use_hash(e) */ d.deptno, d.dname, e.empno, e.ename
from dept d, emp e
where d.deptno = e.deptno
```

수행하는 작업

부서·사원의 등치 관계를 해시 테이블 탐색으로 처리한다.

주요 키워드와 구문의 역할

키워드·구문	역할
USE_HASH(e)	해시 조인 방식 유도

원문에 따른 처리 과정

1. 작은 Build 집합의 조인 키를 해시해 테이블을 만든다.
2. Probe 집합을 읽으며 같은 해시 버킷의 키를 확인한다.

원문 결과·설명

원문 계획은 HASH JOIN과 두 테이블의 Full Scan을 제시한다.

반드시 이해할 부분

- 메모리 초과·해시 키 중복이 성능을 떨어뜨린다.

스칼라 서브쿼리로 부서명 조회

원문 PDF 597쪽

SQL 원문

```
select empno, ename, sal, hiredate
     , (select d.dname from dept d where d.deptno = e.deptno) dname
from emp e
where sal >= 2000
```

수행하는 작업

급여 조건을 만족한 사원마다 부서명을 하나의 값으로 구한다.

주요 키워드와 구문의 역할

키워드·구문	역할
SELECT 목록의 서브쿼리	각 사원에 대응하는 단일 값 반환
e.deptno	메인 쿼리에서 전달하는 입력

원문에 따른 처리 과정

1. 입력 부서번호에 대한 캐시값이 있으면 재사용한다.
2. 없으면 부서를 조회하고 입력·출력을 캐시에 보관한다.

원문 결과·설명

원문은 대응 부서를 못 찾으면 dname이 NULL이라고 설명한다. 이 코드는 원문의 스칼라 조회만 전사했다.

반드시 이해할 부분

- 입력 종류가 적어야 캐시 효과가 크다.
- 실제 계획이 Unnesting됐는지도 확인한다.

상품별 집계 후 상품 조인

원문 PDF 601, 602쪽

SQL 원문

```
select t2.상품명, t1.판매수량, t1.판매금액
from (select 상품코드, sum(판매수량) 판매수량, sum(판매금액) 판매금액
      from 일별상품판매
      where 판매일자 between '20090101' and '20091231'
      group by 상품코드) t1, 상품 t2
where t1.상품코드 = t2.상품코드
```

수행하는 작업

일별 판매를 상품 단위로 먼저 줄여 상품 테이블 조인 횟수를 줄인다.

주요 키워드와 구문의 역할

키워드·구문	역할
인라인 뷰	조인 전 집계된 집합 정의
GROUP BY 상품코드	결과가 필요한 상품 단위로 축약

원문에 따른 처리 과정

1. 연간 판매 365000행을 읽는다.
2. 상품코드별 1000행으로 집계한다.
3. 상품을 1000번 찾아 결합한다.

원문 결과·설명

원문에서 조인 단계 읽기는 730002→2101블록, 전체 소요시간은 약 13.8→5.5초로 감소한다. 이는 원문 실험 결과다.

반드시 이해할 부분

- 집계 단위를 먼저 맞춰 조인 일량을 줄인다.

부등호 자기조인으로 누적매출 계산

원문 PDF 606쪽

SQL 원문

```
select t1.지점, t1.판매월, min(t1.매출) 매출, sum(t2.매출) 누적매출
from 월별지점매출 t1, 월별지점매출 t2
where t2.지점 = t1.지점
  and t2.판매월 <= t1.판매월
group by t1.지점, t1.판매월
order by t1.지점, t1.판매월;
```

수행하는 작업

기준 월 이하의 같은 지점 매출을 모두 합해 누적값을 구한다.

주요 키워드와 구문의 역할

키워드·구문	역할
<= 조인	기준 월 이전과 현재 월 연결
MIN(t1.매출)	반복된 기준행의 매출 한 값 선택
SUM(t2.매출)	기간 내 매출 누계

원문에 따른 처리 과정

1. 같은 지점을 연결한다.
2. 상대 월이 기준 월 이하인 행만 남긴다.
3. 기준 지점·월별로 상대 매출을 합한다.

원문 결과·설명

원문은 윈도우 SUM의 누적 결과와 같은 결과를 얻는 대안으로 설명한다.

반드시 이해할 부분

- 등치 이외 연산자도 업무 관계에 맞게 조인에 사용한다.

선분이력의 임의 시점 조회

원문 PDF 608쪽

SQL 원문

```
select 연체개월수, 연체금액
from 고객별연체금액
where 고객번호 = :cust_num
  and :dt between 시작일자 and 종료일자
```

수행하는 작업

특정 고객의 지정 시점에 유효한 연체 정보를 찾는다.

주요 키워드와 구문의 역할

키워드·구문	역할
BETWEEN	조회 시점이 유효기간 안에 있는지 검사
:dt	과거·현재·미래의 조회 기준일

원문에 따른 처리 과정

1. 고객을 제한한다.
2. 조회일을 포함하는 이력 구간을 선택한다.

원문 결과·설명

원문은 :dt에 '20040815'를 넣는 예시를 설명한다. 결과값은 별도로 추측하지 않는다.

반드시 이해할 부분

- 최종행의 99991231 등치 조회는 미래 예약 입력이 없다는 전제가 필요하다.

제5장 SQL 옵티마이저

과목 III. SQL 고급 활용 및 튜닝

원문 PDF 618-665쪽

전체 처리속도 최적화 모드 지정

원문 PDF 620쪽

SQL 원문

```
alter system set optimizer_mode = all_rows;
alter session set optimizer_mode = all_rows;
```

수행하는 작업

원문에서 전체 처리속도 최적화를 시스템과 세션 레벨에서 지정하는 두 문장을 옮겼다.

주요 키워드와 구문의 역할

키워드·구문	역할
optimizer_mode	옵티마이저의 최적화 목표를 지정한다.
all_rows	전체 결과를 끝까지 읽는 전제의 자원 효율을 목표로 한다.

원문에 따른 처리 과정

1. 시스템 또는 세션 범위에서 최적화 목표를 지정한다.
2. 그 목표를 전제로 실행계획 후보를 평가한다.

원문 결과·설명

원문에는 이 두 설정문에 대한 실행 결과 표가 없다.

반드시 이해할 부분

- 최초 응답속도 최적화와 전체 처리속도 최적화의 전제가 다르다.

로그인 입력값을 바인드 변수로 전달

원문 PDF 631, 632쪽

SQL 원문

```
SELECT *
FROM CUSTOMER
WHERE LOGIN_ID = :LOGIN_ID;
```

수행하는 작업

사용자마다 SQL 문자열을 새로 만드는 대신 로그인 ID만 매개변수로 전달한다.

주요 키워드와 구문의 역할

키워드·구문	역할
:LOGIN_ID	SQL 본문을 바꾸지 않고 실행 시 전달하는 입력값이다.

원문에 따른 처리 과정

1. 최초 실행에서 하드 파싱으로 계획을 만든다.
2. 이후 같은 SQL은 캐시된 계획을 찾아 새 입력값을 바인딩한다.

원문 결과·설명

원문은 바인드 SQL 2만 회 수행의 트레이스에서 라이브러리 캐시 미스가 한 번 발생한 사실을 제시한다. 고객 조회 결과 행은 별도로 제시하지 않는다.

반드시 이해할 부분

- SQL을 공유해도 Parse Call 자체가 자동으로 한 번이 되는 것은 아니다.
- OLTP의 반복 실행에서는 계획 공유가 시스템 확장성에 영향을 준다.

데이터 분포에 따라 UNION ALL로 실행계획 분기

원문 PDF 634쪽

SQL 원문

```
select /*+ FULL(a) */ *
from 아파트매물 a
where :CITY in ('서울시', '경기도')
  and 도시 = :CITY
union all
select /*+ INDEX(a IDX01) */ *
from 아파트매물 a
where :CITY not in ('서울시', '경기도')
  and 도시 = :CITY;
```

수행하는 작업

도시별 데이터 비중 차이로 유리한 액세스 경로가 달라지는 경우, 바인드 변수에 따른 경로 분리를 보여 준다.

주요 키워드와 구문의 역할

키워드·구문	역할
FULL(a)	데이터 비중이 큰 값의 분기에서 Full Scan을 유도한다.
INDEX(a IDX01)	그 밖의 값 분기에서 해당 인덱스 사용을 유도한다.
UNION ALL	서로 다른 입력 조건으로 나눈 두 분기를 연결한다.

원문에 따른 처리 과정

1. 입력 도시가 서울시·경기도인지 각 분기의 조건으로 구분한다.
2. 선택도가 높은 집합과 낮은 집합에 서로 다른 액세스 힌트를 적용한다.

원문 결과·설명

원문은 서울시·경기도는 Full Scan, 강원도·제주도 등은 인덱스 경유가 유리한 상황을 설명한다. 실행시간이나 조회 행 결과는 제시하지 않는다.

반드시 이해할 부분

- 바인드 변수의 평균 분포 가정을 보완하는 원문 사례다.
- 분포와 경로 비용 차이라는 전제를 먼저 확인한다.

IN 서브쿼리와 Unnesting의 출발 구조

원문 PDF 641, 642쪽

SQL 원문

```
select *
from emp
where deptno in (select deptno from dept)
```

수행하는 작업

부서 집합에 포함된 사원만 남기는 쿼리로, 필터 방식과 조인 변환 방식의 차이를 설명한다.

주요 키워드와 구문의 역할

키워드·구문	역할
IN	사원 부서번호가 서브쿼리 결과에 포함되는지 검사한다.
UNNEST / NO_UNNEST	원문에서 제시하는 서브쿼리 변환 유도·방지 힌트다.

원문에 따른 처리 과정

1. 변환하지 않으면 사원 로우의 부서번호를 서브플랜에 넘겨 반복 필터링한다.
2. Unnesting하면 같은 레벨의 조인 구조로 풀고 액세스·조인 방식을 평가한다.

원문 결과·설명

원문은 미변환 계획의 FILTER와 DEPT_PK INDEX UNIQUE SCAN, 변환 계획의 NESTED LOOPS를 대비한다.

반드시 이해할 부분

- 서브쿼리 쪽 유일성이 보장되지 않으면 중복 확대를 막는 UNIQUE 또는 세미 조인이 필요하다.

집계 뷰 안으로 조건절 밀어 넣기

원문 PDF 648, 649쪽

SQL 원문

```
select deptno, avg_sal
from (select deptno, avg(sal) avg_sal
      from emp
      group by deptno) a
where deptno = 30
```

수행하는 작업

외부 부서 조건을 집계 전에 적용해 집계 대상과 액세스량을 줄이는 Predicate Pushdown을 보여 준다.

주요 키워드와 구문의 역할

키워드·구문	역할
GROUP BY deptno	부서별 평균 급여를 구한다.
deptno = 30	외부에 기술됐으나 변환을 통해 뷰 내부 액세스 조건이 된다.

원문에 따른 처리 과정

1. 외부 부서 조건을 내부 쿼리 블록으로 전달한다.

2. 원문의 EMP_DEPTNO_IDX로 해당 부서 범위를 읽고 집계한다.

원문 결과·설명

원문 계획에 INDEX RANGE SCAN과 access(DEPTNO=30)이 나타난다. 전체 사원을 먼저 집계한 뒤 필터링하는 구조를 피한 것이다.

반드시 이해할 부분

- 뷰를 합치지 않더라도 조건을 일찍 적용해 일량을 줄일 수 있다.

조인 관계를 이용한 조건절 이행

원문 PDF 653, 654쪽

SQL 원문

```
select *
from dept d, emp e
where e.job = 'MANAGER'
  and e.deptno = 10
  and d.deptno = e.deptno
```

수행하는 작업

사원 조건이 조인 키를 따라 부서 테이블 필터로 이행되는 원리를 보여 준다.

주요 키워드와 구문의 역할

키워드·구문	역할
e.deptno = 10	사원 쪽에 명시한 상수 조건이다.
d.deptno = e.deptno	d.deptno=10이라는 조건을 추론할 관계다.

원문에 따른 처리 과정

1. 두 부서번호가 같고 사원 부서가 10임을 이용한다.
2. 부서 테이블에도 10 조건을 추가해 각각 먼저 필터링할 수 있게 한다.

원문 결과·설명

원문의 Predicate 정보에서 DEPT_PK의 DEPTNO=10 접근 조건이 확인된다.

반드시 이해할 부분

- 조건절 이행은 조인량을 줄이고 추가 인덱스 경로를 고려하게 한다.

OR 조건을 중복 없는 UNION ALL로 변환

원문 PDF 656, 657쪽

SQL 원문

```
select * from emp
where job = 'CLERK'
union all
select * from emp
where deptno = 20
  and LNNVL(job='CLERK')
```

수행하는 작업

job 조건과 deptno 조건을 나누어 서로 다른 인덱스를 활용하는 원문의 OR-Expansion 형태다.

주요 키워드와 구문의 역할

키워드·구문	역할
UNION ALL	분기별 액세스 경로를 선택할 수 있게 한다.
LNNVL(job='CLERK')	첫 분기에 포함되는 로우를 빼고 조건이 UNKNOWN인 NULL도 원래 의미에 맞게 다룬다.

원문에 따른 처리 과정

1. 첫 분기는 job이 CLERK인 집합을 읽는다.

2. 둘째 분기는 부서 20 중 첫 분기와 중복되지 않는 집합을 읽는다.

원문 결과·설명

원문 계획에는 EMP_JOB_IDX와 EMP_DEPTNO_IDX가 각각 사용된다. 두 번 액세스하므로 교집합이 큰 경우 비용이 늘 수 있다.

반드시 이해할 부분

- OR를 단순 UNION ALL로 바꾸면 중복 결과가 생길 수 있다.

- LNNVL은 조건이 FALSE 또는 UNKNOWN일 때 TRUE를 반환한다.

범위가 공집합이면 하위 액세스 생략

원문 PDF 660, 661쪽

SQL 원문

```
select *
from emp
where sal between :mn and :mx
```

수행하는 작업

하한과 상한의 관계를 먼저 검사하는 내부 필터 추가를 설명하는 원문 쿼리다.

주요 키워드와 구문의 역할

키워드·구문	역할
BETWEEN :mn AND :mx	입력한 급여 범위를 검색한다.
FILTER	원문 계획에서 :mn<=:mx를 검사해 하위 액세스 여부를 결정한다.

원문에 따른 처리 과정

1. 테이블 접근 전에 바인드 하한이 상한 이하인지 검사한다.

2. 조건이 거짓이면 결과가 공집합이므로 TABLE ACCESS FULL 단계를 실행하지 않는다.

원문 결과·설명

원문은 :mn=5000, :mx=100으로 실행했을 때 블록 I/O가 발생하지 않은 통계를 제시한다.

반드시 이해할 부분

- 실행계획에 있는 모든 단계가 항상 수행되는 것은 아니다.

제6장 고급 SQL 튜닝

과목 III. SQL 고급 활용 및 튜닝

원문 PDF 667-762쪽

DISTINCT 대신 연월별 과금 존재 확인

원문 PDF 677쪽

SQL 원문

```
select 연월
from 연월테이블 a
where 연월 <= :yyyymm
  and exists (
    select 'x'
    from 과금
    where 과금연월 = a.연월
      and 지역 like :reg || '%'
  )
```

수행하는 작업

많은 과금 행을 중복 제거하는 대신 연월마다 조건에 맞는 첫 행만 확인한다.

주요 키워드와 구문의 역할

키워드·구문	역할
EXISTS	첫 일치를 찾으면 TRUE를 반환하고 내부 탐색을 끝낸다.
과금연월 = a.연월	외부 연월별로 [과금연월+지역] 인덱스를 탐색할 근거다.

원문에 따른 처리 과정

1. 연월 테이블에서 대상 연월을 읽는다.
2. 해당 월·지역 과금이 존재하는지 확인한다.
3. 존재한 연월만 결과로 남긴다.

원문 결과·설명

원문 트레이스는 35행 출력, 논리적 읽기 82, 경과시간 0.01초를 보인다. 이는 원문의 해당 데이터·환경에서 얻은 수치다.

반드시 이해할 부분

- 중복 제거가 필요한 이유를 존재 확인으로 대체할 수 있는지 검토한다.

정렬 후 10행 제한과 Top N Stopkey

원문 PDF 684, 685쪽

SQL 원문

```
select * from (
  select 거래일시, 체결건수, 체결수량, 거래대금
  from 시간별종목거래
  where 종목코드 = 'KR123456'
    and 거래일시 >= '20080304'
  order by 거래일시
)
where rownum <= 10
```

수행하는 작업

조건에 맞는 거래를 거래일시 순서로 최대 10건 가져온다.

주요 키워드와 구문의 역할

키워드·구문	역할
ORDER BY 거래일시	필요한 결과 순서를 명시한다.
ROWNUM <= 10	상위 10건에서 멈추는 조건이다.

원문에 따른 처리 과정

1. [종목코드+거래일시] 인덱스가 있으면 정렬 순서대로 범위를 읽는다.
2. COUNT(STOPKEY)에서 10건을 읽고 멈춘다.
3. 해당 정렬 인덱스가 없으면 Top N Sort로 후보 집합 메모리를 줄일 수 있다.

원문 결과·설명

원문 계획은 INDEX RANGE SCAN과 COUNT(STOPKEY)를 보이며 SORT ORDER BY는 나타나지 않는다.

반드시 이해할 부분

- Stopkey는 조기 접근 종료, Top N Sort는 정렬 비용 축소라는 차이가 있다.

SQL Server Heap 적재의 TABLOCK 사례

원문 PDF 692, 693쪽

SQL 원문

```
insert into t_heap with (TABLOCK)
select * from t_source
```

수행하는 작업

원문에서 설명하는 최소 로깅의 Heap INSERT 사례다.

주요 키워드와 구문의 역할

키워드·구문	역할
TABLOCK	테이블 Lock을 사용하며 원문 조건에서 최소 로깅에 연결된다.

원문에 따른 처리 과정

1. 원문이 전제하는 Bulk-logged 또는 Simple 복구 모델 등 조건을 확인한다.
2. 소스 집합을 Heap 테이블에 적재한다.

원문 결과·설명

원문은 SQL Server 2008 기준 기능과 테이블 X Lock으로 동시 INSERT가 막힌다는 제약을 설명한다. 실행 통계는 제시하지 않는다.

반드시 이해할 부분

- 힌트만 보고 적용하지 않고 복구 모델·동시성·백업 조건을 함께 읽는다.

집계 소스와 MERGE를 이용한 고객 정보 갱신

원문 PDF 699쪽

SQL 원문

```
merge into 고객 c
using (
  select 고객번호, max(거래일시) 거래일시, sum(거래금액) 거래금액
  from 거래
  where 거래일시 >= trunc(add_months(sysdate,-1))
  group by 고객번호
) t
on (c.고객번호 = t.고객번호)
when matched then update
set c.최종거래일시 = t.거래일시,
    c.최근거래금액 = t.거래금액
```

수행하는 작업

고객별 거래를 한 번 집계한 소스로 일치 고객의 최종 거래일시와 최근 거래금액을 갱신한다.

주요 키워드와 구문의 역할

키워드·구문	역할
USING	갱신값을 제공할 고객별 집계 집합이다.
ON	고객번호가 같은 대상·소스를 대응시킨다.
WHEN MATCHED THEN UPDATE	일치한 고객만 갱신한다.

원문에 따른 처리 과정

1. 기간 조건에 맞는 거래를 고객별로 집계한다.
2. 고객 테이블과 집계 집합을 연결한다.
3. 대응되는 고객 행의 두 칼럼을 변경한다.

원문 결과·설명

원문은 수정 가능 조인 뷰의 대안으로 이 MERGE 형태를 제시하며 변경 행 수나 시간은 제시하지 않는다.

반드시 이해할 부분

- 집계값 조회와 EXISTS를 별도 반복하는 비효율을 줄이는 목적이다.

원격 서버에서 조인·집계해 전송량 축소

원문 PDF 714쪽

SQL 원문

```
select /*+ driving_site(b) */ channel_id,
       sum(quantity_sold) auantity_cold
from order a, sales@lk_sales b
where a.order_date between :1 and :2
  and b.order_no = a.order_no
group by channel_id
```

수행하는 작업

날짜로 줄인 주문을 원격에 보내 조인·집계하고 작은 최종 결과를 받도록 유도한다.

주요 키워드와 구문의 역할

키워드·구문	역할
sales@lk_sales	DB 링크를 통해 참조하는 원격 테이블이다.
DRIVING_SITE(b)	원격 테이블 쪽에서 쿼리를 처리하도록 유도한다.

원문에 따른 처리 과정

1. 원문은 원격 sales 50만 건을 가져오는 기존 계획을 분석한다.
2. 날짜 조건으로 줄인 로컬 주문 939건을 원격으로 보낸다.
3. 원격 조인·집계 후 5건의 결과를 받는다.

원문 결과·설명

원문은 전송하는 원본 집합을 줄이고 최종 5건을 받는 구조로 바뀐 사실을 제시한다. 별칭 auantity_cold는 원문 표기를 유지했다.

반드시 이해할 부분

- 분산 쿼리는 필터 전후·집계 전후의 네트워크 전송량을 함께 비교한다.

범위를 나누어 주문 집계 병렬 처리

원문 PDF 730, 731쪽

SQL 원문

```
select /*+ full(o) parallel(o, 4) */
       count(*) 주문건수, sum(주문수량) 주문수량,
       sum(주문금액) 주문금액
from 주문 o
where 주문일시 between '20100101' and '20101231';
```

수행하는 작업

대상 주문 집합을 병렬로 읽고 집계하는 원문 사례다.

주요 키워드와 구문의 역할

키워드·구문	역할
FULL(o)	병렬 테이블 스캔 경로를 함께 지정한다.
PARALLEL(o, 4)	원문에서 병렬도 4로 작업을 요청한다.

원문에 따른 처리 과정

1. QC가 병렬 서버 집합을 할당하고 작업 범위를 나눈다.
2. 각 서버가 범위별로 읽고 집계한다.
3. QC가 산출물을 통합해 최종 결과를 보낸다.

원문 결과·설명

원문은 병렬 지시와 액세스 경로를 함께 맞추는 법을 설명한다. 주문 합계의 실제 숫자는 제시하지 않는다.

반드시 이해할 부분

- DOP를 높이는 것과 전체 배치 윈도우를 단축하는 것은 같은 목표가 아니다.

CASE 조건부 집계로 납입방법 행을 칼럼으로 변환

원문 PDF 744쪽

SQL 원문

```
INSERT INTO 월별요금납부실적
  (고객번호, 납입월, 지로, 자동이체, 신용카드, 핸드폰, 인터넷)
SELECT 고객번호, 납입월
     , NVL(SUM(CASE WHEN 납입방법코드 = 'A' THEN 납입금액 END), 0) 지로
     , NVL(SUM(CASE WHEN 납입방법코드 = 'B' THEN 납입금액 END), 0) 자동이체
     , NVL(SUM(CASE WHEN 납입방법코드 = 'C' THEN 납입금액 END), 0) 신용카드
     , NVL(SUM(CASE WHEN 납입방법코드 = 'D' THEN 납입금액 END), 0) 핸드폰
     , NVL(SUM(CASE WHEN 납입방법코드 = 'E' THEN 납입금액 END), 0) 인터넷
FROM 월별납입방법별집계
WHERE 납입월 = '200903'
GROUP BY 고객번호, 납입월;
```

수행하는 작업

같은 월별 집계를 방법마다 반복 조회하지 않고 한 번 읽어 고객·월별 여러 금액 칼럼을 만든다.

주요 키워드와 구문의 역할

키워드·구문	역할
CASE WHEN	납입방법에 맞는 금액만 해당 칼럼 집계에 제공한다.
SUM / NVL	합산하고 집계값이 NULL이면 0으로 표시한다.

원문에 따른 처리 과정

1. 해당 월의 집계 데이터를 읽는다.
2. 납입방법별 CASE 식을 계산하며 고객·월별 SUM을 수행한다.
3. 결과를 월별요금납부실적에 입력한다.

원문 결과·설명

원문은 방법별 다섯 번 접근·조인하는 SQL의 대안으로 제시하며 실제 입력 결과 수치는 싣지 않는다.

반드시 이해할 부분

- One SQL 안에서도 동일 데이터의 중복 읽기를 줄이는 것이 핵심이다.

UNION ALL 후 집계로 계획·실적 M:M 관계 해소

원문 PDF 749쪽

SQL 원문

```
select 상품, 연월,
       nvl(sum(계획수량), 0) as 계획수량,
       nvl(sum(실적수량), 0) as 실적수량
from (
  select 상품, 계획연월 as 연월, 계획수량,
         to_number(null) as 실적수량
  from 부서별판매계획
  where 계획연월 between '200901' and '200903'
  union all
  select 상품, 판매연월 as 연월,
         to_number(null) as 계획수량, 판매수량
  from 채널별판매실적
  where 판매연월 between '200901' and '200903'
) a
group by 상품, 연월;
```

수행하는 작업

상품·연월 기준으로 양쪽 집합이 여러 행인 상태에서 직접 조인해 수량이 곱해지는 문제를 피한다.

주요 키워드와 구문의 역할

키워드·구문	역할
TO_NUMBER(NULL)	반대쪽 수량 칼럼을 숫자형 NULL로 맞춘다.
UNION ALL	계획과 실적을 행으로 이어 붙인다.
GROUP BY 상품, 연월	같은 기준에서 두 종류 수량을 각각 합산한다.

원문에 따른 처리 과정

1. 계획 분기와 실적 분기의 출력 칼럼을 맞춘다.
2. 두 집합을 합친다.
3. 상품·연월별 계획·실적을 따로 합산하고 없는 값은 0으로 표시한다.

원문 결과·설명

원문은 상품 A·B·C의 200901~200903별 결과를 제시하며 계획만 있거나 실적만 있는 조합도 포함된다.

반드시 이해할 부분

- 서로 다른 상세 단위를 먼저 맞추지 않고 M:M 조인하면 집계가 중복된다.

페이지 끝의 한 행을 더 읽어 다음 페이지 확인

원문 PDF 750, 751쪽

SQL 원문

```
SELECT *
FROM (
  SELECT ROWNUM NO, 거래일시, 체결건수,
         체결수량, 거래대금, COUNT(*) OVER () CNT
  FROM (
    SELECT 거래일시, 체결건수, 체결수량, 거래대금
    FROM 시간별종목거래
    WHERE 종목코드 = :isu_cd
      AND 거래일시 >= :trd_time
    ORDER BY 거래일시
  )
  WHERE ROWNUM <= :page*:pgsize+1
)
WHERE NO BETWEEN (:page-1)*:pgsize+1 AND :pgsize*:page
```

수행하는 작업

전체 건수를 세지 않고 필요한 페이지와 다음 페이지 존재 여부를 함께 얻는 원문의 기본 페이징 패턴이다.

주요 키워드와 구문의 역할

키워드·구문	역할
ROWNUM 상한 +1	현재 페이지 뒤 데이터 존재를 확인할 한 행을 더 읽는다.
COUNT(*) OVER () CNT	제한된 내부 집합의 건수를 제공한다.
NO BETWEEN	현재 페이지에 해당하는 행만 결과로 남긴다.

원문에 따른 처리 과정

1. 종목·거래일시 조건과 정렬을 적용한다.
2. 페이지 끝 번호보다 한 행 더 읽고 내부 번호와 건수를 계산한다.
3. 바깥에서 해당 페이지 구간을 반환한다.

원문 결과·설명

원문은 pgsize=10, page=3이면 31행을 읽고 21~30번째만 반환한다고 설명한다.

반드시 이해할 부분

- 주로 앞쪽 페이지를 보는 경우의 표준 패턴이며 뒤쪽 반복 조회에는 위치 조건 방식도 검토한다.

최근 비NULL 상태코드를 윈도우로 전달

원문 PDF 755, 756쪽

SQL 원문

```
select 일련번호, 측정값
     , last_value(상태코드 ignore nulls)
       over(order by 일련번호
            rows between unbounded preceding and current row) 상태코드
from 장비측정
order by 일련번호
```

수행하는 작업

상태가 바뀔 때만 저장한 상태코드를 각 측정 행의 가장 최근 비NULL 값으로 표시한다.

주요 키워드와 구문의 역할

키워드·구문	역할
LAST_VALUE ... IGNORE NULLS	프레임 안에서 마지막 비NULL 상태값을 사용한다.
UNBOUNDED PRECEDING ... CURRENT ROW	처음부터 현재 행까지를 계산 범위로 정한다.

원문에 따른 처리 과정

1. 일련번호 순서로 측정 행을 본다.
2. 현재 행까지 나타난 상태코드 중 가장 최근 비NULL을 출력한다.

원문 결과·설명

원문은 전체 집합을 파일로 내려받는 경우의 대안으로 이 SQL을 제시한다. 상태코드를 이어 표시하는 목적 외의 결과는 추가하지 않는다.

반드시 이해할 부분

- 일부 조회의 인덱스 스칼라 접근과 전체 조회의 윈도우 처리를 구분한다.

제7장 Lock과 트랜잭션 동시성 제어

과목 III. SQL 고급 활용 및 튜닝

원문 PDF 764-786쪽

Oracle 잠금 대기 방식 지정

원문 PDF 766쪽

SQL 원문

```
select * from t where no = 1 for update nowait
select * from t where no = 1 for update wait 3
```

수행하는 작업

읽을 행에 변경용 잠금을 시도하며 대기하지 않거나 최대 3초 기다리는 두 대안이다.

주요 키워드와 구문의 역할

키워드·구문	역할
FOR UPDATE	갱신할 목적으로 행을 잠근다.
NOWAIT / WAIT 3	즉시 예외 또는 지정 시간 뒤 예외가 되도록 대기를 통제한다.

원문에 따른 처리 과정

1. 대상 행의 Lock을 시도한다.
2. 다른 변경자의 Lock 때문에 얻지 못하면 각 대기 정책에 따라 예외 처리한다.

원문 결과·설명

원문은 두 줄을 선택 가능한 대기 정책으로 제시하며 조회 결과를 싣지 않는다.

반드시 이해할 부분

- 두 줄은 연속 업무 절차가 아니라 대안이다.
- Lock 예외 이후의 애플리케이션 조치가 필요하다.

SQL Server Update Lock으로 읽은 뒤 갱신

원문 PDF 768쪽

SQL 원문

```
begin tran

select 적립포인트, 방문횟수, 최근방문일시, 구매실적
from 고객 with (updlock)
where 고객번호 = :cust_num

-- 새로운 적립포인트 계산

update 고객 set 적립포인트 = :적립포인트
where 고객번호 = :cust_num

commit
```

수행하는 작업

처음 읽을 때 U Lock을 얻어 같은 고객을 변경할 다른 트랜잭션을 순서대로 처리한다.

주요 키워드와 구문의 역할

키워드·구문	역할
UPDLOCK	갱신을 전제로 읽으며 U끼리의 동시 획득을 막는다.
BEGIN TRAN / COMMIT	읽기·계산·변경을 하나의 트랜잭션으로 묶는다.

원문에 따른 처리 과정

1. 첫 변경자가 읽을 때 U를 얻는다.
2. 두 번째 변경자의 U는 기다린다.
3. 첫 변경자가 X로 전환해 갱신하고 커밋하면 후행 작업이 진행한다.

원문 결과·설명

원문은 S를 함께 얻은 뒤 X로 전환할 때의 교착 가능성을 줄이는 흐름을 설명한다. 실제 적립포인트 결과는 제시하지 않는다.

반드시 이해할 부분

- U는 S와 호환되지만 U와는 호환되지 않는다.
- 콜론 바인드 표기는 원문의 애플리케이션 변수 표기를 유지했다.

트랜잭션 격리 수준 지정

원문 PDF 776쪽

SQL 원문

```
set transaction isolation level serializable;
```

수행하는 작업

트랜잭션 시작 전에 원문이 설명하는 Serializable 격리 수준을 지정한다.

주요 키워드와 구문의 역할

키워드·구문	역할
SET TRANSACTION ISOLATION LEVEL	해당 트랜잭션의 격리 수준을 명시한다.
SERIALIZABLE	원문의 표에서 세 가지 비일관성 읽기를 모두 허용하지 않는 수준이다.

원문에 따른 처리 과정

1. 업무가 요구하는 일관성과 DBMS 구현을 확인한다.
2. 트랜잭션 시작 전에 수준을 설정한다.

원문 결과·설명

원문은 격리성 상승의 예로 제시하며, Lock 방식에서는 공유 Lock 유지로 동시성이 낮아질 수 있음을 설명한다.

반드시 이해할 부분

- 설정만으로 연결이 나뉜 논리적 업무 전체의 충돌 제어가 자동 해결되지는 않는다.

변경일시 비교를 이용한 낙관적 동시성 제어

원문 PDF 780쪽

SQL 원문

```
select 적립포인트, 방문횟수, 최근방문일시, 구매실적, 변경일시
into :a, :b, :c, :d, :mod_dt
from 고객
where 고객번호 = :cust_num;

-- 새로운 적립포인트 계산

update 고객
set 적립포인트 = :적립포인트, 변경일시 = SYSDATE
where 고객번호 = :cust_num
  and 변경일시 = :mod_dt;
```

수행하는 작업

읽는 동안 잠그지 않고 갱신할 때 최종 변경일시가 처음 값과 같은지 확인한다.

주요 키워드와 구문의 역할

키워드·구문	역할
:mod_dt	처음 읽은 최종 변경일시다.
변경일시 = :mod_dt	다른 사용자의 중간 변경 여부를 검사하는 조건이다.

원문에 따른 처리 과정

1. 조회한 값과 변경일시를 보관한다.
2. 새 적립포인트를 계산한다.
3. 처음과 같은 변경일시인 행만 갱신하며 충돌 시 변경 행 수를 확인해 조치한다.

원문 결과·설명

원문은 앞 사례에서 SQL%ROWCOUNT=0이면 다른 사용자의 변경을 알리는 처리를 제시한다. 실제 변경 숫자는 제시하지 않는다.

반드시 이해할 부분

- 낙관적 제어는 잠금을 늦추는 대신 변경 여부 검사를 생략하지 않는다.

보완 B08 → 180쪽

4. 핵심 개념 비교 및 구조화 자료

제1장 데이터 모델링의 이해

과목 I. 데이터 모델링의 이해

원문 PDF 9-76쪽

논리적·물리적 데이터 독립성

기준	논리적 독립성	물리적 독립성
보호하는 경계	개념 스키마와 외부 스키마 사이	내부 스키마와 개념·외부 스키마 사이
변경 대응	논리 구조 변화가 응용에 영향을 주지 않도록 지원	물리 저장 구조 변화가 개념 구조·응용에 영향을 주지 않도록 지원
연결 관리	외부·개념 사상	개념·내부 사상

식별자관계와 비식별자관계

기준	식별자관계	비식별자관계
자식에서의 위치	주식별자 구성에 포함	일반속성에 포함
업무 관계	강한 종속 관계	약한 종속 또는 독립 식별 필요
장점과 유의점	상속 기준으로 직접 조회할 수 있지만 PK 속성이 누적될 수 있다.	PK를 단순화하지만 상위 기준이 차단되면 조인이 늘어날 수 있다.
판단 요인	부모 식별자가 자식 식별과 아래 단계에도 필요한가	독립 PK·상이한 생명주기·선택 관계 등이 필요한가

제2장 데이터 모델과 SQL

과목 I. 데이터 모델링의 이해

원문 PDF 78-118쪽

정규화와 반정규화의 검토 기준

기준	정규화	반정규화
방향	중복을 줄이고 종속성에 맞게 분리	성능을 위해 중복을 허용
갱신과 품질	한 사실의 관리 위치를 모아 변경 부하·불일치 가능성을 줄임	원본 변화 때 중복값 갱신과 정합성 관리가 필요
조회	분리된 정보에 조인이 필요할 수 있음	조인을 줄이거나 인덱스 접근을 개선할 수 있음
순서	기본적으로 먼저 수행	필요성과 다른 대안을 검증한 후 적용

NULL을 포함하는 계산의 위치

형태	원문의 15,000과 NULL 사례	의미
금액-취소금액	NULL	NULL이 연산에 그대로 포함
NVL(금액-취소금액,0)	0	이미 NULL이 된 최종 식을 0으로 대체
NVL(금액,0)-NVL(취소금액,0)	15,000	각 속성을 대체한 후 계산

본질식별자와 인조식별자

기준	본질식별자	인조식별자
구성 근거	업무에서 발생한 구분 속성	인위적으로 만든 번호
원문 입력 사례	주문번호+주문순번 중복은 PK로 차단	새 번호가 붙으면 업무상 동일 행도 PK에 위배되지 않음
조회 인덱스 사례	주문번호+상품번호 PK를 활용	번호 PK 외 주문번호+상품번호 인덱스 추가 필요
선택 기준	업무 규칙과 실제 사용을 우선 확인	과도한 복합 구성 등 필요가 있을 때 비용과 비교

제1장 SQL 기본

과목 II. SQL 기본과 활용

원문 PDF 122-270쪽

WHERE·HAVING·ORDER BY

기준	WHERE	HAVING	ORDER BY
목적	개별 행 제한	그룹 결과 제한	최종 결과 정렬
논리 시점	그룹화 전	그룹화 후	SELECT 후
집계함수	동일 단계의 집계조건으로 사용 불가	그룹 조건으로 사용	그룹 결과에 맞는 집계식으로 정렬 가능
별칭	SELECT보다 먼저 해석	SELECT보다 먼저 해석	SELECT 별칭 재사용 가능

보완 B03 → 175쪽

NATURAL·USING·ON

기준	NATURAL JOIN	USING	ON
연결 대상	동일 이름의 모든 칼럼	지정한 동일 이름 칼럼	명시한 조건의 칼럼·표현식
다른 이름	직접 지정 불가	동일 이름 필요	다른 이름 연결 가능
조인 칼럼 접두사	허용하지 않음	허용하지 않음	모호하지 않게 지정
SELECT * 공통 칼럼	한 열로 처리	지정 공통 칼럼을 한 열로 처리	각 테이블 칼럼을 별도 표시
원문의 SQL Server	미지원으로 설명	미지원으로 설명	지원 문법으로 설명

단일행과 집계의 NULL 처리

대상	동작	원문에서 확인할 점
산술식의 NULL	계산 결과 NULL	더하기 전 COMM을 0으로 대체
SUM·AVG 입력 중 일부 NULL	그 값은 집계 대상에서 제외	AVG의 분모는 NULL 제외 건수
COUNT(*)	전체 행 수	COUNT(HEIGHT)와 다름
없는 행을 NVL로 처리	행이 생기지 않음	JSC 조회 공집합
그룹 없는 MAX의 공집합 결과	NULL 한 결과	NVL(MAX(MGR),9999) 사례

제2장 SQL 활용

과목 II. SQL 기본과 활용

원문 PDF 272-388쪽

집계 수준을 만드는 함수 비교

기준	ROLLUP	CUBE	GROUPING SETS
집계 범위	계층에 따른 소계·총계	가능한 모든 조합	명시한 집계 집합
인수 순서	결과 의미가 달라짐	집계 데이터는 동일	집계 데이터는 동일
복합 괄호	하나의 항목	하나의 항목으로 지정 가능	각 괄호가 하나의 집계 집합
학습 포인트	왼쪽부터 유지되는 수준	원문 기준 2^N 수준과 연산 부하	필요한 집계만 열거

순위·비율 함수의 기준

함수	계산 대상	동률·경계 포인트
RANK	순위	동률 다음 순위 건너뜀
DENSE_RANK	서로 다른 값의 등수	동률 다음 등수 연속
ROW_NUMBER	개별 행 번호	동률도 고유 번호. 세부 정렬 필요
RATIO_TO_REPORT	값 / 파티션 합계	행 비율의 합은 1
PERCENT_RANK	상대 순위 위치	첫 순서 0. 마지막 동률이면 1 미만 가능
CUME_DIST	현재 순서까지 누적 행 비율	동률의 마지막 행까지 포함
NTILE	행을 N개 그룹으로 분배	나머지는 앞 그룹부터 배분

서브쿼리의 위치와 역할

위치·형태	주요 역할	결과 제약
WHERE 중첩	메인 행의 조건 검사	반환 형태에 맞는 연산자
SELECT 스칼라	행마다 값 제공	한 행·한 칼럼, 여러 행 오류
FROM 인라인 뷰	중간 결과를 테이블처럼 사용	출력 열·정렬·후속 처리 기준
HAVING	집계 그룹의 조건 검사	그룹 집계와 비교할 값

제3장 관리 구문

과목 II. SQL 기본과 활용

원문 PDF 390-461쪽

관리 언어 분류

분류	대상	대표 명령
DML	테이블의 행	INSERT, UPDATE, DELETE, MERGE
TCL	트랜잭션	COMMIT, ROLLBACK, SAVEPOINT
DDL	객체 구조	CREATE, ALTER, RENAME, DROP, TRUNCATE
DCL	사용자·권한	CREATE USER, GRANT, REVOKE, DROP USER

보완 B01 → 173쪽 / B02 → 174쪽

DELETE·TRUNCATE·DROP

기준	DELETE	TRUNCATE TABLE	DROP TABLE
대상	조건에 맞는 행 또는 전체 행	전체 행	구조와 모든 자료
구조 유지	유지	유지	제거
WHERE	사용 가능	없음	없음
분류	DML	원문은 DDL로 분류	DDL
취소 문맥	미커밋 변경은 롤백	원문 일반 설명과 SQL Server 명시적 트랜잭션 예외를 함께 확인	원문은 객체 제거로 설명

권한의 구분

구분	예	역할
시스템 권한	CREATE SESSION, CREATE TABLE	접속·종류별 작업 허용
객체 권한	SELECT ON MENU	특정 객체의 특정 작업 허용
ROLE	LOGIN_TABLE	여러 권한을 묶어 전달
SQL Server 서버 역할	로그인의 인스턴스 작업	데이터베이스 역할과 범위 구별
SQL Server 데이터베이스 역할	사용자의 DB 내부 작업	스키마·객체 권한과 연결

제1장 SQL 수행 구조

과목 III. SQL 고급 활용 및 튜닝

원문 PDF 465-503쪽

I/O 분류의 기준

구분	첫 번째 개념	두 번째 개념
논리 / 물리	캐시를 경유해 요청한 블록 읽기	캐시에 없어 디스크에서 읽은 블록
시퀀셜 / 랜덤	논리·물리 순서에 따라 연속 접근	특정 레코드를 찾아 개별 블록 접근
Single / MultiBlock	한 요청에서 한 블록	한 요청에서 인접 여러 블록

메모리 역할 비교

영역	저장 내용	핵심 관계
DB 버퍼 캐시	데이터 블록	Dirty 버퍼 → DBWR → 데이터 파일
공유 풀	메타 정보, SQL·실행계획	실행계획 재사용 → 하드 파싱 감소
로그 버퍼	변경 로그	LGWR → 로그 파일 → 커밋 보장
PGA	프로세스 고유 정보·정렬	공유 직렬화 비용 없이 사용

제2장 SQL 분석 도구

과목 III. SQL 고급 활용 및 튜닝

원문 PDF 504-529쪽

계획과 측정의 차이

대상	시점·성격	알 수 있는 내용
예상 실행계획	실행 전 추정	예상 경로·행 수·비용
Call 통계	실제 수행 전체	단계별 호출·CPU·시간·읽기
Row Source 통계	실제 실행 단계별	출력 행·반복 횟수·읽기 위치
AWR	선택 구간의 시스템 관찰	부하·효율·대기·주요 SQL

제3장 인덱스 튜닝

과목 III. SQL 고급 활용 및 튜닝

원문 PDF 530-579쪽

비슷한 저장 구조의 차이

구조	데이터 배치	주요 비용·조건
일반 B*Tree	키·ROWID와 테이블이 분리	테이블 랜덤 접근
Oracle 클러스터 인덱스	키별 자료를 같은 블록·체인에 배치	키당 클러스터 접근; 변경 패턴 주의
SQL Server 클러스터형	리프가 전체 행 데이터	테이블당 하나; 정렬 유지·분할
Oracle IOT	PK 순 리프가 행 데이터	PK 기준; Secondary의 Logical ROWID
해시 클러스터	같은 해시값 자료를 함께 저장	등치 검색에 한정

보완 B04 → 176쪽

두 가지 칼럼 추가 효과

목표	영향받는 단계	효과
탐색 키 순서 개선	인덱스 스캔	필요한 리프 범위 감소
필터 칼럼 뒤에 추가	테이블 접근	인덱스에서 먼저 걸러 방문 감소
필요 칼럼 모두 포함	테이블 접근	인덱스만으로 처리

제4장 조인 튜닝

과목 III. SQL 고급 활용 및 튜닝

원문 PDF 580-617쪽

세 조인 방식의 비용 구조

기준	NL	소트 머지	해시
주 처리	Outer 행별 Inner 탐색	양쪽 정렬 후 머지	Build 생성 후 Probe
주요 비용	반복 수직·랜덤 접근	정렬·양쪽 스캔	해시 생성·메모리·충돌
유리한 조건	소량·부분범위·적절한 인덱스	정렬 활용·스캔 중심 처리	충분히 작은 Build·대량 조인
부분범위	행별 가능	정렬된 Outer 등 제한 조건	Build 완료 뒤 Probe 단계

이력의 기준 시점

요구	조건의 의미	주의
임의 시점	조회일 BETWEEN 시작 AND 종료	양쪽 이력에 같은 기준 적용
현재·예약 없음	종료일='99991231'	최종행 관리 약속 필요
현재·예약 있음	오늘 BETWEEN 시작 AND 종료	미래 최종행과 현재 유효행 구분
거래 당시	거래일 BETWEEN 시작 AND 종료	현재 정보 조인과 결과 의미가 다름

제5장 SQL 옵티마이저

과목 III. SQL 고급 활용 및 튜닝

원문 PDF 618-665쪽

최적화 목표의 차이

기준	전체 처리속도	최초 응답속도
전제	최종 결과까지 모두 읽음	일부 결과를 읽고 멈춤
목표	전체 자원·처리량 효율	첫 결과의 빠른 응답
주의	일부 결과만 필요한 상황과 구분	끝까지 읽으면 전체 비용이 늘 수 있음

보완 B05 → 177쪽 / B06 → 178쪽

파싱과 커서 재사용의 단계

구분	계획 검색·생성	줄이는 비용
하드 파싱	캐시 미스 후 최적화·계획 생성	계획 공유로 반복 발생을 줄임
소프트 파싱	캐시에서 같은 SQL과 계획 검색	하드 파싱보다 가벼우나 반복 비용 존재
애플리케이션 커서 캐싱	이미 열린 커서를 유지·재사용	반복 Parse Call과 수행 메모리 준비 비용

쿼리 변환이 구조를 바꾸는 위치

변환	변경 위치	조건·주의
Unnesting	서브쿼리를 조인 레벨로 평탄화	중복 확대 방지와 비용 확인
뷰 Merging	뷰 블록과 참조 블록 병합	집계·집합·분석 연산 제약 고려
Pushdown / Pullup	블록 사이에서 필터 조건 이동	결과 의미를 유지할 때 적용
조인 조건 Pushdown	NL 조인의 외부 값을 내부 뷰로 전달	넘긴 값에 대해서만 내부 집계·접근
조건절 이행	관계에서 새 조건 추론	조인 전 필터링과 접근 경로 확대
조인 제거	결과에 불필요한 테이블 제거	유일성·참조 관계·NULL 조건 보장
OR-Expansion	OR를 UNION ALL 분기로 분리	중복과 NULL 의미 유지, 추가 접근 비용 평가

제6장 고급 SQL 튜닝

과목 III. SQL 고급 활용 및 튜닝

원문 PDF 667-762쪽

소트 튜닝 기법의 절감 대상

기법	줄이는 작업	필요 조건·주의
UNION ALL·EXISTS	불필요한 중복 제거·전체 탐색	결과 의미가 같아야 함
인덱스 NOSORT	별도 정렬	조건과 정렬 키 순서가 맞아야 함
First Row / Top N Stopkey	필요한 값·건수 이후의 접근	알맞은 인덱스와 SQL 형태
Top N Sort	비교 횟수·후보 저장량	전체 대상 읽기는 남을 수 있음
소트 후 출력 가공	정렬 행의 폭	정렬 의미를 유지하는 범위
Sort Area 조정	디스크 기록·재읽기	SQL·대상량 개선 뒤 자원 배분 고려

조회·변경·배치의 목표 차이

상황	우선 목표	원문의 수단
일부 온라인 조회	첫 화면과 필요한 행만 처리	인덱스·부분범위·작은 Fetch·페이지
전체 다운로드	전체 결과 효율적 전송	큰 배열 Fetch·전체 처리 계획
대량 변경	인덱스·로그·버퍼 비용 절감	Direct Path·재적재·MERGE
야간 배치 전체	전체 완료시간·자원 경합 완화	윈도우 조정·적정 병렬도·집합·배열

파티션 인덱스의 두 분류 축

분류 축	구분	의미
테이블 파티션 대응	Local / Global	테이블과 1:1 대응 / 독립 구성
인덱스 키 순서	Prefixed / Nonprefixed	파티션 키가 선두 / 선두 아님 또는 키에 없음
미분할	Nonpartitioned	여러 파티션 방문과 정렬 대체 요구를 고려한 대안

WITH 실행 방식

기준	Materialize	Inline
저장	SQL 실행 동안 임시 집합 생성	별도 물리 임시 집합 생성 안 함
재참조	만든 집합 재사용	참조 위치마다 수행
용도	작은 공통 집합의 반복 접근 절감	코드 단순화와 블록 정의
확인	TEMP TABLE TRANSFORMATION 등	명명만으로 한 번 계산을 보장하지 않음

제7장 Lock과 트랜잭션 동시성 제어

과목 III. SQL 고급 활용 및 튜닝

원문 PDF 764-786쪽

SQL Server Lock 호환성(원문 PDF 770쪽, O=호환, -=불가)

모드	IS	S	U	IX	SIX	X
IS	O	O	O	O	O	-
S	O	O	O	-	-	-
U	O	O	-	-	-	-
IX	O	-	-	O	-	-
SIX	O	-	-	-	-	-
X	-	-	-	-	-	-

Oracle 테이블 Lock 호환성(원문 PDF 772쪽, O=호환, -=불가)

모드	Null	RS	RX	S	SRX	X
Null	O	O	O	O	O	O
RS	O	O	O	O	O	-
RX	O	O	O	-	-	-
S	O	O	-	O	-	-
SRX	O	O	-	-	-	-
X	O	-	-	-	-	-

비관적·낙관적 제어

기준	비관적	낙관적
가정	같은 데이터 동시 변경 가능성이 있음	같은 데이터 동시 변경 가능성이 낮음
읽기 시점	잠그고 완료까지 유지	잠그지 않고 최초 상태 보관
갱신 시점	확보한 잠금으로 변경	처음 값·변경일시와 비교
주의	대기·동시성 저하 통제	0행 갱신 등 충돌 검사와 처리 필수

읽기 일관성의 기준 시점

구분	범위·기준	주의
문장 수준	한 SQL의 쿼리 시작 시점	다음 SQL은 다른 시점일 수 있음
트랜잭션 수준	트랜잭션 시작 시점	자신이 수행한 변경은 읽음
MVCC 공통	Undo로 필요한 과거 버전 구성	Undo I/O·CR 생성 비용, 정보 재사용 제약

블로킹과 교착상태

기준	블로킹	교착상태
상태	선행 Lock 해제를 기다림	세션들이 서로 상대 Lock 해제를 기다림
진행	선행 완료 후 후행 진행 가능	한쪽이 물러나지 않으면 진행 불가
원문 대응	짧고 빠른 트랜잭션·대기 제어	접근 순서 통일·Update Lock 등

5. 전체 핵심 내용 복습 자료

전체 핵심 요약

데이터 모델은 저장할 대상·속성·관계·식별자를 정의한다. SQL은 그 모델의 데이터를 원하는 결과 집합으로 표현한다. 실행계획과 I/O 구조는 결과를 얻는 실제 비용을 설명하며, 튜닝은 불필요한 읽기·정렬·호출을 줄인다. 트랜잭션과 동시성 제어는 그 작업이 함께 수행될 때의 정확성을 지킨다.

과목 I. 데이터 모델링의 이해 / 제1장 데이터 모델링의 이해

원문 PDF 9-76쪽

- 업무를 이해한 뒤 관리할 집합·성격·연관성을 각각 엔터티·속성·관계로 표현하고 식별자로 개별 인스턴스를 구분한다. 개념 모델에서 핵심 구조를 잡고 논리 모델에서 규칙을 구체화한 뒤 물리 모델로 구현한다.
- 좋은 모델은 필요한 정보와 업무 규칙을 빠짐없이 표현하면서 중복을 줄이고, 독립성·통합성·재사용성·의사소통을 지원한다. 모델의 품질은 이후 SQL의 정확성과 변경 범위, 데이터 정합성에 직접 영향을 준다.
- ERD는 관계의 개수와 필수 여부를 양방향으로 읽는다. 부모 식별자가 자식 PK에 들어가는지 일반 FK로 들어가는지에 따라 식별자관계와 비식별자관계가 갈리며, 업무 의존성과 실제 조회 조건을 근거로 균형 있게 선택한다.

과목 I. 데이터 모델링의 이해 / 제2장 데이터 모델과 SQL

원문 PDF 78-118쪽

- 정규화는 속성 의미와 함수 종속성을 기준으로 엔터티를 나누어 중복 변경과 정합성 문제를 줄인다. 분리한 정보는 관계가 상속한 식별자를 매핑키로 조인한다. 성능을 위해 중복을 허용할 때도 조회 이득보다 추가 트랜잭션 비용이 커지지 않는지 검증한다.
- 모델은 테이블 구조뿐 아니라 데이터의 동시 발생과 트랜잭션 범위도 표현한다. 주문과 주문상세처럼 함께 생겨야 하는 정보는 한 커밋 단위로 처리한다. 속성에 NULL을 허용했다면 관련 계산·집계·평균의 업무 의미를 정확히 정의한다.
- 식별자는 업무 규칙과 조회 구조에 영향을 준다. 본질식별자의 중복 제약과 인덱스 활용 이점, 인조식별자의 채번 편의와 추가 품질·인덱스 비용을 함께 비교한다.

과목 II. SQL 기본과 활용 / 제1장 SQL 기본

원문 PDF 122-270쪽

- SELECT는 테이블의 필요한 칼럼과 표현식을 선택하고 WHERE로 행을 제한한다. 문자·숫자·날짜·변환 함수와 CASE는 각 행의 값을 가공한다. NULL 대체는 업무 의미와 적용 위치를 확인하고, 행이 없는 공집합과 구분한다.
- GROUP BY는 조건을 통과한 행을 그룹으로 바꾸며 집계함수는 그룹마다 값을 반환한다. COUNT(*)와 COUNT(표현식), SUM·AVG의 NULL 제외 규칙을 구분한다. HAVING은 집계 후 그룹을 고르고 ORDER BY는 최종 순서를 정한다.
- SQL을 해석할 때는 논리 순서와 칼럼의 유효 범위를 따른다. 인라인 뷰의 선택 밖 칼럼과 그룹의 개별 행 값은 밖에서 임의로 사용할 수 없다. 실제 접근·조인·정렬의 수행 순서는 실행계획으로 확인한다.
- 정규화한 정보를 다시 연결하는 조인은 등가·비등가의 조건과 INNER·OUTER·CROSS의 결과 의미를 함께 이해한다. 표준 조인은 ON·USING으로 연결 조건을 드러내며 NATURAL의 자동 공통 칼럼과 외부 조인의 보존 방향을 특히 주의한다.

과목 II. SQL 기본과 활용 / 제2장 SQL 활용

원문 PDF 272-388쪽

- 서브쿼리는 메인 집합의 행을 유지하며 조건이나 값을 제공한다. 조인으로 행이 늘어나는 관계와 구별하고 단일·다중 행, 다중 칼럼, 연관 여부를 각각 판단한다.
- 집합 연산은 같은 모양의 결과를 합·교·차로 결합한다. 여러 집계 수준은 ROLLUP·CUBE·GROUPING SETS로, 개별 행의 분석은 OVER를 갖는 윈도우 함수로 표현한다.
- 순위와 비율의 정의, 프레임의 포함 범위, 정렬 후 Top N, 시작점과 방향을 갖는 계층 전개가 결과를 결정한다.
- 행·열 전환에서는 집계 기준과 NULL 포함 여부를, 패턴 검색에서는 반복·묶기·앵커·반환값의 종류를 확인한다.

과목 II. SQL 기본과 활용 / 제3장 관리 구문

원문 PDF 390-461쪽

- INSERT·UPDATE·DELETE·MERGE로 행을 바꾸고, COMMIT·ROLLBACK·SAVEPOINT로 논리적 작업 단위의 무결성을 유지한다.
- 구조는 CREATE·ALTER·RENAME·DROP·TRUNCATE로 관리한다. 제약조건과 데이터 유형이 허용되는 값과 테이블 간 참조를 정한다.
- 권한은 접속, 종류별 작업, 특정 객체 작업으로 구분하고 ROLE로 묶는다. SQL Server의 로그인·사용자·스키마와 Oracle 사용자 체계를 구별한다.
- 전체 행 삭제, 구조 삭제, 변경 취소 가능성은 명령 이름만으로 일반화하지 않고 DBMS·트랜잭션 문맥까지 함께 확인한다.

과목 III. SQL 고급 활용 및 튜닝 / 제1장 SQL 수행 구조

원문 PDF 465-503쪽

- SQL은 결과 집합을 선언하며 옵티마이저가 이를 실행 절차로 만든다. 파싱·변환·비용 산정·계획 선택·로우소스 생성 뒤 SQL 엔진이 수행한다.
- 저장공간은 블록·익스텐트·세그먼트·테이블스페이스로 연결되고, 일반 읽기와 변경은 버퍼 캐시를 거친다. 로그 선행 기록과 체크포인트가 장애 복구를 뒷받침한다.
- 성능의 중심은 필요한 결과를 얻기 위해 읽는 블록 수다. 캐시 히트율, 물리적 읽기, I/O 호출 수를 구분하고 논리적 읽기 자체를 최소화한다.

과목 III. SQL 고급 활용 및 튜닝 / 제2장 SQL 분석 도구

원문 PDF 504-529쪽

- 실행 전에는 EXPLAIN PLAN·SHOWPLAN으로 계획을 확인하고, 실행 후에는 트레이스와 단계별 실제 통계로 수행량을 검증한다.
- Query·Current·Disk, Starts·E-Rows·A-Rows, 부모·자식 누적값을 구분하면 어디서 읽기와 반복 호출이 늘어나는지 추적할 수 있다.
- 시스템 분석은 CPU 시간과 대기시간을 분리하고 AWR의 측정 구간·부하·효율·이벤트를 함께 해석한다. 순위와 비율만으로 원인을 단정하지 않는다.

과목 III. SQL 고급 활용 및 튜닝 / 제3장 인덱스 튜닝

원문 PDF 530-579쪽

- B*Tree는 수직 탐색으로 시작점을 찾고 정렬된 리프를 수평 스캔한다. 스캔 방식은 선두 조건·유일성·필요 칼럼·정렬 요구에 따라 달라진다.
- 인덱스에서 적게 읽는 것과 테이블을 적게 방문하는 것은 다른 목표다. 칼럼 순서는 스캔 범위에, 필터 칼럼 추가·Covered 구조·INCLUDE·군집 저장은 테이블 접근에 영향을 준다.
- 선행 범위 조건은 후행 조건의 집약성을 떨어뜨릴 수 있다. IN-List는 반복 탐색 비용과 비교하며, 인덱스 설계는 실제 쿼리 패턴과 전체 시스템 비용으로 결정한다.

과목 III. SQL 고급 활용 및 튜닝 / 제4장 조인 튜닝

원문 PDF 580-617쪽

- NL은 행별 반복 탐색, 소트 머지는 정렬된 집합의 머지, 해시는 Build 해시 테이블의 Probe라는 차이가 있다. 작은 반복 조회·대량 스캔·정렬·메모리의 비용을 비교한다.
- 쿼리 구조를 바꿀 때는 집계 단위와 조인 횟수, 캐시 입력 종류, 배타적 관계의 분기·중복 접근을 함께 검토한다.
- 시간 이력은 기준 시점과 시작·종료의 의미가 우선이다. 거래 당시·현재·최종 이력을 구분하고, 성능 기법의 인덱스 및 예약 입력 조건을 지켜야 한다.

과목 III. SQL 고급 활용 및 튜닝 / 제5장 SQL 옵티마이저

원문 PDF 618-665쪽

- 옵티마이저는 주어진 구조·제약·통계·파라미터 안에서 가장 낮은 예상 비용의 계획을 찾는다. 목표가 전체 처리인지 첫 응답인지부터 구분해야 한다.
- 비용 추정은 분포와 가정에 의존한다. 히스토그램은 불균일 분포를 반영하지만 바인드 변수·칼럼 상관관계·운영 환경에서 오차가 남는다.
- SQL 문자열을 동일하게 유지하고 입력값을 바인딩하면 무거운 최적화를 공유할 수 있다. 커서를 유지하면 그 계획을 찾는 반복 비용까지 줄일 수 있다.
- 쿼리 변환은 블록 구조를 풀거나 조건을 이동해 선택 가능한 계획을 넓히는 작업이다. 결과 건수·중복·NULL·제약조건을 보존한 뒤 비용을 비교해야 한다.

과목 III. SQL 고급 활용 및 튜닝 / 제6장 고급 SQL 튜닝

원문 PDF 667-762쪽

- 소트 튜닝은 모델·불필요 연산을 먼저 보고 인덱스 순서, Top N, 행 폭 축소, 메모리 조정을 차례로 연결한다.
- DML은 테이블 한 번 변경으로 끝나지 않는다. 인덱스·Undo·Redo·캐시·조인 비용을 고려해 Direct Path, 재적재, 조인 UPDATE·MERGE를 선택한다.
- One SQL과 Array Processing은 반복 데이터베이스 Call을 줄인다. 부분범위처리·적정 FetchSize·서버 페이징은 사용하지 않을 결과의 접근·전송까지 줄인다.
- 파티션은 관련 저장영역만 고르고, 병렬은 독립 처리 범위를 나눈다. Local/Global·키 순서·Pruning·재분배·QC를 연결해 계획을 읽어야 한다.
- CASE·복제·UNION ALL·윈도우·WITH는 복잡한 절차를 집합으로 바꾸는 수단이다. 같은 SQL 한 문장 안에서도 중복 읽기와 결과 의미를 별도로 검증한다.

과목 III. SQL 고급 활용 및 튜닝 / 제7장 Lock과 트랜잭션 동시성 제어

원문 PDF 764-786쪽

- Lock은 다중 트랜잭션의 충돌 작업을 직렬화해 무결성을 지킨다. 빠른 SQL·짧은 트랜잭션·일정한 접근 순서가 대기와 교착을 줄이는 기반이다.
- 트랜잭션은 ACID를 가진 업무 단위다. Dirty·Non-Repeatable·Phantom Read를 구분하고 실제 요구에 필요한 격리 수준을 선택해야 한다.
- 비관적 제어는 먼저 잠그고, 낙관적 제어는 변경 직전에 처음 읽은 상태와 비교한다. 둘 다 논리적 업무의 충돌을 처리하는 방법이다.
- MVCC는 Undo로 기준 시점의 값을 만들어 읽는다. 문장 수준은 쿼리 시작, 트랜잭션 수준은 트랜잭션 시작을 기준으로 하며, Undo가 사라지면 Snapshot too old가 발생할 수 있다.

SQL 문법 요약표

조회

문법	원문에 근거한 형태	역할·조건 / 원문 PDF
SELECT·ALL·DISTINCT	SELECT [ALL/DISTINCT] 출력 대상 칼럼명, 출력 대상 칼럼명, ... FROM 출력 대상 칼럼들이 있는 테이블명;	ALL은 중복 포함 기본값, DISTINCT는 출력값 중복 제거 출처: 원문 PDF 139-141쪽
전체 칼럼	SELECT * FROM 테이블명;	테이블의 모든 칼럼 조회 출처: 원문 PDF 141쪽
칼럼 별칭	SELECT PLAYER_NAME AS 선수명, POSITION AS 위치, HEIGHT AS 키, WEIGHT AS 몸무게 FROM PLAYER;	결과 칼럼 레이블에 별칭 부여. AS는 생략 가능 출처: 원문 PDF 142-143쪽

문자

문법	원문에 근거한 형태	역할·조건 / 원문 PDF
문자 합성	CONCAT(문자열1, 문자열2) / 문자열1 || 문자열2 / 문자열1 + 문자열2	문자열 연결. ||는 Oracle, +는 원문 SQL Server 표현 출처: 원문 PDF 145-149쪽
대소문자·문자코드	LOWER(문자열), UPPER(문자열), ASCII(문자), CHR/CHAR(ASCII 번호)	대소문자 변환 및 문자·ASCII 번호 대응 출처: 원문 PDF 148-149쪽
부분문자열·길이	SUBSTR/SUBSTRING(문자열, m[, n]), LENGTH/LEN(문자열)	지정 위치·길이의 부분 문자열 또는 문자열 길이 반환. 벤더별 생략 규칙 확인 출처: 원문 PDF 148-152쪽
앞뒤 문자 제거	LTRIM(문자열[, 지정문자]), RTRIM(문자열[, 지정문자]), TRIM([leading|trailing|both] 지정문자 FROM 문자열)	앞·뒤·양쪽 문자 제거. 지정 문자와 방향의 지원 범위는 원문 벤더 설명을 확인 출처: 원문 PDF 149쪽

숫자

문법	원문에 근거한 형태	역할·조건 / 원문 PDF
절댓값·부호·나머지	ABS(숫자), SIGN(숫자), MOD(숫자1, 숫자2)	절댓값, 음수·0·양수 구분, 나머지 계산 출처: 원문 PDF 153-155쪽
올림·내림·반올림·버림	CEIL/CEILING(숫자), FLOOR(숫자), ROUND(숫자[, m]), TRUNC(숫자[, m])	정수 올림·내림 및 지정 소수 자릿수의 반올림·버림 출처: 원문 PDF 153-156쪽
거듭제곱·제곱근·로그	POWER(숫자1, 숫자2), SQRT(숫자), LOG(숫자1, 숫자2), LN(숫자), EXP(숫자)	거듭제곱·제곱근·로그·자연로그·지수 계산. LOG 인수 순서는 벤더별 차이 출처: 원문 PDF 154-155쪽

날짜

문법	원문에 근거한 형태	역할·조건 / 원문 PDF
현재 날짜·시각	SELECT SYSDATE FROM DUAL; / SELECT GETDATE() AS CURRENTTIME;	Oracle / SQL Server의 현재 날짜·시각 조회 출처: 원문 PDF 157-158쪽
날짜 구성값 추출	EXTRACT(YEAR|MONTH|DAY FROM d), DATEPART(YEAR|MONTH|DAY, d), YEAR(d), MONTH(d), DAY(d)	날짜의 연·월·일 구성값 추출. Oracle / SQL Server 대응 출처: 원문 PDF 157-160쪽

변환

문법	원문에 근거한 형태	역할·조건 / 원문 PDF
Oracle 변환	TO_NUMBER(문자열), TO_CHAR(숫자|날짜[, FORMAT]), TO_DATE(문자열[, FORMAT])	문자→숫자, 숫자·날짜→서식 문자, 문자→날짜로 명시적 변환 출처: 원문 PDF 160-162쪽
CAST·CONVERT	CAST(expression AS data_type[(length)]), CONVERT(data_type[(length)], expression[, style])	지정 데이터 유형으로 명시적 변환 출처: 원문 PDF 160-162쪽

분기

문법	원문에 근거한 형태	역할·조건 / 원문 PDF
SIMPLE CASE	CASE expr WHEN comparison_expr THEN return_expr [ELSE default_expr] END	하나의 표현식을 기준값들과 동등 비교하여 반환값 선택 출처: 원문 PDF 164-165쪽
SEARCHED CASE	CASE WHEN condition THEN return_expr [ELSE default_expr] END	WHEN마다 비교·범위 조건을 작성하여 반환값 선택 출처: 원문 PDF 165-167쪽

NULL

문법	원문에 근거한 형태	역할·조건 / 원문 PDF
NVL·ISNULL	NVL(표현식1, 표현식2) / ISNULL(표현식1, 표현식2)	첫 값이 NULL이면 둘째 값으로 대체 출처: 원문 PDF 168-175쪽
NULLIF·COALESCE	NULLIF(표현식1, 표현식2), COALESCE(표현식1, 표현식2, ...)	두 값이 같으면 NULL, 또는 여러 값 중 NULL 아닌 최초값 선택 출처: 원문 PDF 175-178쪽

조건

문법	원문에 근거한 형태	역할·조건 / 원문 PDF
WHERE와 비교	WHERE 조건식 / =, >, >=, <, <=, <>	그룹화 전 개별 행의 검색 조건 적용 출처: 원문 PDF 179-185쪽
IN·BETWEEN	칼럼명 IN (list), 칼럼명 BETWEEN a AND b	목록 중 하나와 일치 또는 양 끝값을 포함한 범위 출처: 원문 PDF 186-190쪽
LIKE·NULL 판정	칼럼명 LIKE '비교문자열', 칼럼명 IS NULL, 칼럼명 IS NOT NULL	%는 0개 이상 문자, _는 한 문자. NULL은 IS NULL로 판정 출처: 원문 PDF 188-191쪽
논리·부정	AND, OR, NOT / NOT BETWEEN a AND b / NOT IN (list)	조건 결합·부정. 우선순위는 괄호, 비교·SQL 연산, NOT, AND, OR 출처: 원문 PDF 180-181, 192-198쪽

집계

문법	원문에 근거한 형태	역할·조건 / 원문 PDF
COUNT·SUM·AVG·MAX·MIN	COUNT(*), COUNT(표현식), SUM([DISTINCT|ALL] 표현식), AVG([DISTINCT|ALL] 표현식), MAX(표현식), MIN(표현식)	행 수·합계·평균·최대·최소. COUNT(*)와 NULL 제외 집계의 차이 확인 출처: 원문 PDF 199-200쪽
GROUP BY·HAVING	GROUP BY 칼럼이나 표현식 HAVING 그룹조건식	기준별 그룹화 후 그룹의 집계 결과 제한 출처: 원문 PDF 201-206쪽

정렬

문법	원문에 근거한 형태	역할·조건 / 원문 PDF
ORDER BY	ORDER BY 칼럼이나 표현식 [ASC 또는 DESC]	최종 결과 정렬. 기본 ASC, 칼럼·별칭·선택 위치 번호 사용 출처: 원문 PDF 214-223쪽

조인

문법	원문에 근거한 형태	역할·조건 / 원문 PDF
등가·비등가 조인	WHERE 테이블2.칼럼명 = 테이블1.칼럼명 / WHERE 테이블1.칼럼명 BETWEEN 테이블2.칼럼명1 AND 테이블2.칼럼명2	값의 동등 또는 범위 관계로 두 테이블 연결 출처: 원문 PDF 226, 233쪽
INNER JOIN·ON	FROM 테이블1 INNER JOIN 테이블2 ON 테이블2.칼럼명 = 테이블1.칼럼명	명시한 연결 조건을 만족하는 행만 반환. INNER 생략 가능 출처: 원문 PDF 226, 242, 251-257쪽
NATURAL·USING	FROM EMP A NATURAL JOIN DEPT B / FROM DEPT A JOIN DEPT_TEMP B USING (DEPTNO)	모든 공통 이름 또는 지정 공통 이름을 동등 비교. 참여 칼럼 접두사 제한 출처: 원문 PDF 243-250쪽
CROSS JOIN	FROM EMP A CROSS JOIN DEPT B	조건 없는 모든 조합 M×N 생성 출처: 원문 PDF 258-260쪽
LEFT·RIGHT·FULL OUTER JOIN	FROM 테이블1 LEFT|RIGHT|FULL OUTER JOIN 테이블2 ON 조인조건	좌측·우측·양쪽의 연결 실패 행까지 보존. OUTER 생략 가능 출처: 원문 PDF 260-266쪽

서브쿼리·집합 결합

문법	원문에 근거한 형태	역할·조건 / 원문 PDF
단일·다중 행 비교	칼럼 = (서브 쿼리) 칼럼 IN (서브 쿼리)	반환 행 수에 맞게 단일·다중 행 연산자를 대응한다. 출처: 273-278쪽
ALL / ANY / SOME	비교연산자 ALL(서브 쿼리) 비교연산자 ANY(서브 쿼리)	모든 값 / 하나 이상의 값 만족 여부. SOME은 ANY와 같다. 출처: 277쪽
EXISTS	EXISTS (SELECT 1 …)	조건을 만족하는 행의 존재만 확인한다. 출처: 281쪽
다중 칼럼 비교	(TEAM_ID, HEIGHT) IN (SELECT TEAM_ID, MIN(HEIGHT) …)	같은 위치의 칼럼 쌍을 동시에 비교한다. 출처: 279쪽
스칼라·인라인 뷰	SELECT … (SELECT …) … FROM (SELECT …) 별칭	한 값을 제공하는 서브쿼리와 중간 집합을 제공하는 서브쿼리를 구별한다. 출처: 281-284쪽
CREATE / DROP VIEW	CREATE VIEW 뷰명 AS SELECT … DROP VIEW 뷰명;	SELECT 정의를 저장·제거한다. 출처: 286-287쪽
집합 연산	SELECT … UNION [ALL] SELECT … INTERSECT / MINUS / EXCEPT	합집합·교집합·차집합. UNION ALL은 중복을 유지한다. 최종 ORDER BY는 마지막이다. 출처: 288-300쪽

다중 수준 집계

문법	원문에 근거한 형태	역할·조건 / 원문 PDF
ROLLUP	GROUP BY ROLLUP(B.DNAME, A.JOB)	계층적 소계와 총계. 인수 순서가 중요하다. 출처: 302-309쪽
CUBE	GROUP BY CUBE(B.DNAME, A.JOB)	그룹 항목의 모든 조합을 집계한다. 출처: 310-312쪽
GROUPING SETS	GROUP BY GROUPING SETS(B.DNAME, A.JOB)	명시한 집계 집합을 결합한다. 출처: 312-316쪽
GROUPING	GROUPING(표현식)	소계로 묶인 항목을 1, 그 외를 0으로 구별한다. 출처: 305-307쪽

윈도우 분석

문법	원문에 근거한 형태	역할·조건 / 원문 PDF
OVER	함수(인수) OVER ([PARTITION BY 칼럼] [ORDER BY ...] [WINDOWING ...])	파티션·분석 순서·프레임을 정한다. 함수별 사용 조건을 따른다. 출처: 317-318쪽
순위 함수	RANK() / DENSE_RANK() / ROW_NUMBER() OVER (...)	동률 뒤 건너뛰기 / 연속 등수 / 고유 행 번호. 출처: 319-322쪽
집계 함수와 OVER	SUM(SAL) OVER (PARTITION BY MGR) MAX / MIN / AVG / COUNT ... OVER (...)	개별 행을 유지하면서 파티션·프레임 집계를 표시한다. 출처: 323-329쪽
ROWS / RANGE	ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING RANGE BETWEEN 50 PRECEDING AND 150 FOLLOWING	물리적 이웃 행 수 / 정렬 값 범위. 파티션 경계와 동률 포함을 주의한다. 출처: 327-329쪽
첫 값·마지막 값	FIRST_VALUE(칼럼) OVER (...) LAST_VALUE(칼럼) OVER (...)	지정 윈도우의 첫·끝 값을 선택한다. 동률의 세부 정렬과 프레임이 중요하다. 출처: 330-332쪽
이전·이후 값	LAG(SAL, 2, 0) OVER (ORDER BY HIREDATE) LEAD(HIREDATE, 1) OVER (...)	앞·뒤 행 값. 이동 거리와 행이 없을 때의 대체값을 인수로 정한다. 출처: 333-334쪽
비율·분할	RATIO_TO_REPORT / PERCENT_RANK / CUME_DIST / NTILE(N) ... OVER (...)	합계 대비 비율, 상대 순위, 누적 비율, N개 그룹 분배를 구별한다. 출처: 335-338쪽

Top N·계층

문법	원문에 근거한 형태	역할·조건 / 원문 PDF
Oracle ROWNUM	SELECT ... FROM (SELECT ... ORDER BY ...) WHERE ROWNUM <= N	먼저 정렬한 집합에 행 제한을 적용한다. 출처: 339-340쪽
SQL Server TOP	TOP(Expression) [PERCENT] [WITH TIES]	행 수·비율 제한. WITH TIES는 마지막 동순위까지 포함한다. 출처: 341-342쪽
ROW LIMITING	[OFFSET offset ROWS] [FETCH FIRST rowcount ROWS ONLY]	정렬 결과에서 건너뛰기·반환 수 제한. 원문은 NEXT·PERCENT·WITH TIES도 설명한다. 출처: 342-343쪽
셀프 조인	FROM EMP WORKER, EMP MANAGER WHERE MANAGER.EMPNO = WORKER.MGR	동일 테이블을 역할별 별칭으로 구분해 연결한다. 출처: 344-347쪽
Oracle 계층 전개	START WITH MGR IS NULL CONNECT BY MGR = PRIOR EMPNO	시작점과 부모·자식 연결 방향을 정한다. 출처: 348-352쪽
계층 표시·경로	LEVEL / CONNECT_BY_ISLEAF SYS_CONNECT_BY_PATH(칼럼, 구분자) CONNECT_BY_ROOT 칼럼	깊이·리프·경로·루트 표시. NOCYCLE과 형제 정렬도 문맥에 맞게 사용한다. 출처: 348-352쪽
재귀 CTE	WITH CTE AS (앵커 SELECT UNION ALL 재귀 SELECT) SELECT ... FROM CTE	앵커부터 이전 결과를 입력으로 반복하고 전체 결과를 합친다. 출처: 353-357쪽

행·열 변환·패턴

문법	원문에 근거한 형태	역할·조건 / 원문 PDF
PIVOT	PIVOT (SUM(SAL) FOR DEPTNO IN (10, 20, 30))	행 값을 열로 펼쳐 집계한다. 대상 밖 열은 집계 기준이다. 출처: 358-362쪽
UNPIVOT	UNPIVOT [INCLUDE NULLS] (SAL FOR DEPTNO IN (D10_SAL AS 10, D20_SAL AS 20))	열을 행으로 전환한다. NULL 포함 여부를 선택한다. 출처: 362-367쪽
정규 표현식 조건	REGEXP_LIKE(source_char, pattern [, match_param])	패턴 일치 여부로 행을 제한한다. 출처: 379-380쪽
정규 표현식 함수	REGEXP_REPLACE / REGEXP_SUBSTR / REGEXP_INSTR / REGEXP_COUNT	치환 문자열 / 일치 문자열 / 위치 / 횟수 반환. 선택 인수는 함수별로 확인한다. 출처: 380-383쪽

보완 B07 → 179쪽

행 변경

문법	원문에 근거한 형태	역할·조건 / 원문 PDF
INSERT VALUES	INSERT INTO 테이블명 [(칼럼1, 칼럼2, ...)] VALUES (값1, 값2, ...);	한 행 입력. 칼럼과 값을 같은 순서로 대응한다. 출처: 391-392쪽
INSERT SELECT	INSERT INTO 테이블명 [(칼럼1, 칼럼2, ...)] 서브 쿼리;	조회된 여러 행을 입력한다. 출처: 393쪽
UPDATE	UPDATE 테이블명 SET 칼럼 = 값 [, ...] [WHERE 조건];	조건 대상 행을 수정한다. WHERE가 없으면 전체 다. 출처: 394-396쪽
DELETE	DELETE [FROM] 테이블명 [WHERE 조건];	조건 대상 행을 삭제한다. 구조는 유지한다. 출처: 396-397쪽
MERGE	MERGE INTO 타깃 USING 소스 ON (조건) WHEN MATCHED THEN UPDATE ... WHEN NOT MATCHED THEN INSERT ...	소스와 타깃의 일치 여부로 수정·입력을 나눈다. 출처: 398-401쪽

트랜잭션

문법	원문에 근거한 형태	역할·조건 / 원문 PDF
확정·전체 취소	COMMIT; ROLLBACK;	미확정 변경을 확정하거나 시작 전으로 취소한다. 출처: 403-409쪽
Oracle 저장점	SAVEPOINT 저장점명; ROLLBACK TO 저장점명;	지정 시점 이후 변경만 취소한다. 출처: 410-416쪽
SQL Server 저장점	BEGIN TRAN SAVE TRAN 저장점명; ROLLBACK TRAN 저장점명;	명시적 트랜잭션 안에서 부분 취소한다. 출처: 407-413쪽

테이블·제약 정의

문법	원문에 근거한 형태	역할·조건 / 원문 PDF
CREATE TABLE	CREATE TABLE 테이블명 (칼럼명 데이터유형 [기본값] [NOT NULL], ...);	칼럼·유형·기본값·제약조건을 정의한다. 출처: 418-424쪽
키·무결성 제약	PRIMARY KEY / UNIQUE / NOT NULL / CHECK / FOREIGN KEY ... REFERENCES ...	유일성·필수값·값 조건·참조 관계를 제한한다. 출처: 423-424쪽
조회 결과로 생성	CREATE TABLE TEAM_TEMP AS SELECT * FROM TEAM; SELECT * INTO TEAM_TEMP FROM TEAM;	Oracle CTAS / SQL Server SELECT INTO. 원문의 제약 복사 제한을 함께 확인한다. 출처: 426-427쪽
칼럼 추가·삭제	ALTER TABLE ... ADD ... ALTER TABLE ... DROP COLUMN ...	칼럼 추가·삭제. DBMS별 괄호·키워드 차이는 상세 노트 참고. 출처: 428-430쪽
칼럼 변경·이름 변경	ALTER TABLE ... MODIFY (...) ALTER TABLE ... ALTER COLUMN ... RENAME COLUMN ... TO ...	유형·기본값·필수 여부·이름 변경. 기존 자료와의 양립 조건을 확인한다. 출처: 431-435쪽
제약 추가·제거	ALTER TABLE ... ADD CONSTRAINT ... ALTER TABLE ... DROP CONSTRAINT ...	기존 테이블의 무결성 제약을 관리한다. 출처: 435-437쪽
이름·전체 제거	RENAME ... TO ... TRUNCATE TABLE ... DROP TABLE ...	이름 변경 / 구조 유지 전체 행 제거 / 구조·데이터 제거. 출처: 438-442쪽

사용자·권한

문법	원문에 근거한 형태	역할·조건 / 원문 PDF
사용자 생성·삭제	CREATE USER ... IDENTIFIED BY ... DROP USER ... [CASCADE]	Oracle 사용자·객체 관리. SQL Server 로그인·사용자 매핑과 구별한다. 출처: 445-446,457쪽
시스템·객체 권한	GRANT CREATE TABLE TO SQLD; GRANT SELECT ON MENU TO SCOTT; REVOKE ... FROM ...	종류별 작업과 특정 객체의 작업 권한을 부여·회수한다. 출처: 445-455쪽
ROLE	CREATE ROLE LOGIN_TABLE; GRANT ... TO LOGIN_TABLE; GRANT LOGIN_TABLE TO SQLD;	권한을 ROLE에 묶어 사용자에게 부여한다. 출처: 454-458쪽

실행·분석·튜닝

문법	원문에 근거한 형태	역할·조건 / 원문 PDF
Oracle 힌트	select /*+ leading(d) use_nl(e) index(d dept_loc_idx) */ * from emp e, dept d where e.deptno = d.deptno and d.loc = 'CHICAGO'	CHICAGO 부서와 사원을 조인하면서 처리 순서·조인 방식·인덱스 경로를 지정한다. 출처: 498쪽
SQL Server 힌트	select * from dept d with (index(dept_loc_idx)), emp e where e.deptno = d.deptno and d.loc = 'CHICAGO' option (force order, loop join)	Oracle 예시와 대응하는 접근 제어를 SQL Server의 힌트 위치로 표현한다. 출처: 498쪽
예상 실행계획	explain plan set statement_id = 'query1' for select * from emp where empno = 7900; @?/rdbms/admin/utlxpls	사원 조회를 실제 수행하기 전에 예상 실행계획을 저장·출력한다. 출처: 505쪽
SQL 트레이스	alter session set sql_trace = true; select * from emp where empno = 7900; select * from dual; alter session set sql_trace = false;	현재 세션에서 수행한 SQL의 실제 처리정보를 파일에 수집한다. 출처: 512쪽

주요 개념 비교표

I-1 / 논리적·물리적 데이터 독립성

기준	논리적 독립성	물리적 독립성
보호하는 경계	개념 스키마와 외부 스키마 사이	내부 스키마와 개념·외부 스키마 사이
변경 대응	논리 구조 변화가 응용에 영향을 주지 않도록 지원	물리 저장 구조 변화가 개념 구조·응용에 영향을 주지 않도록 지원
연결 관리	외부·개념 사상	개념·내부 사상

I-2 / 정규화와 반정규화의 검토 기준

기준	정규화	반정규화
방향	중복을 줄이고 종속성에 맞게 분리	성능을 위해 중복을 허용
갱신과 품질	한 사실의 관리 위치를 모아 변경 부하·불일치 가능성을 줄임	원본 변화 때 중복값 갱신과 정합성 관리가 필요
조회	분리된 정보에 조인이 필요할 수 있음	조인을 줄이거나 인덱스 접근을 개선할 수 있음
순서	기본적으로 먼저 수행	필요성과 다른 대안을 검증한 후 적용

II-1 / WHERE·HAVING·ORDER BY

기준	WHERE	HAVING	ORDER BY
목적	개별 행 제한	그룹 결과 제한	최종 결과 정렬
논리 시점	그룹화 전	그룹화 후	SELECT 후
집계함수	동일 단계의 집계조건으로 사용 불가	그룹 조건으로 사용	그룹 결과에 맞는 집계식으로 정렬 가능
별칭	SELECT보다 먼저 해석	SELECT보다 먼저 해석	SELECT 별칭 재사용 가능

II-2 / 집계 수준을 만드는 함수 비교

기준	ROLLUP	CUBE	GROUPING SETS
집계 범위	계층에 따른 소계·총계	가능한 모든 조합	명시한 집계 집합
인수 순서	결과 의미가 달라짐	집계 데이터는 동일	집계 데이터는 동일
복합 괄호	하나의 항목	하나의 항목으로 지정 가능	각 괄호가 하나의 집계 집합
학습 포인트	왼쪽부터 유지되는 수준	원문 기준 2^N 수준과 연산 부하	필요한 집계만 열거

II-3 / 관리 언어 분류

분류	대상	대표 명령
DML	테이블의 행	INSERT, UPDATE, DELETE, MERGE
TCL	트랜잭션	COMMIT, ROLLBACK, SAVEPOINT
DDL	객체 구조	CREATE, ALTER, RENAME, DROP, TRUNCATE
DCL	사용자·권한	CREATE USER, GRANT, REVOKE, DROP USER

III-1 / I/O 분류의 기준

구분	첫 번째 개념	두 번째 개념
논리 / 물리	캐시를 경유해 요청한 블록 읽기	캐시에 없어 디스크에서 읽은 블록
시퀀셜 / 랜덤	논리·물리 순서에 따라 연속 접근	특정 레코드를 찾아 개별 블록 접근
Single / MultiBlock	한 요청에서 한 블록	한 요청에서 인접 여러 블록

III-2 / 계획과 측정의 차이

대상	시점·성격	알 수 있는 내용
예상 실행계획	실행 전 추정	예상 경로·행 수·비용
Call 통계	실제 수행 전체	단계별 호출·CPU·시간·읽기
Row Source 통계	실제 실행 단계별	출력 행·반복 횟수·읽기 위치
AWR	선택 구간의 시스템 관찰	부하·효율·대기·주요 SQL

III-3 / 비슷한 저장 구조의 차이

구조	데이터 배치	주요 비용·조건
일반 B*Tree	키·ROWID와 테이블이 분리	테이블 랜덤 접근
Oracle 클러스터 인덱스	키별 자료를 같은 블록·체인에 배치	키당 클러스터 접근; 변경 패턴 주의
SQL Server 클러스터형	리프가 전체 행 데이터	테이블당 하나; 정렬 유지·분할
Oracle IOT	PK 순 리프가 행 데이터	PK 기준; Secondary의 Logical ROWID
해시 클러스터	같은 해시값 자료를 함께 저장	등치 검색에 한정

III-4 / 세 조인 방식의 비용 구조

기준	NL	소트 머지	해시
주 처리	Outer 행별 Inner 탐색	양쪽 정렬 후 머지	Build 생성 후 Probe
주요 비용	반복 수직·랜덤 접근	정렬·양쪽 스캔	해시 생성·메모리·충돌
유리한 조건	소량·부분범위·적절한 인덱스	정렬 활용·스캔 중심 처리	충분히 작은 Build·대량 조인
부분범위	행별 가능	정렬된 Outer 등 제한 조건	Build 완료 뒤 Probe 단계

III-5 / 최적화 목표의 차이

기준	전체 처리속도	최초 응답속도
전제	최종 결과까지 모두 읽음	일부 결과를 읽고 멈춤
목표	전체 자원·처리량 효율	첫 결과의 빠른 응답
주의	일부 결과만 필요한 상황과 구분	끝까지 읽으면 전체 비용이 늘 수 있음

III-6 / 소트 튜닝 기법의 절감 대상

기법	줄이는 작업	필요 조건·주의
UNION ALL·EXISTS	불필요한 중복 제거·전체 탐색	결과 의미가 같아야 함
인덱스 NOSORT	별도 정렬	조건과 정렬 키 순서가 맞아야 함
First Row / Top N Stopkey	필요한 값·건수 이후의 접근	알맞은 인덱스와 SQL 형태
Top N Sort	비교 횟수·후보 저장량	전체 대상 읽기는 남을 수 있음
소트 후 출력 가공	정렬 행의 폭	정렬 의미를 유지하는 범위
Sort Area 조정	디스크 기록·재읽기	SQL·대상량 개선 뒤 자원 배분 고려

III-7 / SQL Server Lock 호환성(원문 PDF 770쪽, O=호환, -=불가)

모드	IS	S	U	IX	SIX	X
IS	O	O	O	O	O	-
S	O	O	O	-	-	-
U	O	O	-	-	-	-
IX	O	-	-	O	-	-
SIX	O	-	-	-	-	-
X	-	-	-	-	-	-

주요 코드 모음

모델과 NULL의 의미

정규화 모델에서 고객의 최근 신용카드 조회

원문 PDF 87쪽

SQL 원문

```
SELECT A.결제수단번호
FROM (
    SELECT B.결제수단번호
    FROM 주문 A, 결제 B
    WHERE A.주문번호 = B.주문번호
      AND A.고객번호 = 1234
      AND B.결제수단구분코드 = '신용카드'
    ORDER BY B.결제일시 DESC
) A
WHERE ROWNUM = 1;
```

수행하는 작업

고객 1234의 신용카드 결제 중 최근 한 건의 결제수단번호를 구한다.

상속된 고객번호를 매핑키로 조인

원문 PDF 92쪽

SQL 원문

```
SELECT B.고객명
FROM 주문 A, 고객 B
WHERE A.주문번호 = '1100001'
  AND A.고객번호 = B.고객번호
```

수행하는 작업

특정 주문의 고객명을 고객 엔터티에서 찾아온다.

기본 조회와 조건·조인

ALL과 DISTINCT로 포지션 조회

원문 PDF 140, 141쪽

SQL 원문

```
SELECT ALL POSITION
FROM PLAYER;

SELECT POSITION
FROM PLAYER;

SELECT DISTINCT POSITION
FROM PLAYER;
```

수행하는 작업

중복 포함 조회의 기본값과 포지션 종류만 남기는 조회를 비교한다.

공집합을 집계한 뒤 대체값 출력

원문 PDF 175쪽

SQL 원문

```
SELECT NVL(MAX(MGR), 9999) AS MGR
FROM EMP
WHERE ENAME = 'JSC';
```

수행하는 작업

검색한 사원이 없어도 원문에서 9999 한 결과를 얻는 과정을 보여준다.

급여 범위로 급여등급을 찾는 비등가 조인

원문 PDF 233, 234, 235쪽

SQL 원문

```
SELECT A.ENAME, A.JOB, A.SAL, B.GRADE
FROM EMP A, SALGRADE B
WHERE A.SAL BETWEEN B.LOSAL AND B.HISAL;
```

수행하는 작업

사원 급여가 속한 등급 범위를 연결한다.

서브쿼리·집합·집계

다중 행은 IN으로 비교

원문 PDF 278쪽

SQL 원문

```
SELECT REGION_NAME AS 연고지명, TEAM_NAME AS 팀명, E_TEAM_NAME AS 영문팀명
FROM TEAM
WHERE TEAM_ID IN (SELECT TEAM_ID
                  FROM PLAYER
                  WHERE PLAYER_NAME = '정현수')
ORDER BY TEAM_NAME;
```

수행하는 작업

동명이인의 소속팀을 모두 조회한다.

UNION으로 두 팀 결과 결합

원문 PDF 291, 292쪽

SQL 원문

```
SELECT TEAM_ID AS 팀코드, PLAYER_NAME AS 선수명, POSITION AS 포지션,
       BACK_NO AS 백넘버, HEIGHT AS 키
FROM PLAYER
WHERE TEAM_ID = 'K02'
UNION
SELECT TEAM_ID AS 팀코드, PLAYER_NAME AS 선수명, POSITION AS 포지션,
       BACK_NO AS 백넘버, HEIGHT AS 키
FROM PLAYER
WHERE TEAM_ID = 'K07';
```

수행하는 작업

두 팀 선수 결과를 합집합으로 결합한다.

ROLLUP으로 부서 소계와 총계 생성

원문 PDF 303, 304쪽

SQL 원문

```
SELECT B.DNAME, A.JOB, COUNT(*) AS EMP_CNT, SUM(A.SAL) AS SAL_SUM
FROM EMP A, DEPT B
WHERE B.DEPTNO = A.DEPTNO
GROUP BY ROLLUP(B.DNAME, A.JOB);
```

수행하는 작업

부서·업무별 집계에 부서 소계와 전체 합계를 더한다.

행 분석과 상위 행 제한

RANK와 DENSE_RANK의 동률 처리

원문 PDF 321쪽

SQL 원문

```
SELECT JOB, ENAME, SAL,
       RANK() OVER (ORDER BY SAL DESC) AS RK,
       DENSE_RANK() OVER (ORDER BY SAL DESC) AS DR
FROM EMP;
```

수행하는 작업

급여 순위에서 동률 뒤 등수의 차이를 비교한다.

ROWS 프레임으로 이웃 행 평균

원문 PDF 327, 328쪽

SQL 원문

```
SELECT MGR, ENAME, HIREDATE, SAL,
       ROUND(AVG(SAL) OVER (PARTITION BY MGR ORDER BY HIREDATE
             ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING)) AS AVG_SAL
FROM EMP;
```

수행하는 작업

같은 관리자 아래에서 입사일 순 이웃 행과 자신의 평균 급여를 구한다.

Oracle 정렬 후 상위 3건

원문 PDF 340쪽

SQL 원문

```
SELECT ENAME, SAL
FROM (SELECT ENAME, SAL
      FROM EMP
      ORDER BY SAL DESC)
WHERE ROWNUM <= 3;
```

수행하는 작업

급여 내림차순으로 정렬한 후보에서 3행을 추출한다.

SQL Server TOP WITH TIES

원문 PDF 342쪽

SQL 원문

```
SELECT TOP(2) WITH TIES
       ENAME, SAL
FROM EMP
ORDER BY SAL DESC;
```

수행하는 작업

급여 상위 2개 위치에 마지막 동순위까지 포함한다.

계층·행열 변환·패턴

Oracle 계층형 질의의 순방향 전개

원문 PDF 349, 350쪽

SQL 원문

```
SELECT LEVEL AS LV, LPAD(' ', (LEVEL - 1) * 2) || EMPNO AS EMPNO, MGR,
       CONNECT_BY_ISLEAF AS ISLEAF
FROM EMP
START WITH MGR IS NULL
CONNECT BY MGR = PRIOR EMPNO;
```

수행하는 작업

관리자가 없는 사원부터 부하 방향으로 계층을 조회한다.

PIVOT으로 부서 값을 열로 전환

원문 PDF 358, 359쪽

SQL 원문

```
SELECT *
FROM (SELECT JOB, DEPTNO, SAL FROM EMP)
PIVOT (SUM(SAL) FOR DEPTNO IN (10, 20, 30))
ORDER BY 1;
```

수행하는 작업

업무별 부서 급여 합계를 열로 펼친다.

UNPIVOT에서 NULL 포함

원문 PDF 364, 365쪽

SQL 원문

```
SELECT JOB, DEPTNO, SAL
FROM T1
UNPIVOT INCLUDE NULLS (SAL FOR DEPTNO IN (D10_SAL AS 10, D20_SAL AS 20))
ORDER BY 1, 2;
```

수행하는 작업

부서별 급여 열을 행으로 바꾸면서 NULL 값도 포함한다.

REGEXP_LIKE로 이름 패턴 검색

원문 PDF 380쪽

SQL 원문

```
SELECT FIRST_NAME, LAST_NAME
FROM HR.EMPLOYEES
WHERE REGEXP_LIKE(FIRST_NAME, '^Ste(v|ph)en$');
```

수행하는 작업

Ste로 시작하고 v 또는 ph 다음 en으로 끝나는 이름을 찾는다.

변경·트랜잭션·무결성

MERGE의 일치·불일치 분기

원문 PDF 400쪽

SQL 원문

```
MERGE
INTO TEAM T
USING TEAM_TMP S
ON (T.TEAM_ID = S.TEAM_ID)
WHEN MATCHED THEN
    UPDATE
    SET T.REGION_NAME = S.REGION_NAME,
        T.TEAM_NAME = S.TEAM_NAME,
        T.DDD = S.DDD,
        T.TEL = S.TEL
WHEN NOT MATCHED THEN
    INSERT (T.TEAM_ID, T.REGION_NAME, T.TEAM_NAME, T.STADIUM_ID, T.DDD, T.TEL)
    VALUES (S.TEAM_ID, S.REGION_NAME, S.TEAM_NAME, S.STADIUM_ID, S.DDD, S.TEL);
```

수행하는 작업

원문의 임시 소스 TEAM_TMP로 TEAM 자료를 입력·수정한다.

Oracle 저장점까지 부분 롤백

원문 PDF 412쪽

SQL 원문

```
SAVEPOINT SVPT2;
UPDATE PLAYER SET HEIGHT = 100;
ROLLBACK TO SVPT2;
```

수행하는 작업

키 전체 수정 작업을 저장점 이후 범위에서 취소한다.

생성 뒤 외래키 추가

원문 PDF 435, 436쪽

SQL 원문

```
ALTER TABLE PLAYER
ADD CONSTRAINT PLAYER_FK
FOREIGN KEY (TEAM_ID) REFERENCES TEAM (TEAM_ID);
```

수행하는 작업

선수의 팀 ID가 팀 테이블을 참조하도록 제약한다.

권한 관리

ROLE로 권한을 묶어 부여

원문 PDF 456쪽

SQL 원문

```
CREATE ROLE LOGIN_TABLE;
GRANT CREATE SESSION, CREATE TABLE TO LOGIN_TABLE;
GRANT LOGIN_TABLE TO SQLD;
```

수행하는 작업

접속·테이블 생성 권한을 ROLE에 묶어 SQLD에 전달한다.

제1장 SQL 수행 구조

Single Block 인덱스 읽기 실험

원문 PDF 494쪽

SQL 원문

```
create table t
as
select * from all_objects;

alter table t add
constraint t_pk primary key(object_id);

select /*+ index(t) */ count(*)
from t where object_id > 0
```

수행하는 작업

테스트 테이블과 PK 인덱스를 만들고 인덱스 스캔의 I/O 요청을 관찰한다.

제2장 SQL 분석 도구

예상 계획을 저장하고 출력

원문 PDF 505쪽

SQL 원문

```
explain plan set statement_id = 'query1' for
select * from emp where empno = 7900;

@?/rdbms/admin/utlxpls
```

수행하는 작업

사원 조회를 실제 수행하기 전에 예상 실행계획을 저장·출력한다.

제3장 인덱스 튜닝

선두 조건에 의한 Range Scan

원문 PDF 533쪽

SQL 원문

```
create index emp_deptno_idx on emp(deptno);

set autotrace traceonly explain
select * from emp where deptno = 20;
```

수행하는 작업

부서번호를 선두 키로 만든 인덱스의 범위 접근을 확인한다.

제4장 조인 튜닝

NL 조인의 선행 집합과 후행 탐색

원문 PDF 581쪽

SQL 원문

```
select /*+ ordered use_nl(d) */ e.empno, e.ename, d.dname
from emp e, dept d
where d.deptno = e.deptno
```

수행하는 작업

사원 한 행마다 해당 부서를 찾아 조인하는 순서를 유도한다.

제5장 SQL 옵티마이저

전체 처리속도 최적화 모드 지정

원문 PDF 620쪽

SQL 원문

```
alter system set optimizer_mode = all_rows;
alter session set optimizer_mode = all_rows;
```

수행하는 작업

원문에서 전체 처리속도 최적화를 시스템과 세션 레벨에서 지정하는 두 문장을 옮겼다.

제6장 고급 SQL 튜닝

DISTINCT 대신 연월별 과금 존재 확인

원문 PDF 677쪽

SQL 원문

```
select 연월
from 연월테이블 a
where 연월 <= :yyyymm
  and exists (
    select 'x'
    from 과금
    where 과금연월 = a.연월
      and 지역 like :reg || '%'
  )
```

수행하는 작업

많은 과금 행을 중복 제거하는 대신 연월마다 조건에 맞는 첫 행만 확인한다.

제7장 Lock과 트랜잭션 동시성 제어

Oracle 잠금 대기 방식 지정

원문 PDF 766쪽

SQL 원문

```
select * from t where no = 1 for update nowait
select * from t where no = 1 for update wait 3
```

수행하는 작업

읽을 행에 변경용 잠금을 시도하며 대기하지 않거나 최대 3초 기다리는 두 대안이다.

핵심 내용 체크리스트

과목 I. 데이터 모델링의 이해 / 제1장 데이터 모델링의 이해

[ ] 개념·논리·물리 모델링의 목적과 상세 수준을 설명할 수 있다.

[ ] 3단계 스키마와 두 사상, 두 독립성의 관계를 이해한다.

[ ] 엔터티 성립 조건과 관계·속성의 예외를 구분한다.

[ ] 기본·설계·파생속성과 도메인을 구분한다.

[ ] 관계차수와 선택사양을 두 방향에서 읽을 수 있다.

[ ] 식별자 네 특성과 네 가지 분류 기준을 구분한다.

[ ] 식별자·비식별자관계 선택이 SQL에 주는 영향을 이해한다.

과목 I. 데이터 모델링의 이해 / 제2장 데이터 모델과 SQL

[ ] 다중값·부분 종속·이행 종속을 각각 어느 정규형에서 해결하는지 안다.

[ ] 최근 결제와 송장번호 사례에서 반정규화의 이득과 추가 비용을 설명할 수 있다.

[ ] 식별자 상속과 조인키의 연결을 이해한다.

[ ] 계층형 자기참조와 배타적 관계의 조인 조건을 구분한다.

[ ] 모델의 필수 관계가 한 커밋 단위를 요구하는 사례를 이해한다.

[ ] NULL 연산 전후 처리와 평균 분모의 차이를 이해한다.

[ ] 인조식별자의 중복 데이터·추가 인덱스 비용을 설명할 수 있다.

과목 II. SQL 기본과 활용 / 제1장 SQL 기본

[ ] 테이블·행·칼럼·필드와 PK·FK를 구분한다.

[ ] SELECT의 ALL·DISTINCT·별칭·표현식 의미를 이해한다.

[ ] 주요 문자·숫자·날짜·변환 함수의 역할을 구분한다.

[ ] SIMPLE CASE와 SEARCHED CASE를 구분한다.

[ ] NVL·NULLIF·COALESCE와 NULL·공집합 차이를 이해한다.

[ ] 문자 상수·LIKE 와일드카드·BETWEEN·다중 IN·논리 우선순위를 이해한다.

[ ] COUNT(*)와 COUNT(표현식), 집계에서 NULL의 역할을 이해한다.

[ ] WHERE와 HAVING, GROUP BY와 ORDER BY를 구분한다.

[ ] 논리적 SELECT 순서와 인라인 뷰·그룹의 칼럼 범위를 이해한다.

[ ] 등가·비등가 조인과 테이블 별칭·연결 조건을 이해한다.

[ ] NATURAL·USING·ON의 제약과 차이를 설명할 수 있다.

[ ] INNER·LEFT·RIGHT·FULL·CROSS의 결과 행 보존을 구분한다.

과목 II. SQL 기본과 활용 / 제2장 SQL 활용

[ ] 조인과 서브쿼리가 만드는 결과 단위를 구분한다.

[ ] 단일·다중 행과 다중 칼럼의 조건을 이해한다.

[ ] UNION과 UNION ALL의 중복 의미를 구별한다.

[ ] ROLLUP·CUBE·GROUPING SETS의 집계 수준을 구별한다.

[ ] GROUPING으로 소계 행을 식별하는 이유를 이해한다.

[ ] 윈도우의 파티션·정렬·프레임을 각각 설명할 수 있다.

[ ] RANK·DENSE_RANK·ROW_NUMBER의 동률 처리를 구분한다.

[ ] 정렬 후 행 제한과 WITH TIES의 의미를 이해한다.

[ ] PRIOR의 위치와 계층 전개 방향을 연결한다.

[ ] 재귀 CTE의 앵커·반복·종료·결과 결합을 이해한다.

[ ] PIVOT 집계 기준과 UNPIVOT의 NULL 처리를 구분한다.

[ ] 정규 표현식 조건·치환·문자열·위치·횟수 반환을 구별한다.

과목 II. SQL 기본과 활용 / 제3장 관리 구문

[ ] INSERT의 칼럼·값 대응과 생략 시 DEFAULT·NULL을 이해한다.

[ ] UPDATE 서브쿼리의 NULL 반환 위험과 대상 제한을 이해한다.

[ ] DELETE와 MERGE의 대상 식별을 설명할 수 있다.

[ ] 트랜잭션의 네 특성과 COMMIT·ROLLBACK 효과를 구분한다.

[ ] 저장점의 범위와 이후 저장점 무효화를 이해한다.

[ ] SQL Server 세 트랜잭션 모드를 구분한다.

[ ] PK·UNIQUE·NOT NULL·CHECK·FK를 구분한다.

[ ] CTAS의 제약 복사 제한을 기억한다.

[ ] 칼럼 추가·삭제·변경 조건과 DEFAULT의 적용 시점을 이해한다.

[ ] DELETE·TRUNCATE·DROP의 범위를 구별한다.

[ ] 로그인·사용자·스키마 및 시스템·객체 권한을 구별한다.

[ ] ROLE과 서버·데이터베이스 역할의 범위를 이해한다.

과목 III. SQL 고급 활용 및 튜닝 / 제1장 SQL 수행 구조

[ ] Oracle의 데이터베이스와 인스턴스를 구분할 수 있다.

[ ] 블록·익스텐트·세그먼트·테이블스페이스 관계를 설명할 수 있다.

[ ] WAL, Log Force at Commit, Fast Commit의 연결을 이해한다.

[ ] SGA·PGA·UGA·CGA의 역할을 구분한다.

[ ] 파싱에서 실행까지 엔진별 역할을 설명한다.

[ ] 비용이 실측값이 아님을 이해한다.

[ ] BCHR의 분모에 Disk를 다시 더하지 않는다.

[ ] 인덱스 사용 여부보다 실제 블록 접근량을 평가한다.

과목 III. SQL 고급 활용 및 튜닝 / 제2장 SQL 분석 도구

[ ] AutoTrace 여섯 설정의 실제 수행 여부를 구분한다.

[ ] DISPLAY와 DISPLAY_CURSOR의 대상을 구분한다.

[ ] Query·Current·Disk 및 부모·자식 누적값을 해석한다.

[ ] Starts와 선행 집합 건수의 관계를 이해한다.

[ ] SQL Server의 세 가지 STATISTICS 옵션을 구분한다.

[ ] SQL*Net message from client의 Idle 성격을 이해한다.

[ ] AWR 구간을 분석 목적에 맞게 선택한다.

[ ] 상위 대기를 부하·CPU 상태와 함께 해석한다.

과목 III. SQL 고급 활용 및 튜닝 / 제3장 인덱스 튜닝

[ ] 수직·수평 탐색과 키·ROWID의 역할을 이해한다.

[ ] Range·Full·Unique·Skip·Fast Full·Descending 스캔을 구분한다.

[ ] Oracle의 모든 NULL 키 누락 조건을 이해한다.

[ ] 불균형과 Skew·Sparse를 구분한다.

[ ] 비트맵·함수기반·리버스 키의 장단점을 설명한다.

[ ] 클러스터 인덱스·클러스터형·IOT를 구분한다.

[ ] ROWID 접근의 캐시 검색과 손익분기점 원리를 이해한다.

[ ] Covered와 INCLUDE 및 일반 복합 키의 차이를 안다.

[ ] IN-List의 리프 절감과 수직 탐색 증가를 비교한다.

[ ] 인덱스 설계에서 조회와 DML·공간·관리 비용을 함께 고려한다.

과목 III. SQL 고급 활용 및 튜닝 / 제4장 조인 튜닝

[ ] NL의 세 부하지점과 선행 건수의 영향을 설명한다.

[ ] Prefetch·배치 I/O와 출력 순서의 관계를 이해한다.

[ ] 소트 머지의 정렬 비용과 제한적 부분범위처리를 구분한다.

[ ] Build·Probe·Grace·Recursive 해시 조인을 설명한다.

[ ] Build 메모리 크기와 중복 키 및 수행빈도를 함께 평가한다.

[ ] 스칼라 캐시와 Unnesting에 따른 경로 차이를 이해한다.

[ ] 집계 후 조인으로 줄어드는 반복 횟수를 확인한다.

[ ] 배타적 관계의 UNION ALL 중복 범위를 인덱스 순서로 판단한다.

[ ] 점이력·선분이력과 현재·거래 시점을 구분한다.

[ ] ROWID 튜닝의 인덱스 의존성과 변경 관리 조건을 이해한다.

과목 III. SQL 고급 활용 및 튜닝 / 제5장 SQL 옵티마이저

[ ] 선택도·카디널리티·I/O Call 기준 비용의 관계를 설명할 수 있다.

[ ] 인덱스 비용 식의 수직 탐색·수평 탐색·테이블 랜덤 접근을 구분한다.

[ ] 리터럴 SQL이 동시 사용자 환경에서 파싱 부하를 만드는 이유를 이해한다.

[ ] Static/Dynamic SQL과 바인드 변수 사용 여부를 별도로 판단한다.

[ ] 바인드 Peeking의 최초 입력값 의존성과 계획 분기의 목적을 이해한다.

[ ] 서브쿼리 변환에서 일반 조인·UNIQUE·세미 조인을 구분한다.

[ ] 조인 제거와 OR-Expansion에서 중복·NULL·제약조건을 확인한다.

과목 III. SQL 고급 활용 및 튜닝 / 제6장 고급 SQL 튜닝

[ ] 메모리·Onepass·Multipass 소트와 실제 정렬 없는 SORT AGGREGATE를 구분한다.

[ ] UNION ALL·EXISTS·COUNT 생략의 결과 보장 조건을 설명한다.

[ ] Top N Stopkey와 Top N Sort의 I/O 차이를 이해한다.

[ ] 대량 DML의 인덱스·Undo·Redo 비용 및 Lock·백업 제약을 확인한다.

[ ] 키 보존 조인 뷰와 MERGE를 통한 갱신의 목적을 이해한다.

[ ] User·Recursive 및 Parse·Execute·Fetch Call의 두 분류를 구분한다.

[ ] 전체 전송과 일부 조회에 맞는 배열 크기·페이징을 선택한다.

[ ] 파티션 유형과 정적·동적 Pruning을 실행계획에 연결한다.

[ ] Local/Global과 Prefixed/Nonprefixed를 구분한다.

[ ] QC·서버 집합·테이블 큐·데이터 재분배의 관계를 이해한다.

[ ] PQ_DISTRIBUTE의 분배 방식과 조인 방식이 다른 개념임을 안다.

[ ] CASE·UNION ALL·윈도우·WITH가 중복 접근을 줄이는 이유를 설명한다.

과목 III. SQL 고급 활용 및 튜닝 / 제7장 Lock과 트랜잭션 동시성 제어

[ ] 공유·배타·Update Lock의 호환성을 설명한다.

[ ] 블로킹과 교착상태를 구분하고 원자성을 유지하며 대기를 줄인다.

[ ] Oracle 테이블 RX Lock이 다른 RX를 막는다는 오해를 구분한다.

[ ] ACID와 세 가지 비일관성 읽기의 관계를 설명한다.

[ ] 네 격리 수준의 허용 현상과 제품별 지원 범위를 구분한다.

[ ] 비관적 선점과 낙관적 변경 검사의 차이를 설명한다.

[ ] MVCC에서 미커밋뿐 아니라 기준 이후 커밋도 과거 버전으로 읽는 이유를 이해한다.

[ ] 문장과 트랜잭션 수준의 기준 시점을 구분한다.

[ ] Snapshot too old와 Undo 재사용·긴 쿼리·잦은 커밋의 관계를 이해한다.

핵심 보완 찾아보기

원래 회차와 문제 순서는 그대로 따라갑니다. 아래 보완 자료는 해당 개념이 헷갈리는 날의 학습 25분 안에서 본문과 바꾸어 읽습니다. 추가 과제로 더하지 않습니다.

노란색은 정의·공식·판단 조건입니다. 「헷갈리는 지점」은 혼동하기 쉬운 경계를 설명합니다. 강조 개수는 출제 빈도나 배점을 뜻하지 않습니다.

B01. PERCENT_RANK의 마지막 값이 항상1은 아니다

핵심 개념 비교 | 순위·비율 함수 | 본문 138쪽 → 보완 자료의 같은 번호

B02. 스칼라 서브쿼리는 0행·1행·여러행을 나눠 본다

서브쿼리의 위치와 역할 | 본문 138쪽 → 보완 자료의 같은 번호

B03. NULL 대체 위치가 계산 의미를 바꾼다

NULL을 포함하는 계산의 위치 | 본문 136쪽 → 보완 자료의 같은 번호

B04. 실행계획의 예상·실측·반복 횟수를 구분한다

계획과 측정의 차이 | 본문 140쪽 → 보완 자료의 같은 번호

B05. 조인 방식 변경 전후에 행 의미를 검증한다

세 조인 방식의 비용 구조 | 본문 141쪽 → 보완 자료의 같은 번호

B06. 복합 인덱스의 탐색·필터·정렬 역할을 분리한다

두 가지 칼럼 추가 효과 | 본문 141쪽 → 보완 자료의 같은 번호

B07. Top N은 인원·동률·최종 정렬을 명시한다

Top N·행 제한 | 본문 153쪽 → 보완 자료의 같은 번호

B08. 낙관적 잠금은 버전 조건과 변경 행 수를 함께 쓴다

낙관적 동시성 제어 | 본문 134쪽 → 보완 자료의 같은 번호

정답·해설은 보완 문제 뒤의 별도 절에서 확인합니다. PDF 책갈피의 「핵심 보완」에서 각 주제로 바로 이동할 수 있습니다.

B01. PERCENT_RANK의 마지막 값이 항상1은 아니다

연결: 핵심 개념 비교 | 순위·비율 함수 | 본문 138쪽

PERCENT_RANK=(RANK-1)/(n-1), 단일행 파티션은 0이다.

원리와 판단 조건

동률 행은 같은 RANK를 갖는다. 최종 동률 집단의 RANK가 n보다 작으면 마지막 행의 PERCENT_RANK도 1보다 작다.

CUME_DIST는 현재 정렬값까지의 누적 행 수를 n으로 나누며 동률 끝까지 포함한다. 최종 동률 집단은 모두 1이다.

PERCENT_RANK는 순위의 상대적 위치다. 금액이 파티션 합계에서 차지하는 비율인 RATIO_TO_REPORT와 구분한다.

적용·손풀이

점수 100,90,90을 DESC 정렬하면 RANK는 1,2,2다. PERCENT_RANK는 0,0.5,0.5이고 CUME_DIST는 1/3,1,1이다. 마지막 값이 1이라는 설명은 두 행 이상이고 마지막 정렬값이 유일한 경우에 성립하며 일반 규칙이 아니다.

헷갈리는 지점

PERCENT_RANK와 CUME_DIST를 같은 값의 백분위 함수로 취급하지 않는다.

직접 확인

한 행뿐인 파티션의 PERCENT_RANK와 CUME_DIST는?

참고: PostgreSQL 순위·분포 함수 (www.postgresql.org)

B02. 스칼라 서브쿼리는 0행·1행·여러행을 나눠 본다

연결: 서브쿼리의 위치와 역할 | 본문 138쪽

스칼라 문맥은 한 열·최대 한 행이며 0행이면 NULL로 취급한다.

원리와 판단 조건

한 값이 필요한 문맥에서 여러 행이 나오면 PostgreSQL·Oracle 등에서는 오류가 난다. SQLite의 동작을 다른 제품에 일반화하지 않는다.

COALESCE는 0행으로 인한 NULL을 대체할 수 있어도 여러 행 오류를 해결하지 않는다.

최대값이 필요하면 MAX처럼 의미를 정의한다. LIMIT 1로 아무 행이나 고르면 원하는 값이라는 보장이 없으므로 정렬 기준을 함께 확인한다.

적용·손풀이

고객 1의 계좌 잔액이 100,200일 때 SELECT (SELECT balance FROM account WHERE cid=1)은 여러 행 문제다. 최대 잔액을 원하면 SELECT MAX(balance)...로 200을 구한다. 계좌가 없는 고객의 스칼라 서브쿼리 결과는 NULL이다.

헷갈리는 지점

DISTINCT나 LIMIT 1로 오류를 없애는 것과 요구한 값을 구하는 것은 다르다.

직접 확인

MAX 집계에 입력 행이 없을 때 스칼라 결과는?

참고: PostgreSQL 스칼라 서브쿼리 (www.postgresql.org)

B03. NULL 대체 위치가 계산 의미를 바꾼다

연결: NULL을 포함하는 계산의 위치 | 본문 136쪽

연산 전체를 0으로 대체하는 것과 각 입력을 0으로 대체하는 것은 다르다.

원리와 판단 조건

금액 15000, 취소금액 NULL이면 금액-취소금액은 NULL이다. COALESCE(금액-취소금액,0)은 0이다.

COALESCE(금액,0)-COALESCE(취소금액,0)은 15000이다. 미확인 취소를 0으로 보겠다는 업무 가정이 추가된다.

SUM(x-y)와 SUM(x)-SUM(y)도 NULL의 위치에 따라 달라진다. 먼저 데이터 사전에서 NULL의 의미를 확인한다.

적용·손풀이

행 (금액,취소)=(100,NULL),(200,50)에서 SUM(금액-취소)=150, SUM(금액)-SUM(취소)=250이다. 앞은 첫 행의 차액 NULL을 제외하고, 뒤는 각 열의 알려진 값을 따로 합산한다.

헷갈리는 지점

수식이 일반 대수에서 같다는 이유로 NULL이 있는 SQL에서도 같다고 판단하지 않는다.

직접 확인

미확인 취소액을 0원으로 바꾸는 식은 단순한 오류 수정인가?

참고: PostgreSQL 조건식과 COALESCE (www.postgresql.org)

B04. 실행계획의 예상·실측·반복 횟수를 구분한다

연결: 계획과 측정의 차이 | 본문 140쪽

예상 비용은 시간이 아니며 actual rows는 loops와 함께 읽는다.

원리와 판단 조건

EXPLAIN은 예상 계획을 보여 주고 EXPLAIN ANALYZE는 실제 실행을 수반한다. 변경문에서는 분석 과정의 실행 효과를 고려한다.

PostgreSQL의 actual time과 rows는 여러 번 실행된 노드에서 반복당 평균으로 표시된다. rows=3,loops=100이면 총 출력은 약 300행이다.

상위 노드 시간에 하위 작업이 포함될 수 있어 노드 시간을 모두 더하면 중복될 수 있다. 추정 행 수 오차와 I/O·대기도 함께 점검한다.

적용·손풀이

Nested Loop의 안쪽 노드가 actual rows=2,loops=1000이면 총 출력은 약 2000행이다. 바깥쪽 실제 행 수가 예상보다 100배 많으면 후행 접근도 크게 늘 수 있다. 첫 추정 오차가 생긴 노드부터 찾는다.

헷갈리는 지점

cost=1000을 1000ms로 읽거나 rows=2만 보고 작업이 작다고 판단하지 않는다.

직접 확인

하위 노드 10ms와 상위 노드 12ms를 합쳐 22ms로 보고하면?

참고: PostgreSQL EXPLAIN 해석 (www.postgresql.org)

B05. 조인 방식 변경 전후에 행 의미를 검증한다

연결: 세 조인 방식의 비용 구조 | 본문 141쪽

NL·Hash·Merge는 실행 전략이며 요구한 결과의 의미는 유지되어야 한다.

원리와 판단 조건

NL은 선행 행마다 후행 탐색을 수행하는 형태이므로 선행 건수와 후행 접근 경로의 영향을 크게 받는다.

Hash Join은 해시 테이블을 만드는 build 측의 크기와 메모리·분할 비용을 고려한다. Merge Join은 정렬 입력을 이용하므로 필요한 정렬 비용도 본다.

EXISTS를 JOIN으로 바꾸면 오른쪽 중복으로 왼쪽 행이 늘 수 있다. UNION을 UNION ALL로 바꾸는 것도 중복 제거 의미를 바꾼다.

적용·손풀이

A의 키 1이 한 행이고 B의 키 1이 세 행이면 INNER JOIN 결과는 3행이다. WHERE EXISTS는 A의 한 행을 남긴다. JOIN이 더 빠를 것 같다는 이유로 서로 교체할 수 없다.

헷갈리는 지점

튜닝 검증을 시간 비교만으로 마치지 않는다. 중복·NULL·외부 조인·동률 처리 결과도 확인한다.

직접 확인

A와 B에 같은 값의 행이 하나씩 있을 때 UNION과 UNION ALL의 결과 행 수는?

참고: PostgreSQL 집합 연산 (www.postgresql.org)

B06. 복합 인덱스의 탐색·필터·정렬 역할을 분리한다

연결: 두 가지 칼럼 추가 효과 | 본문 141쪽

읽는 인덱스 범위의 축소와 테이블 방문 감소는 다른 효과다.

원리와 판단 조건

선두 동등 조건 뒤 첫 범위 조건은 B-tree 탐색 구간을 좁히는 전형적 형태다. 뒤의 열 조건은 인덱스 내부 필터로 테이블 방문을 줄일 수 있다.

필요한 모든 열을 포함해도 항상 인덱스만 읽는 것은 아니다. PostgreSQL index-only scan에는 가시성 확인 같은 조건도 관련된다.

인덱스 열 추가는 쓰기 유지비·용량·캐시 비용을 늘릴 수 있다. 한 SQL의 읽기 성능만 보고 무제한 추가하지 않는다.

적용·손풀이

인덱스 (customer_id,ts,status)에서 customer_id=7 AND ts>=시작 AND status=완료라면 고객과 시간 범위를 찾은 뒤 status로 테이블 방문을 줄일 수 있다. 후행 status 조건이 탐색 구간까지 같은 비율로 좁힌다고 보장할 수는 없다.

헷갈리는 지점

인덱스에 열이 있다는 것과 첫 탐색 범위를 줄인다는 것을 같은 의미로 쓰지 않는다.

직접 확인

SELECT에 없는 열을 인덱스에 추가하면 효과가 없는가?

참고: PostgreSQL 복합 인덱스 규칙 (www.postgresql.org)

참고: PostgreSQL Index-Only Scans (www.postgresql.org)

B07. Top N은 인원·동률·최종 정렬을 명시한다

연결: Top N·행 제한 | 본문 153쪽

상위 N행과 상위 N개 값 수준은 다르다. 동률을 유지할 순위 키와 최종 출력의 정렬 키를 구분한다.

원리와 판단 조건

정확히 N행은 ROW_NUMBER() OVER(ORDER BY amount DESC,id ASC)로 고른다. 상위 N개 금액 수준은 DENSE_RANK() OVER(ORDER BY amount DESC)로 고른다. id는 후자의 순위 키에 넣지 않고 최종 출력 정렬에만 사용해야 금액 동률이 유지된다.

Oracle ROWNUM 제한을 같은 질의 블록의 ORDER BY보다 먼저 적용하는 실수를 피한다. 대상 엔진의 행 제한 문법을 확인한다.

NULL 금액을 마지막에 둘지, 같은 금액에서 작은 id를 먼저 둘지까지 정렬 기준을 완성한다.

적용·손풀이

id1:100,id2:90,id3:90에서 amount DESC,id ASC의 ROW_NUMBER<=2는 id1,2다. 동률을 포함한 두 금액 수준은 id1,2,3이다. RANK<=2도 이 예에서는 세 행이나 100,100,90에서는 두 행이어서 DENSE_RANK와 다르다.

헷갈리는 지점

한 예제에서 RANK와 DENSE_RANK 결과가 같았다고 항상 대체할 수 있다고 판단하지 않는다.

직접 확인

100,100,90에서 RANK<=2와 DENSE_RANK<=2는?

참고: PostgreSQL 윈도 함수 기초 (www.postgresql.org)

B08. 낙관적 잠금은 버전 조건과 변경 행 수를 함께 쓴다

연결: 낙관적 동시성 제어 | 본문 134쪽

읽은 버전이 그대로일 때만 저장하고 실패하면 새 상태로 재판단한다.

원리와 판단 조건

version=5인 행을 읽고 값을 계산했다면 WHERE id=1 AND version=5로 조건을 두고 성공 시 version=6으로 증가시킨다.

다른 거래가 먼저 버전을 바꿨으면 0행 갱신된다. 기존 계산을 강제로 덮어쓰지 않고 재읽기·재계산·사용자 조정 등 충돌 정책을 따른다.

변경일시를 버전으로 쓰면 정밀도·갱신 규칙·동일 시각 가능성을 점검한다. 전용 증가 버전은 판정 의도를 명확히 한다.

적용·손풀이

잔액 100·version=5를 T1,T2가 읽었다. T1이 110·version=6으로 바꾸면 T2의 UPDATE … WHERE version=5는 0행이다. T2는 영향 행 수를 확인해야 한다. 이미 전송한 알림 같은 외부 부작용은 재시도 때 중복되지 않게 별도로 설계한다.

헷갈리는 지점

버전 조건 없이 UPDATE … WHERE id=1만 쓰면서 낙관적 잠금이라고 부르지 않는다.

직접 확인

갱신 0행이면 반드시 동시 수정 충돌인가?

참고: PostgreSQL UPDATE와 영향 행 수 (www.postgresql.org)

보완 문제 정답·해설

문제를 먼저 풀고 첫 오류를 확인합니다. 풀이를 닫은 뒤 같은 조건을 다시 설명할 수 있으면 다음 회차로 넘어갑니다.

B01. PERCENT_RANK의 마지막 값이 항상1은 아니다

각각 0과 1이다. n=1인 경계에서 PERCENT_RANK를 직접 나눗셈 오류가 발생하는 식으로 해석하지 않는다.

B02. 스칼라 서브쿼리는 0행·1행·여러행을 나눠 본다

집계 결과 한 행의 NULL이다. 서브쿼리 자체가 0행인 경우와 값은 같아도 행 생성 원리는 다르다.

B03. NULL 대체 위치가 계산 의미를 바꾼다

아니다. 값의 의미를 변경하는 업무 가정이다. 미확인과 취소 없음이 같은 상태인지 먼저 정한다.

B04. 실행계획의 예상·실측·반복 횟수를 구분한다

상위 시간에 하위 작업이 포함될 수 있어 틀릴 수 있다. 엔진별 측정 정의와 총 실행시간을 확인한다.

B05. 조인 방식 변경 전후에 행 의미를 검증한다

각각 1과 2다. 중복 보존이 요구되는 경우 성능만을 이유로 연산자를 바꾸면 안 된다.

B06. 복합 인덱스의 탐색·필터·정렬 역할을 분리한다

아니다. WHERE 필터·정렬·조인 조건에 기여할 수 있다. 실제 효과와 유지 비용은 실행계획 및 측정으로 확인한다.

B07. Top N은 인원·동률·최종 정렬을 명시한다

RANK는 1,1,3이라 두 행, DENSE_RANK는 1,1,2라 세 행이다.

B08. 낙관적 잠금은 버전 조건과 변경 행 수를 함께 쓴다

행 부재·삭제도 가능하다. 요구에 따라 원인을 판정하되 0행을 성공으로 간주하지 않는다.